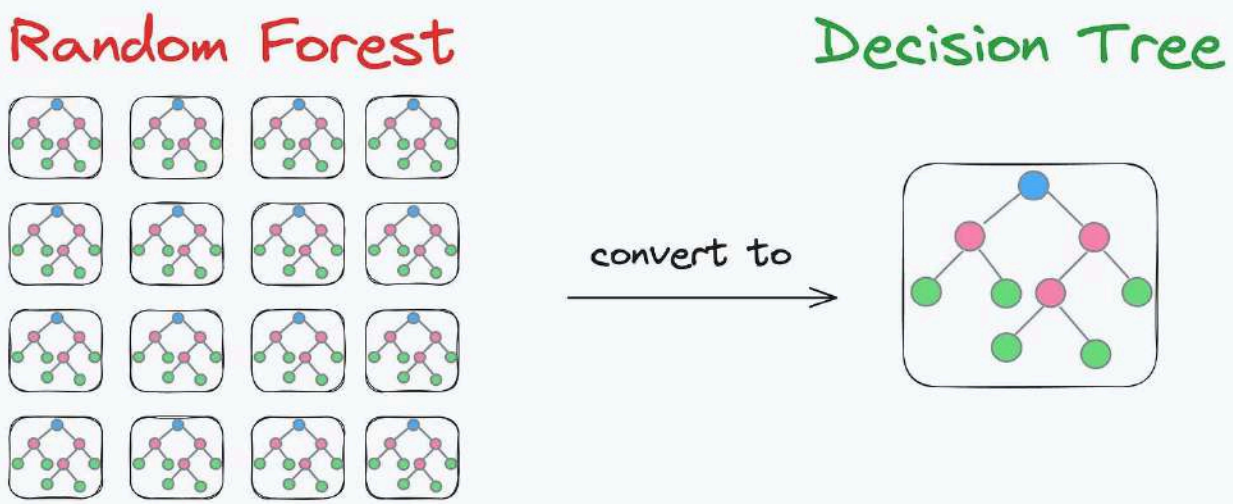
Decision Trees and Ensemble Methods

Condense Random Forest into a Decision Tree

There's an interesting technique, using which, we can condense an entire random forest model into a single decision tree.



The benefits?

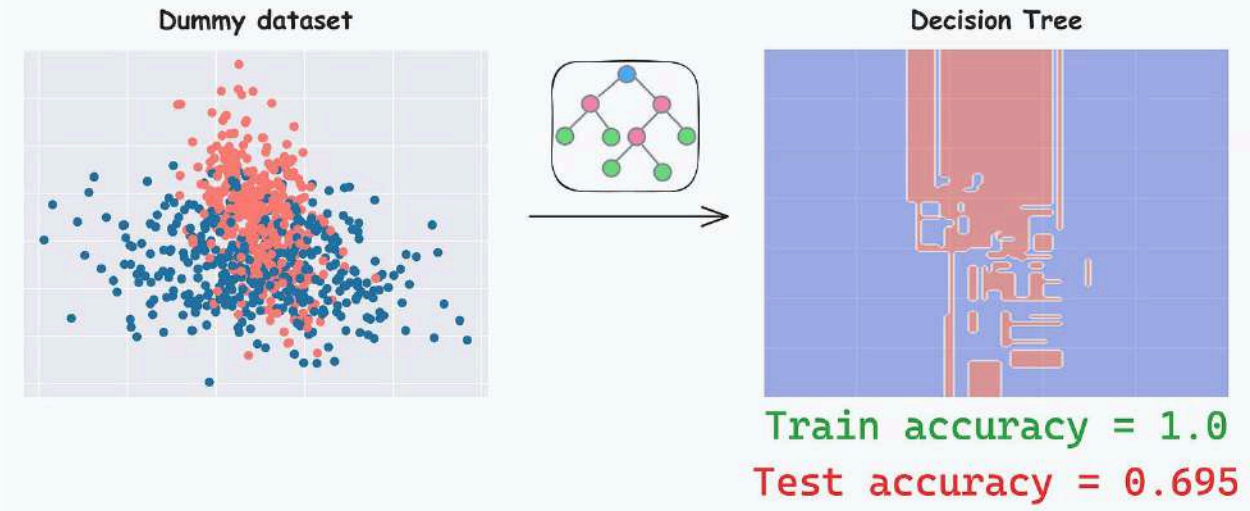
This technique can:

- Decrease the prediction run-time.
- Improve interpretability.
- Reduce the memory footprint.
- Simplify the model.
- Preserve the generalization power of the random forest model.

Let's understand in this chapter.

Technique Walkthrough

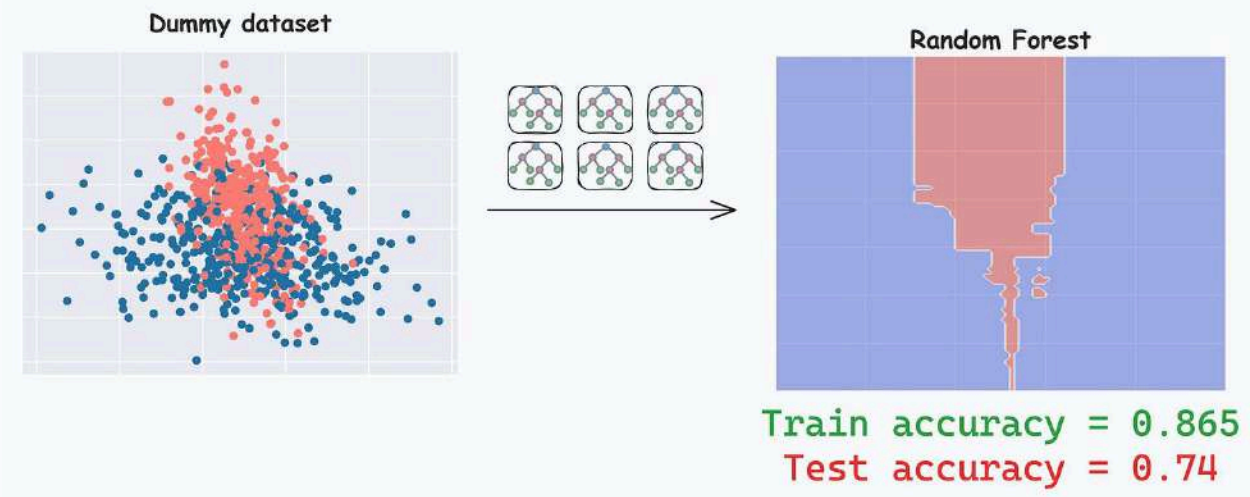
Let's fit a decision tree model on the following dummy dataset. It produces a decision region plot shown on the right.



It's clear that there is high overfitting.

In fact, we must note that, by default, a decision tree can always 100% overfit any dataset (we will use this information shortly). This is because it is always allowed to grow until all samples have been classified correctly.

This overfitting problem is resolved by a random forest model, as depicted below:

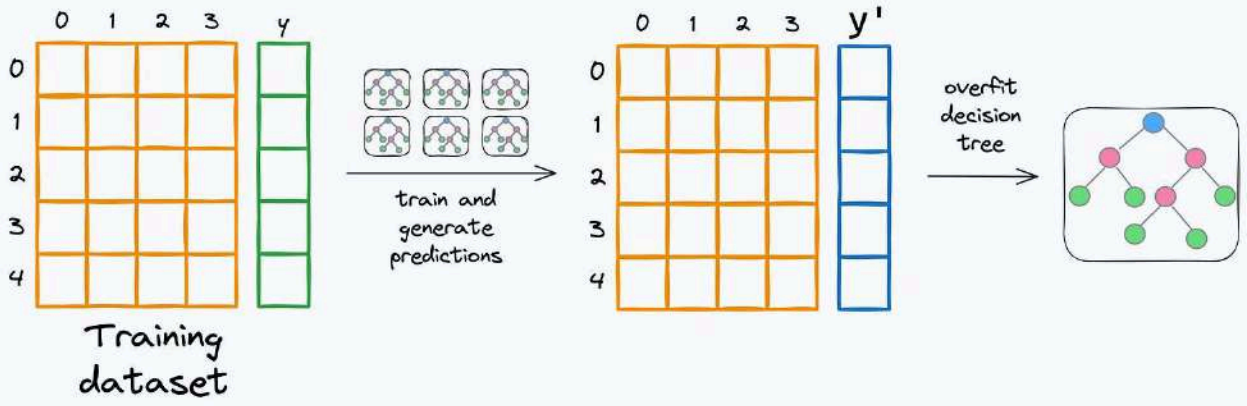


This time, the decision region plot suggests that we don't have a complex decision boundary. The test accuracy has also improved (69.5% to 74%).

Now, here's an interesting thing we can do.

We know that the random forest model has learned some rules that generalize on unseen data.

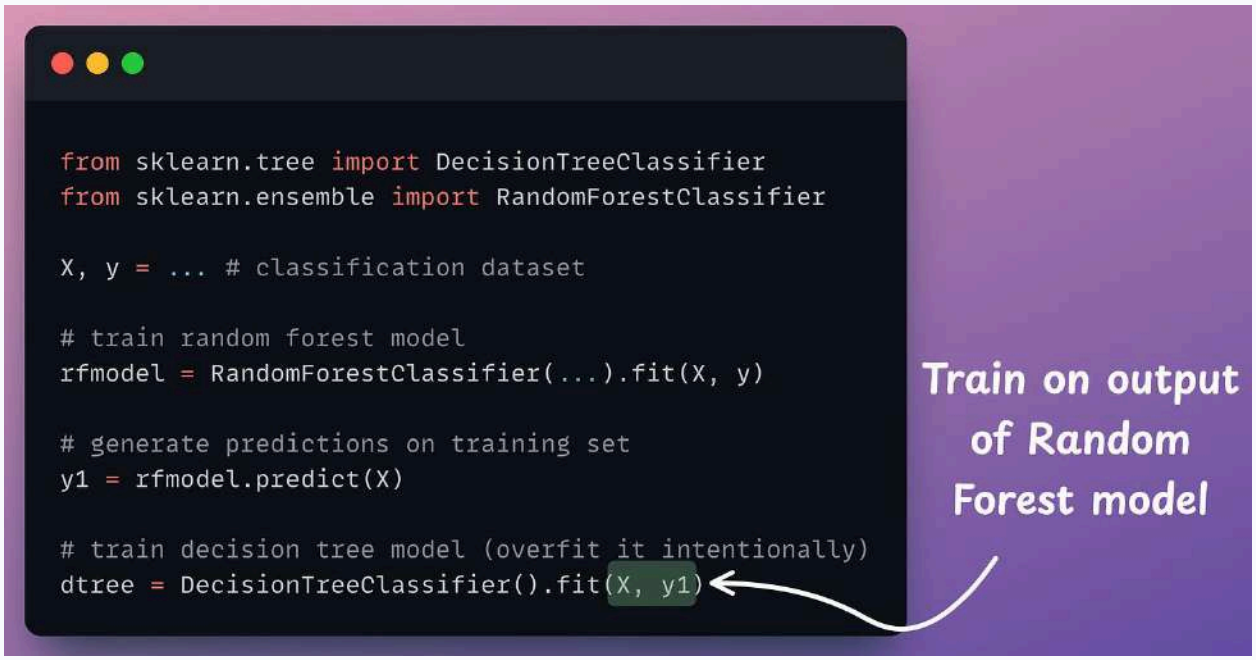
So, how about we train a decision tree on the predictions generated by the random forest model on the training set?



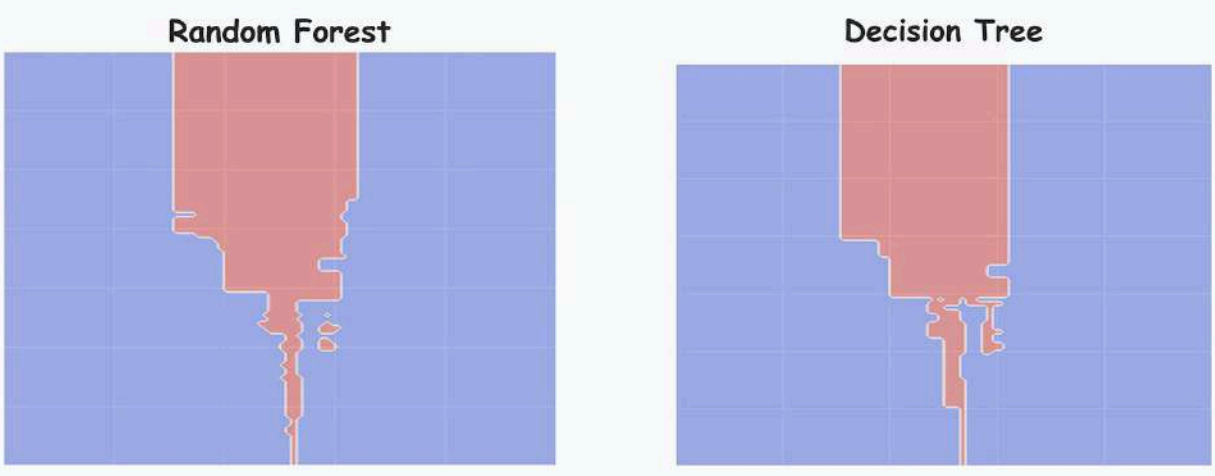
More specifically, given a dataset (X, y):

- Train a random forest model. This will learn some rules from the training set which are expected to generalize on unseen data (due to Bagging).
- Generate predictions on X, which produces the output y'. These predictions will capture the essence of the rules learned by the random forest model.
- Finally, train a decision tree model on (X, y'). Here, we want to intentionally overfit this mapping as this mapping from (X) to (y') is a proxy for the rules learned by the random forest model.

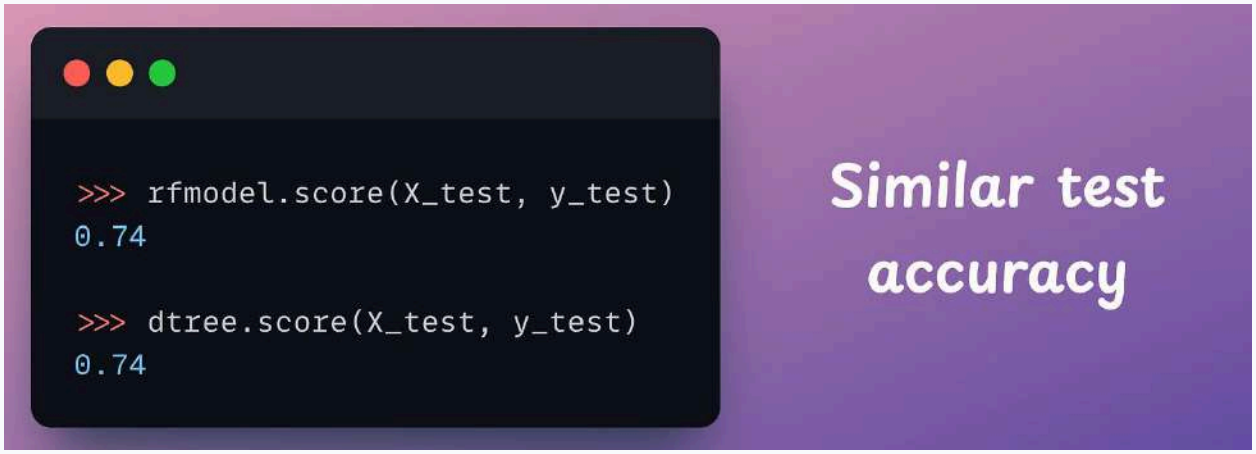
This idea is implemented below:



The decision region plot we get with the new decision tree is pretty similar to what we saw with the random forest earlier:



Measuring the test accuracy of the decision tree and random forest model, we notice them to be similar too:



In fact, this approach also significantly reduces the run-time, as depicted below:

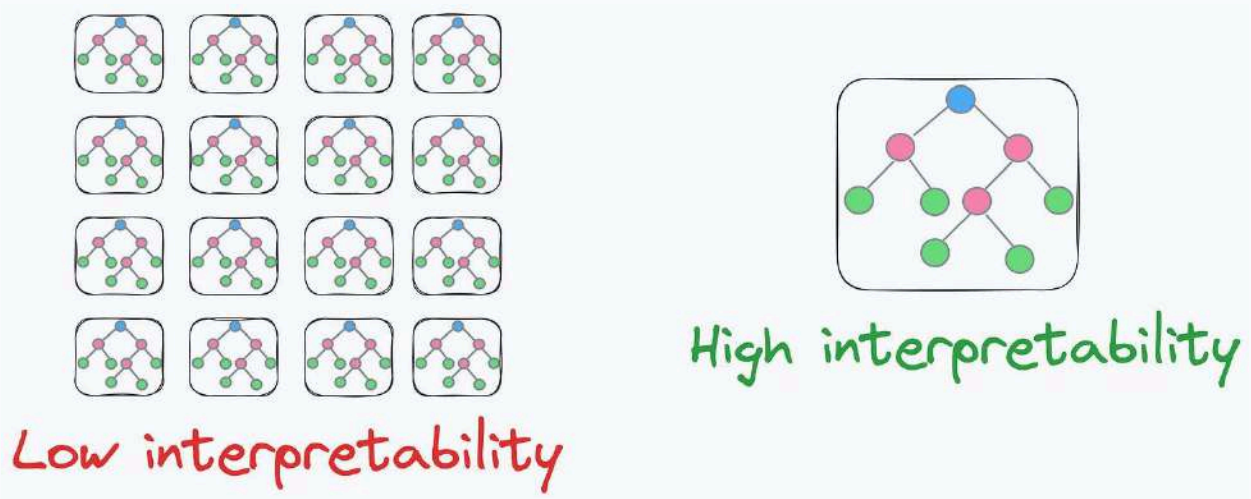


Isn't that cool?

Another rationale for considering doing this is that it adds interpretability.

This is because if we have 100 trees in a random forest, there's no way we can interpret them.

However, if we have condensed it to a decision tree, now we can inspect it.



A departing note

I devised this very recently. I also tested this approach on a couple of datasets, and they produced promising results.

But it won't be fair to make any conclusions based on just two instances.

While the idea makes intuitive sense, I understand there could be some potential flaws that are not evident right now.

So, I not saying that you should adopt this technique right away.

Instead, it is advised to test this approach on your random forest use cases. Considering reverting back to me with what you discovered.

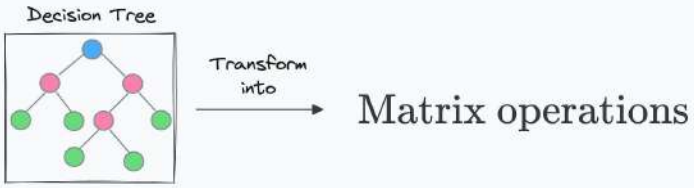
The code for this chapter is available here: https://bit.ly/3XSPejD.

In this next chapter, let's understand a technique to transform a decision tree into matrix operations which can run on GPUs.

Transform Decision Tree into Matrix Operations

Inference using a decision tree is an iterative process. We traverse a decision tree by evaluating the condition at a specific node in a layer until we reach a leaf node.

In this chapter, let's learn a superb technique that to represent inferences from a decision tree in the form of matrix operations.

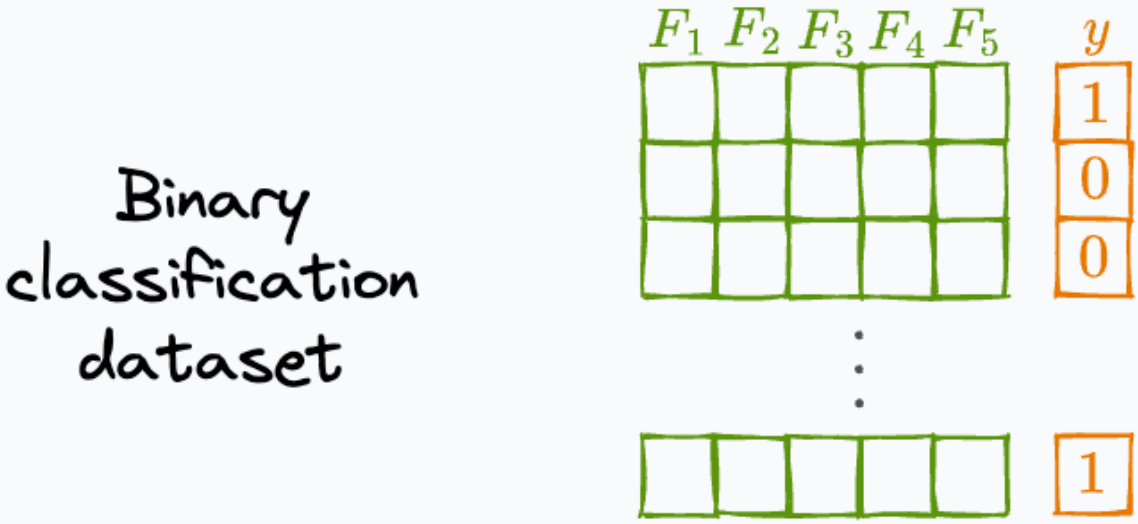


As a result:

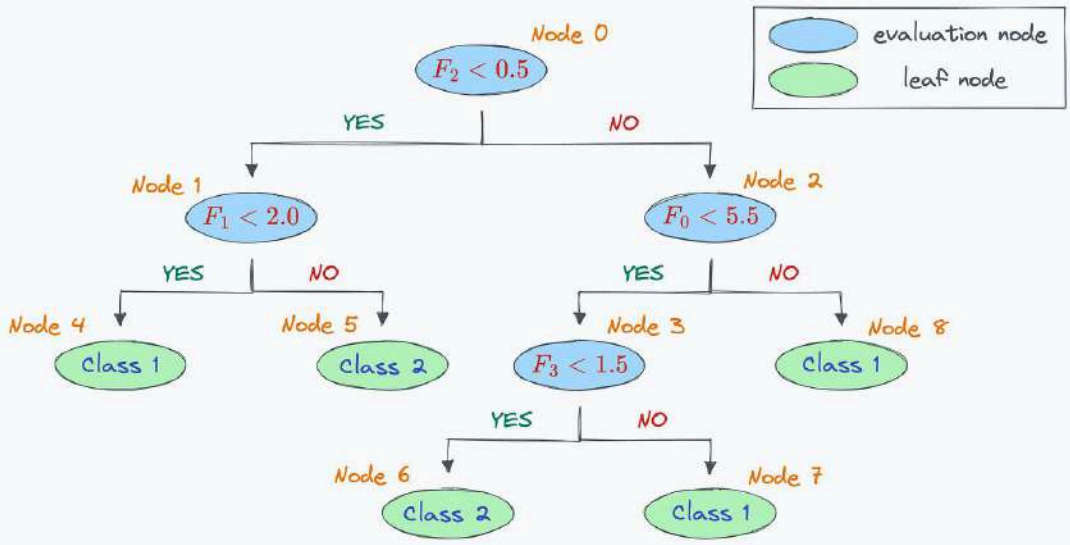
1. It makes inference much faster as matrix operations can be radically parallelized.
2. These operations can be loaded on a GPU for even faster inference, making them more deployment-friendly.

Setup

Consider a binary classification dataset with 5 features.



Let's say we get the following tree structure after fitting a decision tree on the above dataset:



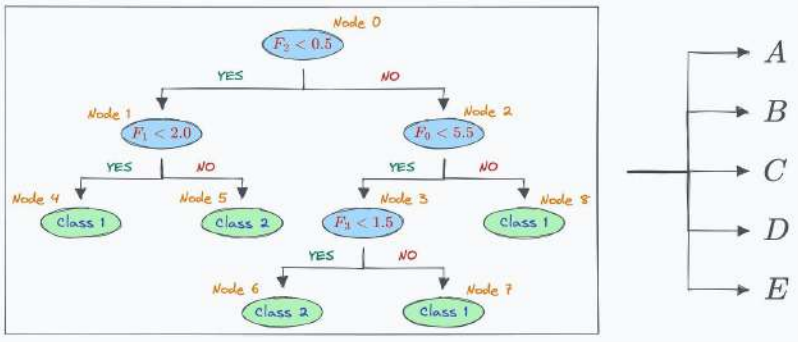
Notations

Before proceeding ahead, let's assume that:

- m → Total features in the dataset (5 in the dataset above).
- e → Total evaluation nodes in the tree (4 blue nodes in the tree above).
- l → Total leaf nodes in the tree (5 green nodes in the tree).
- c → Total classes in the dataset (2 in the dataset above).

Tree to Matrices

The core idea in this conversion is to derive five matrices (A, B, C, D, E) that capture the structure of the decision tree.



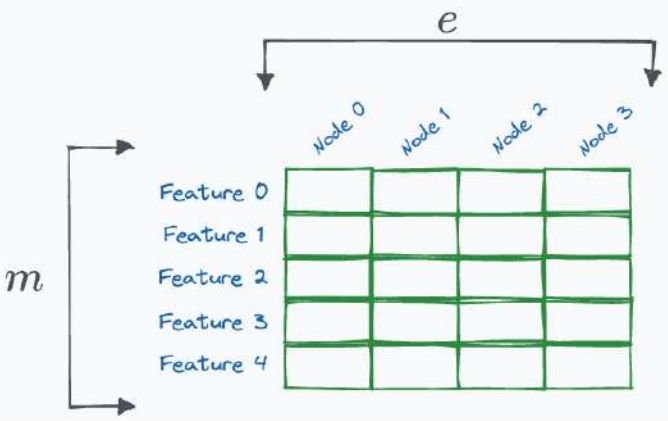
Let's understand them one by one!

Note: The steps we shall be performing below might not make immediate sense so please keep reading. You'll understand everything once we have derived all 5 matrices.

#1) Matrix A

This matrix captures the relationship between input features and evaluation nodes (blue nodes above).

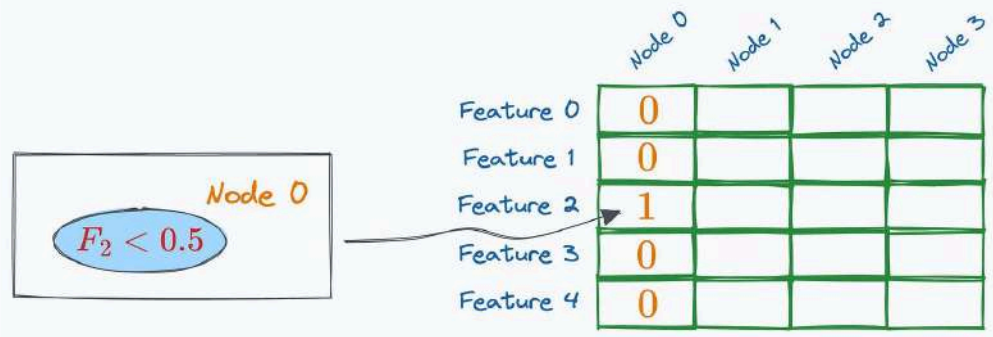
So it's an (m×e) shaped matrix.



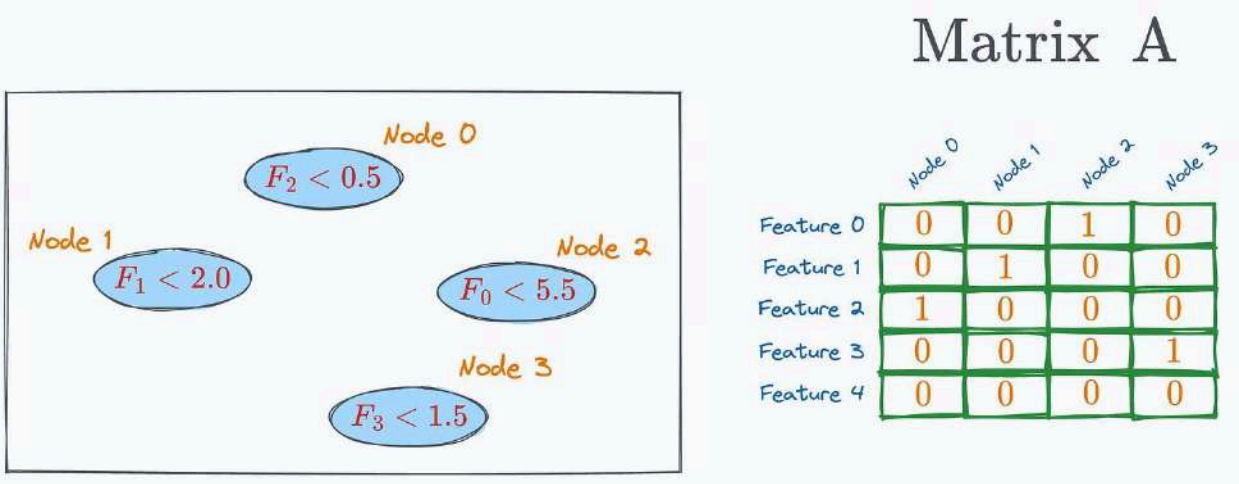
A specific entry is set to "1" if the corresponding node in the column evaluates the corresponding feature in the row. For instance, in our decision tree, "Node 0" evaluates "Feature 2".



Thus, the corresponding entry will be "1" and all other entries will be "0."

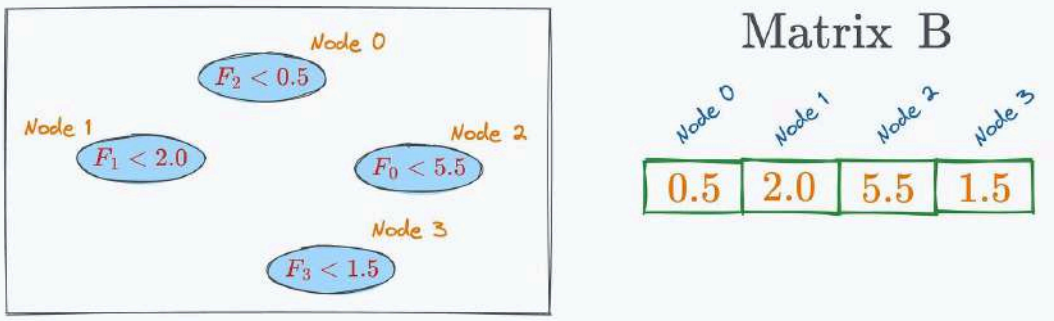


Filling out the entire matrix this way, we get:



#2) Matrix B

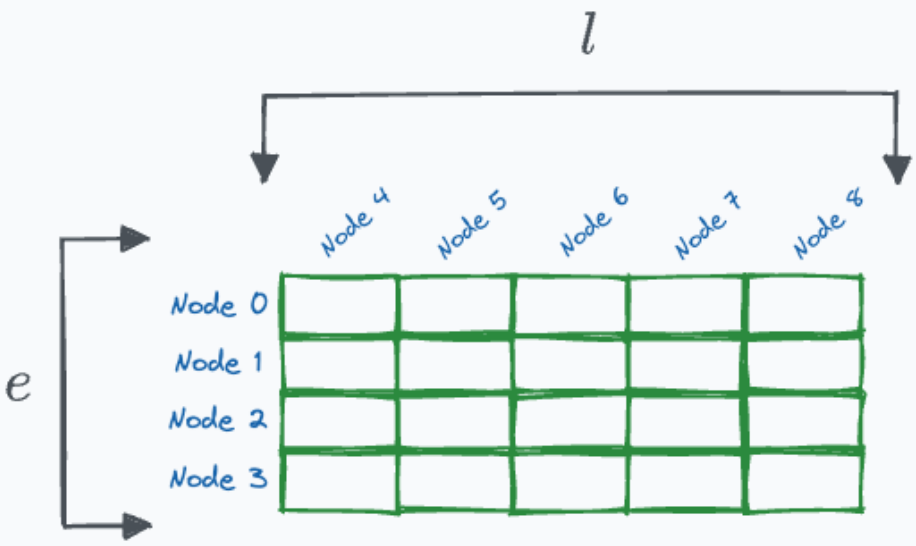
The entries of matrix B are the threshold value at each node. Thus, its shape is 1×e.



This is vector though, but the terminology is not important here.

#3) Matrix C

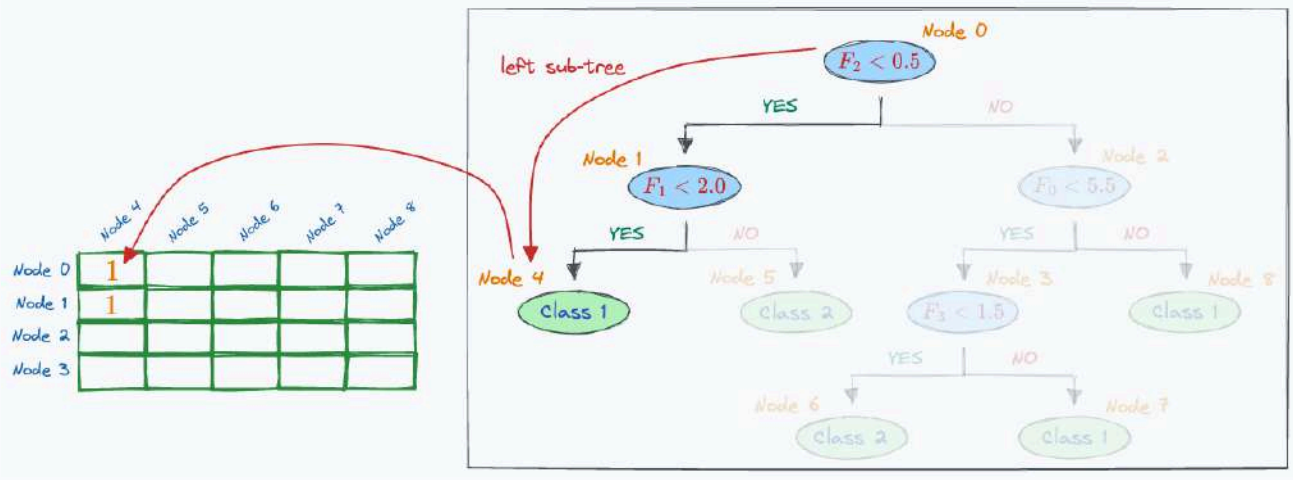
This is a matrix between every pair of leaf nodes and evaluation nodes. Thus, its dimensions are e×l.



A specific entry is set to:

- "1" if the corresponding leaf node in the column lies in the left sub-tree of the corresponding evaluation node in the row.
- "-1" if the corresponding leaf node in the column lies in the right sub-tree of the corresponding evaluation node in the row.
- "0" if the corresponding leaf node and evaluation node have no link.

For instance, in our decision tree, the "leaf node 4" lies on the left sub-tree of both "evaluation node 0" and "evaluation node 1". Thus, the corresponding values will be 1.



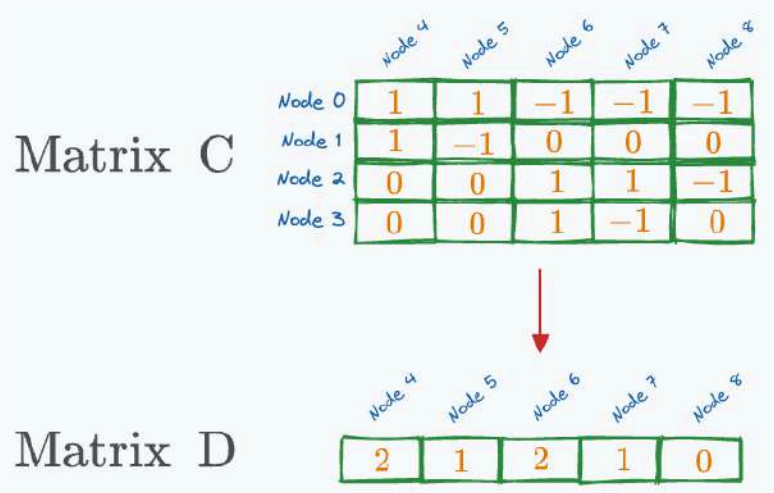
Filling out the entire matrix this way, we get:

Matrix C

	Node 4	Node 5	Node 6	Node 7	Node 8
Node 0	1	1	−1	−1	−1
Node 1	1	−1	0	0	0
Node 2	0	0	1	1	−1
Node 3	0	0	1	−1	0

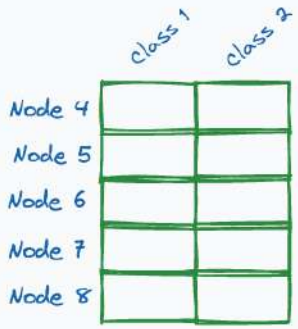
#4) Matrix (or Vector) D

The entries of vector D are the sum of non-negative entries in every column of Matrix C:



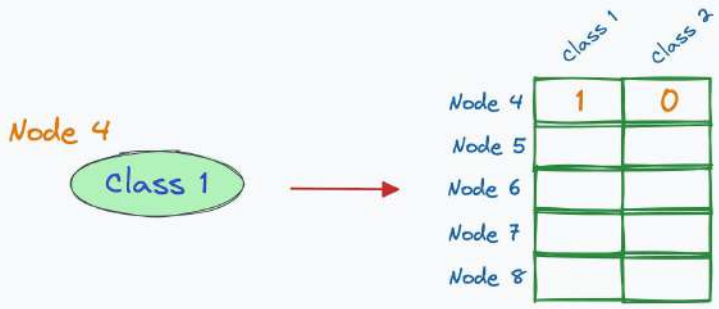
#5) Matrix E

Finally, this matrix holds the mapping between leaf nodes and their corresponding output labels. Thus, its dimensions are $l \times c$.

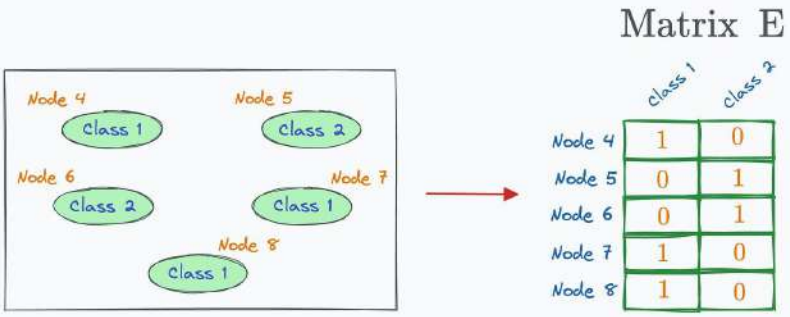


If a leaf node classifies a sample to "Class 1", the corresponding entry will be 1, and the other cell entry will be 0.

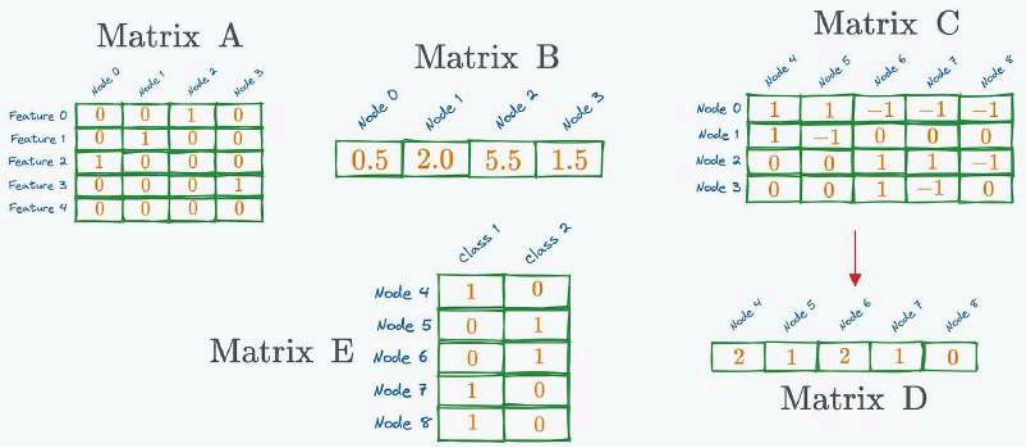
For instance, "lead node 4" outputs "Class 1", thus the corresponding entries for the first row will be (1,0):



We repeat this for all other leaf nodes to get the following matrix as Matrix E:



With this, we have compiled our decision tree into matrices. To recall, these are the five matrices we have created so far:



- Matrix A captures which input feature was used at each evaluation node.
- Matrix B stores the threshold of each evaluation node.
- Matrix C captures whether a leaf node lies on the left or right sub-tree of a specific evaluation node or has no relation to it.
- Matrix D stores the sum of non-negative entries in every column of Matrix C.
- Finally, Matrix E maps from leaf nodes to their class labels.

Inference using matrices

Say this is our input feature vector X (5 dimensions):

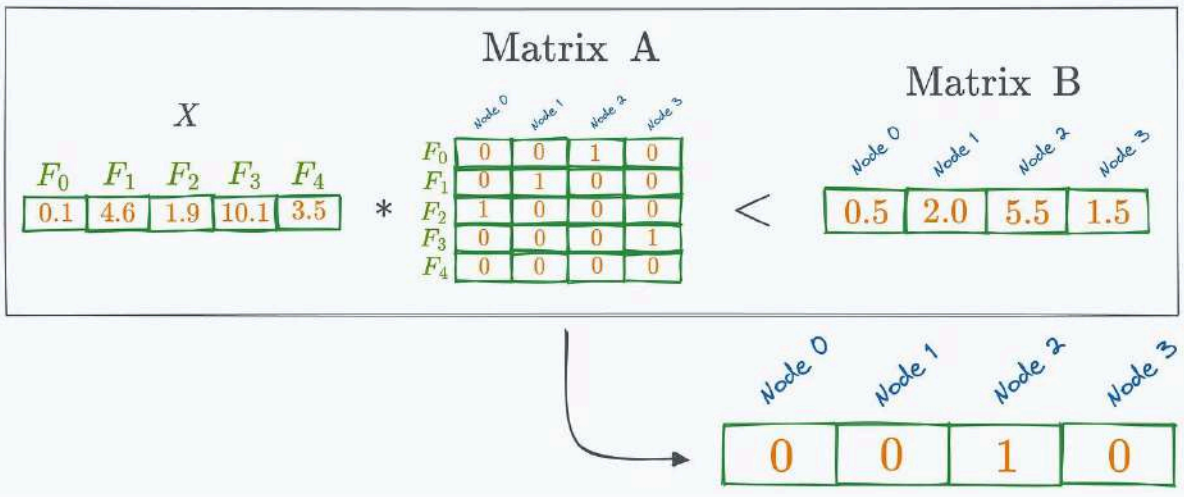
X

F_0	F_1	F_2	F_3	F_4
0.1	4.6	1.9	10.1	3.5

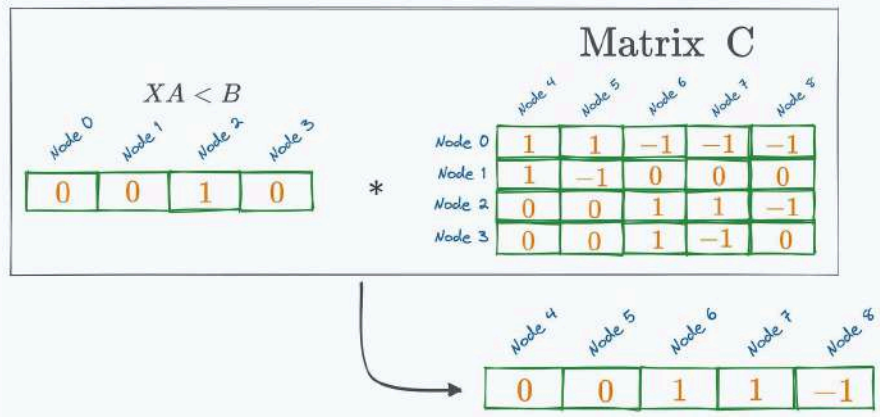
The whole inference can now be done using just these matrix operations:

$$(((XA < B) * C) == D) * E$$

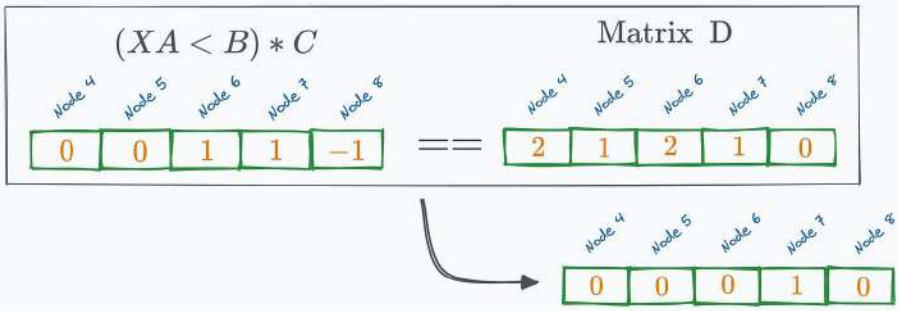
- XA < B gives:



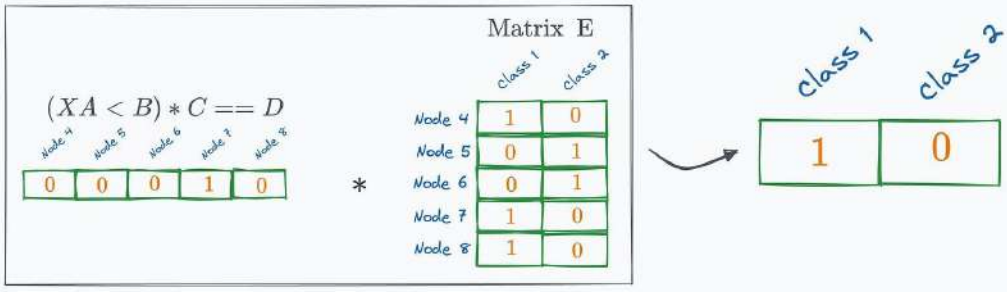
- The above result multiplied by C gives:



- The above result, when matched to D, gives:



- Finally, multiplying with E, we get:

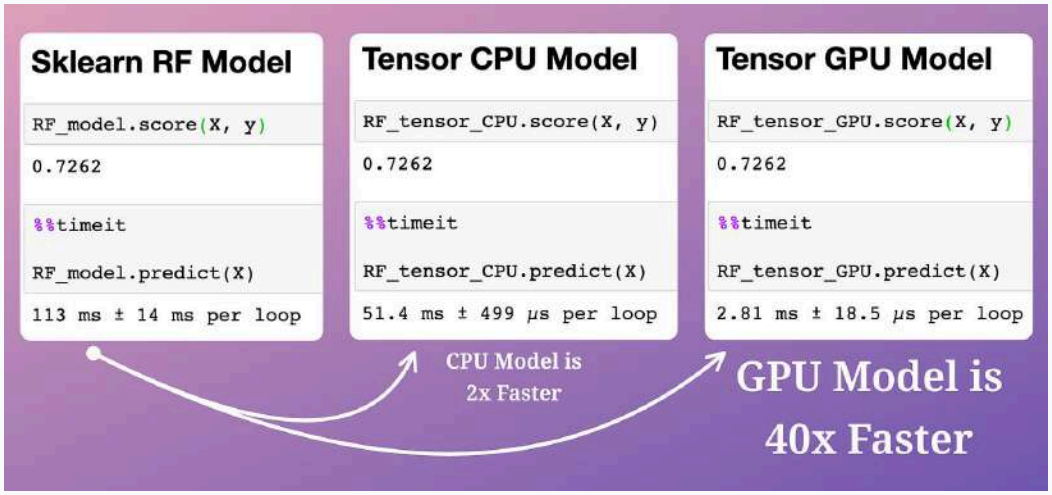


The final prediction comes out to be "Class 1," which is indeed correct! Notice that we carried out the entire inference process using only matrix operations:

$$(((XA < B) * C) == D) * E$$

As a result, the inference operation can largely benefit from parallelization and GPU capabilities.

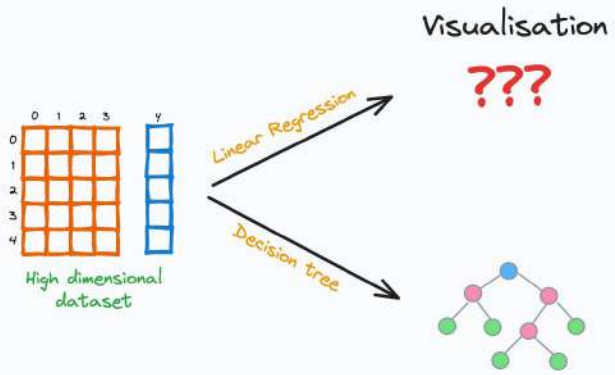
The run-time efficacy of this technique is evident from the image below:



- Here, we have trained a random forest model.
- The compiled model runs:
    - Over twice as fast on a CPU (Tensor CPU Model).
    - ~40 times faster on a GPU, which is huge (Tensor GPU Model).
- All models have the same accuracy — indicating no loss of information during compilation.

Interactively Prune a Decision Tree

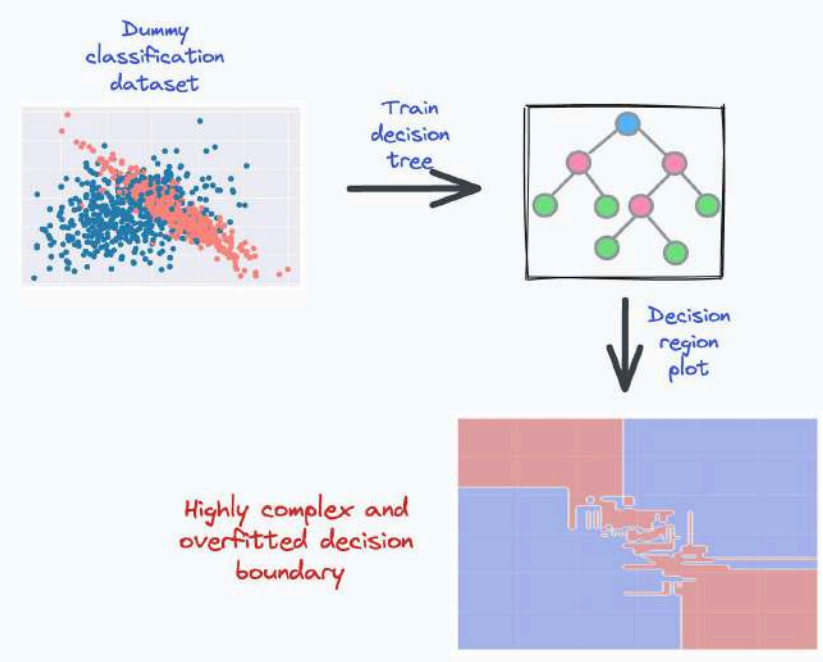
One thing I always appreciate about decision trees is their ease of visual interpretability. No matter how many features our dataset has, we can ALWAYS visualize and interpret a decision tree.



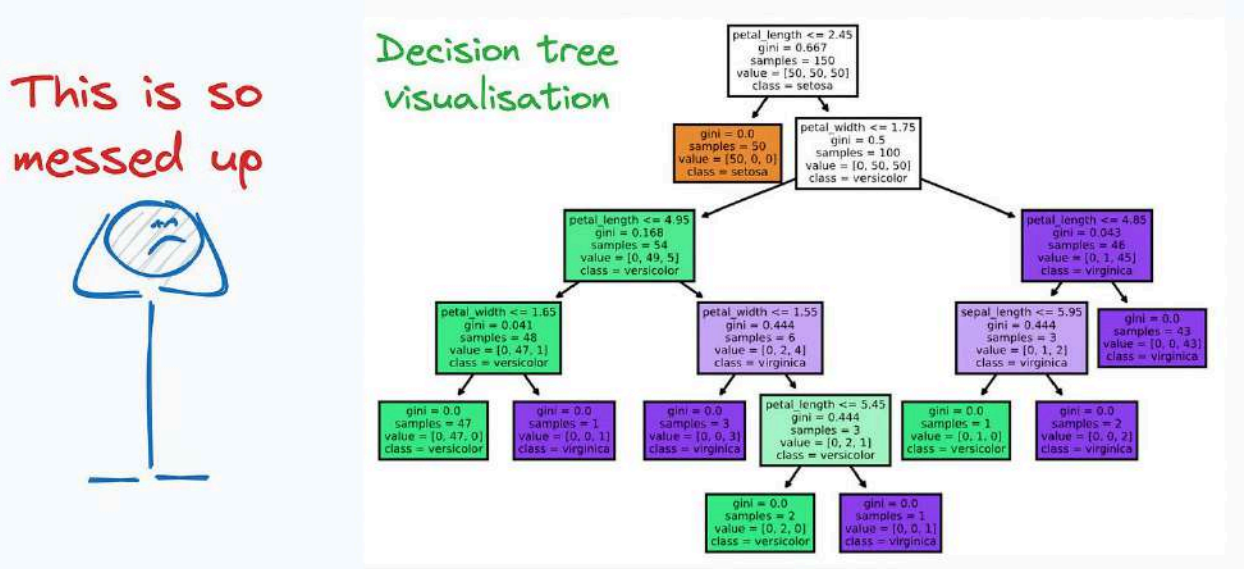
This is not always possible with other intuitive and simple models like linear regression. But decision trees stand out in this respect. Nonetheless, one thing I often find a bit time-consuming and somewhat hit-and-trial-driven is pruning a decision tree.

Why prune?

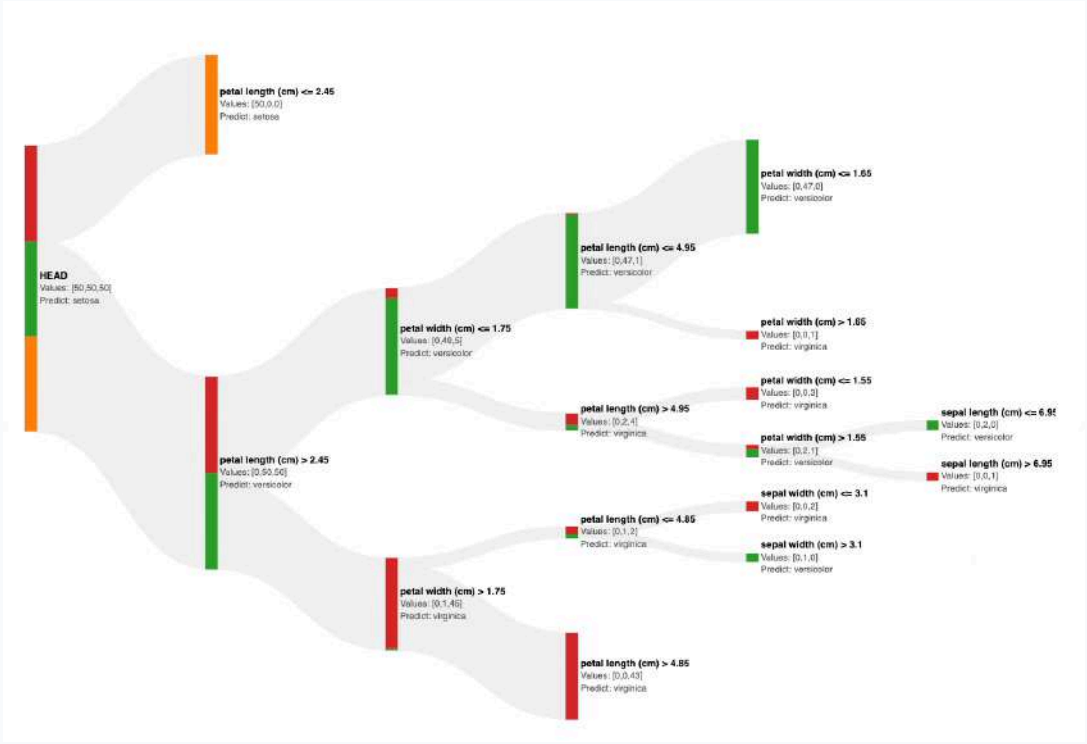
The problem is that under default conditions, decision trees ALWAYS 100% overfit the dataset, as depicted in this image:



Thus, pruning is ALWAYS necessary to reduce model variance. Scikit-learn already provides a method to visualize them as shown below:

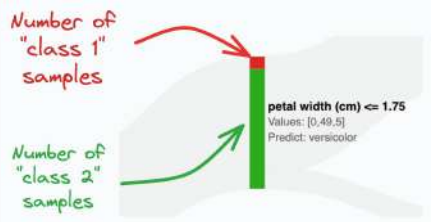


But the above visualisation is pretty non-elegant, tedious, messy, and static (or non-interactive). I recommend using an interactive Sankey diagram to prune decision trees. This is depicted below:

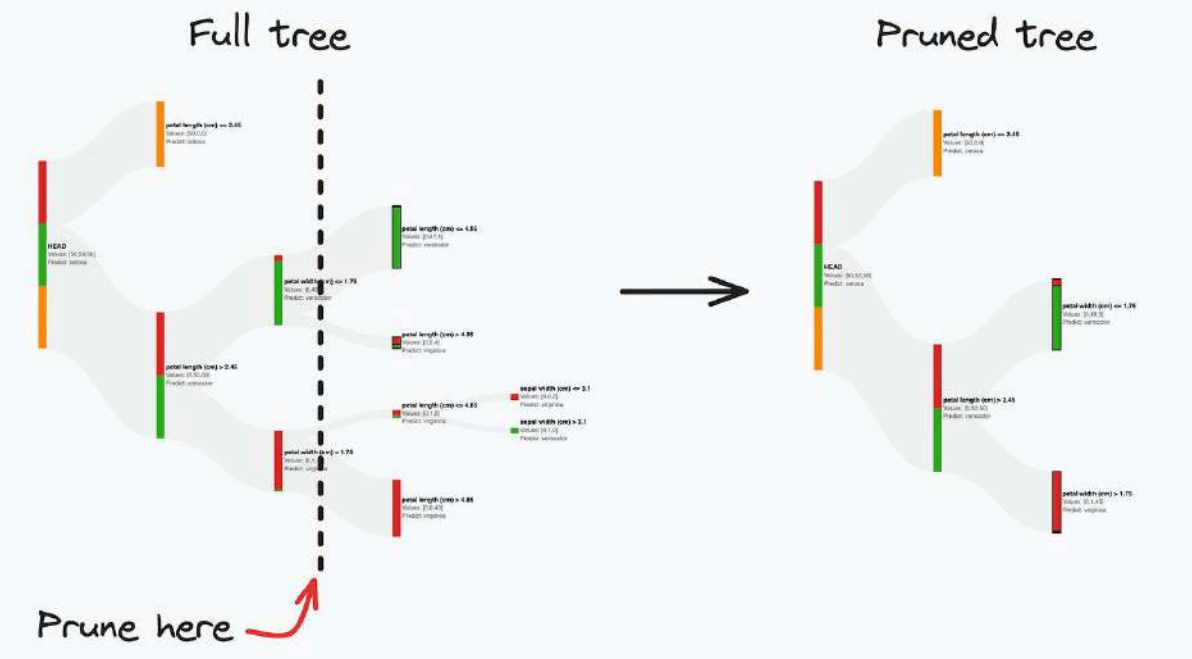


As shown above, the Sankey diagram allows us to interactively visualize and prune a decision tree by collapsing its nodes.

Also, the number of data points from each class is size and color-encoded in each node, as shown below.



This instantly gives an estimate of the node's impurity, based on which, we can visually and interactively prune the tree in seconds. For instance, in the full decision tree shown below, pruning the tree at a depth of two appears reasonable:



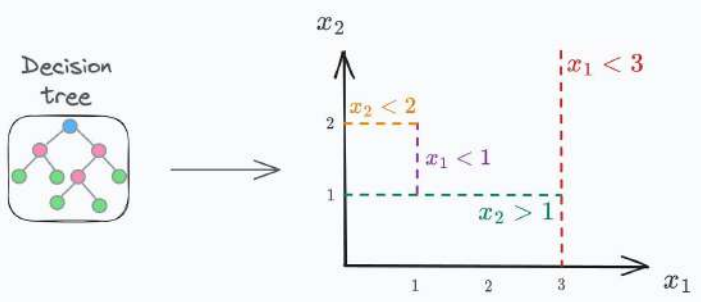
Next, we can train a new decision tree after obtaining an estimate for hyperparameter values. This will help us reduce the variance of the decision tree.

You can download the code notebook for the interactive decision tree here: https://bit.ly/4bBwY1p. Instructions are available in the notebook.

Next, let's understand a point of caution when using decision trees.

Why Decision Trees Must Be Thoroughly Inspected After Training

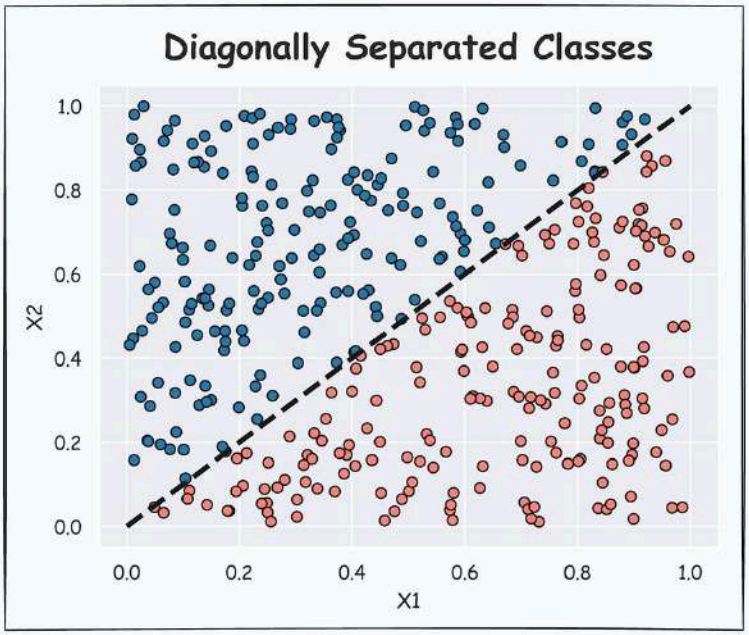
If we were to visualize the decision rules (the conditions evaluated at every node) of ANY decision tree, we would ALWAYS find them to be perpendicular to the feature axes, as depicted in the image.



In other words, every decision tree progressively segregates feature space based on such perpendicular boundaries to split the data.

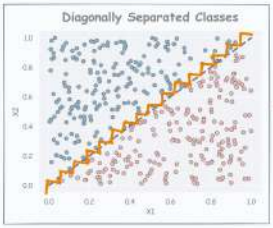
Of course, this is not a "problem" per se.

In fact, this perpendicular splitting is what makes it so powerful to perfectly overfit any dataset. However, this also brings up a pretty interesting point that is often overlooked when fitting decision trees. More specifically, what would happen if our dataset had a diagonal decision boundary, as depicted below:

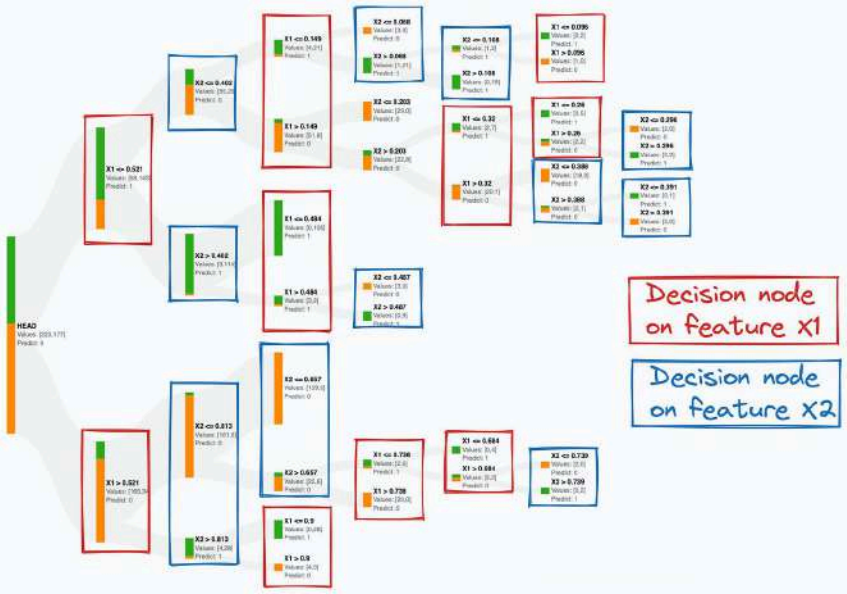


Dummy dataset with diagonal decision boundary

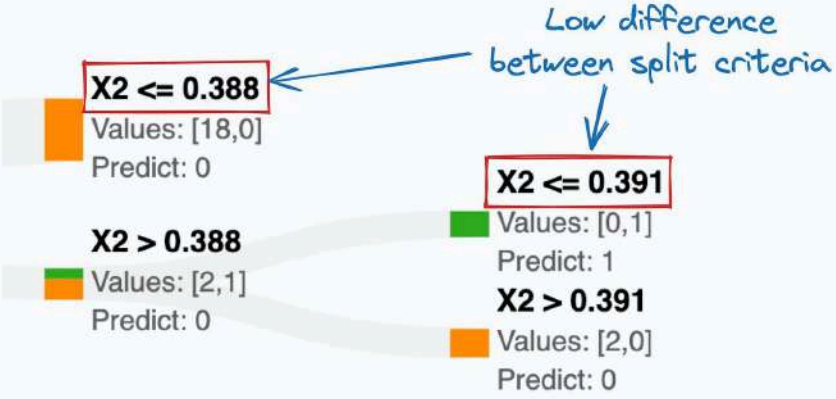
It is easy to guess that in such a case, the decision boundary learned by a decision tree is expected to appear as follows:



In fact, if we plot this decision tree, we notice that it creates so many splits just to fit this easily separable dataset, which a model like logistic regression, support vector machine (SVM), or even a small neural network can easily handle:



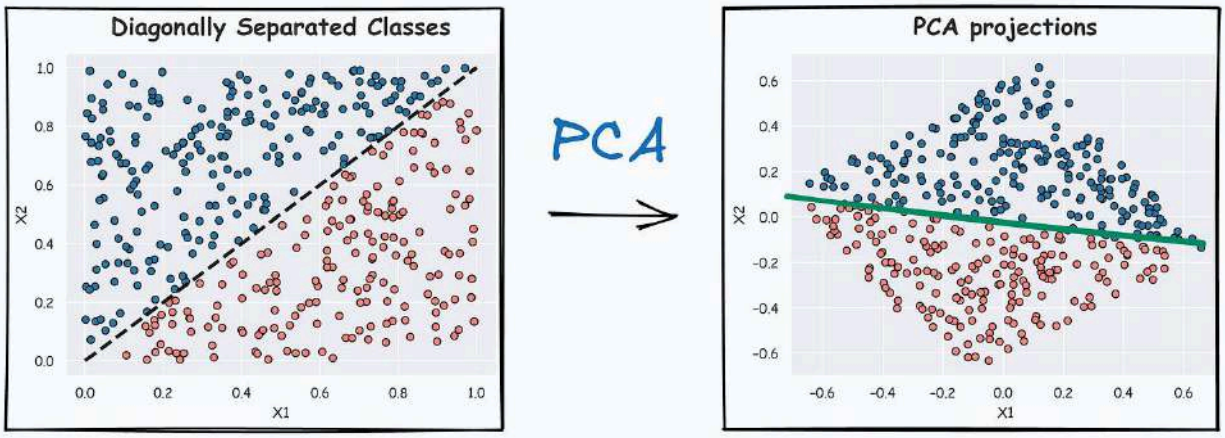
It becomes more evident if we zoom into this decision tree and notice how close the thresholds of its split conditions are:



This is a bit concerning because it clearly shows that the decision tree is meticulously trying to mimic a diagonal decision boundary, which hints that it might not be the best model to proceed with. To double-check this, I often do the following:

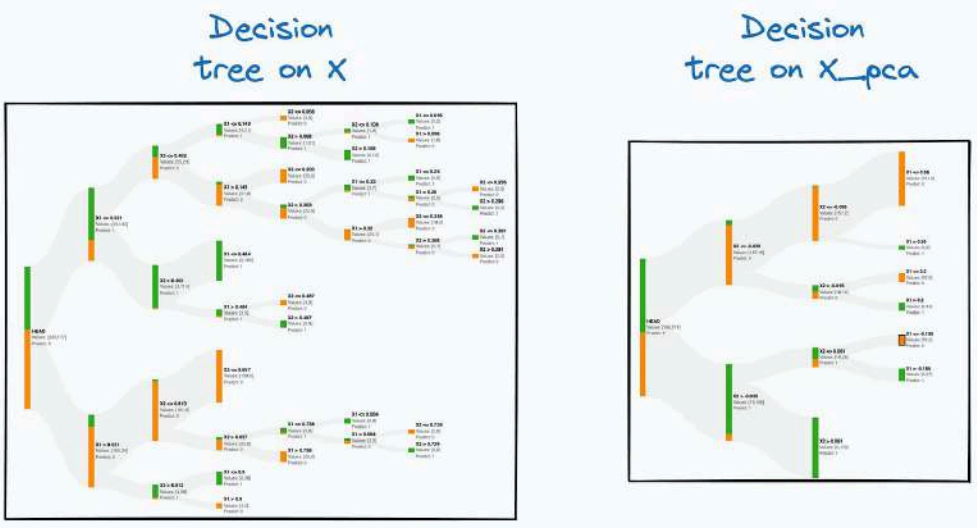
- Take the training data (X, y);
    - Shape of X: (n, m).
    - Shape of y: (n, 1).
- Run PCA on X to project data into an orthogonal space of m dimensions. This will give X_pca, whose shape will also be (n, m).
- Fit a decision tree on X_pca and visualize it (thankfully, decision trees are always visualizable).
- If the decision tree depth is significantly smaller in this case, it validates that there is a diagonal separation.

For instance, the PCA projections on the above dataset are shown below:



It is clear that the decision boundary on PCA projections is almost perpendicular to the X2` feature (the 2nd principal component).

Fitting a decision tree on this X_pca drastically reduces its depth, as depicted below:



This lets us determine that we might be better off using some other algorithm instead.

Or, we can spend some time engineering better features that the decision tree model can easily work with using its perpendicular data splits.

At this point, if you are thinking, why can't we use the decision tree trained on X_pca?

While nothing stops us from doing that, do note that PCA components are not interpretable, and maintaining feature interpretability can be important at times. Thus, whenever you train your next decision tree model, consider spending some time inspecting what it's doing.

Before ending this chapter

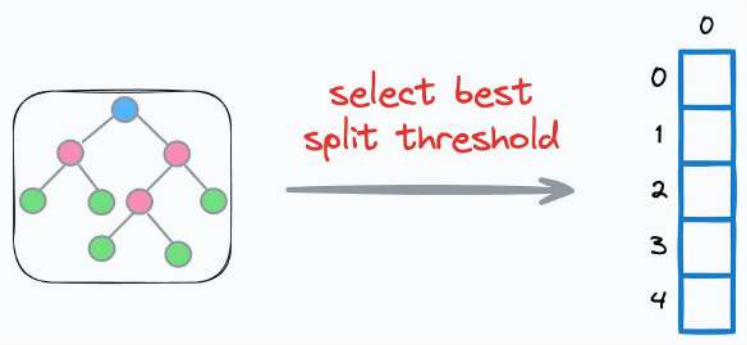
I don't intend to discourage the use of decision trees. They are the building blocks of some of the most powerful ensemble models we use today.

My point is to bring forward the structural formulation of decision trees and why/when they might not be an ideal algorithm to work with.

Decision Trees ALWAYS Overfit!

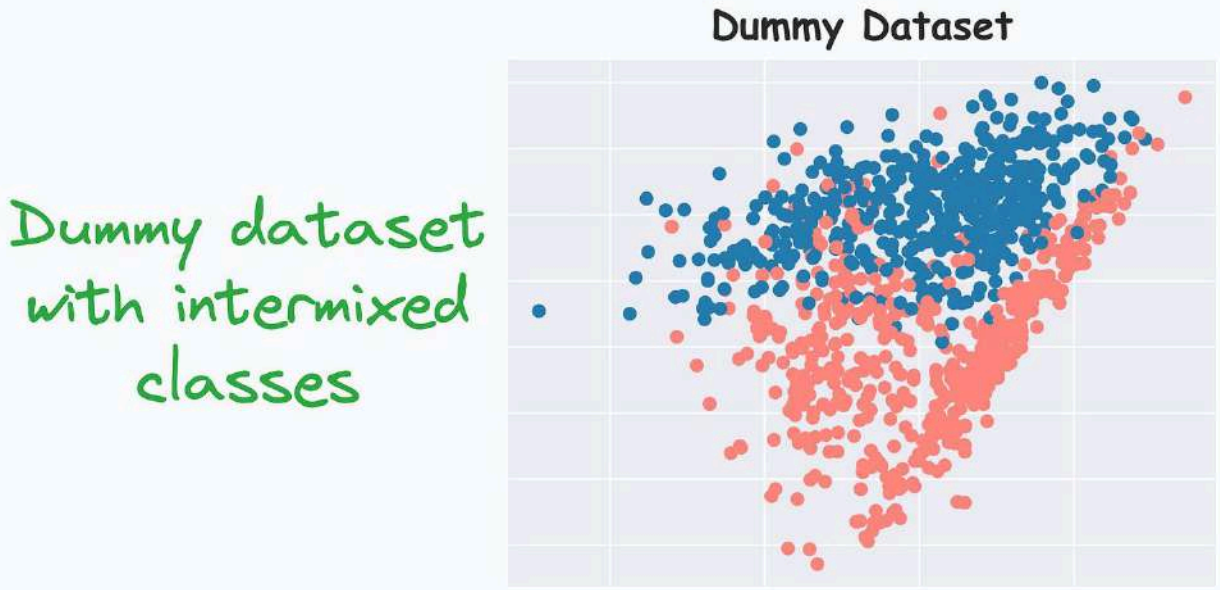
In addition to the above inspection, there's one more thing you need to be careful of when using decision trees. This is about overfitting.

The thing is that, by default, a decision tree (in sklearn's implementation, for instance), is allowed to grow until all leaves are pure. This happens because a standard decision tree algorithm greedily selects the best split at each node.



This makes its nodes more and more pure as we traverse down the tree. As the model correctly classifies ALL training instances, it leads to 100% overfitting, and poor generalization.

For instance, consider this dummy dataset:



Fitting a decision tree on this dataset gives us the following decision region plot:



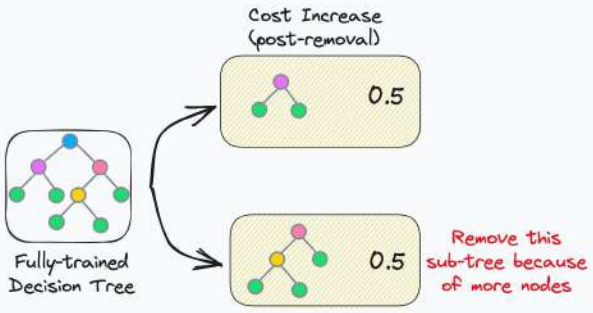
It is pretty evident from the decision region plot, the training and test accuracy that the model has entirely overfitted our dataset.

Cost-complexity-pruning (CCP) is an effective technique to prevent this.

CCP considers a combination of two factors for pruning a decision tree:

- Cost (C): Number of misclassifications
- Complexity (C): Number of nodes

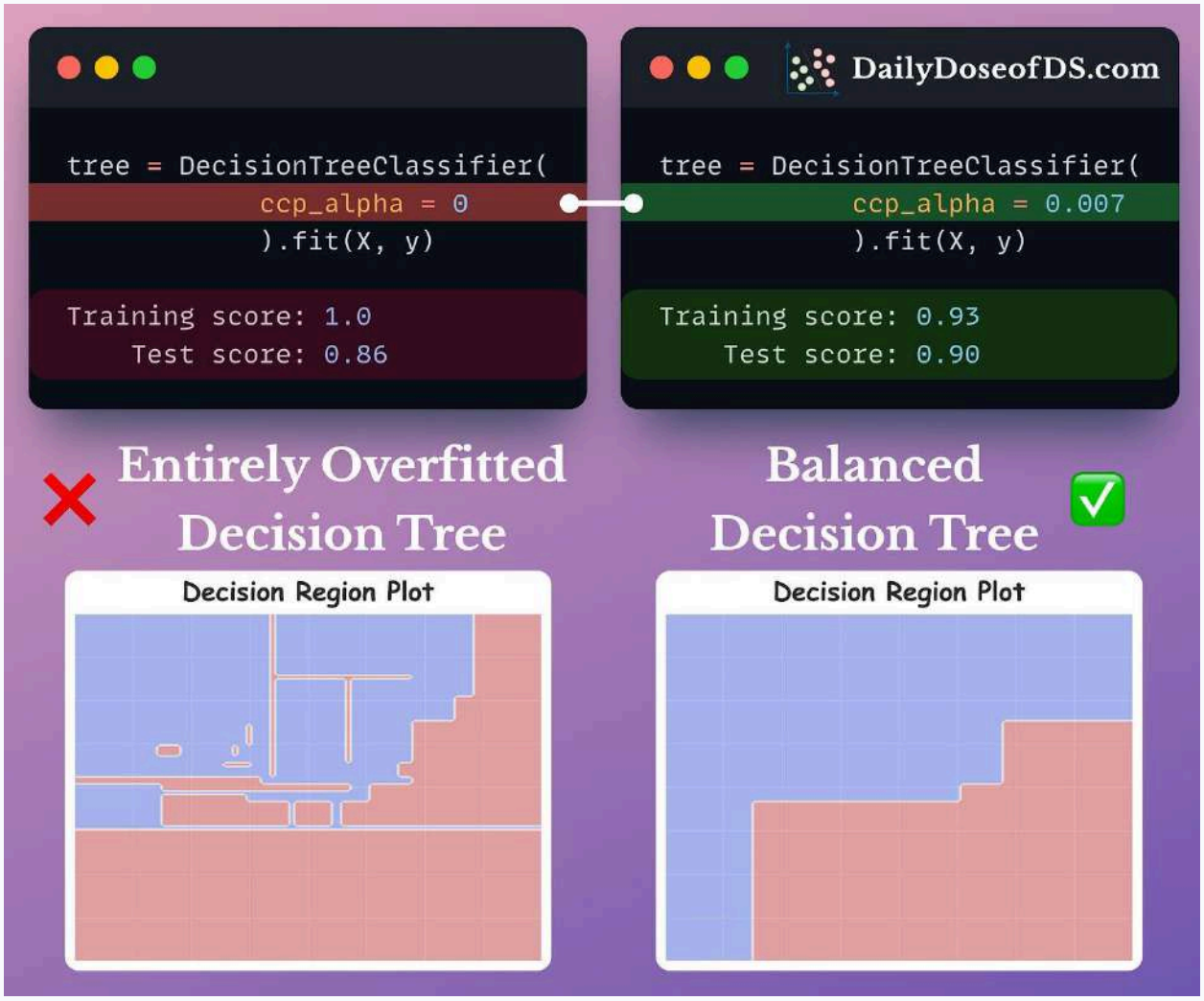
The core idea is to iteratively drop sub-trees, which, after removal, lead to a minimal increase in classification cost AND a maximum reduction of complexity (or nodes). In other words, if two sub-trees lead to a similar increase in classification cost, then it is wise to remove the sub-tree with more nodes.



In sklearn, you can control cost-complexity-pruning using the ccp_alpha parameter:

- large value of ccp_alpha → results in underfitting
- small value of ccp_alpha → results in overfitting

The objective is to determine the optimal value of ccp_alpha, which gives a better model. The effectiveness of cost-complexity-pruning is evident from the image below:



As depicted above, CCP results in a much simpler and acceptable decision region plot.

OOB Validation in Random Forest

After training an ML model on a training set, we always keep a held-out validation/test set for evaluation. I am sure you already know the purpose, so we won't discuss that.

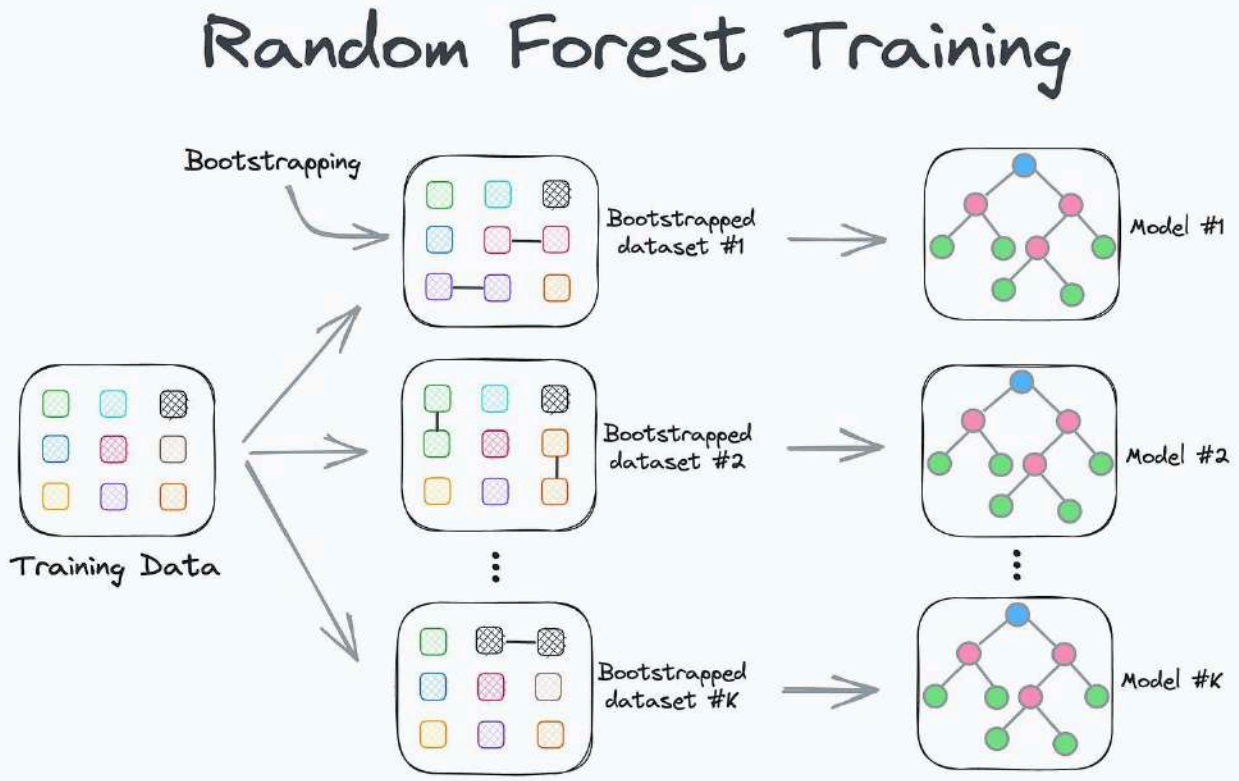
But do you know that random forests are an exception to that? In other words, one can somewhat "evaluate" a random forest using the training set itself.

Let's understand how.

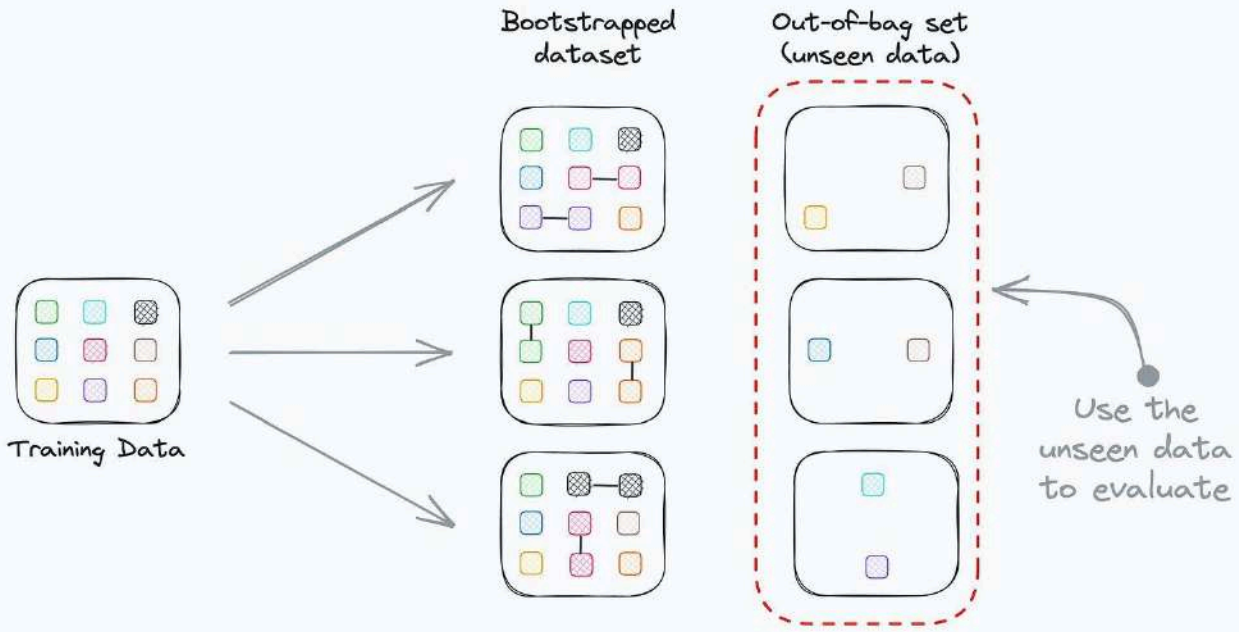
To recap, a random forest is trained as follows:

- First, we create different subsets of data with replacement (this process is called bootstrapping).
- Next, we train one decision tree per subset.
- Finally, we aggregate all predictions to get the final prediction.

This process is depicted below:

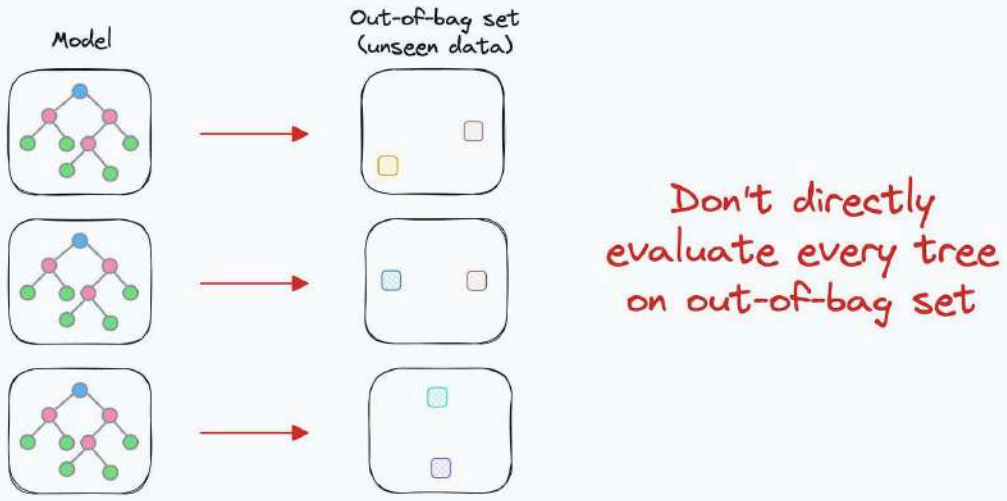


If we look closely above, every subset has some missing data points from the original training set.



We can use these observations to validate the model. This is also called out-of-bag validation. Calculating the out-of-bag score for the whole random forest is simple too.

But one thing to remember is that we CAN NOT evaluate individual decision trees on their specific out-of-bag sample and generate some sort of "aggregated score" for the entire random forest model.

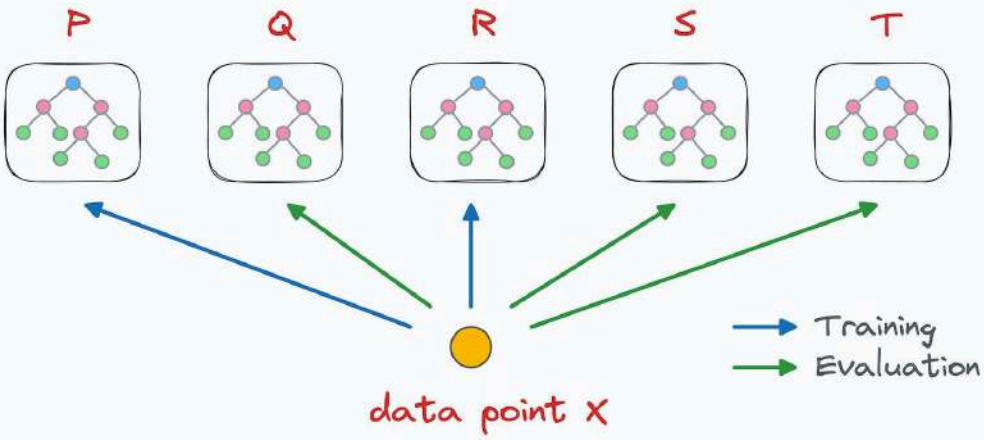


This is because a random forest is not about what a decision tree says individually. Instead, it's about what all decision trees say collectively.

So here's how we can generate the out-of-bag score for the random forest model. For every data point in the training set:

- Gather predictions from all decision trees that did not use it as a training data point.
- Aggregate predictions to get the final prediction.

For instance, consider a RF model with 5 decision trees → (P, Q, R, S, T). Say a specific data point X was used as a training data in decision trees P and R.



So we shall gather the out-of-bag prediction for data point X from decision trees Q, S and T.

After obtaining out-of-bag predictions for all samples, we score them to get the out-of-bag score.

See...this technique allowed us to evaluate a random forest model on the training set. Of course, I don't want you to blindly adopt out-of-bag validation without understanding some of its advantages and considerations.

I have found out-of-bag validation to be particularly useful in the following situations:

- In low-data situations, out-of-bag validation prevents data splitting whilst obtaining a good proxy for model validation.
- In large-data situations, traditional cross-validation techniques are computationally expensive. Here, out-of-bag validation provides an efficient alternative. This is because, by its very nature, even cross-validation provides an out-of-fold metric. Out-of-bag validation is also based on a similar principle.

And, of course, an inherent advantage of out-of-bag validation is that it guarantees no data leakage. Luckily, out-of-bag validation is also neatly tied in sklearn's random forest implementation.



The most significant consideration about out-of-bag score is to use it with caution for model selection, model improvement, etc.

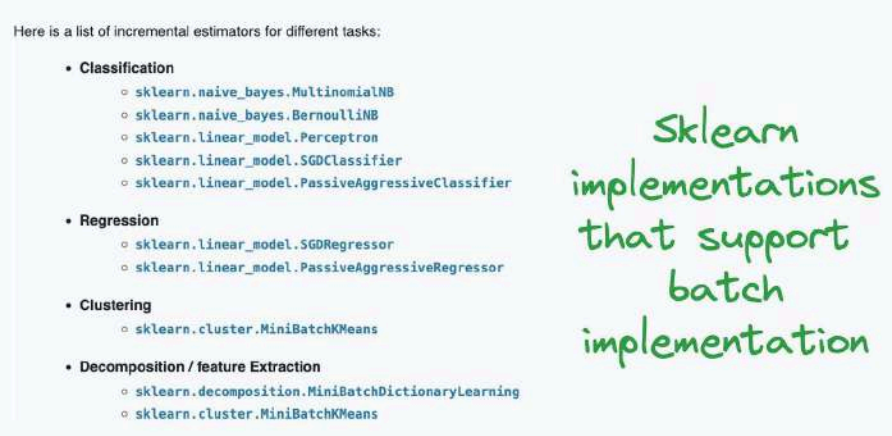
This is because if we do, we typically tend to overfit the out-of-bag score as the model is essentially being tuned to perform well on the data points that were left out during its training. And if we consistently improve the model based on the out-of-bag score, we obtain an overly optimistic evaluation of its generalization performance.

If I were to share just one lesson here based on my experience, it would be that if we don't have a true (and entirely different) held-out set for validation, we will overfit to some extent.

The decisions made may be too specific to the out-of-bag sample and may not generalize well to new data.

Train Random Forest on Large Datasets

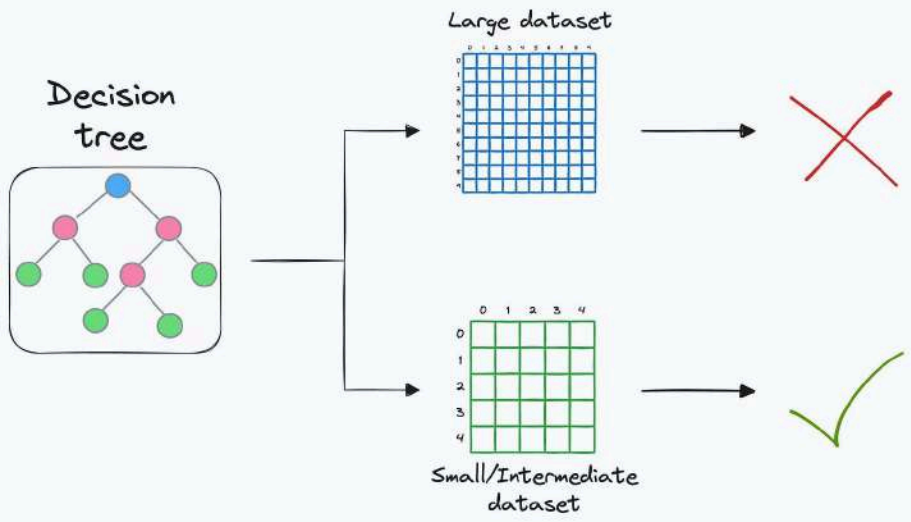
Most classical ML algorithms cannot be trained with a batch implementation. This limits their usage to only small/intermediate datasets. For instance, this is the list of sklearn implementations that support a batch API:



It's pretty small, isn't it?

This is concerning because, in the enterprise space, the data is primarily tabular. Classical ML techniques, such as tree-based ensemble methods, are frequently used for modeling.

However, typical implementations of these models are not "big-data-friendly" because they require the entire dataset to be present in memory.



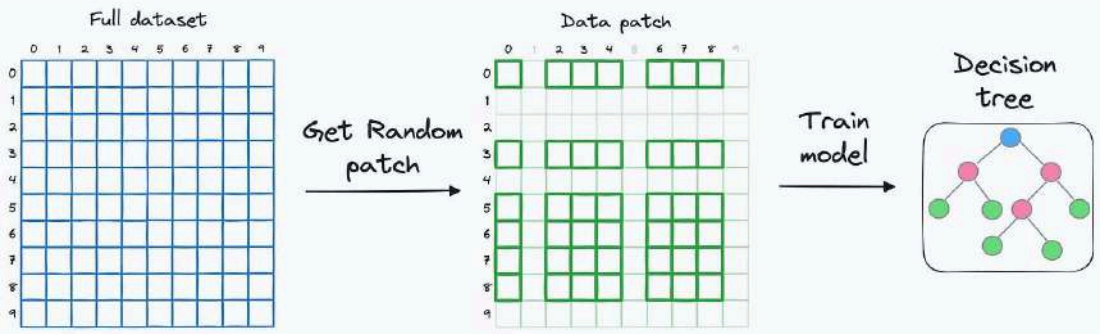
There are two ways to approach this:

1. Use big-data frameworks like Spark MLlib to train them.
2. There's one more way, which Dr. Gilles Louppe discussed in his PhD thesis — Understanding Random Forests.

Here's what he proposed.

Random Patches

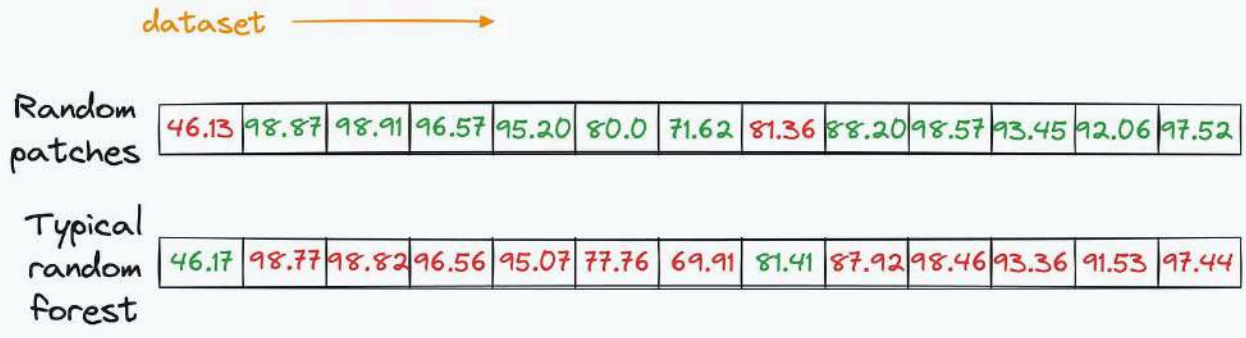
Before explaining, note that this approach will only work in an ensemble setting. So, you would have to train multiple models. The idea is to sample random data patches (rows and columns) and train a tree model on them.



Repeat this step multiple times by generating different patches of data randomly to obtain the entire random forest model.

The efficacy?

The thesis presents many benchmarks (check pages 174 and 178 if you need more details) on 13 datasets, and the results are shown below:



From left to right → Cifar10, mnist3v8, mnist4v9, mnist, isolet, arcene, breast2, madelon, marti, reged, second, this, and sido.

From the above image, it is clear that in most cases, the random patches approach performs better than the traditional random forest.

In other cases, there wasn't a significant difference in performance.

And this is how we can train a random forest model on large datasets that do not fit into memory.

Why does it work?

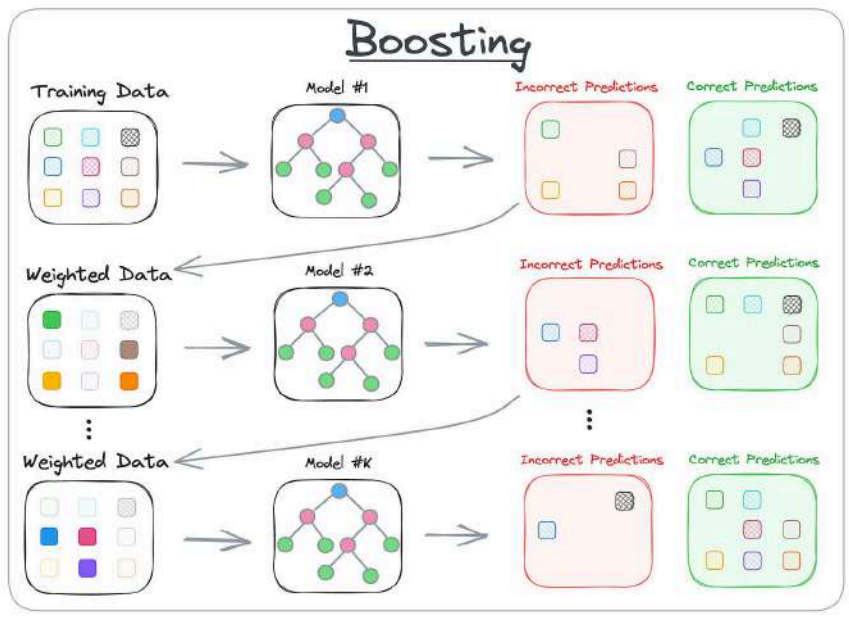
While the thesis did not provide a clear intuition behind this, I can understand why such an approach would still be as effective as random forest.

In a gist, building trees that are as different as possible guarantees a greater reduction in variance.

In this case, the dataset overlap between any two trees is NOT expected to be huge compared to the typical random forest. This aids in the Bagging objective.

A Visual Guide to AdaBoost

The following visual summarizes how Boosting models work:



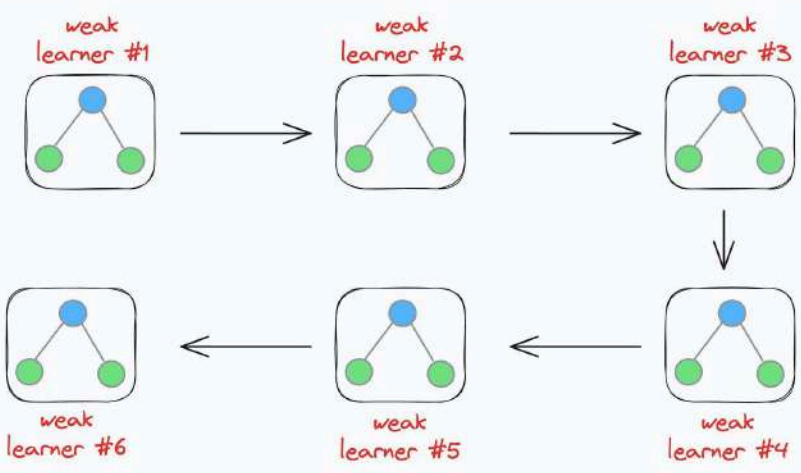
- Boosting is an iterative training process.
- The subsequent model puts more focus on misclassified samples from the previous model.
- The final prediction is a weighted combination of all predictions

However, many find it difficult to understand how this model is precisely trained and how instances are reweighed for subsequent models. AdaBoost is a common Boosting model, so in this chapter, let's understand how it works.

AdaBoost internal working

The core idea behind Adaboost is to train many weak learners to build a more powerful model. This technique is also called ensembling.

Specifically talking about Adaboost, the weak classifiers progressively learn from the previous model's mistakes, creating a powerful model when considered as a whole.



These weak learners are usually decision trees.

Let me make it more clear by implementing AdaBoost using the `DecisionTreeClassifier` class from sklearn.

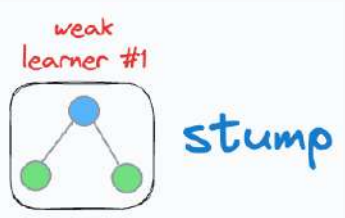
Consider we have the following classification dataset:

feature 1	feature 2	feature 3	target	weight
1.2	2.3	True	class A	0.2
-0.3	0.6	False	class B	0.2
0.7	1.8	False	class A	0.2
4.5	-3.5	True	class B	0.2
-0.4	0.4	True	class A	0.2

To begin, every row has an equal weight, and it is equal to (1/n), where n is the number of training instances.

Step 1: Train a weak learner

In Adaboost, every decision tree has a unit depth, and they are also called stumps.



Thus, we define DecisionTreeClassifier with a max_depth of 1, and train it on the above dataset.

Custom-Adaboost.py

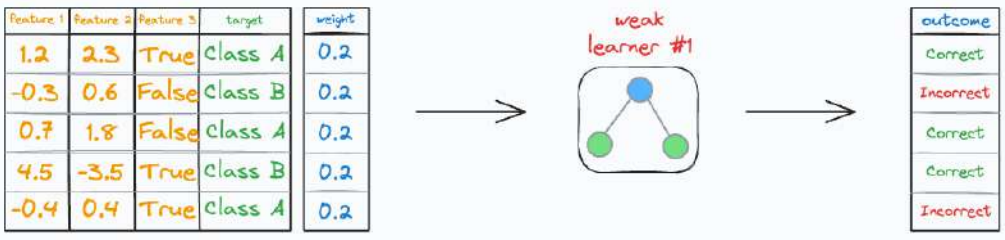
```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(max_depth=1)

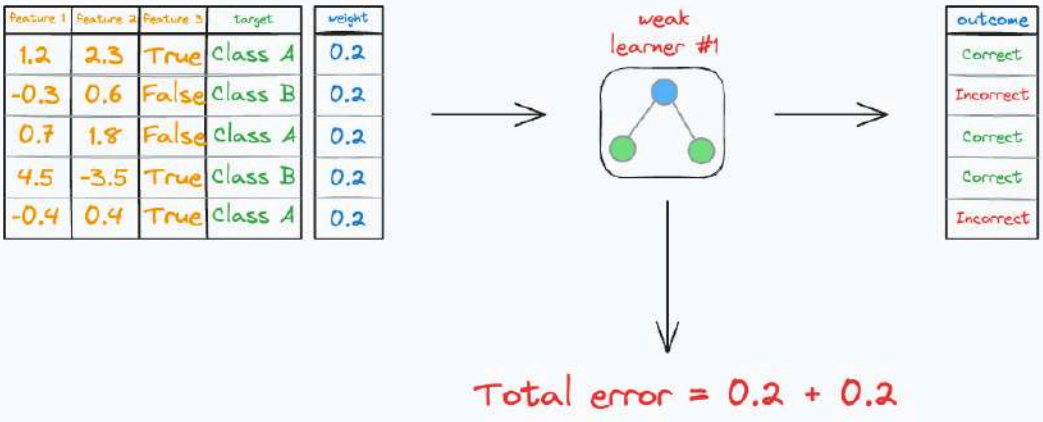
tree.fit(X, y)
```

Step 2: Calculate the learner's cost

Of course, there will be some correct and incorrect predictions.



The total cost (or error/loss) of this specific weak learner is the sum of the weights of the incorrect predictions. In our case, we have two errors, so the total error is:



Now, as discussed above, the idea is to let the weak learn progressively learn from previous learner's mistakes. So, going ahead, we want the subsequent model to focus more on the incorrect predictions produced earlier.

Here's how we do this:

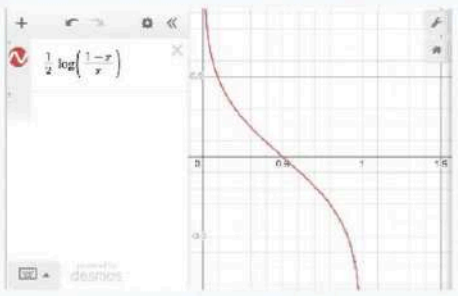
Step 3: Calculate the learner's importance

First, we determine the importance of the weak learner. Quite intuitively, we want the importance to be inversely related to the above error.

- If the weak learner has a high error, it must have a lower importance.
- If the weak learner has a low error, it must have a higher importance.

One choice is the following function:

$$\text{Importance} = \frac{1}{2} \cdot log(\frac{1 - \text{Total error}}{\text{Total error}})$$



- This function is only defined between [0,1].
- When the error is high (~1), this means there were no correct predictions → This gives a negative importance to the weak learner.
- When the error is low (~0), this means there were no incorrect predictions → This gives a positive importance to the weak learner.

If you feel there could be a better function to use here, you are free to use that and call it your own Boosting algorithm.

Now, we have the importance of the learner.

$$\text{Importance} = \frac{1}{2} \cdot log(\frac{1 - \text{Total error}}{\text{Total error}})$$

The importance value is used during model inference to weigh the predictions from the weak learners. So the next step is to...

Step 4: Reweigh the training instances

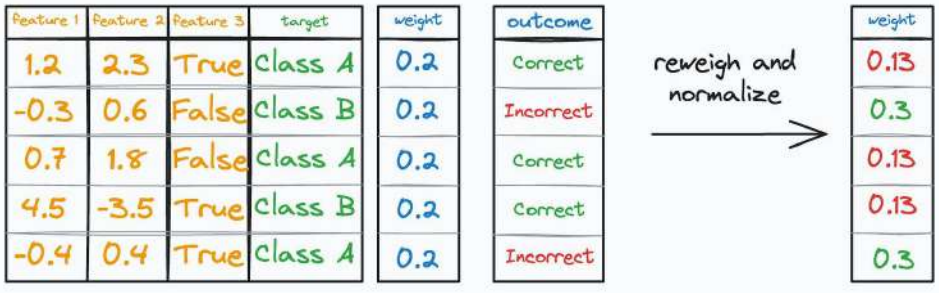
All the correct predictions are weighed down as follows:

$$w := w * e^{\text{-Importance}}$$

And all the incorrect predictions are weighed up as follows:

$$w := w * e^{\text{Importance}}$$

Once done, we normalize the new weights to add up to one.



That's it!

Step 5: Sample from reweighed dataset

From step 4, we have the reweighed dataset.

We sample instances from this dataset in proportion to the new weights to create a new dataset.



Next, go back to step 1 — Train the next weak learner.



And repeat the above process over and over for some pre-defined max iterations.

That's how we build the AdaBoost model.

All we have to do is consider the errors from the previous model, reweigh and sample the training instances for the next model, and repeat.

Dimensionality Reduction

The Utility of 'Variance' in PCA

The core objective of PCA is to retain the maximum variance of the original data while reducing the dimensionality. The rationale is that if we retain variance, we will retain maximum information.

But why?

Many people struggle to intuitively understand the motivation for using "variance" here. In other words:

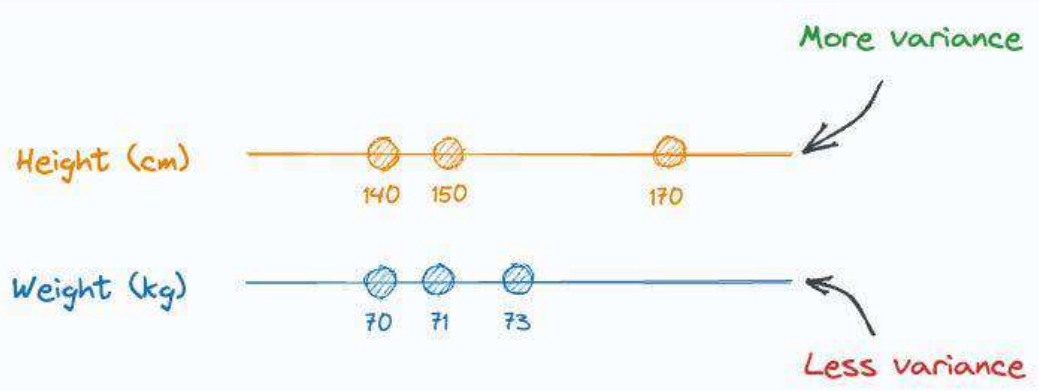
Why retaining maximum variance is an indicator for retaining maximum information?

This chapter provides an intuitive explanation of this.

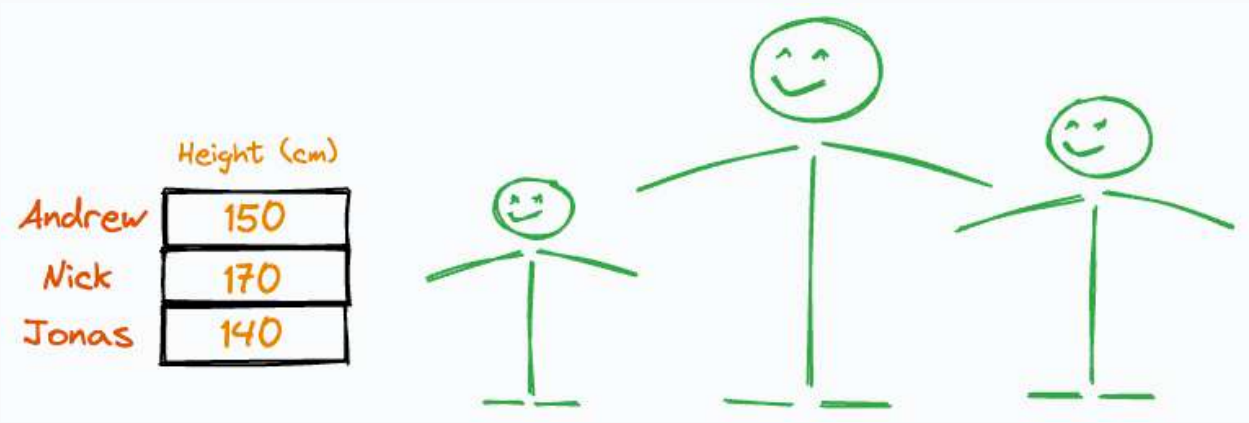
Imagine someone gave us the following weight and height information about three individuals:

	Height (cm)	Weight (kg)
Andrew	150	71
Nick	170	73
Jonas	140	70

It's clear that the height column has more variation than weight.



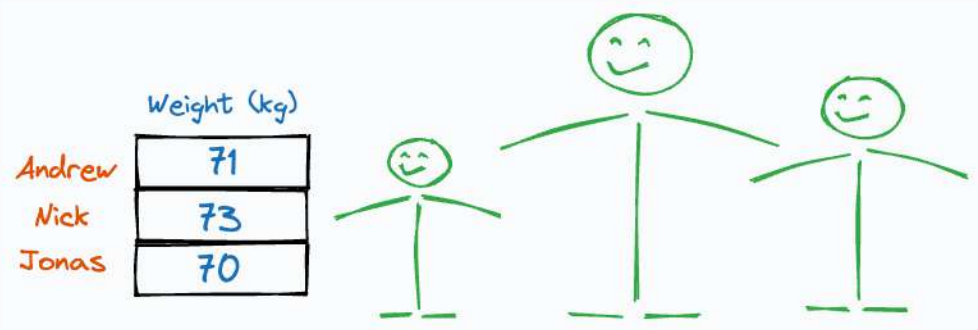
Thus, even if we discard the weight column, we can still identify these people solely based on their heights.



Dropping the weight column

- The one in the middle is Nick.
- The leftmost person is Jonas.
- The rightmost one is Andrew.

That was super simple. But what if we discarded the height column instead?



Can you identify them now?

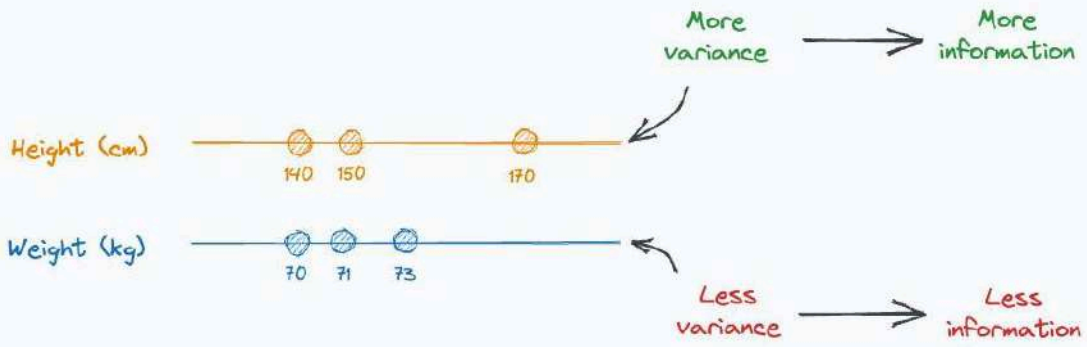
No, right?

And why?

This is because their heights have more variation than their weights.

And it's clear from the above example that, typically, if a column has more variation, it holds more information.

That is the core idea PCA is built around, and that is why it tries to retain maximum data variance. Simply put, PCA is devised on the premise that more variance means more information.

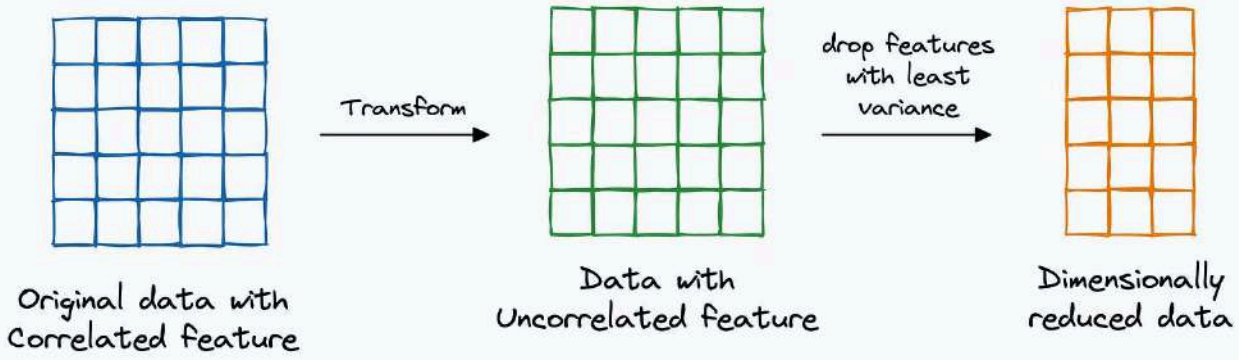


More variance means more information and less variance means less information Thus, during dimensionality reduction, we can (somewhat) say that we are retaining maximum information if we retain maximum variance.

Of course, as we are using variance, this also means that it can be easily influenced by outliers. That is why we say that PCA is influenced by outliers.

As a concluding note, always remember that when using PCA, we don't just measure column-wise variance and drop the columns with the least variance.

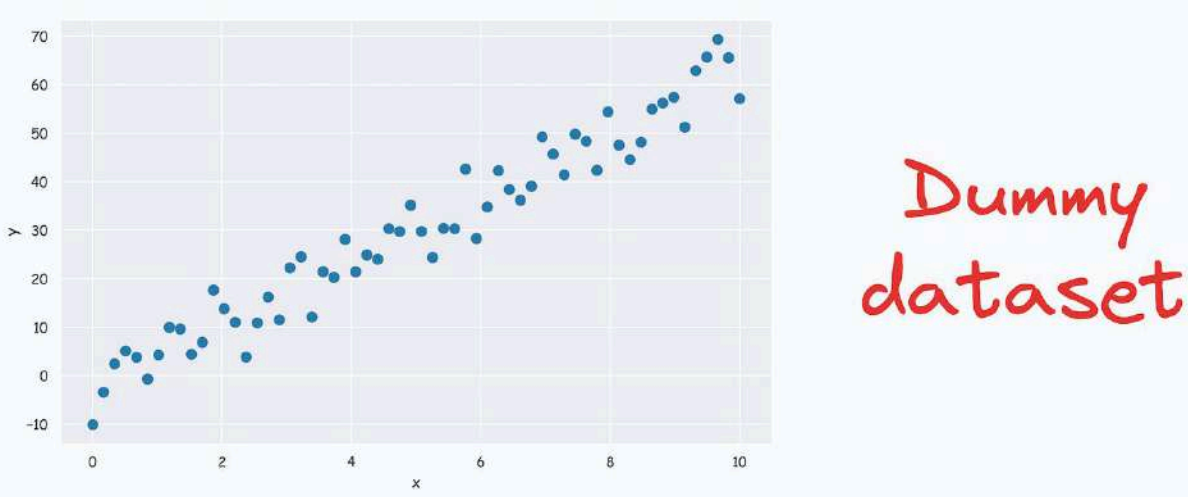
Instead, we must first transform the data to create uncorrelated features. After that, we drop the new features based on their variance.



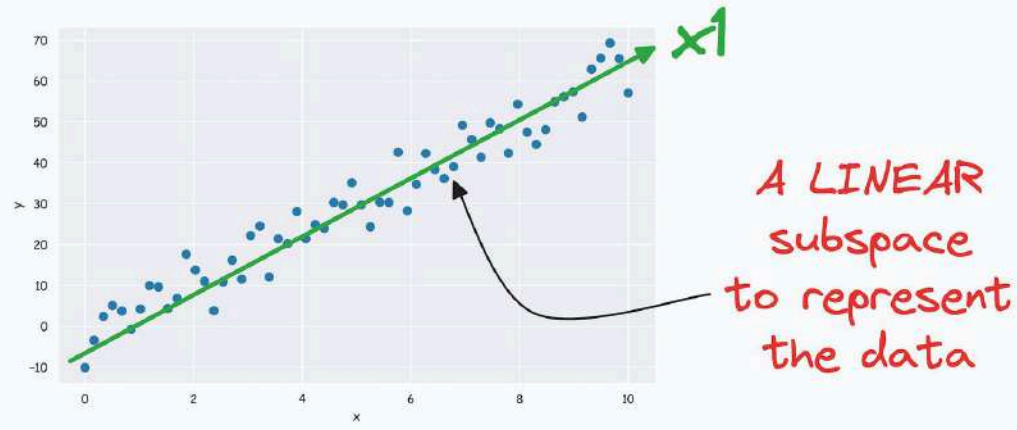
KernelPCA vs. PCA

During dimensionality reduction, principal component analysis (PCA) tries to find a low-dimensional linear subspace that the given data conforms to.

For instance, consider the following dummy dataset:

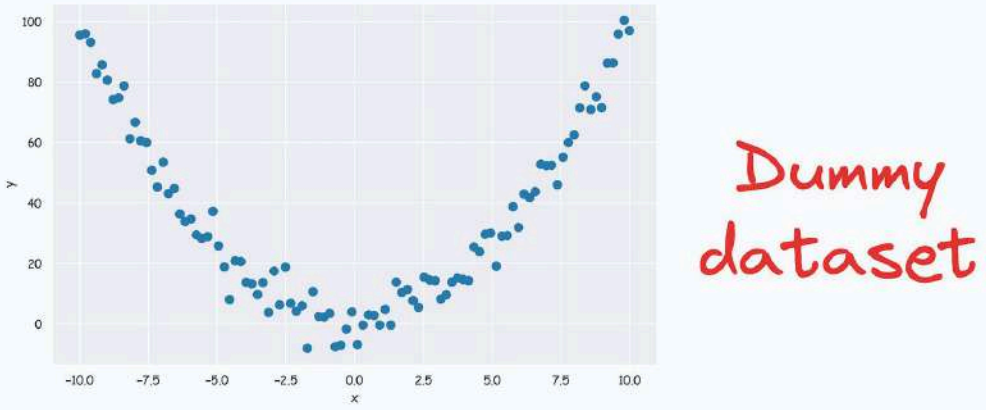


It's pretty clear from the above visual that there is a linear subspace along which the data could be represented while retaining maximum data variance. This is shown below:



But what if our data conforms to a low-dimensional yet non-linear subspace.

For instance, consider the following dataset:



Do you see a low-dimensional non-linear subspace along which our data could be represented?

No?

Don't worry. Let me show you!



The above curve is a continuous non-linear and low-dimensional subspace that we could represent our data given along.

Okay...so why don't we do it then?

The problem is that PCA cannot determine this subspace because the data points are non-aligned along a straight line.

In other words, PCA is a linear dimensionality reduction technique.

Thus, it falls short in such situations.

Nonetheless, if we consider the above non-linear data, don't you think there's still some intuition telling us that this dataset can be reduced to one dimension if we can capture this non-linear curve.



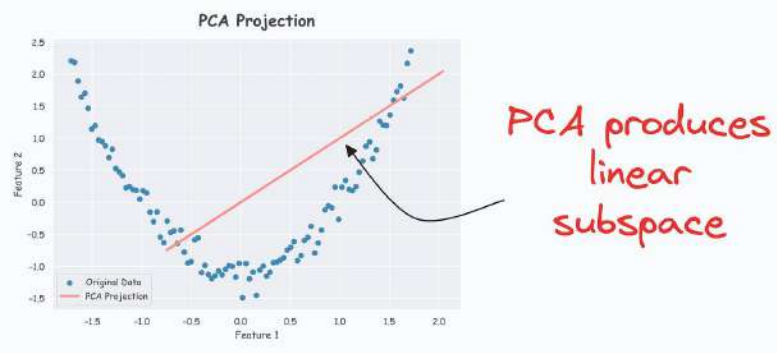
KernelPCA (which uses the kernel trick) precisely addresses this limitation of PCA.

The idea is pretty simple:

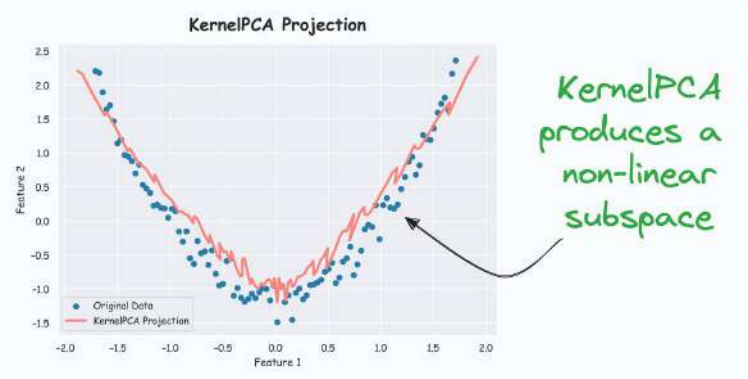
- Project the data to another high-dimensional space using a kernel function, where the data becomes linearly representable. Sklearn provides a KernelPCA wrapper, supporting many popularly used kernel functions.
- Apply the standard PCA algorithm to the transformed data.

The efficacy of KernelPCA over PCA is evident from the demo below.

As shown below, even though the data is non-linear, PCA still produces a linear subspace for projection:



However, KernelPCA produces a non-linear subspace:



What's the catch, you might be wondering?

The catch is the run time.

Please note that the run time of PCA is already cubically related to the number of dimensions.

$$O(nm^2 + m^3)$$

n -> data points
m -> dimensions

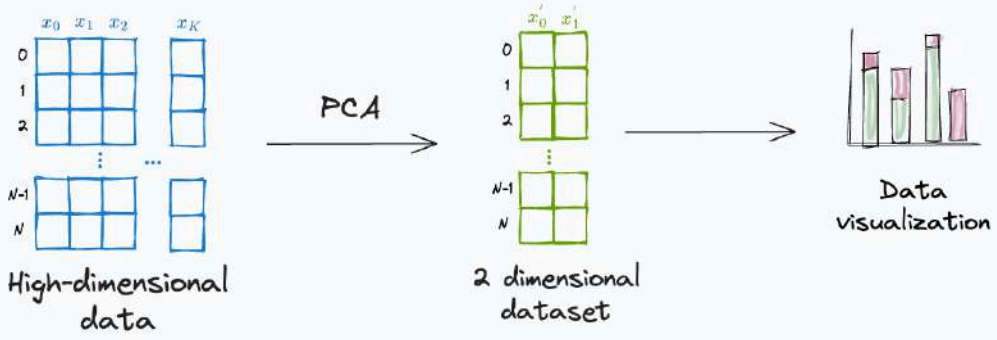
KernelPCA involves the kernel trick, which is quadratically related to the number of data points (n).

Thus, it increases the overall run time.

This is something to be aware of when using KernelPCA.

PCA is not a Visualization Technique

PCA, by its very nature, is a dimensionality reduction technique. Yet, at times, many use PCA for visualizing high-dimensional datasets. This is done by projecting the given data into two dimensions and visualizing it.



While this may appear like a fair thing to do, there's a big problem here that often gets overlooked.

To understand this problem, we first need to understand a bit about how PCA works.

How PCA works?

The core idea in PCA is to linearly project the data to another space using the eigenvectors of the covariance matrix.

$$X' = X\lambda$$ Transform data

Why eigenvectors?

1. It creates uncorrelated features, which is useful because if features are independent, the features with the least variance can be dropped for dimensionality reduction.

2. It ensures that new features collectively preserve the original data variance.

Coming back to the visualization topic...

As discussed above, after applying PCA, each new feature captures a fraction of the original data variance. Thus, if we intend to use PCA for visualization by projecting the data to 2-dimensions...

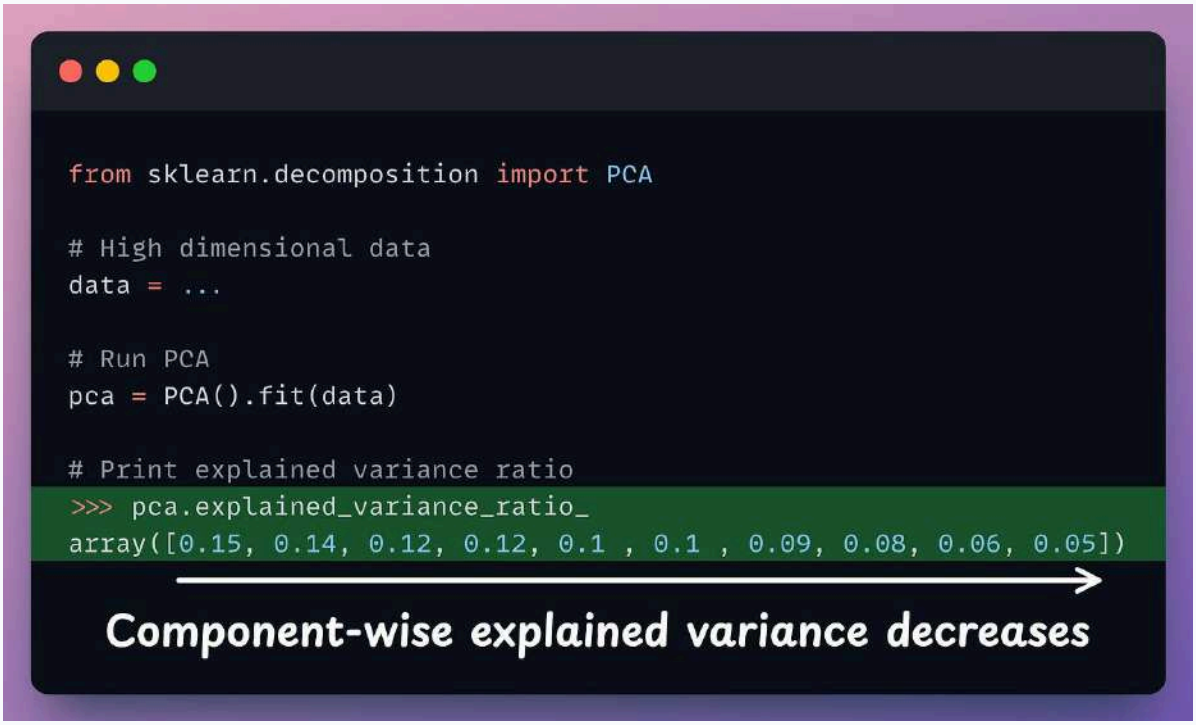
...then this visualization will only be useful if the first two principal components collectively capture most of the original data variance.

If they don't, then the two-dimensional visualization will be highly misleading and incorrect.

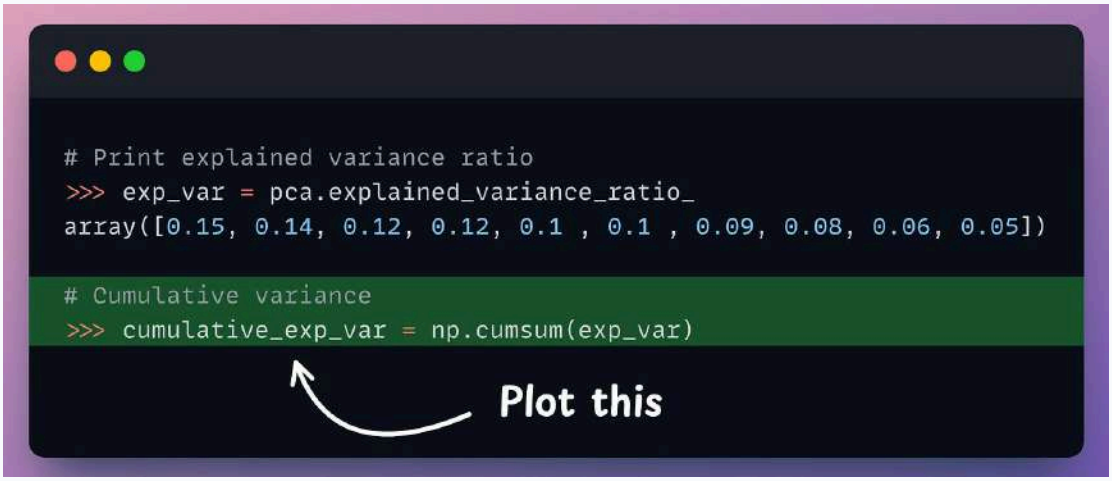
How to determine the variance contribution of the first two principal components?

We can avoid this mistake by plotting a cumulative explained variance (CEV) plot. As the name suggests, it plots the cumulative variance explained by principal components.

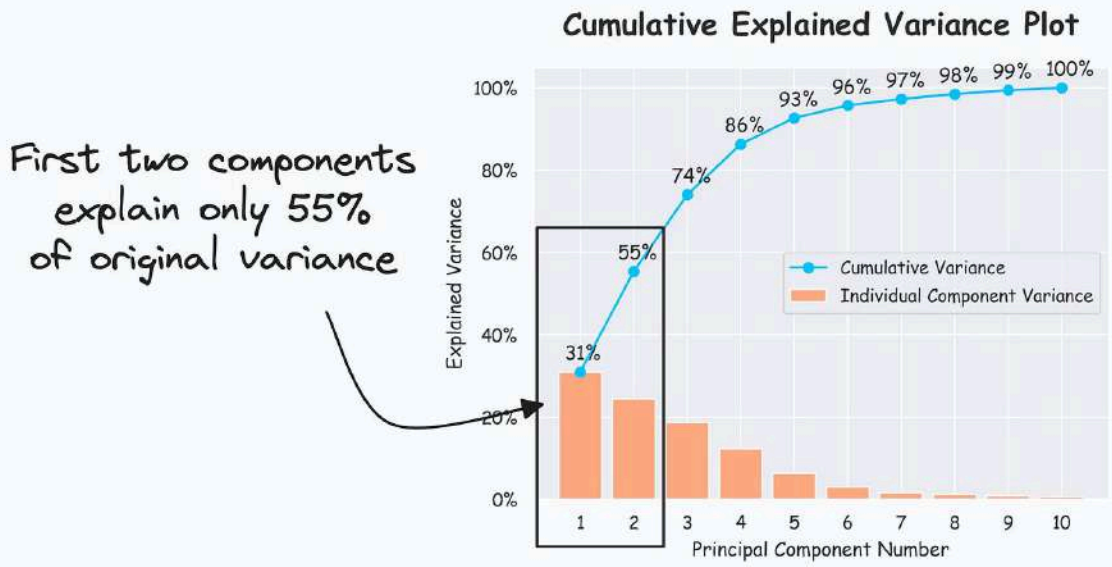
In sklearn, for instance, the explained variance fraction is available in the `explained_variance_ratio_` attribute:



We can create a cumulative plot of explained variance and check whether the first two components explain the majority of variance.

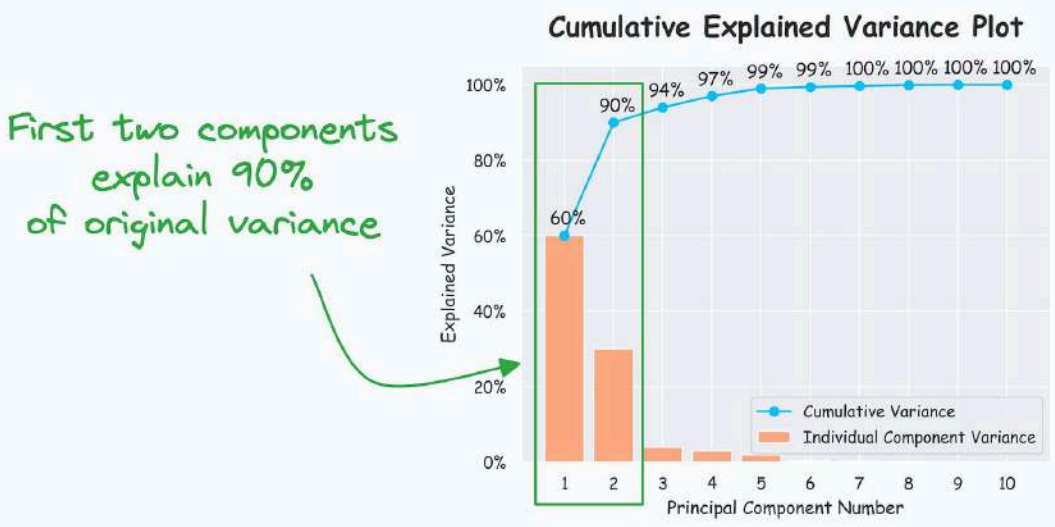


For instance, in the plot below, the first two components only explain 55% of the original data variance.



Thus, visualizing this dataset in 2D using PCA may not be a good choice because plenty of data variance is missing.

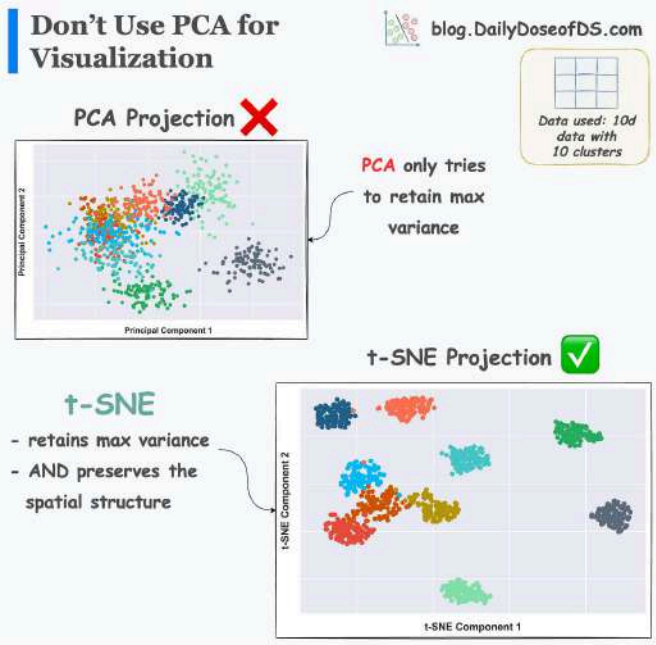
However, in the below plot, the first two components explain 90% of the original data variance.



Thus, using PCA for visualization looks like a fair thing to do. As a takeaway, use PCA for 2D visualization only when the above plot suggests so.

If it does not, then refrain from using PCA for 2D visualization and use other techniques specifically meant for visualization, like t-SNE, UMAP, etc. The efficacy of t-SNE over PCA is depicted below:

2D projections created by PCA do not consider local structure. Instead, its principal components primarily focus on preserving the maximum variance. But t-SNE projections provide much more clarity into data clusters. The clusters produced by t-SNE are well separated. But those in PCA have a significant overlap, and they hardly convey anything.



t-SNE vs. SNE — What's the difference?

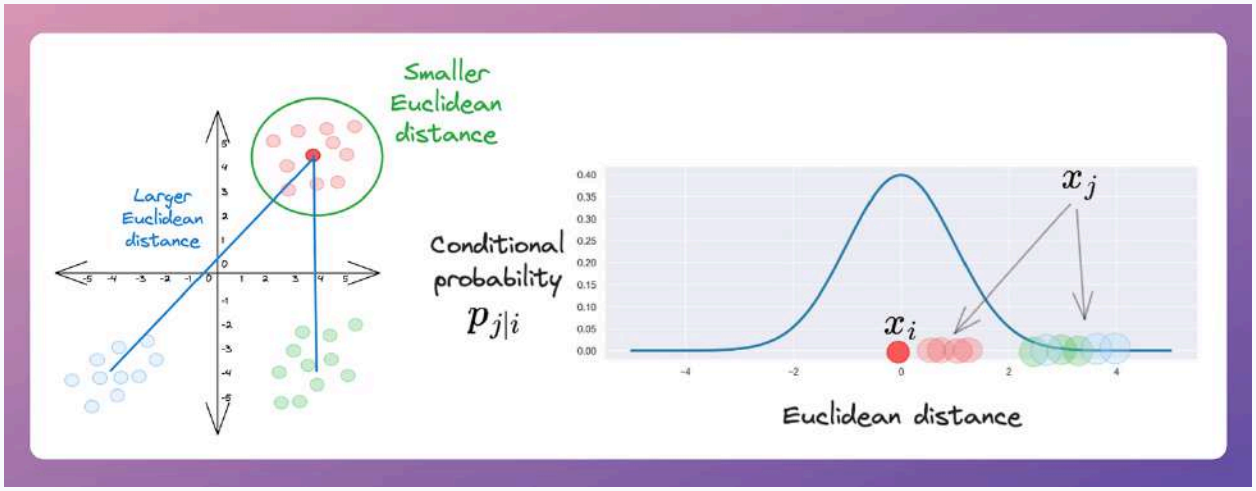
In this chapter, let's continue discussing the t-SNE algorithm.

The t-SNE algorithm is an improved version of the SNE algorithm, both used for dimensionality reduction.



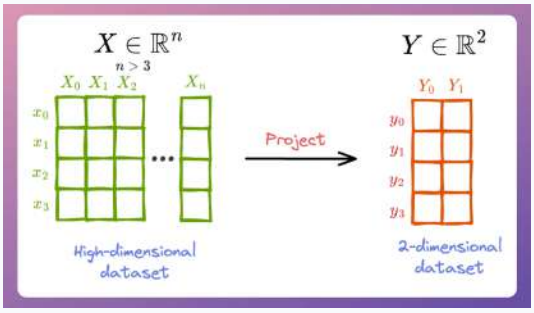
The core idea of SNE (not t-SNE) is the following:

- Step 1) For every point (i) in the given high-dimensional data, convert the high-dimensional Euclidean distances to all other points (j) into conditional Gaussian probabilities.
    - For instance, consider the marked red point in the dataset on the left below.

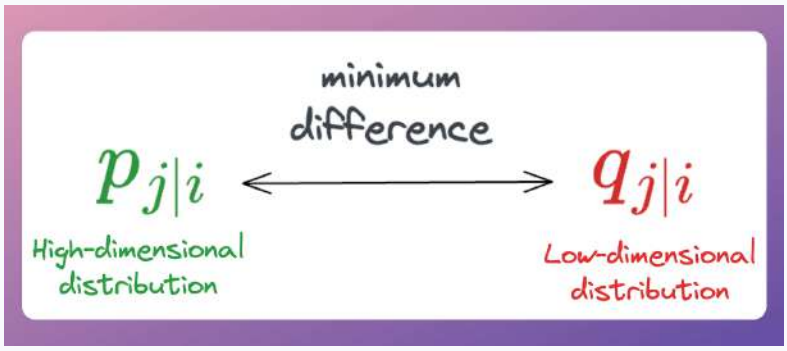


    - Converting Euclidean distance to all other points into Gaussian probabilities (the distribution on the right above) shows that other red points have a higher probability of being its neighbor than other points.

- Step 2) For every data point x_i, randomly initialize its counterpart y_i in 2-dimensional space. These will be our projections.



- Step 3) Just like we defined conditional probabilities in the high-dimensional space in Step 1, we define the conditional probabilities in the low-dimensional space, using Gaussian distribution again.
- Step 4) Now, every data point (i) has a high-dimensional probability distribution and a corresponding low-dimensional distribution:



  - The objective is to match these two probability distributions. Thus, we can make the positions of counterpart y_i's learnable such that this difference is minimized.
  - Using KL divergence as a loss function helps us achieve this. It measures how much information is lost when we use distribution Q to approximate distribution P.

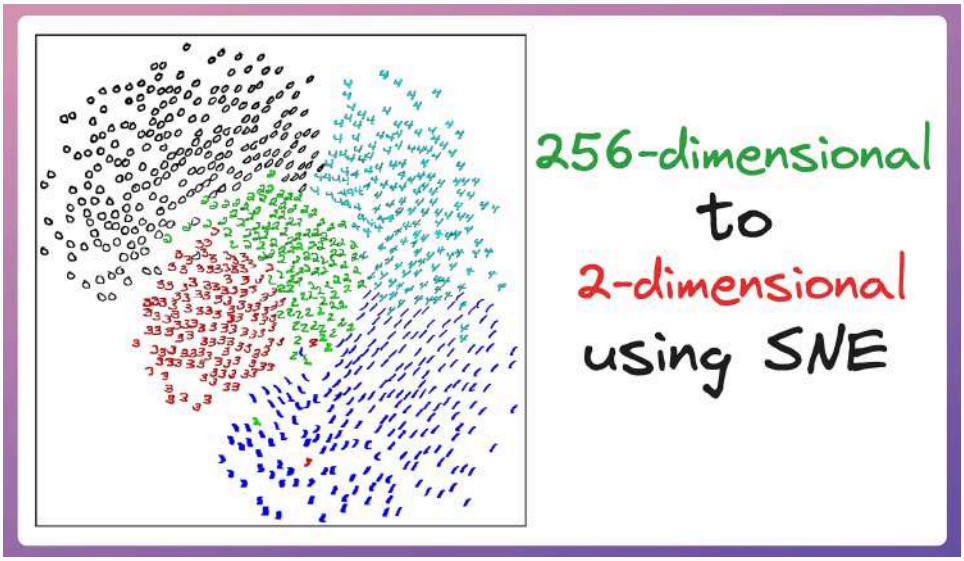
$$D_{KL}(P \parallel Q) = \sum_{x} P(x) \cdot \log\left(\frac{P(x)}{Q(x)}\right)$$

- Ideally, we want to have the minimum loss value (which is zero), and this will be achieved when P=Q.

$$D_{KL}(P \parallel Q) = \sum_{x} P(x) \cdot \log\left(\frac{P(x)}{Q(x)}\right) \quad \text{if } P = Q \Rightarrow loss = 0$$

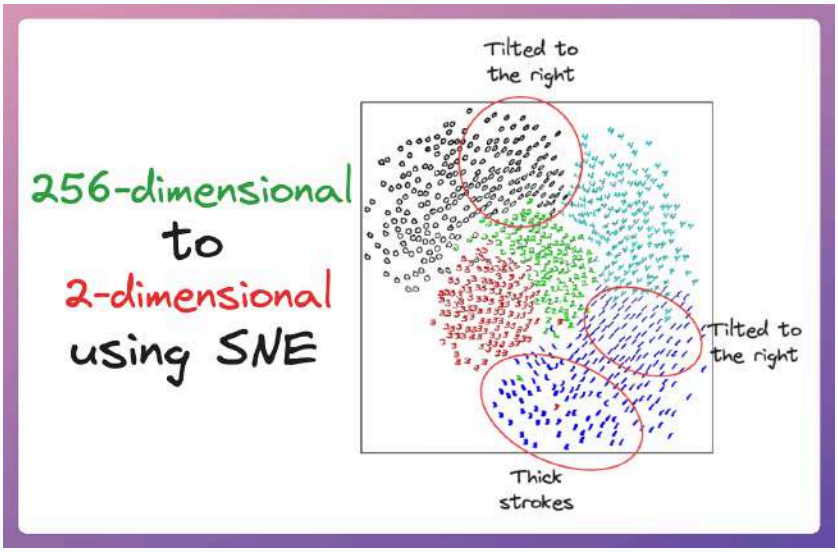
The model can be trained using gradient descent, and it works pretty well.

For instance, the following image depicts a 2-dimensional visualization produced by the SNE algorithm on 256-dimensional handwritten digits:

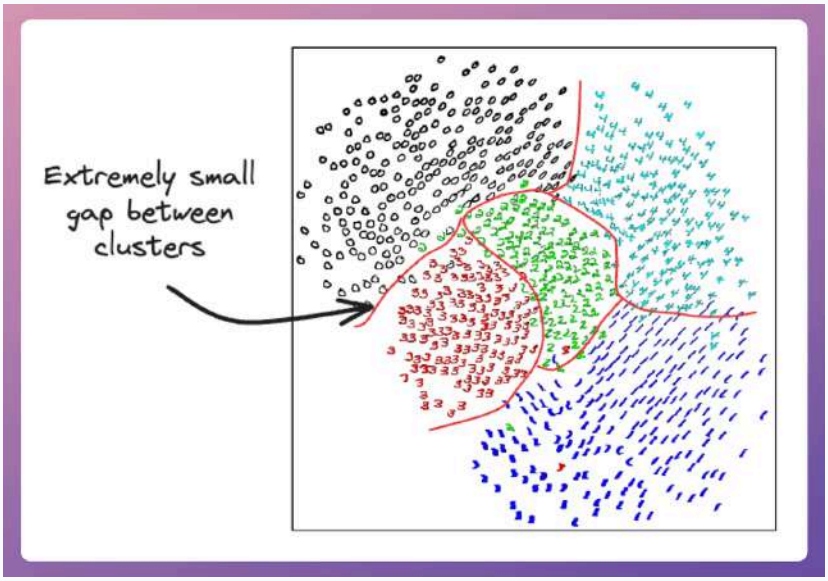


SNE produces good clusters.

What's even more astonishing is that properties like orientation, skew, and strokethickness vary smoothly across the space within each cluster. This is depicted below:



Nonetheless, it has some limitations, which the t-SNE algorithm addresses. Notice that the clusters produced by SNE are not well separated.



Here, it could be fair to assume that the original data clusters, the one in the 256-dimensional space, most likely would have been well separated. Thus:

- All zeros must have been together but well separated from other digits.
- All ones must have been together but well separated from other digits.
- And so on.

Yet, SNE still produces tightly packed clusters. This is also called the "crowding problem."

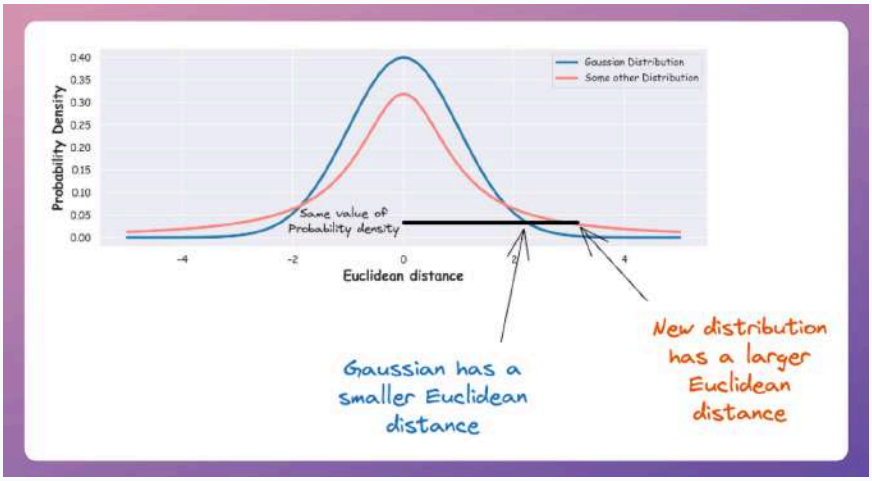
To eliminate this problem, t-SNE was proposed, standing for t-distributed Stochastic Neighbor Embedding (t-SNE).

Here's the difference.

Recall that in SNE, we used a Gaussian distribution to define the low-dimensional conditional probabilities. But it is not producing well-separated clusters.

One solution is to use some other probability distribution, such that for distant points, we get the same value of the conditional probability as we would have obtained from a Gaussian distribution but at a larger Euclidean distance.

Let me simplify that a bit. Compare the following two distributions:

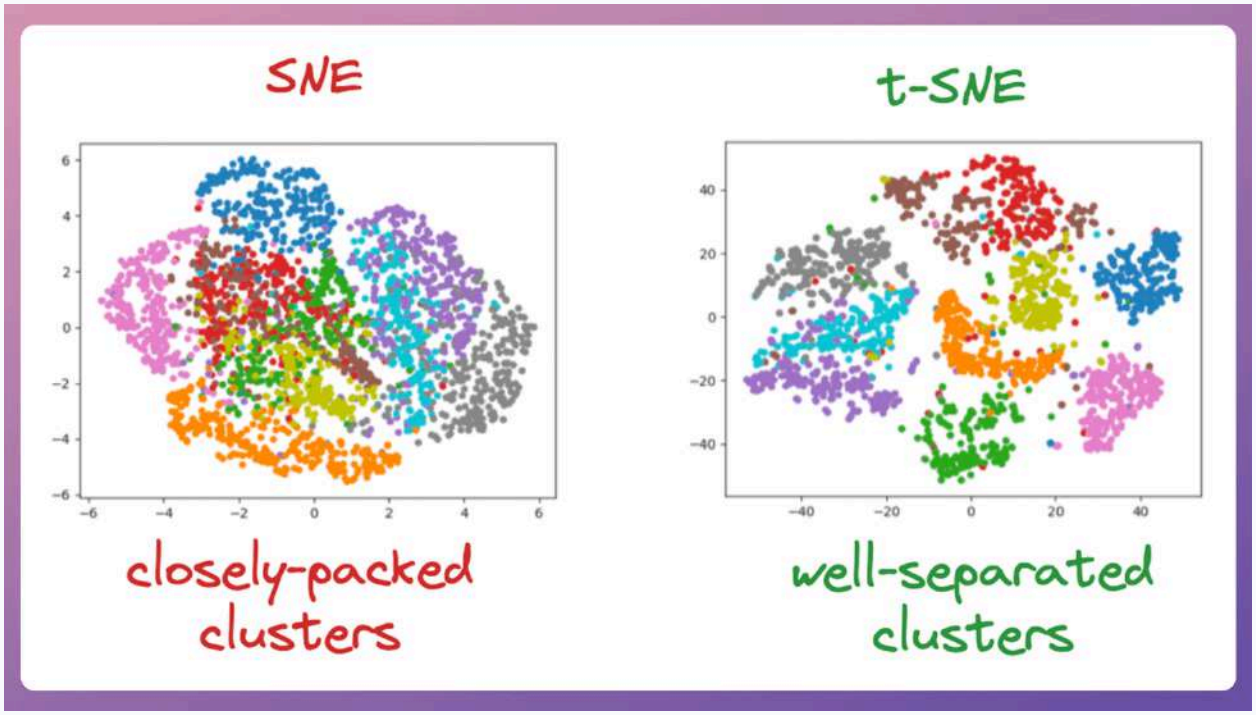


Notice that Gaussian achieves a specific value of low probability density at a smaller distance. But t-distribution achieves it at a larger distance.

This is precisely what we intend to achieve.

We need a heavier tail distribution so that we can still minimize the difference between the two probability distributions but at a larger distance in the low-dimensional space.

The Student t-distribution is a perfect fit for it. The following image depicts the difference this change brings:



As shown above:

- SNE produces closely packed clusters.
- t-SNE produces well-separated clusters.

And that's why t-distribution is used in t-SNE.

That said, besides producing well-separated clusters, using the Student t-distribution has many more advantages.

For instance, it is computationally much faster to evaluate the density of a point under a Student t-distribution than under a Gaussian.

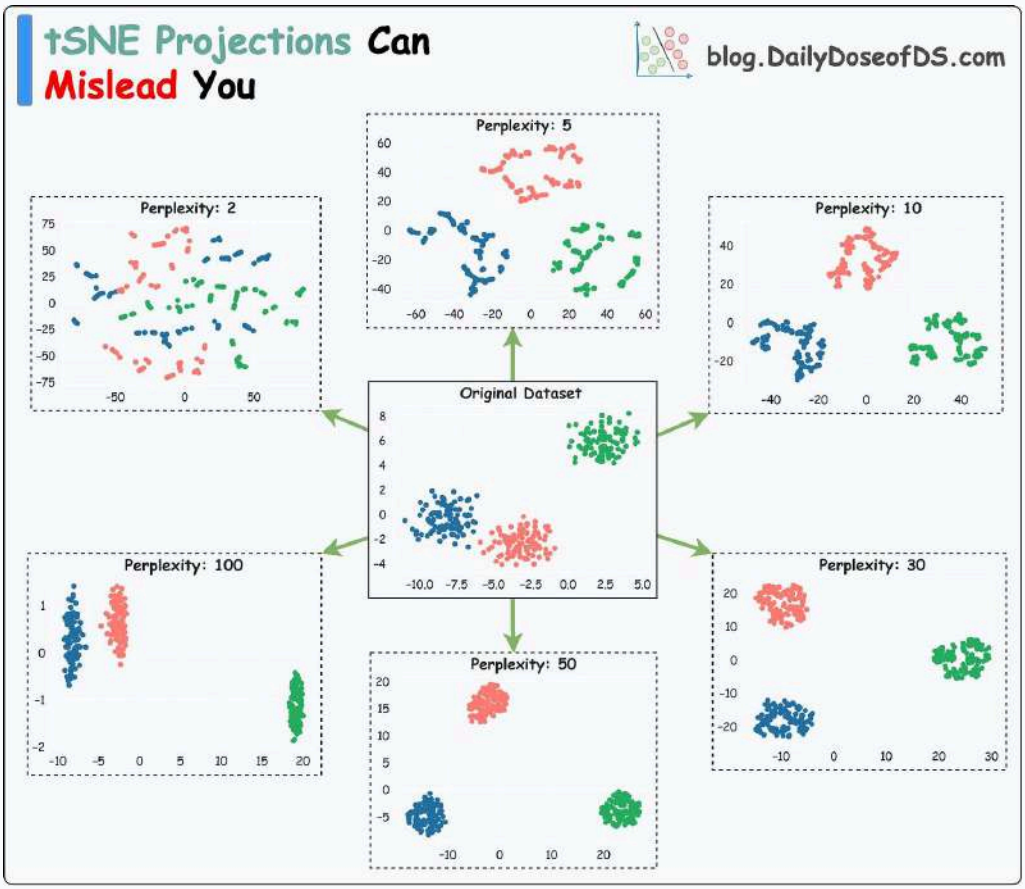
How To Avoid Getting Misled by t-SNE Projections?

From the above discussion, t-SNE sounds promising, doesn't it? While the algorithm is quite powerful, many consistently make misleading conclusions from the t-SNE projections of their high-dimensional data.

In this chapter, I want to point out a few of these mistakes so that you don't make those mistakes ever.

To begin, the performance of the t-SNE algorithm is primarily reliant on perplexity — a hyperparameter of t-SNE. That is why it is considered the most important hyperparameter in the t-SNE algorithm.

Simply put, the perplexity value provides a rough estimate for the number of neighbors a point may have in a cluster. And different values of perplexity create very different low-dimensional cluster spaces, as depicted below:



As shown above, most projections do depict the original clusters. However, they vary significantly in shape.

There are five takeaways from the above image:

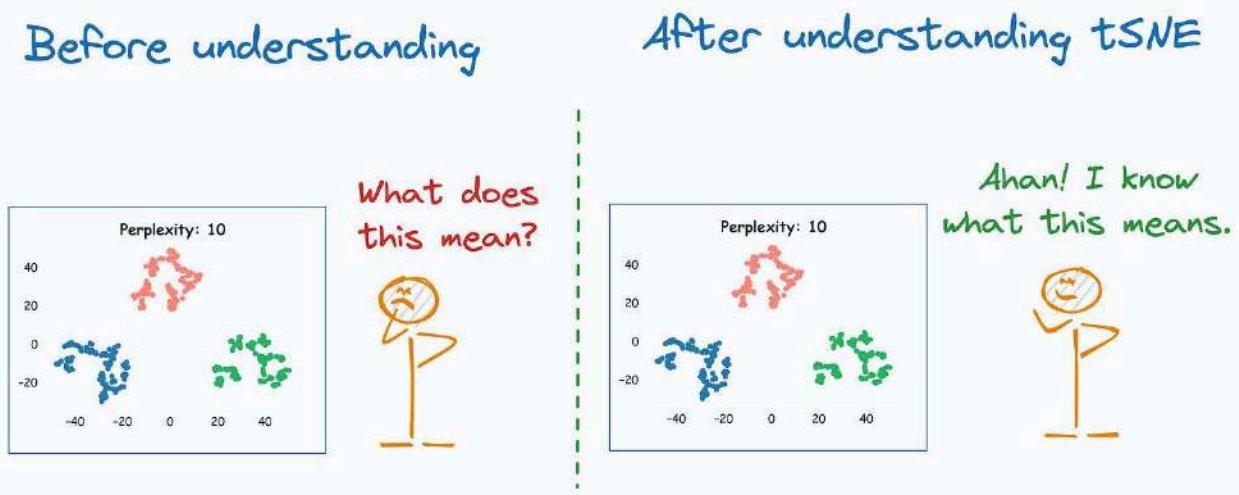
- NEVER make any conclusions about the original cluster shape by looking at these projections.
    - Different projections have different low-dimensional cluster shapes, and they do not resemble the original cluster shape.
    - For low perplexity values (5 and 10), cluster shapes significantly differ from the original ones.
    - Although, in this case, the clusters were color-coded, which provided more clarity. But it may not always be the case, as tSNE is an unsupervised algorithm.
- Cluster sizes in a t-SNE plot do not convey anything either.
- The dimensions (or coordinates of data points) created by t-SNE in low dimensions have no inherent meaning.
    - The axes tick labels of the low-dimensional plots are different and somewhat random.
    - Similar to PCA's principal components, they offer no interpretability.
- The distances between clusters in a projection do not mean anything.
    - In the original dataset, the blue and red clusters are close.
    - Yet, most projections do not preserve the global structure of the original dataset.
- Strange things happen at `perplexity=2` and `perplexity=100`.
    - At `perplexity=2`, the low-dimensional mapping conveys nothing.
        - As discussed earlier, the perplexity value provides a rough estimate of the number of neighbors a point may have in a cluster.
        - t-SNE tries to maintain approx. 2 points per cluster. That is why the distortion.

- At `perplexity=100`, the global structure is preserved, but the local structure gets distorted.
- Thus, tweaking the perplexity hyperparameter is extremely critical here.
- That is why I mentioned above that it is the most important hyperparameter of this algorithm.

As a concluding note, it is found that the ideal perplexity values typically lie in the range `[5,50]`. So try experimenting in that range and see what looks promising.

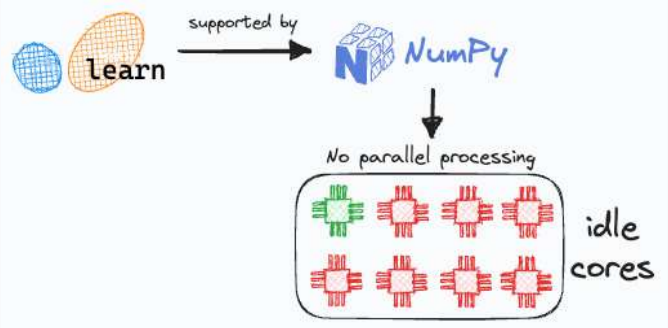
Next time you use t-SNE, consider the above points, as these plots can get tricky to interpret. This is especially true if you don't understand the internal workings of this algorithm.

Nonetheless, understanding the algorithm will massively help you develop an intuition on its interpretability.

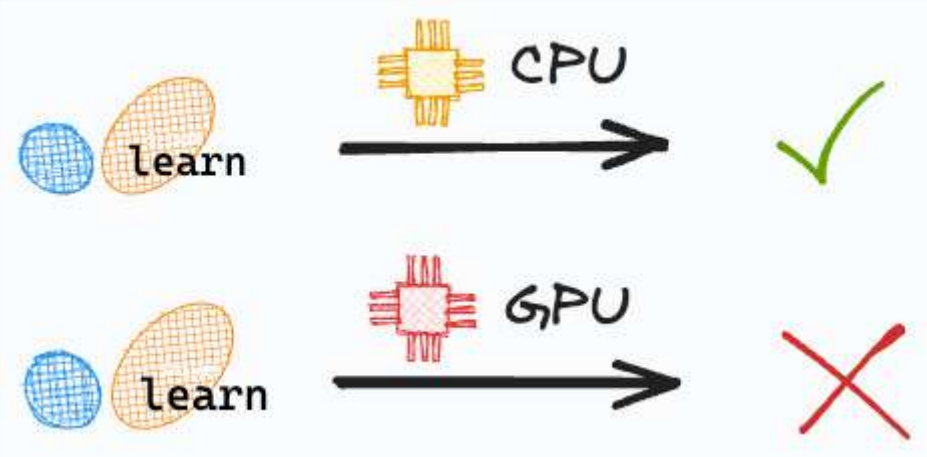


Accelerate tSNE with GPU

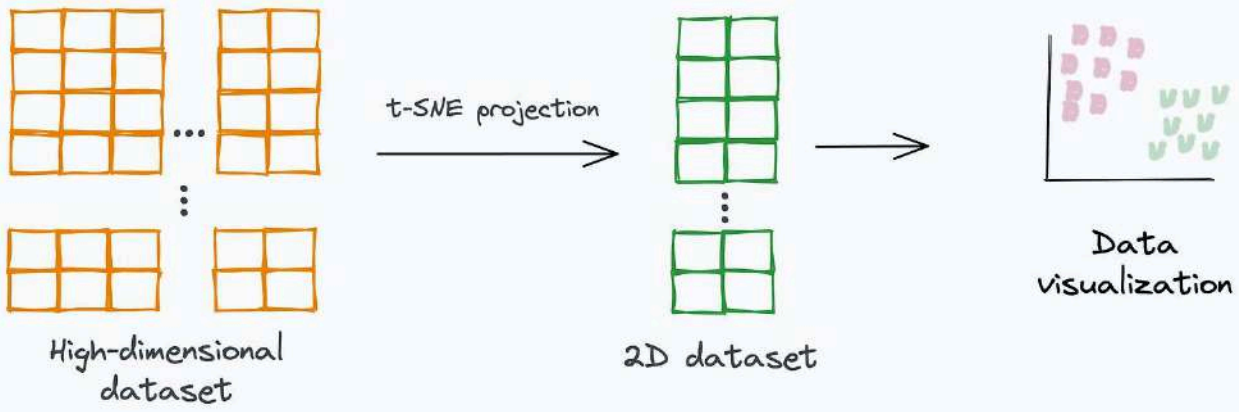
Sklearn implementations are driven by NumPy, which typically run on a single core of a CPU. Thus, the ability to run parallelized operations is quite limited at times.



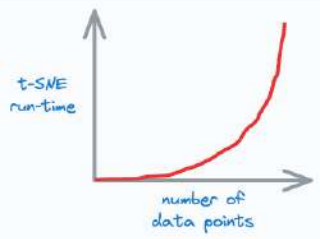
Another major limitation is that scikit-learn models cannot run on GPUs.



This bottleneck provides massive room for run-time improvement. The same applies to the tSNE algorithm, which is among the most powerful dimensionality reduction techniques to visualize high-dimensional datasets.



Because the biggest issue with tSNE is that its run-time is quadratically related to the number of data points.

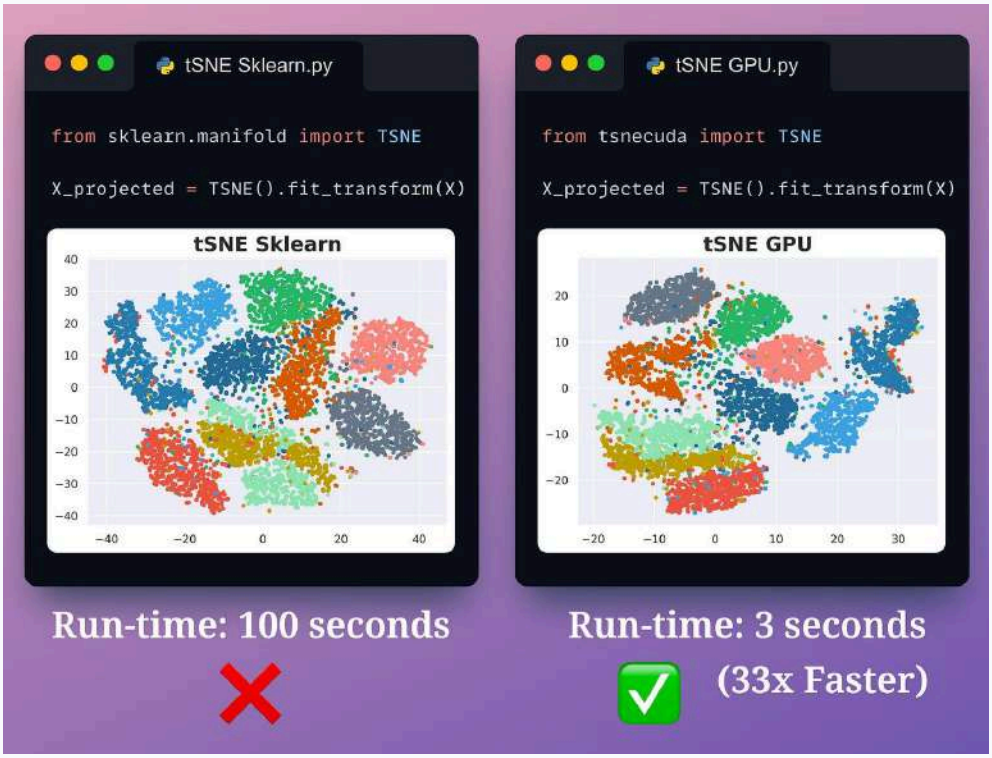


Thus, beyond, say, 20k-25k data points, it becomes pretty difficult to use tSNE from Sklearn implementations. There are two ways to handle this:

- Either keep waiting.
- Or use optimized implementations that could be possibly accelerated with GPU.

I was stuck due to the same issue in one of my projects, and I found a pretty handy solution.

tSNE-CUDA is an optimized CUDA version of the tSNE algorithm, which, as the name suggests, can leverage hardware accelerators. As a result, it provides immense speedups over the standard Sklearn implementation:



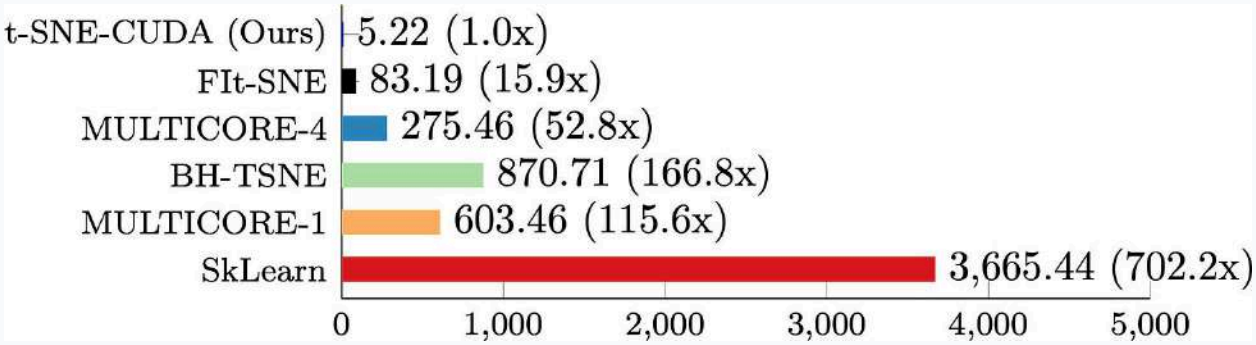
As depicted above, the GPU-accelerated implementation:

- Is 33 times faster than the Sklearn implementation.
- Produces similar quality clustering as the Sklearn implementation.

Do note that this implementation only supports `n_components=2`, i.e., you can only project to two dimensions. As per the docs, the authors have no plans to support more dimensions, as this will require significant changes to the code.

But in my opinion, that doesn't matter because, for more than 99% of cases, tSNE is used to obtain 2D projections. So we are good here.

Before I conclude this chapter, I also found the following benchmarking results by the authors:



It depicts that on the CIFAR-10 training set (50k images), tSNE-CUDA is 700x Faster than Sklearn, which is an insane speedup.

Here's my Colab notebook for you to get started: https://bit.ly/3WdEone.

Scale tSNE to Millions of Data Points With openTSNE

tSNE-CUDA discussed above provides immense speedups over the standard Sklearn implementation using a GPU. But what if you don't have access to a GPU?

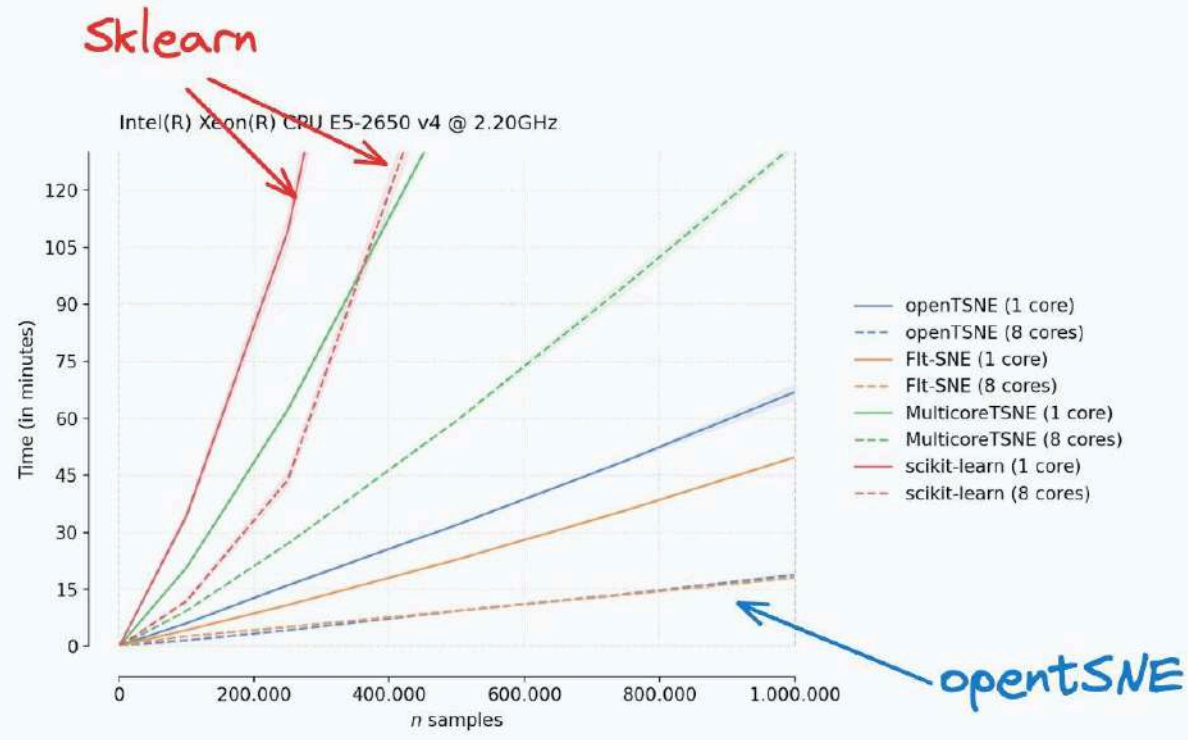
openTSNE is another optimized Python implementation of t-SNE, which provides massive speed improvements and enables us to scale t-SNE to millions of data points — a place where Sklearn implementation may never reach.



As depicted above, the openTSNE implementation:

- is 20 times faster than the Sklearn implementation.
- produces similar quality clustering as the Sklearn implementation.

The authors have also provided the following benchmarking results:



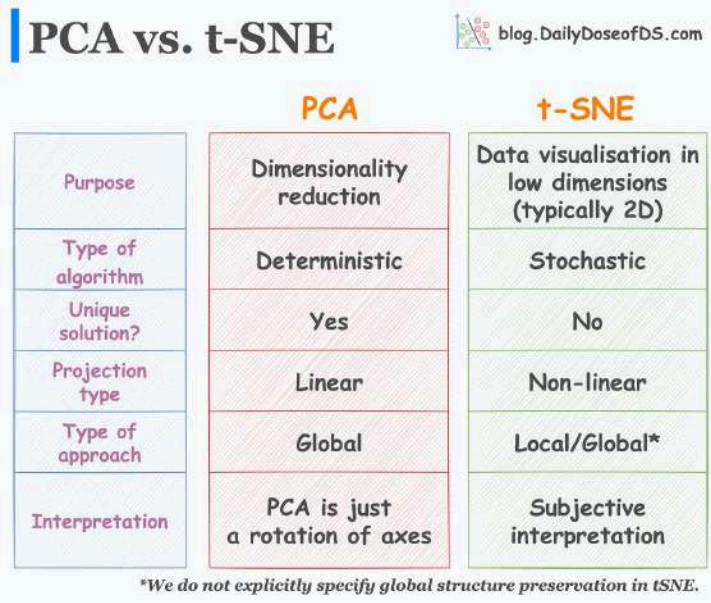
As depicted above, openTSNE can produce low dimensional visualization of a million data points in just ~15 minutes.

However, it is clear from their benchmarks that the run-time of the Sklearn implementation has already reached a couple of hours with just ~250k data points.

Download the notebook here to try out openTSNE: https://bit.ly/3zAHCZ8.

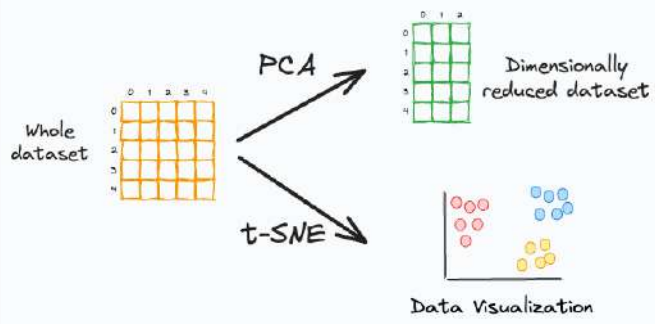
PCA vs. t-SNE

Finally, it would be good to look at the core differences between PCA and t-SNE algorithms. The following table for you which neatly summarizes the major differences between the two algorithms:



#1) Purpose

- While many interpret PCA as a data visualization algorithm, it is primarily a dimensionality reduction algorithm.
- t-SNE, however, is a data visualization algorithm. We use it to project high-dimensional data to low dimensions (primarily 2D).



#2) Type of algorithm

- PCA is a deterministic algorithm. Thus, if we run the algorithm twice on the same dataset, we will ALWAYS get the same result.
- t-SNE, however, is a stochastic algorithm. Thus, rerunning the algorithm can lead to entirely different results.

#3) Uniqueness of solution

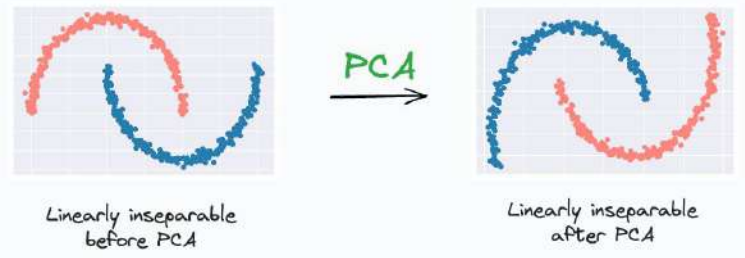
As far as uniqueness and interpretation of results is concerned...



- PCA always has a unique solution for the projection of data points. Simply put, PCA is just a rotation of axes such that the new features we get are uncorrelated.
- t-SNE, as discussed above, can provide entirely different results, and its interpretation is subjective in nature.

#4) Projection type

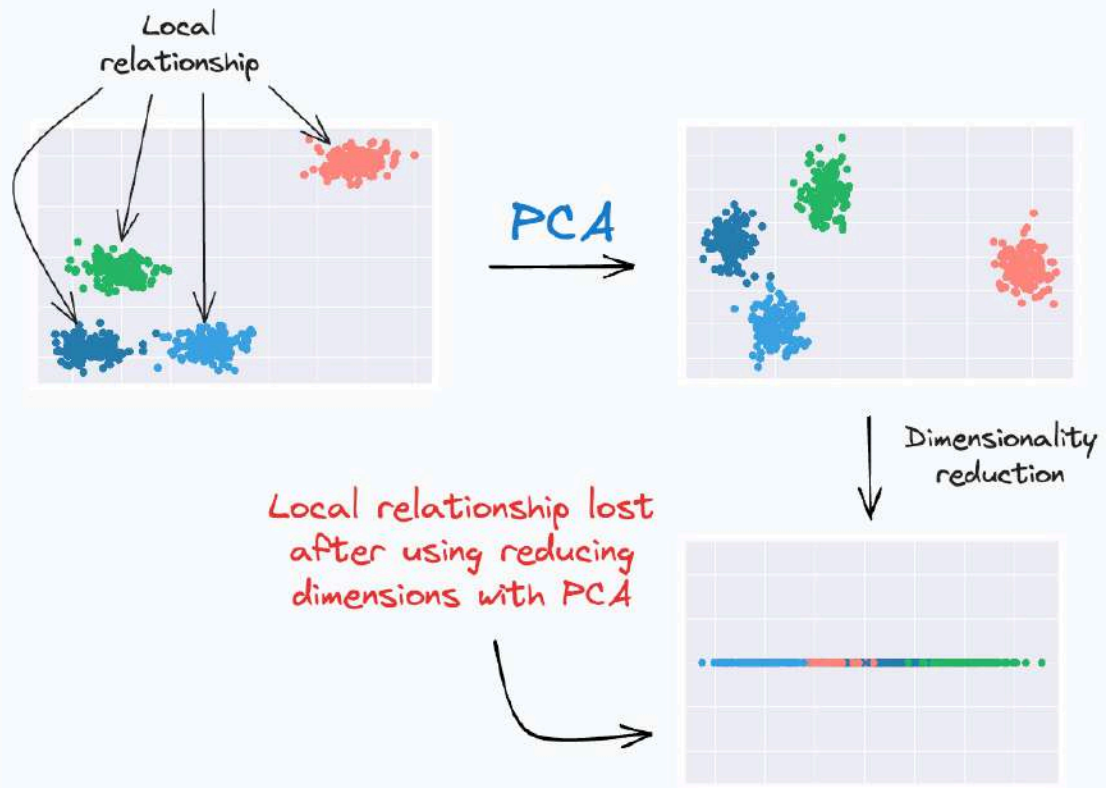
- PCA is a linear dimensionality reduction approach. It can only find a linear subspace to project the given dataset. KernelPCA addresses this, which we covere earlier in this section.



- t-SNE is a non-linear approach. It can handle non-linear datasets.

#5) Projection approach

- PCA only aims to retain the global variance of the data. Thus, local relationships (such as clusters) are often lost after projection, as shown below:



- t-SNE preserves local relationships. Thus, data points in a cluster in the high-dimensional space are much more likely to lie together in the low-dimensional space.
    - In t-SNE, we do not explicitly specify global structure preservation. But it typically does create well-separated clusters.
    - Nonetheless, as discussed in the previous chapter, the distance between two clusters in low-dimensional space is NEVER an indicator of cluster separation in high-dimensional space.

Clustering

Beyond KMeans: 6 Must-Know Types of Clustering Algorithms

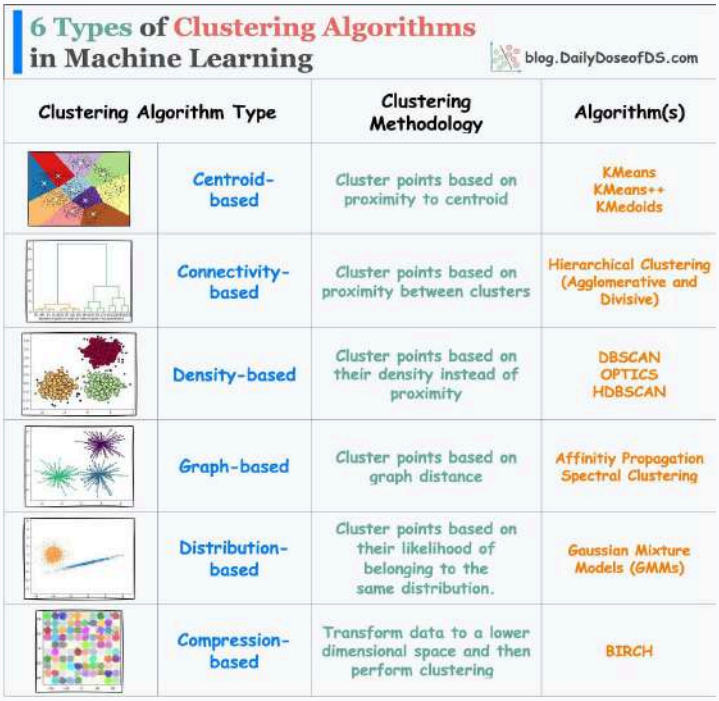
Clustering is one of the core branches of unsupervised learning in ML. The first (and sometimes the only) clustering algorithm folks learn is KMeans. Yet, it is important to note that KMeans is not a universal solution to all clustering problems. In fact, there's a whole world of clustering algorithms beyond KMeans, which we must be familiar with. The visual below summarizes 6 different types of clustering algorithms:

Centroid-based: Cluster data based on proximity to centroids.

Connectivity-based: Cluster points based on proximity between clusters.

Density-based: Cluster points based on their density. It is more robust to clusters with varying densities and shapes than centroid-based clustering.

Graph-based: Cluster points based on graph distance.

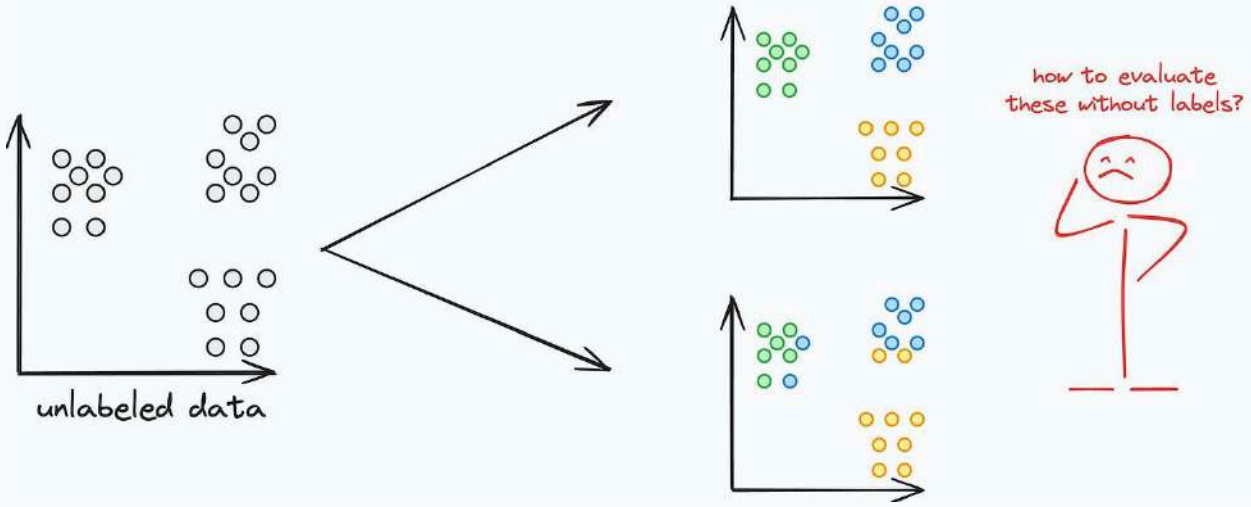


Distribution-based: Cluster points based on their likelihood of belonging to the same distribution. Gaussian Mixture Model in one example. We will discuss Gaussian mixture models (GMMs) in an upcoming chapter.

Compression-based: Transform data to a lower dimensional space and then perform clustering

Intrinsic Measures for Clustering Evaluation

Without labeled data, it is not possible to objectively measure how well the clustering algorithm has grouped similar data points together and separated dissimilar ones.



Also, most clustering datasets are multi-dimensional, so directly visualizing the results is not possible.

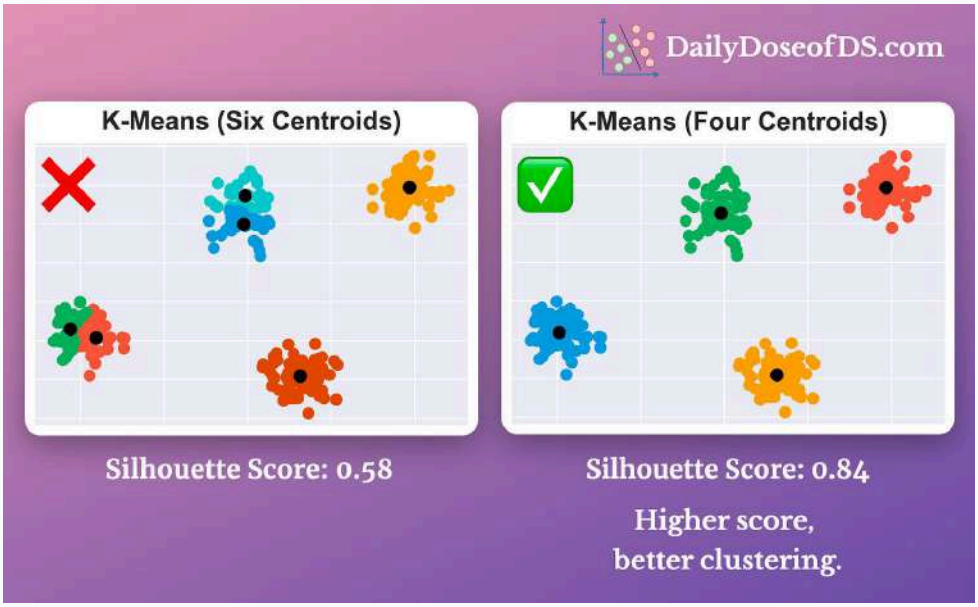
Thus, we must rely on intrinsic measures to determine clustering quality in such cases.

The way I like to use them is as follows:

- Say I am using KMeans:
    - Run KMeans with a range of k values.
    - Evaluate the performance.
    - Select the value of k based on the most promising cluster quality metric.

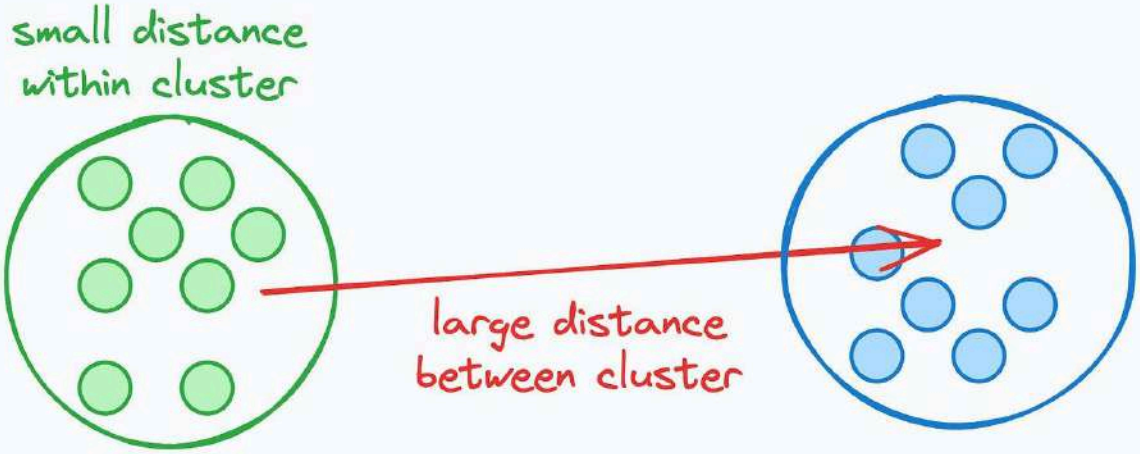
Let's understand a few of these metrics.

#1) Silhouette Coefficient:



The Silhouette Coefficient indicates how well each data point fits into its assigned cluster.

The idea is that if the average distance to all data points in the same cluster is small but that to another cluster is large, this intuitively indicates that the clusters are well separated and somewhat reliable.



It is measured as follows:

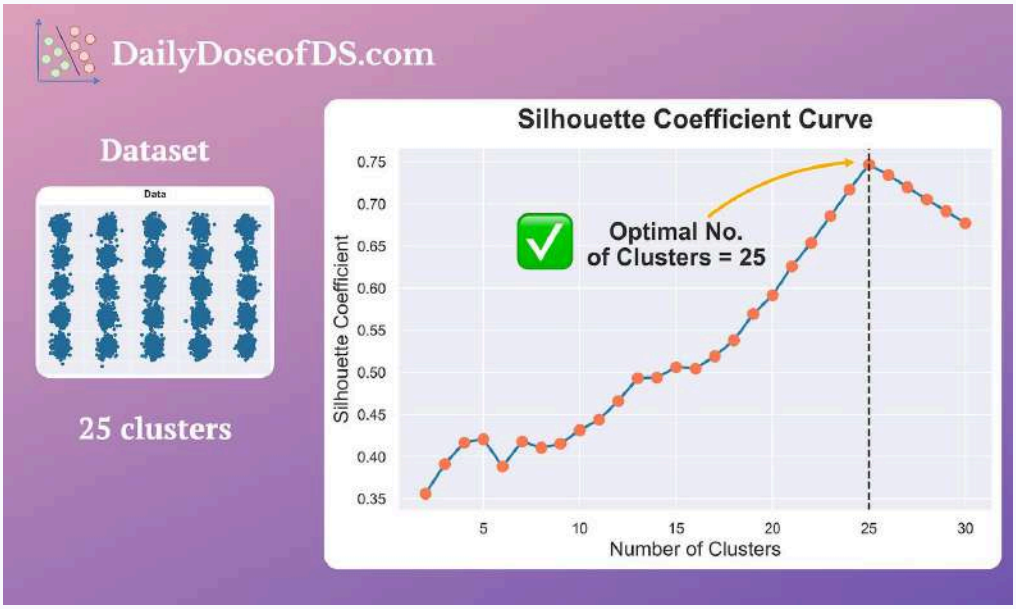
- For every data point:
  - find the average distance to all other points within its cluster → A
  - find the average distance to all points in the nearest cluster → B
  - point's score = (B-A)/max(B, A)
- Compute the average of all individual scores to get the overall clustering score.

Now consider this:

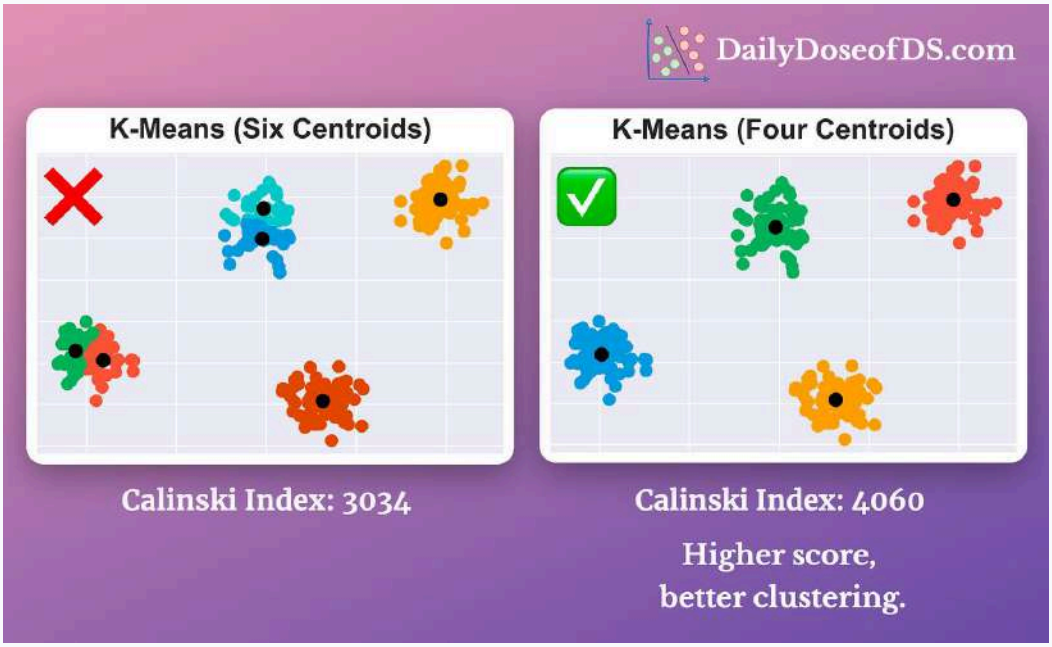
- If, for all data points, B is much greater than A, this would mean that:
  - The average distance to points in the closest cluster is large.
  - The average distance to points in the same cluster is small.
  - Thus, the overall score will be close to 1, indicating that the clusters are well separated.

So, a higher Silhouette Coefficient would mean that, generally speaking, all data points fit into their ideal clusters.

Measuring it across a range of centroids (k) can reveal which clustering results are most promising:



#2) Calinski-Harabasz Index



The issue with the Silhouette score is that its run-time grows quadratically with the number of data points.

Calinski-Harabasz Index is another metric that is measured quite similarly to the Silhouette Coefficient.

Here's what it computes:

- A → sum of squared distance between all centroids and overall dataset center.
- B → sum of squared distance between all points and their specific centroid.
- metric is computed as A/B (with an additional scaling factor).

Yet again, is A is much greater than B:

- The distance of centroids to the dataset center is large.
- The distance of data points to their specific centroid is small
- Thus, it will result in a higher score, indicating that the clusters are well separated.

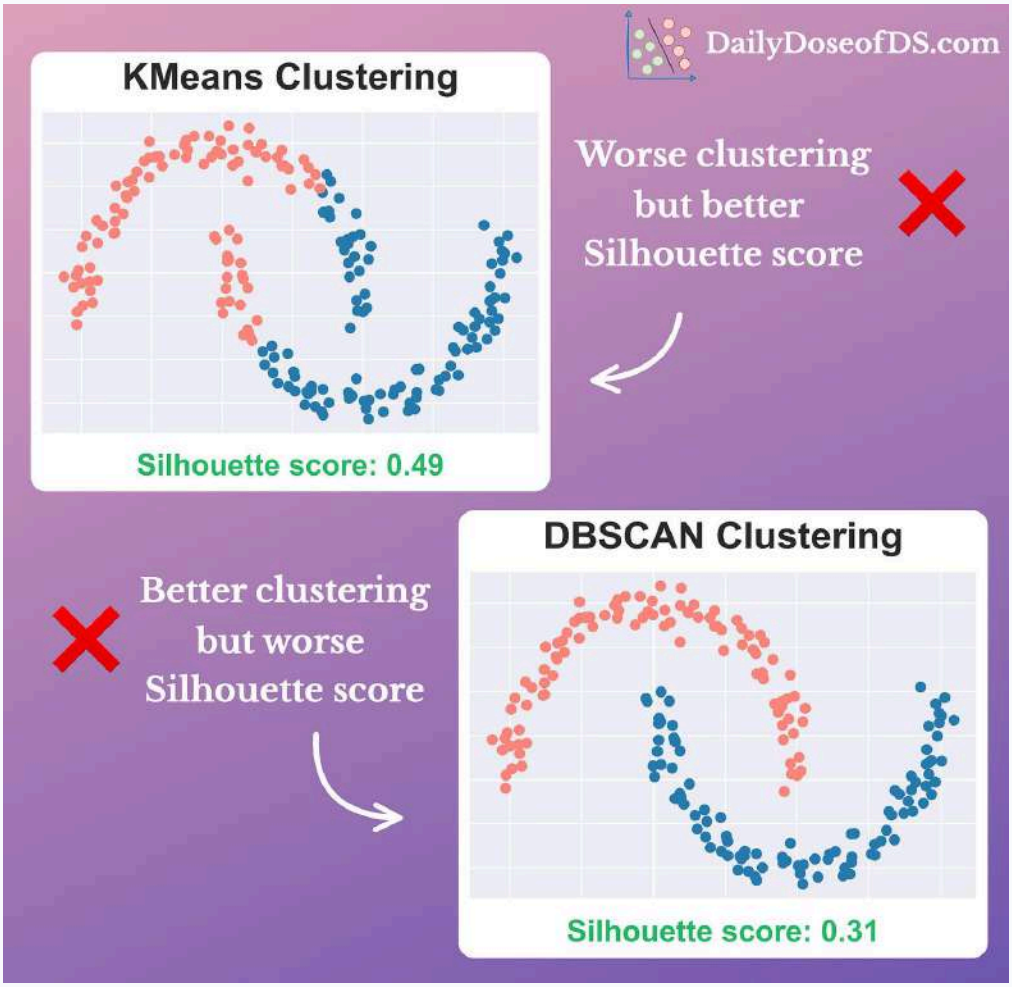
The reason I prefer the Calinski-Harabasz Index over the Silhouette Coefficient is that:

- It is relatively much faster to compute than the Silhouette Coefficient.
- It makes the same intuitive sense for interpretation as the Silhouette Coefficient.

#3) DBCV

The issue with the Silhouette score and Calinski-Harabasz index is that they are typically higher for convex (or somewhat spherical) clusters.

However, using them to evaluate arbitrary-shaped clustering, like those obtained with density-based clustering, can produce misleading results. This is evident from the image below:



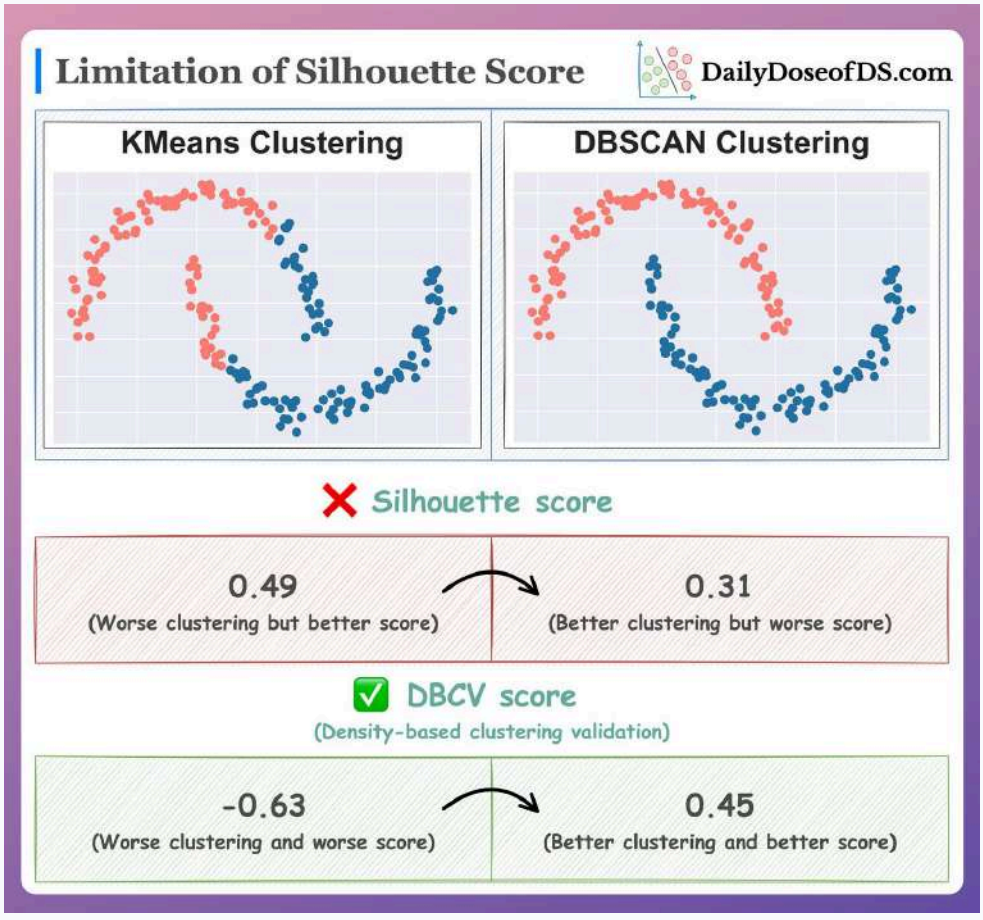
As depicted above, while the clustering output of KMeans is worse, the Silhouette score is still higher than Density-based clustering.

DBCV — density-based clustering validation is a better metric in such cases.

As the name suggests, it is specifically meant to evaluate density-based clustering. Simply put, DBCV computes two values:

- The density within a cluster
- The density overlap between clusters

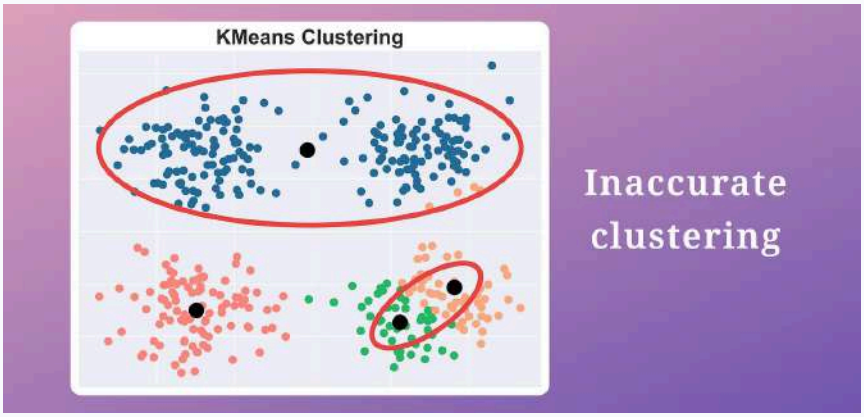
A high density within a cluster and a low density overlap between clusters indicate good clustering results. The effectiveness of DBCV is also evident from the image below:



With DBCV, the score for the clustering output of KMeans is worse, and that of density-based clustering is higher.

Breathing KMeans vs KMeans

KMeans is widely used for its simplicity and effectiveness as a clustering algorithm. Yet, we all know that its performance is entirely dependent on the centroid initialization step. Thus, it is likely that we may obtain inaccurate clusters, as depicted below:

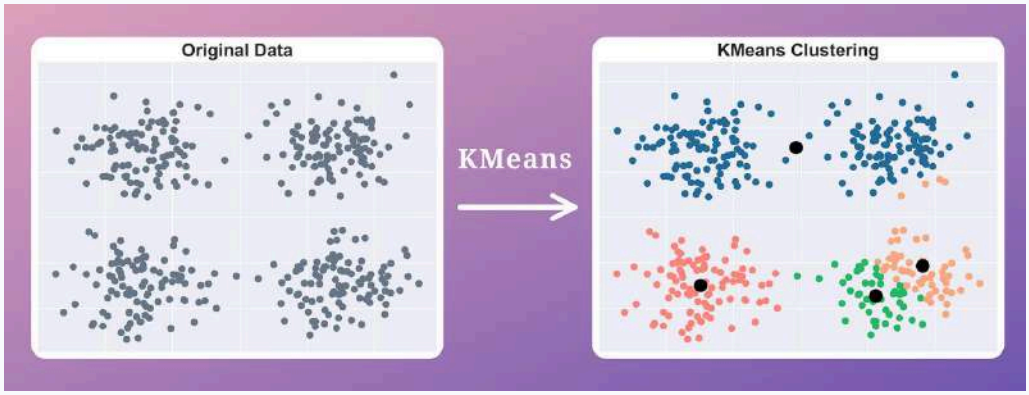


Of course, rerunning with different initialization does help at times. But I have never liked the unnecessary run-time overhead it introduces.

In this chapter, let's learn a supercharged upgrade to KMeans, which addresses this issue while also producing better clustering results. It's called the **Breathing KMeans** algorithm.

Step 1: Run Kmeans

First, we run the usual KMeans clustering only once, i.e., without rerunning the algorithm with a different initialization.



Clustering results of KMeans after running the algorithm without repetition and until convergence

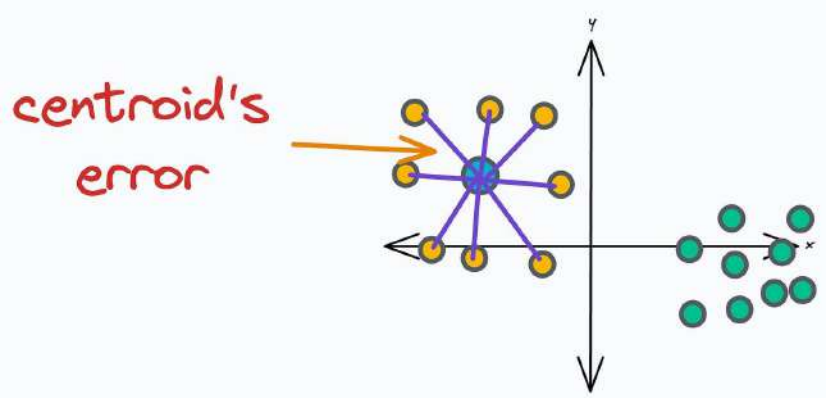
This gives us the location of "k" centroids, which may or may not be accurate.

Step 2: Breathe in step

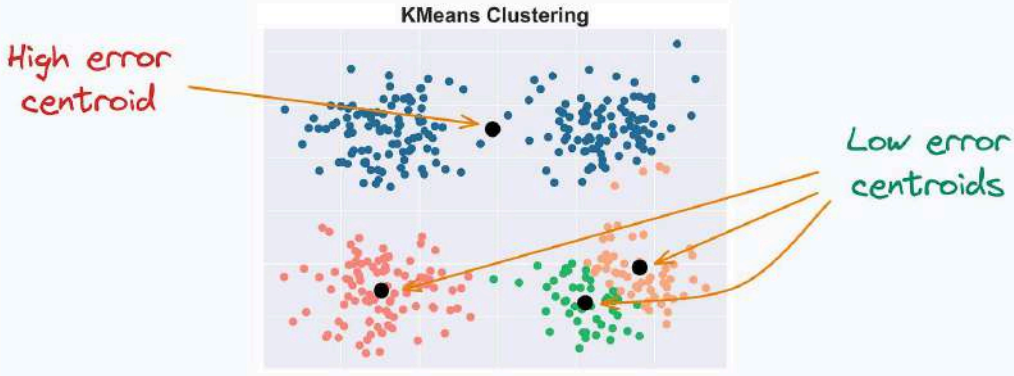
To the "k" centroids obtained from Step 1, we add "m" new centroids.

As per the research paper of Breathing Kmeans, m=5 was found to be good value. Now, you might be thinking, where do we add these "m" centroids?

The addition of new centroids is decided based on the error associated with a centroid. Simply put, a centroid's error is the sum of the squared distance to the points associated with that centroid.



Thus, we add "m" centroids in the vicinity of centroids with high error. Let's understand more intuitively why this makes sense.



In the above KMeans clustering results:

- The centroid at the top has a high error.
- All other centroids have relatively low error.

Intuitively speaking, if a centroid has a very high error, it is possible that multiple clusters are associated with it. Thus, we would want to split this cluster. Adding new centroids near clusters with high error will precisely fulfill this objective.

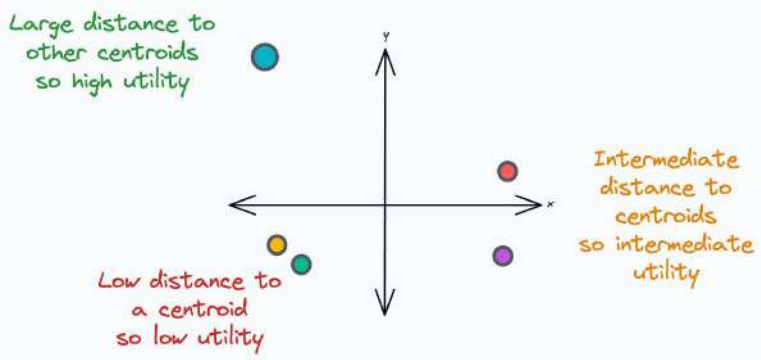
After adding "m" new centroids, we get a total of "k+m" centroids.

Finally, we run KMeans again with "k+m" centroids only once. This gives us the location of "k+m" centroids.

Step 3: Breathe out step

Next, we want to remove "m" centroids from the "k+m" centroids obtained above. Here, you might be thinking, which "m" centroids should we remove? The removal of centroids is decided based on the "utility" of a centroid.

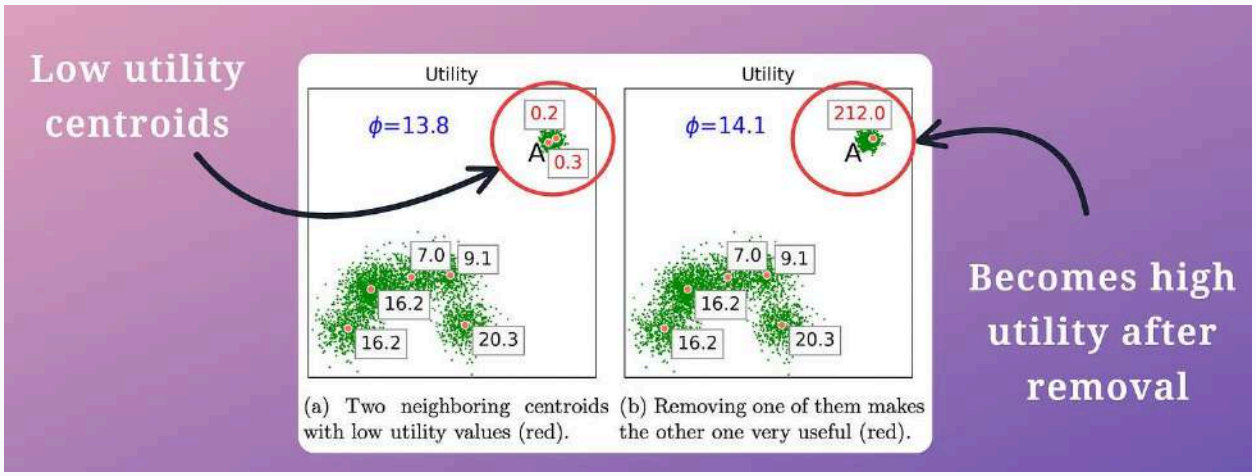
Simply put, a centroid's utility is proportional to its distance from all other centroids. The greater the distance, the more isolated it will be. Hence, the more the utility.



This makes intuitive sense as well. If two centroids are pretty close, they will have low utility.

Thus, they are likely in the same cluster, and we must remove one of them.

This is demonstrated below:



After removing one of the low-utility centroids, the other centroid becomes very useful. So, in practice, after removing one centroid, we update the utility values of all other centroids.

We repeat the process until all "m" low-utility centroids have been removed. This gives back "k" centroids.

Finally, we run KMeans with these "k" centroids only once.

Step 4: Decrease m by 1.

Step 5: Repeat Steps 2 to 4 until m=0.

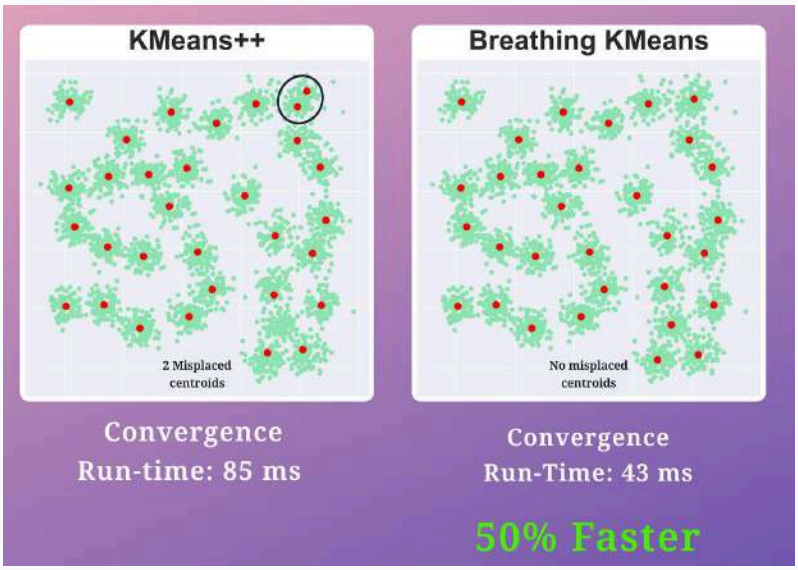
Done!

These repeated breathing cycles (breathe-in and breathe-out steps) almost always provide a faster and better solution than standard KMeans with repetitions.

In each cycle:

- New centroids are added at "good" locations. This helps in splitting clusters occupied by a single centroid.
- Low-utility centroids are removed. This helps in eliminating centroids that are likely in the same cluster.

As a result, it is expected to converge to the optimal solution faster. The effectiveness of Breathing KMeans over KMeans is evident from the image below:



- KMeans produced two misplaced centroids
- Breathing KMeans accurately clustered the data with a 50% run-time improvement.

There is also an open-source implementation of Breathing KMeans, with a sklearn-like API. To get started, install the bkmeans library:

Python
```
pip install bkmeans
```

Next, run the algorithm as follows:

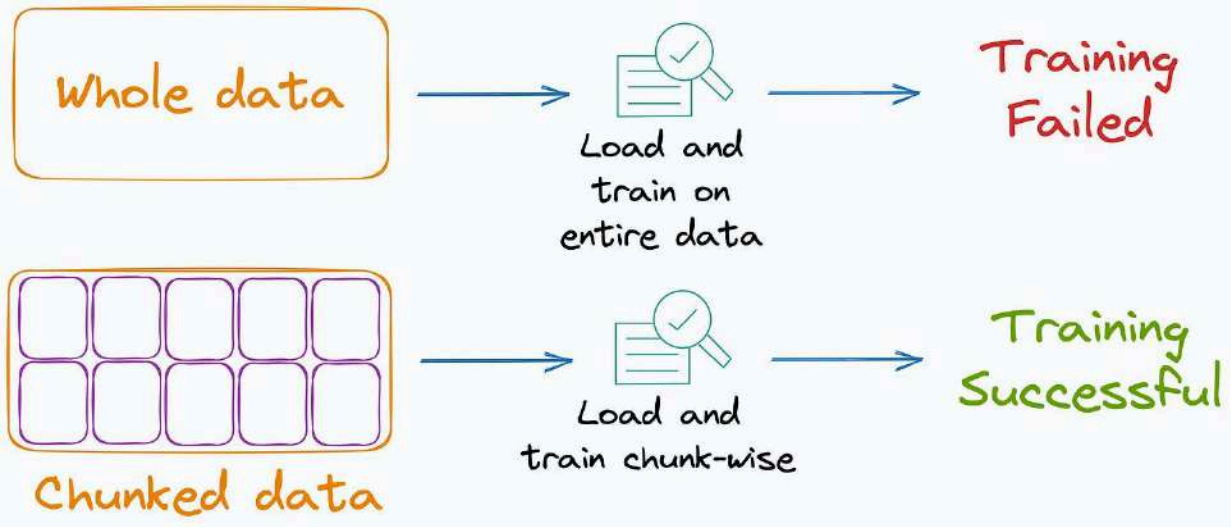
Python
```
from bkmeans import BKMeans

bkm = BKMeans(n_clusters = 100)

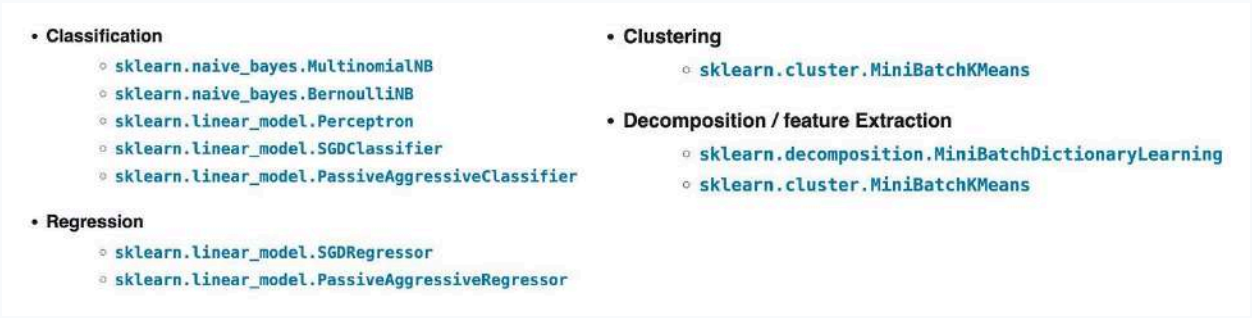
bkm.fit(X)
```

How Does MiniBatchKMeans Works?

Many ML algorithms implemented by Sklearn support incremental learning, i.e., mini-batch-based training. This makes them quite useful when the entire data does not fit into memory.



These are the supported algorithms for your reference:



If you look closely, there's KMeans in that list too. Being a popular clustering, it's good that we can run it on large datasets if needed.

But when I first learned that KMeans can be implemented in a mini-batch fashion, I wondered how exactly a batch implementation of KMeans might work?

Giving it a thought for a minute or so gave me an idea, which I wish to share with you in this chapter.

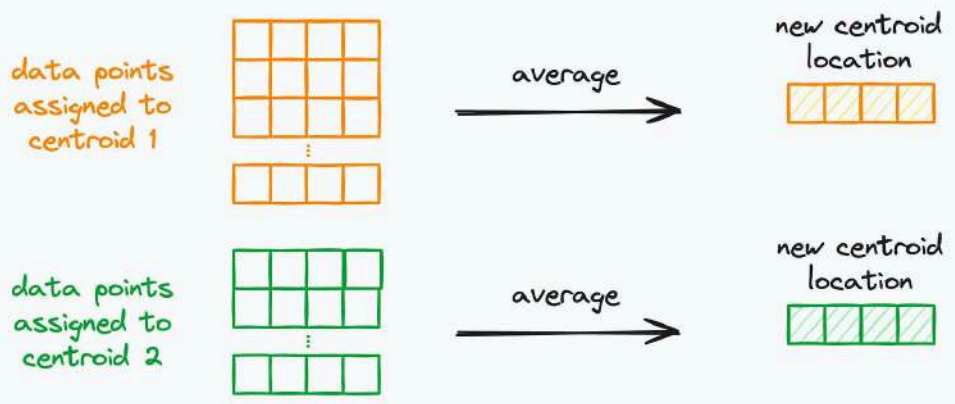
The source of overhead in KMeans

Before understanding the mini-batch implementation, we must know the issue with the usual KMeans implementation.

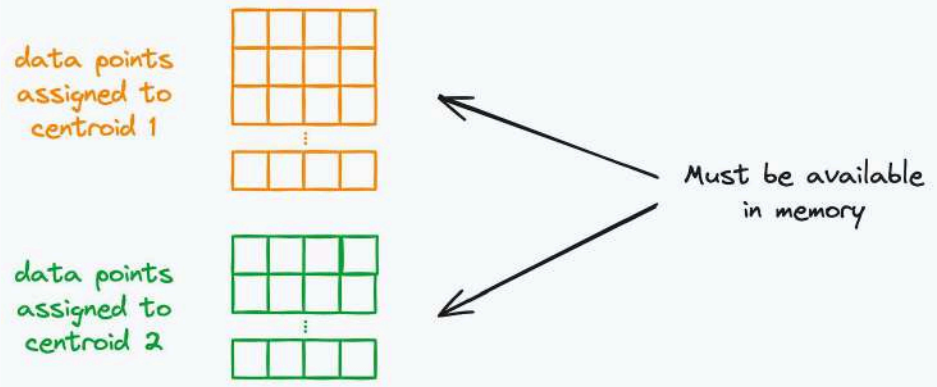
To recall, KMeans works as follows:

- Step 1) Initialize centroids.
- Step 2) Find the nearest centroid for each point.
- Step 3) Reassign centroids as the average of points assigned to them.
- Step 4) Repeat until convergence.

In this implementation, the primary memory bottleneck originates from Step 3. This is because, in this step, we compute the average of all points assigned to a centroid to shift the centroid.



Thus, traditionally, all data points assigned to a centroid must be available in memory to take an average over.



But in our case, all data points DO NOT fit into memory at once. Thus, direct averaging isn't feasible. Nonetheless, here's how we can smartly address this issue.

Before I proceed further:

Honestly speaking, I haven't checked the precise implementation of MiniBatchKMeans in Sklearn. The docs don't tell much either.

The solution I am about to share is the one I came up with when I was trying to figure out how a mini-batch KMeans can be possibly implemented.

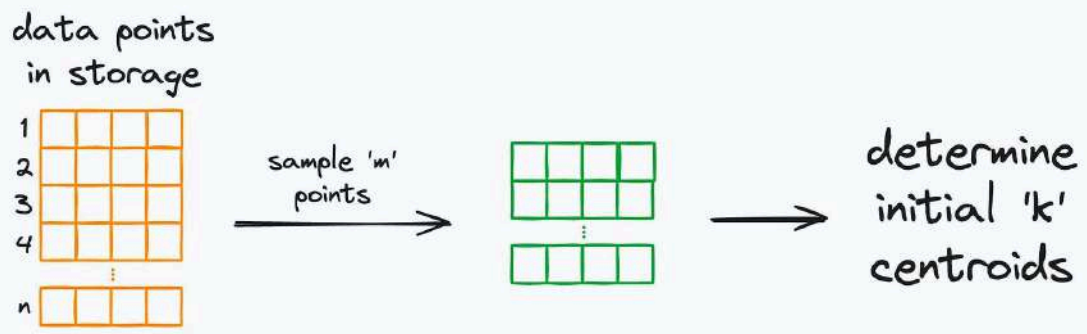
So if there's a perfect overlap with Sklearn, great!

If not, it's fine because the solution I am about to share is perfectly viable too.

MiniBatchKMeans

We begin the algorithm by selecting a value of k — the number of centroids.

Next, we need an initial location for the centroids so we can sample m (m << n) random data points to initialize the centroids.



Now, recall the bottleneck we discussed above — All data points must be available in memory to average over.

Thus, to address this, we need a way to keep track of the data points assigned to each centroid as we load and process each mini-batch.

We cannot maintain a growing list of data points assigned to each centroid because that would eventually take up the memory of the entire dataset.

So here's how we can smartly fulfill this objective.

We can utilize the following mathematical property that relates sum and average:

$$\text{average} = \frac{\text{sum}}{\text{n}}$$

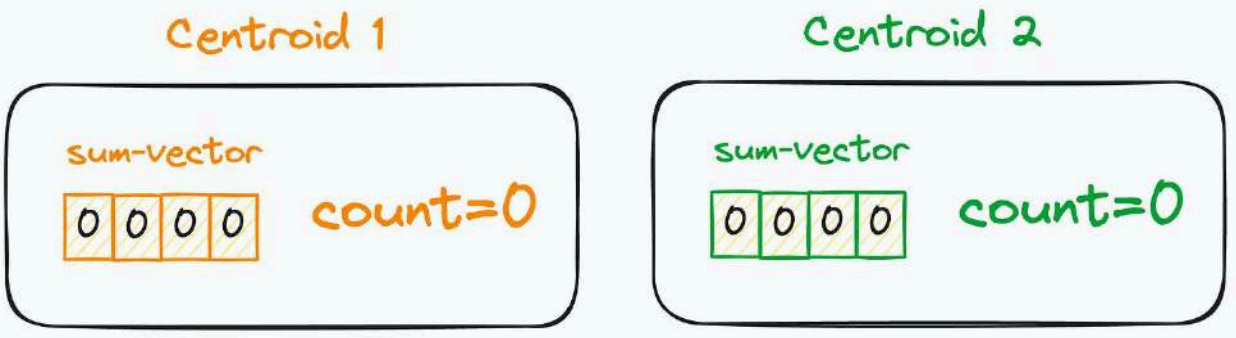
How?

See, in a typical KMeans setting, we average over all data points assigned to a centroid to determine its new location. But as discussed above, we can't keep all data points in memory at once.

So how about we do this in the mini-batch setting of KMeans:

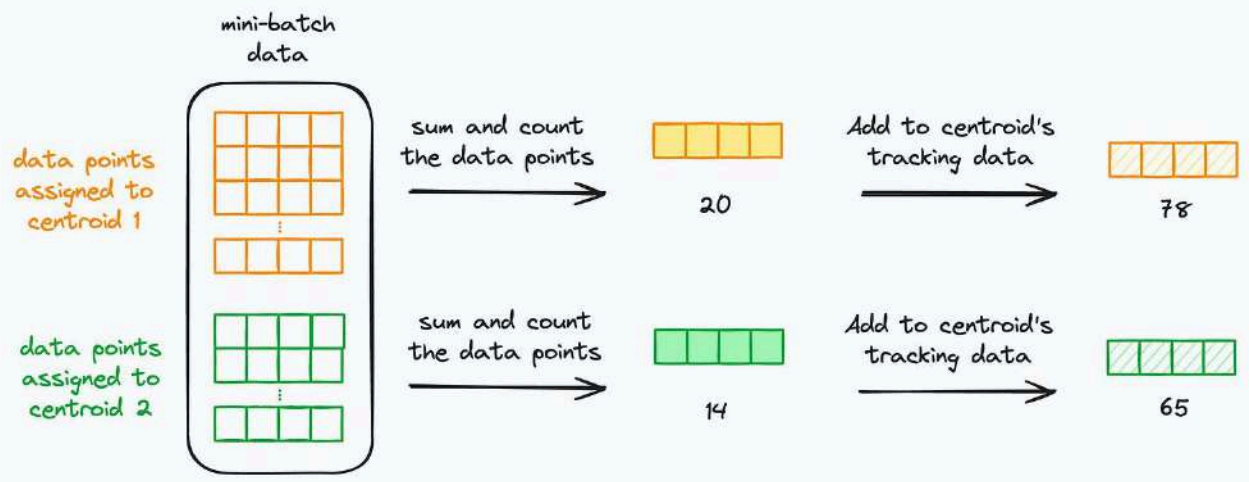
For every centroid:

- We maintain a "sum-vector", which stores the sum of the vectors of all data points assigned to that centroid. To begin, this will be a zero vector.
- A "count" variable, which tracks the number of data points assigned to a centroid. To begin, the count will be zero.

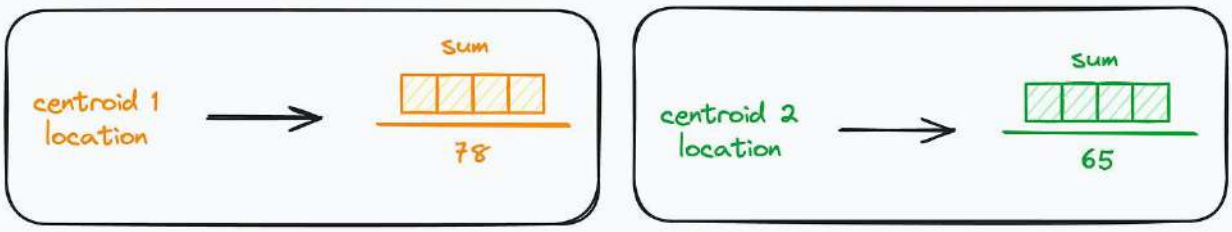


While iterating through mini-batches:

- Every "sum-vector" of a centroid will continue to accumulate the sum of all data points that have been assigned to it so far.
- Every "count" variable of a centroid will continue to track the number of data points that have been assigned to it so far.



After iterating through all mini-batches, we can perform a scalar division between sum-vector and count to get the average vector:



This average will precisely tell us the new location of the centroid as if all data points were present in memory. We can repeat this until convergence.

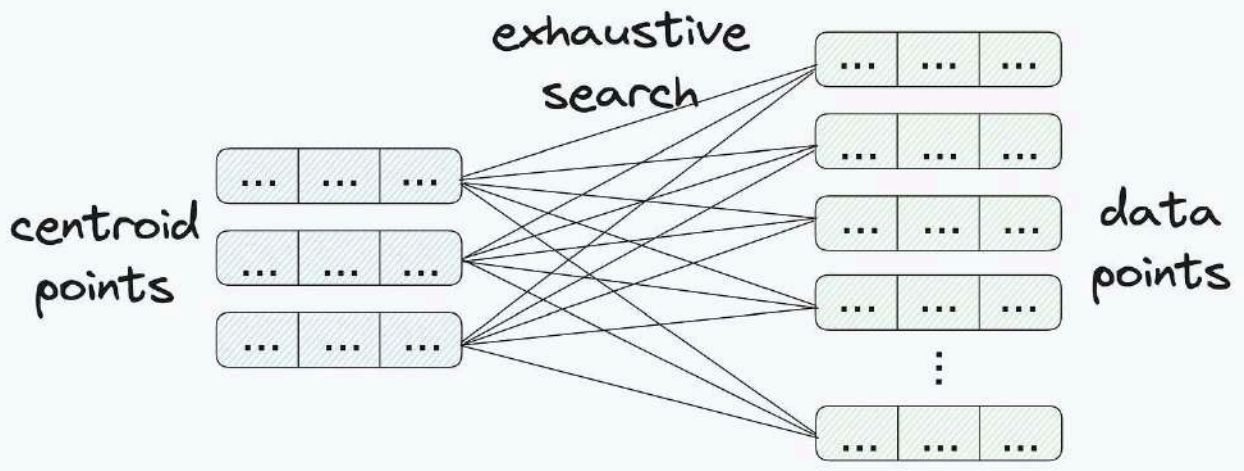
Also, before starting the next iteration of MiniBatchKMeans, we must reset the sum-vector to a zero vector and count to zero. And that's how a mini-batch version of KMeans can be implemented.

ANN-driven KMeans with Faiss

As discussed in the previous chapter, KMeans is trained as follows:

- Step 1) Initialize centroids
- Step 2) Find the nearest centroid for each point
- Step 3) Reassign centroids
- Step 4) Repeat until convergence

But in this implementation, "Step 2" has a run-time bottleneck, as this step involves a brute-force and exhaustive search. In other words, this finds the distance of every data point from every centroid.



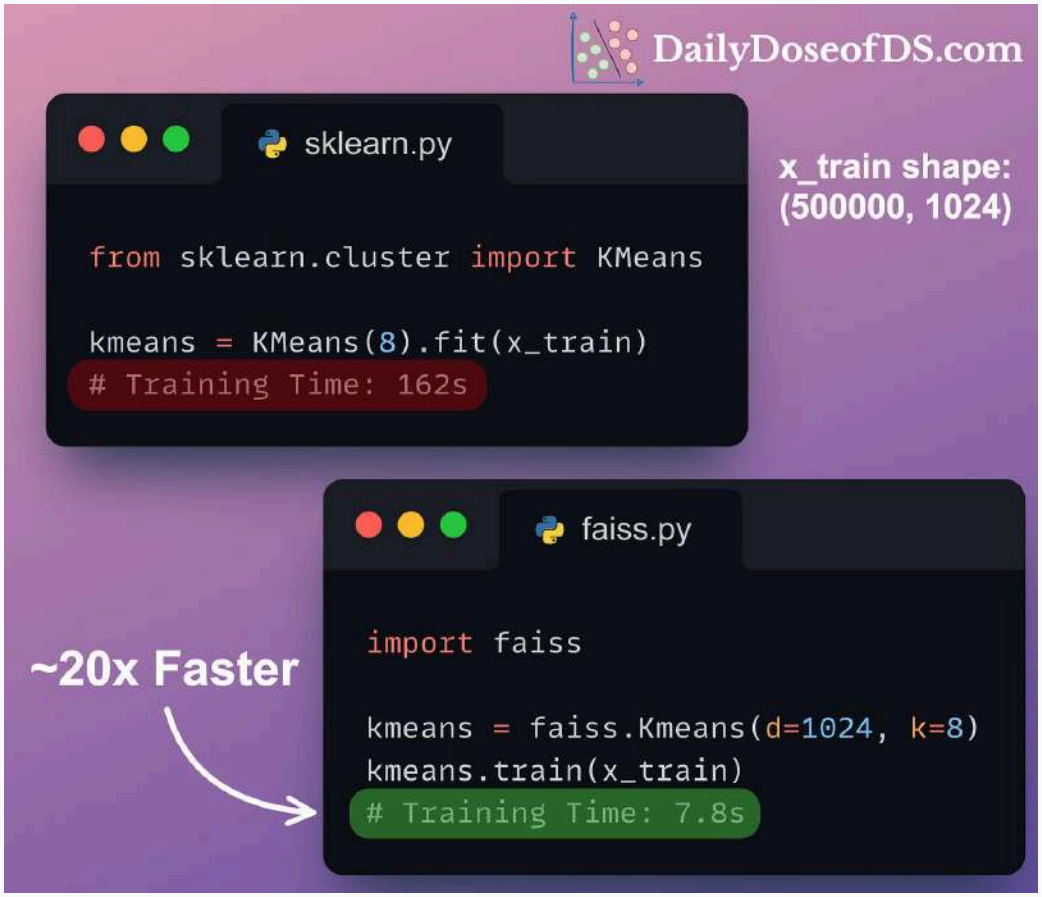
As a result, this step isn't optimized, and it takes plenty of time to train and predict. This is especially challenging with large datasets.

To speed up KMeans, one of the implementations I usually prefer, especially on large datasets, is Faiss by Facebook AI Research.

To elaborate further, Faiss provides a much faster nearest-neighbor search using approximate nearest-neighbor search algorithms.

It uses an "Inverted Index," which is an optimized data structure to store and index the data point.

This makes performing clustering extremely efficient, especially on large datasets, which is also evident from the image below:



As shown above, on a dataset of 500k data points (1024 dimensions), Faiss is roughly 20x faster than KMeans from Sklearn, which is an insane speedup.

What's more, Faiss can also run on a GPU, which can further speed up your clustering run-time performance.

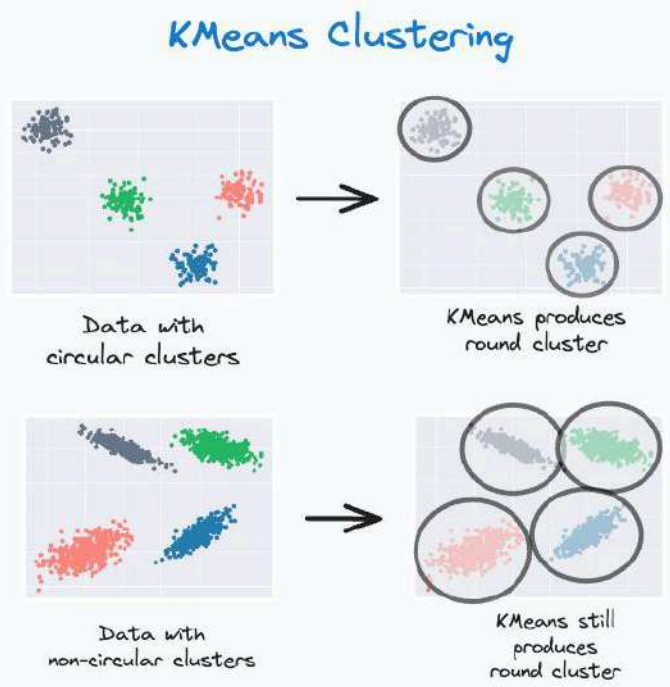
Get started with Faiss here: https://github.com/facebookresearch/faiss.

KMeans vs. Gaussian Mixture Models

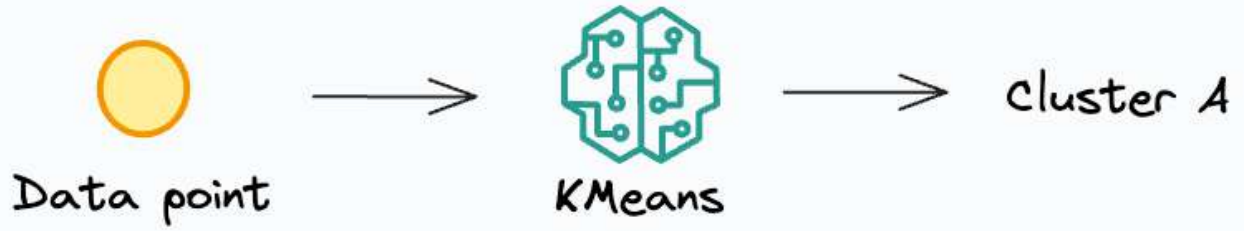
I like to think of Gaussian Mixture Models as a more generalized version of KMeans, which addresses some of the widely known limitations of KMeans.

Limitations of KMeans

To begin, KMeans can only produce globular clusters. For instance, as shown below, even if the data has non-circular clusters, it still produces round clusters.

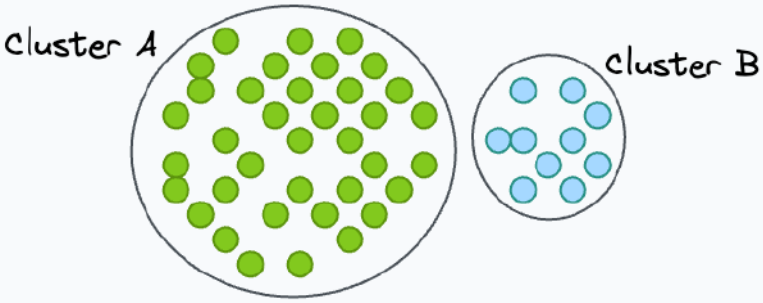


Moreover, it performs a hard assignment. There are no probabilistic estimates of each data point belonging to each cluster.

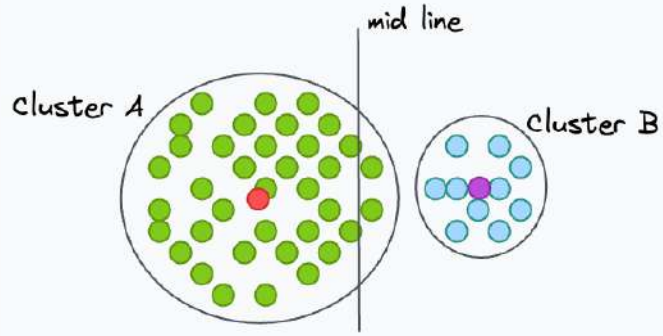


Lastly, it only relies on distance-based measures to assign data points to clusters.

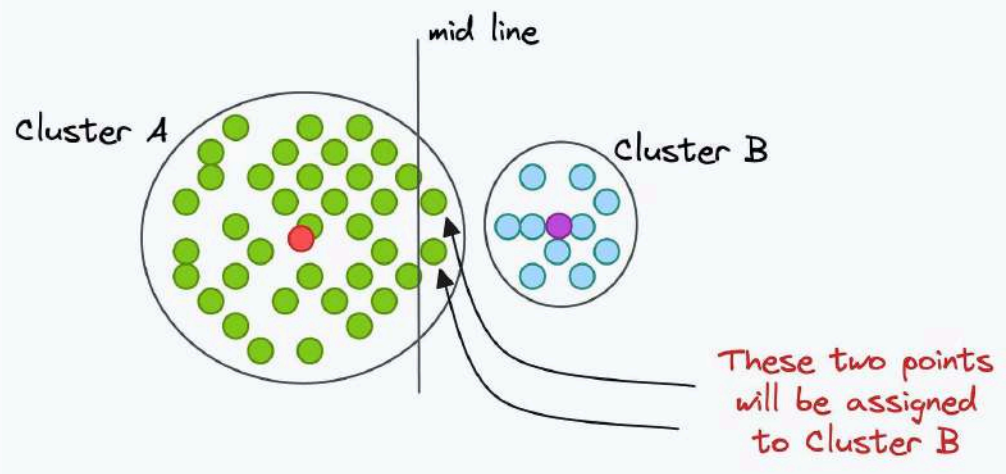
To understand this point better, consider two clusters in 2D — A and B. Cluster A has a higher spread than B.



Now consider a line that is mid-way between centroids of A and B.



Although A has a higher spread, even if a point is slightly right to the midline, it will get assigned to cluster B.

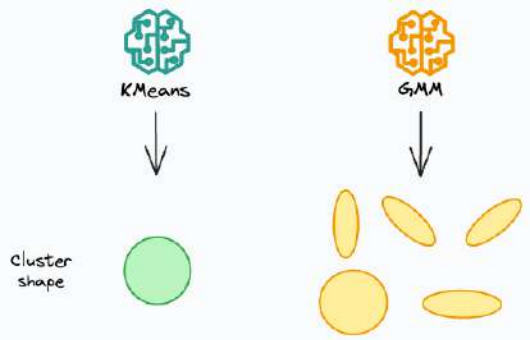


Ideally, however, cluster A should have had a larger area of influence.

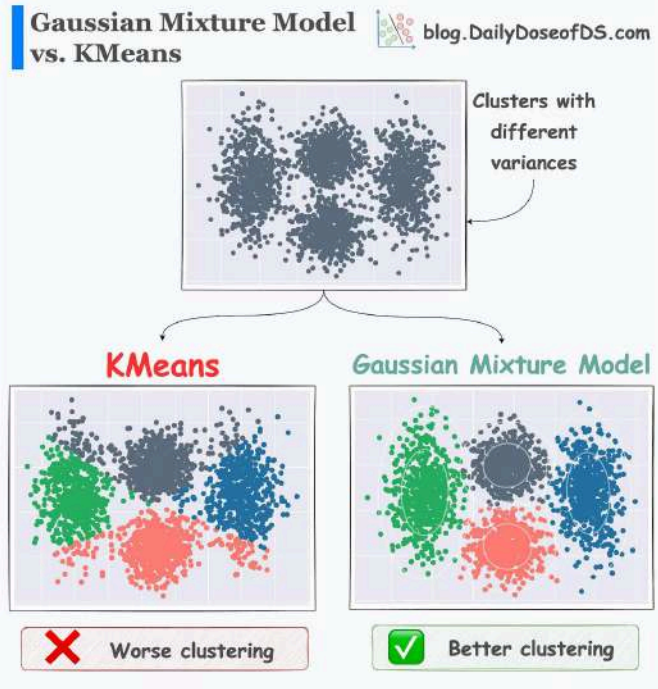
Gaussian Mixture Models

These limitations often make KMeans a non-ideal choice for clustering. Gaussian Mixture Models are often a superior algorithm in this respect. As the name suggests, they can cluster a dataset that has a mixture of many Gaussian distributions. They can be thought of as a more flexible twin of KMeans.

The primary difference is that, KMeans learns centroids but Gaussian mixture models learn a distribution. For instance, in 2 dimensions, KMeans can only create circular clusters but GMM can create oval-shaped clusters.



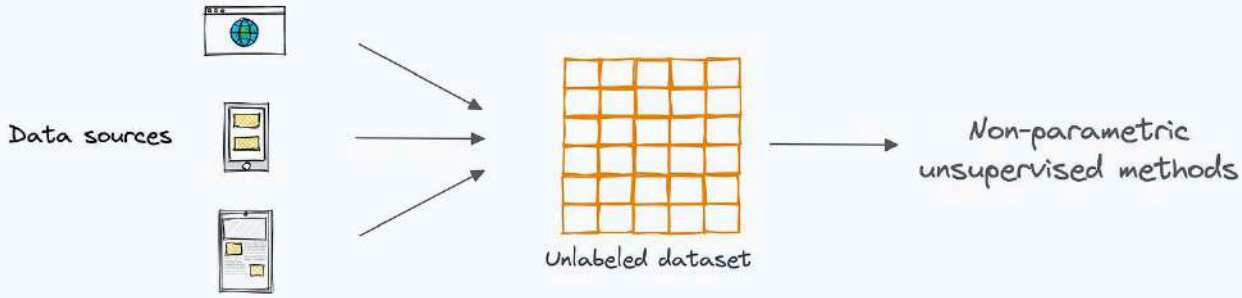
The effectiveness of GMMs over KMeans is evident from the image below.



- KMeans just relies on distance and ignores the distribution of each cluster.
- GMM learns the distribution and produces better clustering.

DBSCAN++: A Faster and Scalable Alternative to DBSCAN

Non-parametric unsupervised algorithms are widely used in industry to analyze large datasets. This is because in many practical applications, gathering true labels is pretty difficult or infeasible.



In such cases:

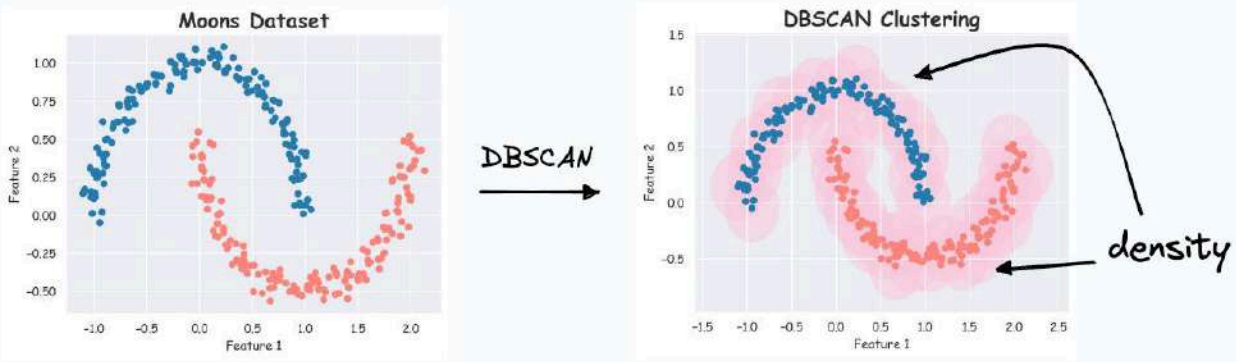
- Either data teams annotate the data, which can be practically impossible at times,
- Or use unsupervised methods to identify patterns.

While KMeans is widely used here due to its simplicity and effectiveness as a clustering algorithm, it has many limitations as we also discussed in the previous chapter:

- It does not account for cluster covariance.
- It can only produce spherical clusters. As shown below, even if the data has non-circular clusters, it still produces round clusters.

Density-based algorithms, like DBSCAN, quite effectively address these limitations.

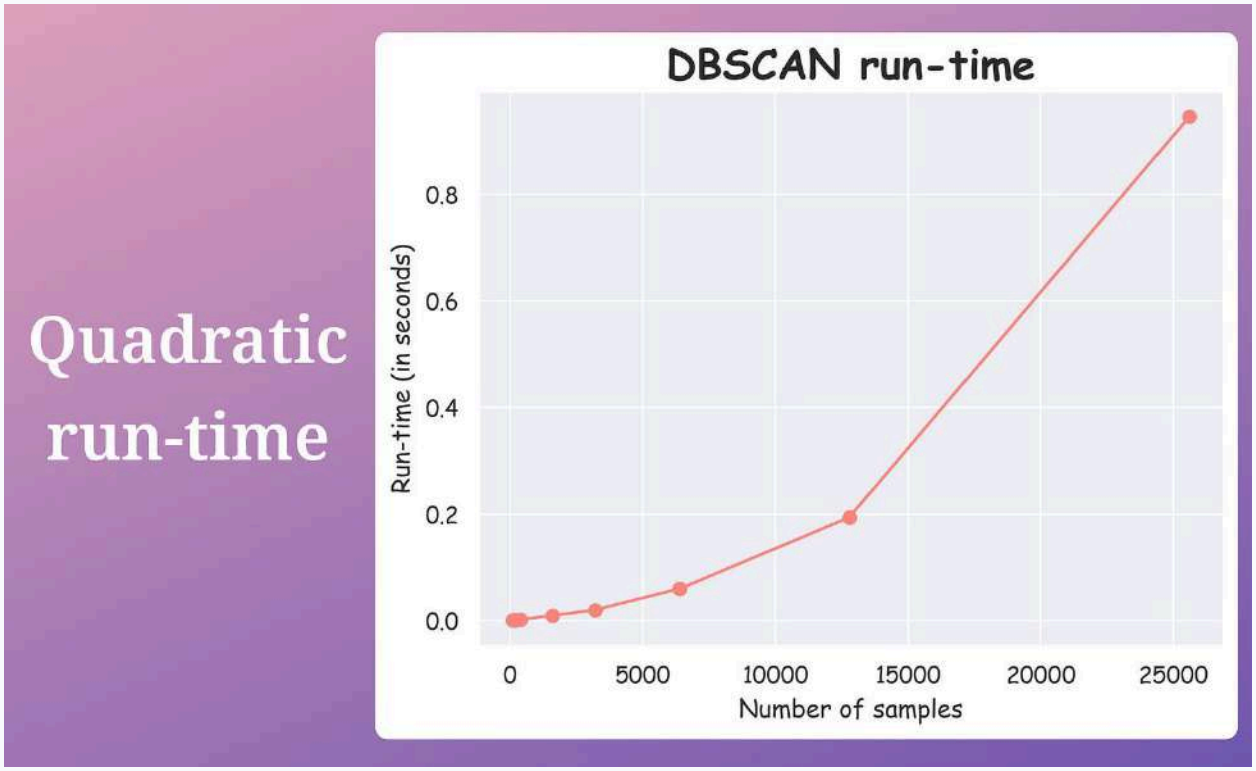
The core idea behind DBSCAN is to group together data points based on "density", i.e., points that are close to each other in a high-density region and are separated by lower-density regions.



But unfortunately, with the use of DBSCAN, we again run into an overlooked problem. One of the things that makes DBSCAN infeasible to use at times is its run-time.

Until recently, it was believed that DBSCAN had a run-time of O(nlogn), but it was proven to be $O(n^2)$ in the worst case.

In fact, this can also be verified from the figure below. It depicts the run-time of DBSCAN with the number of samples:

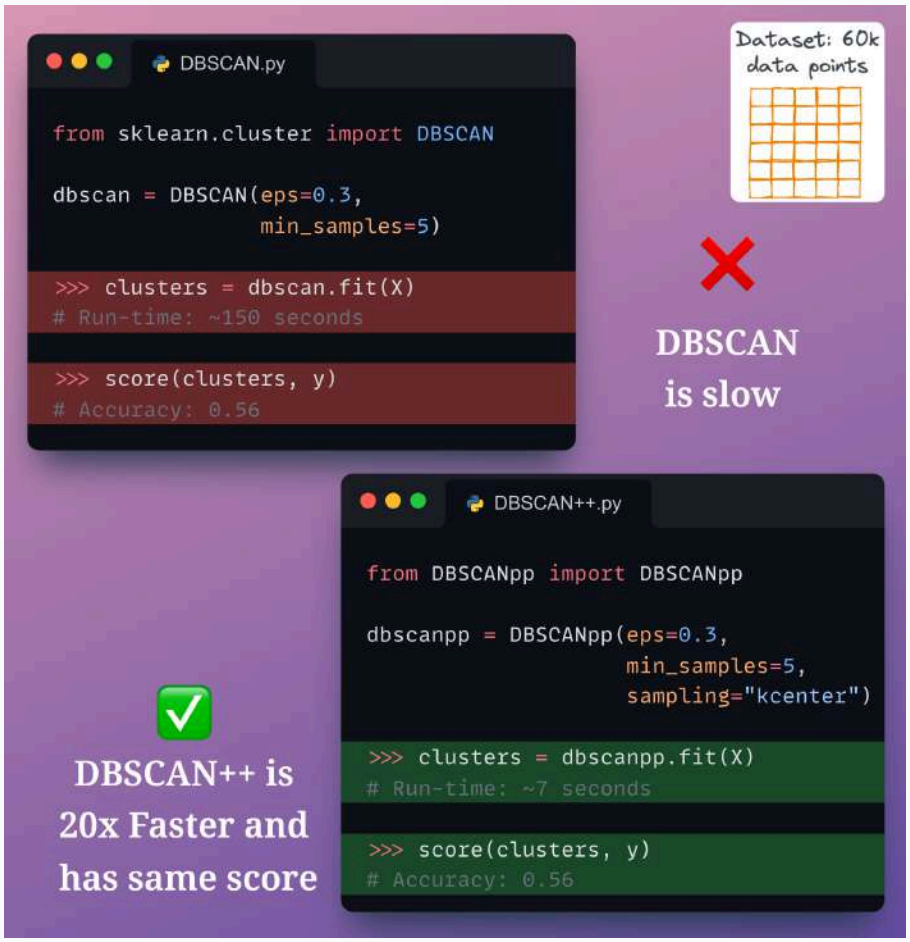


It is clear that DBSCAN has a quadratic relation with the dataset's size.

Thus, there is an increasing need to establish more efficient versions of DBSCAN.

DBSCAN++ is a major step towards a fast and scalable DBSCAN.

Simply put, DBSCAN++ is based on the observation that we only need to compute the density estimates for a subset "m" of the "n" data points in the given dataset, where "m" can be much smaller than "n" to cluster properly. The effectiveness of DBSCAN++ is evident from this image.



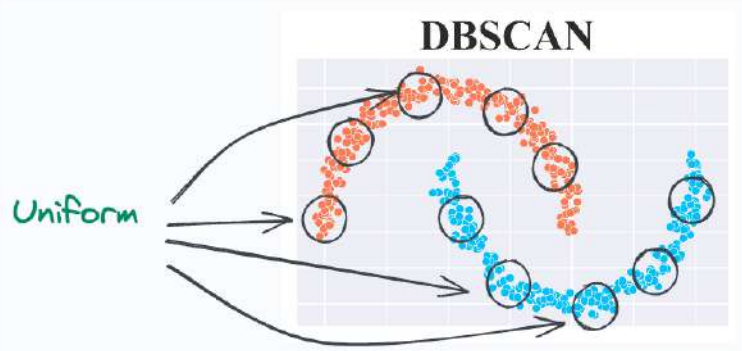
As depicted above, on a dataset of 60k data points:

- DBSCAN++ is 20x faster than DBSCAN.
- DBSCAN++ produces better clustering scores than DBSCAN.

HDBSCAN vs. DBSCAN

In this chapter, let's dive into HDBSCAN and understand how it differs from DBSCAN.

Like any other algorithm, DBSCAN also has some limitations. To begin, DBSCAN assumes that the local density of data points is (somewhat) globally uniform. This is governed by its eps parameter.

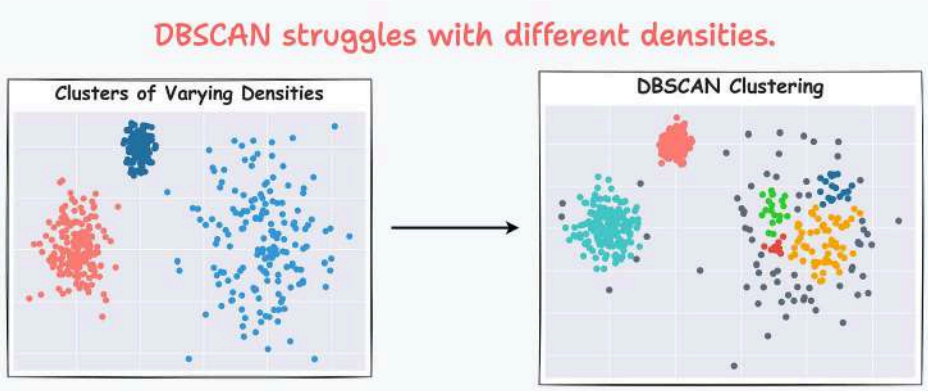


Thus, it may struggle to identify clusters with varying densities. This may need several hyperparameter tuning attempts to get promising results.

HDBSCAN can be a better choice for density-based clustering. It relaxes the assumption of local uniform density, which makes it more robust to clusters of varying densities by exploring many different density scales.

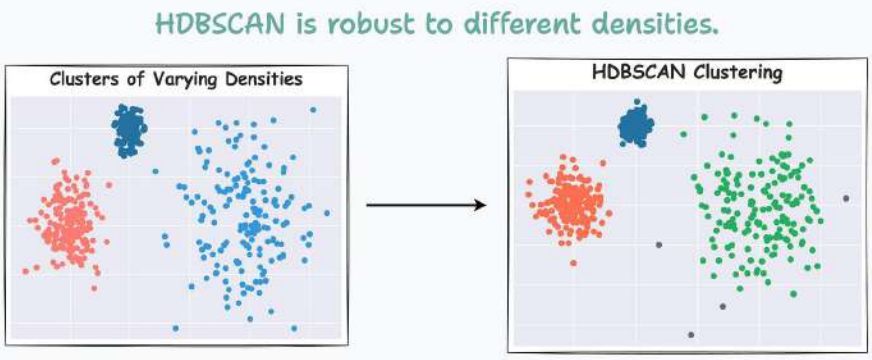
For instance, consider the clustering results obtained with DBSCAN on the dummy dataset below, where each cluster has different densities:

It is clear that DBSCAN produces bad clustering results.



Now compare it with HDBSCAN results depicted below:

On a dataset with three clusters, each with varying densities, HDBSCAN is found to be more robust.

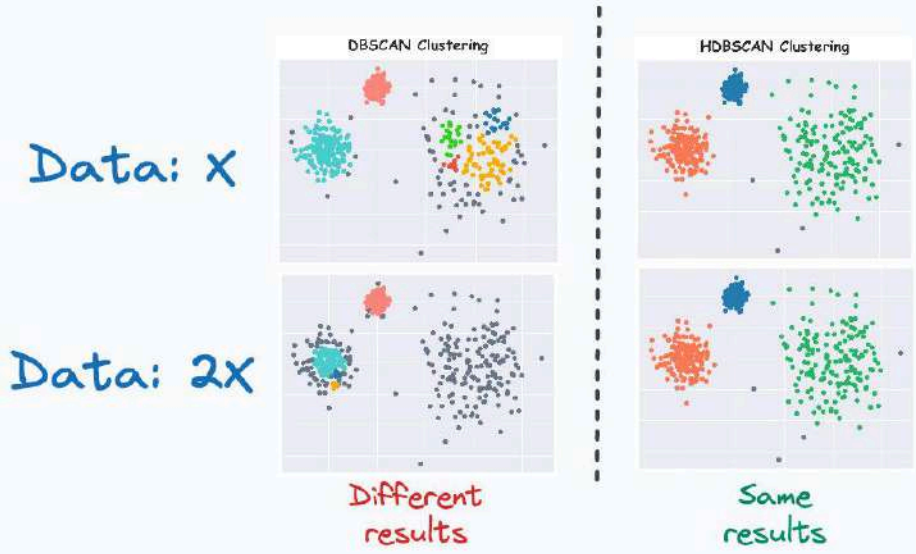


There's one more thing I love about HDBSCAN:

- DBSCAN is a scale variant algorithm. Thus, clustering results for data X, 2X, 3X, etc., can be entirely different.
- On the other hand, HDBSCAN is scale-invariant. So, clustering results remain the same across different scales of data.

We have DBSCAN on the left, and we can see that the results vary with the scale of the data.

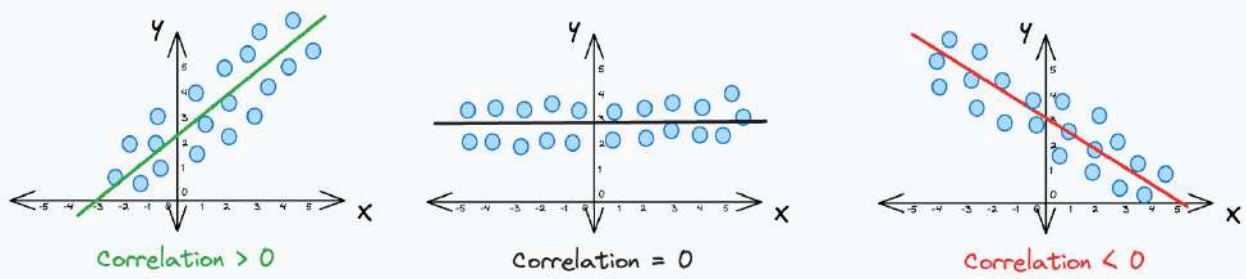
However, clustering from HDBSCAN (on the right) remains unaltered with the scale for HDBSCAN.



Correlation analysis

Correlation != Predictiveness

Correlation measures how two features vary with one another linearly (or monotonically).

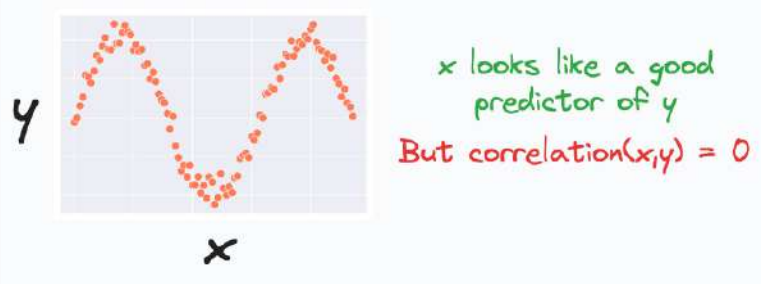


This makes correlation symmetric: corr(A, B) = corr(B, A).

Yet, associations are often asymmetric. For instance, given a date, it is easy to tell the month. But given a month, you can never tell the date.



Correlation, being symmetric, entirely ignores this notion. What's more, it is not meant to quantify how well a feature can predict the outcome, as demonstrated below:

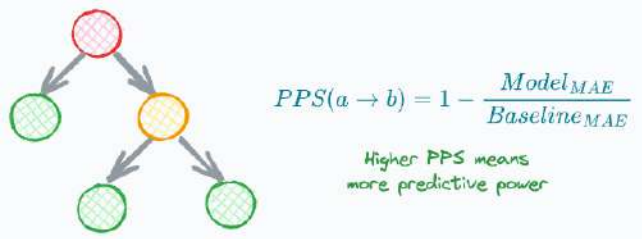


Yet, at times, it is misinterpreted as a measure of "predictiveness". Lastly, correlation is mostly limited to numerical data. But categorical data is equally important for predictive models.

The Predictive Power Score (PPS) addresses each of these limitations. As the name suggests, it measures the predictive power of a feature.

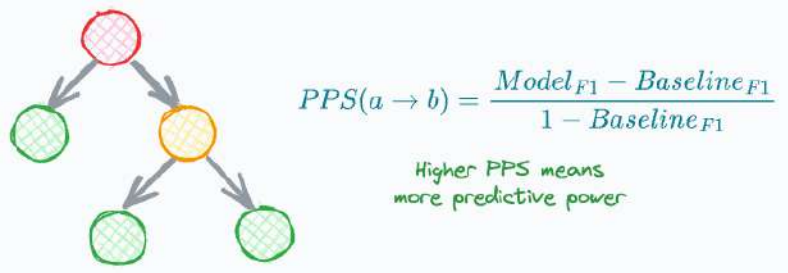
PPS(a → b) is calculated as follows:

- If the target (b) is numeric:
    - Train a Decision Tree Regressor that predicts b using a.



    - Find PPS by comparing its MAE to the MAE of a baseline model (median prediction).

- If the target (b) is categorical:
    - Train a Decision Tree Classifier that predicts b using a.



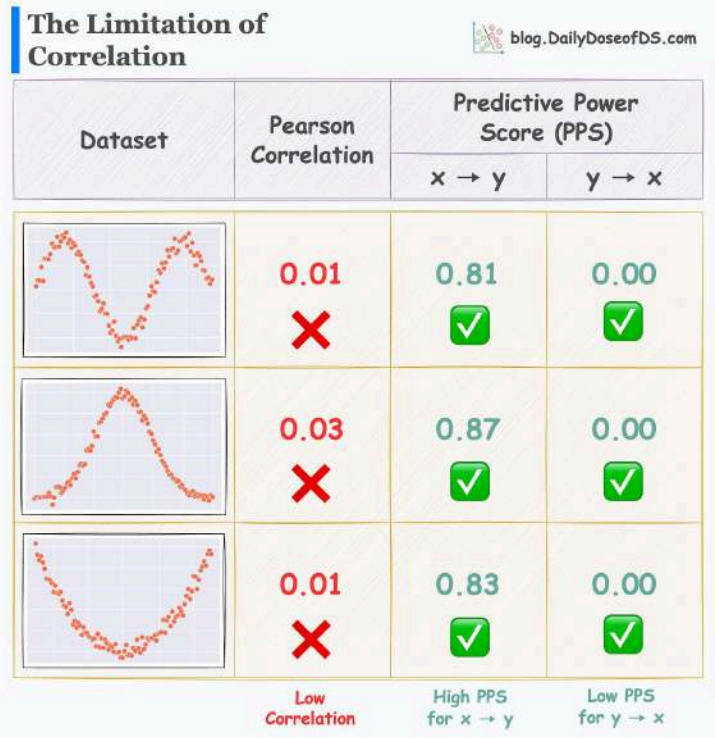
    - Find PPS by comparing its F1 to the F1 of a baseline model (random or most frequent prediction).

Thus, PPS:

- is asymmetric, meaning PPS(a, b) != PPS(b, a).
- can be used on categorical targets (b).

- can be used to measure the predictive power of categorical features (a).
- works well for linear and non-linear relationships.
- works well for monotonic and non-monotonic relationships.

Its effectiveness is evident from the image below. For all three datasets:



- Correlation is low.
- PPS (x → y) is high.
- PPS (y → x) is zero.

That being said, it is important to note that correlation has its place.

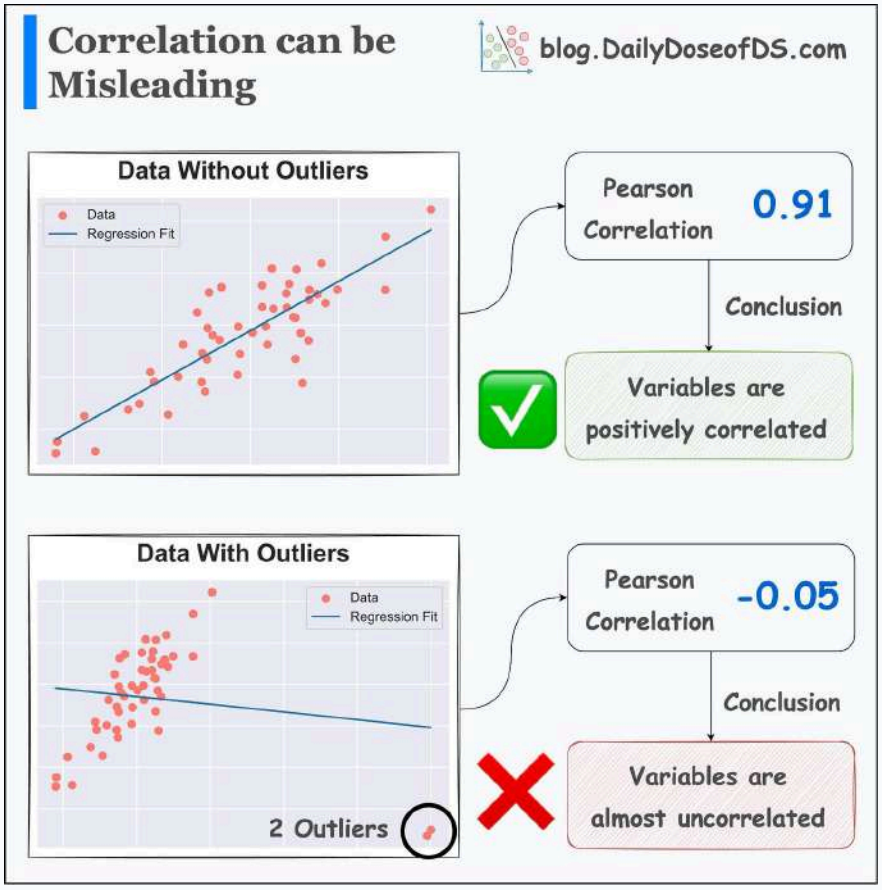
When selecting between PPS and correlation, first set a clear objective about what you wish to learn about the data:

- Do you want to know the general monotonic trend between two variables? Correlation will help.
- Do you want to know the predictiveness of a feature? PPS will help.

Get started with PPS: https://github.com/8080labs/ppscore.

Beware of Summary Statistics

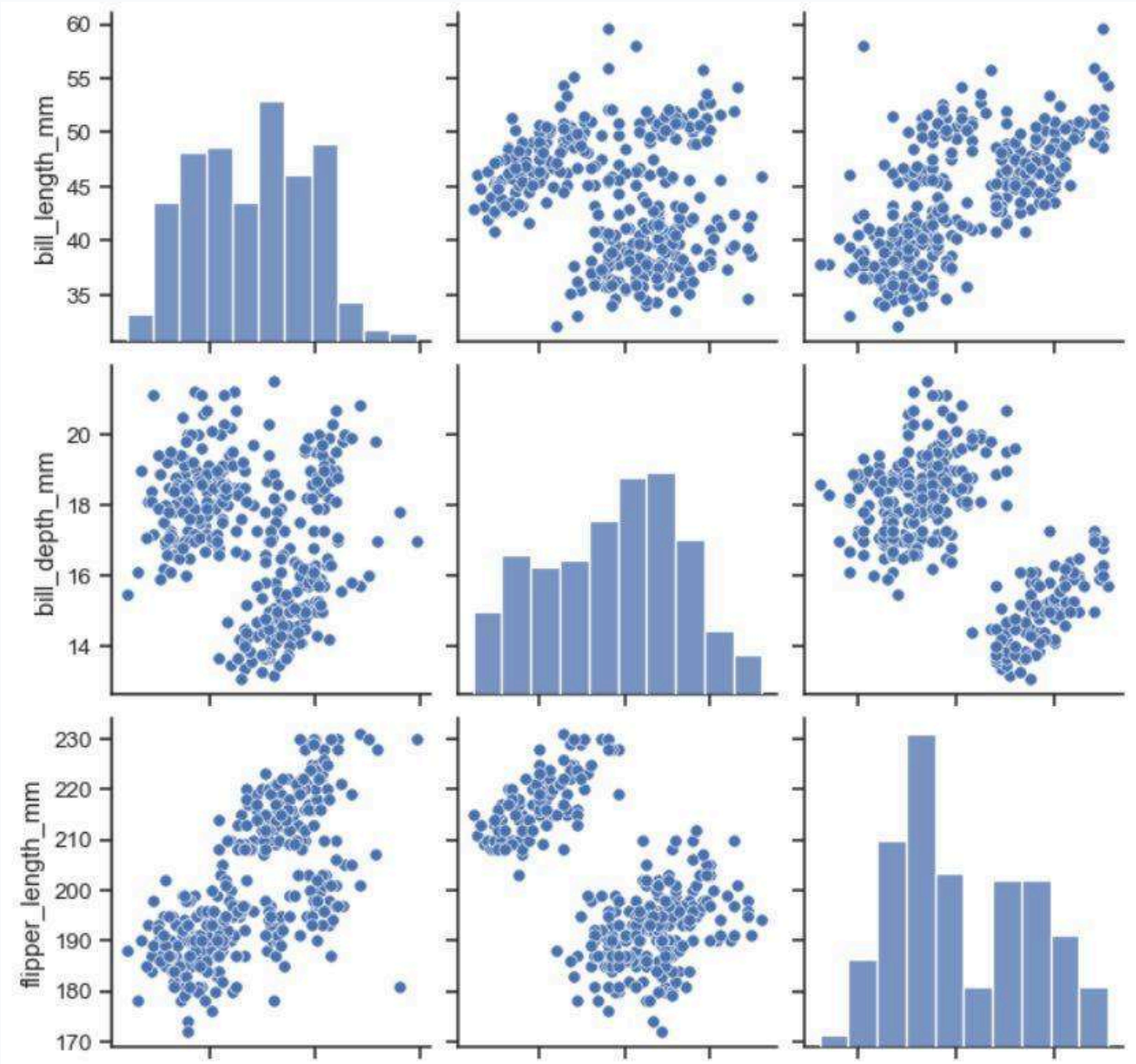
Many data scientists solely rely on the correlation matrix to study the association between variables. But unknown to them, the obtained statistic can be heavily driven by outliers. This is evident from the image below.



Adding just two outliers drastically changed the correlation coefficient and the regression fit. Thus, plotting the data is highly important.

This can save you from drawing wrong conclusions, which you may have drawn otherwise by solely looking at the summary statistics.

One thing that I often do when using a correlation matrix is creating a PairPlot as well (shown below).



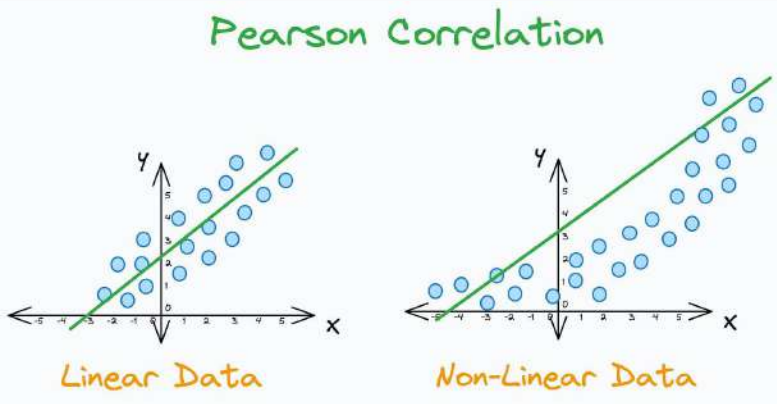
This lets me infer if the scatter plot of two variables and their corresponding correlation measure resonate with each other or not. We shall continue this discussion in the next chapter.

Pearson correlation can only measure linear association

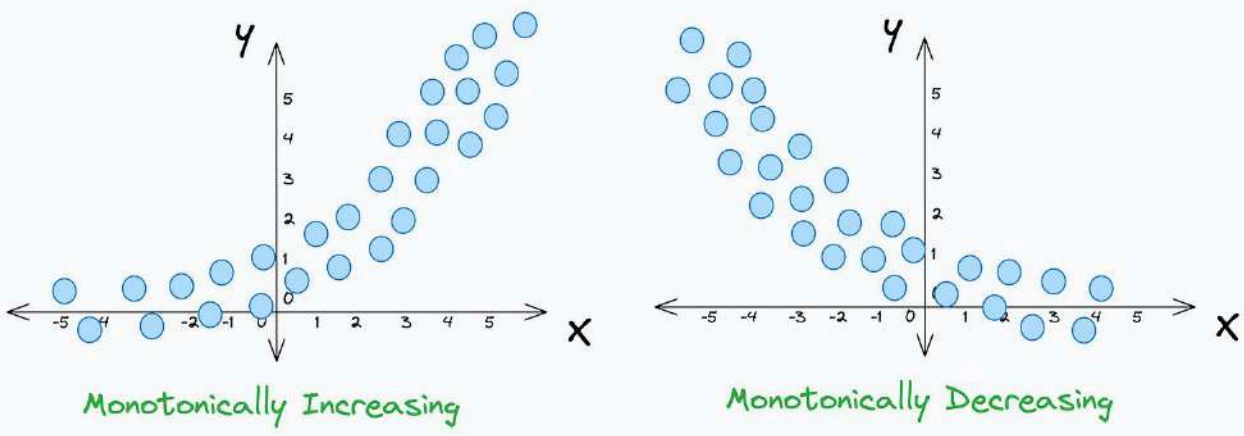
Pearson correlation is commonly used to determine the association between two continuous variables. Many frameworks (like Pandas, for instance) have it as their default correlation metric.

Yet, unknown to many, Pearson correlation:

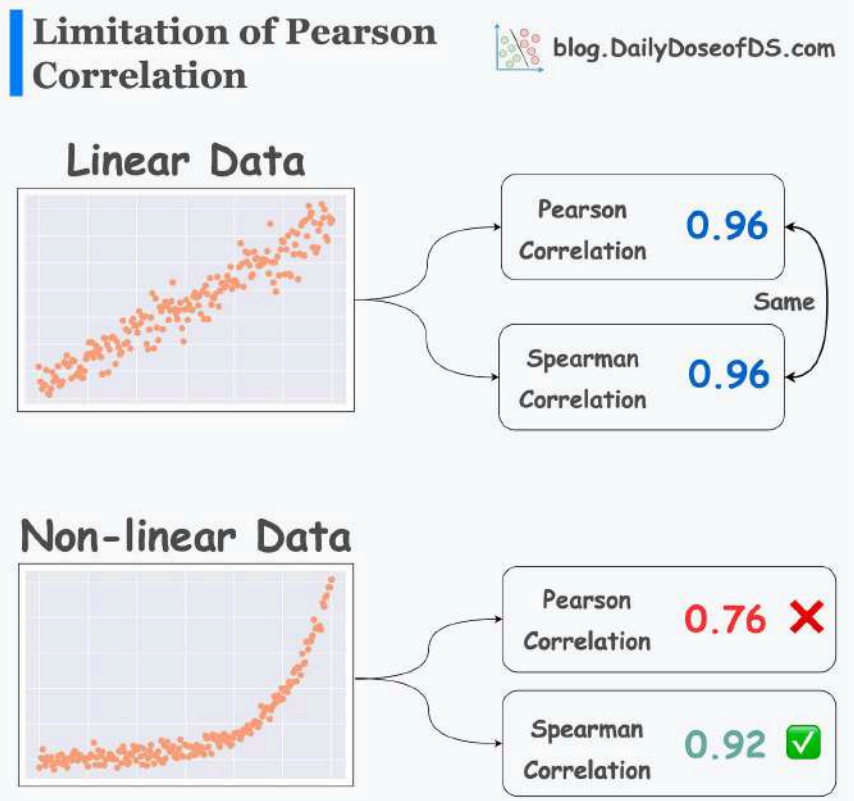
- Only measures the linear relationship.
- Penalizes a non-linear yet monotonic association.



Instead, Spearman correlation is a better alternative. It assesses monotonicity, which can be linear as well as non-linear.



This is evident from the illustration below:



Pearson vs. Spearman on linear and non-linear data

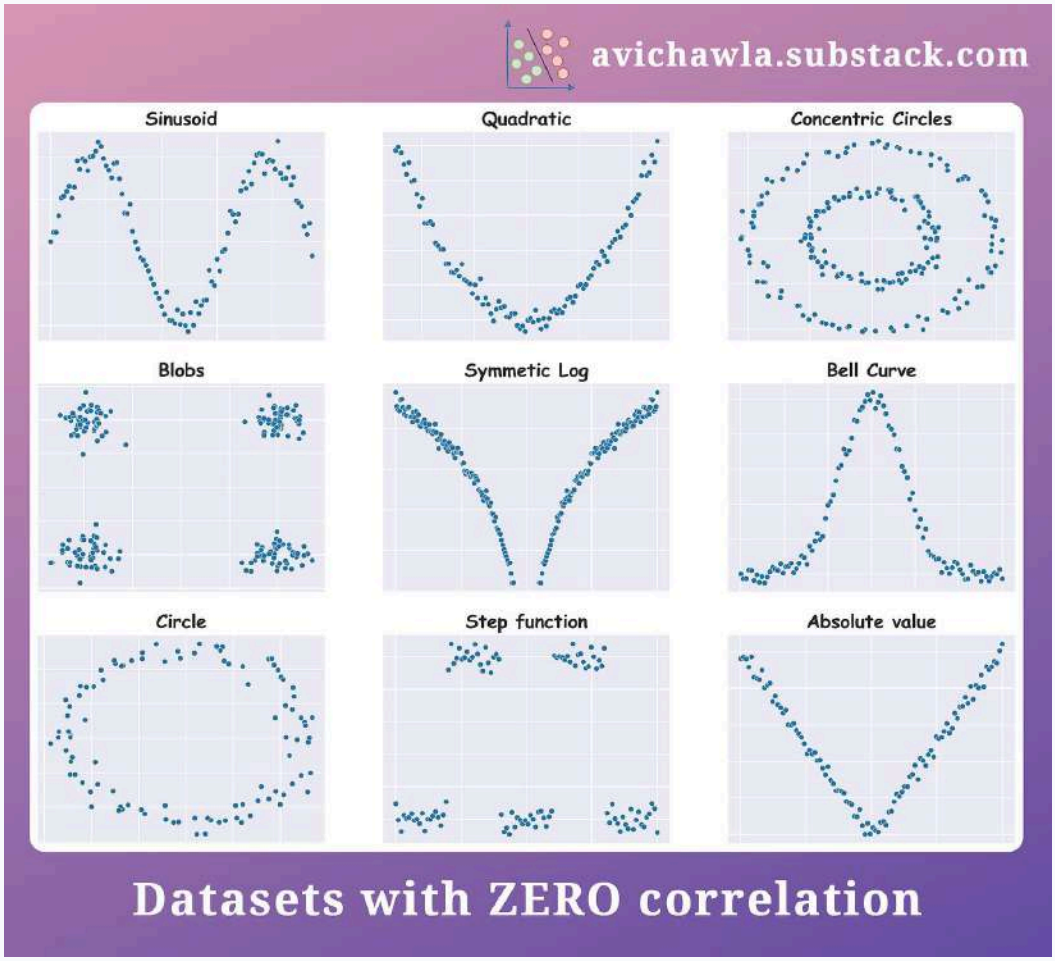
- Pearson and Spearman correlation is the same on linear data.
- But Pearson correlation underestimates a non-linear association.

Spearman correlation is also useful when data is ranked or ordinal, which will shall discuss in the next chapter.

Also, before we end this chapter, remember to always be cautious before drawing any conclusions using summary statistics. We also saw this in the last chapter.

While analyzing data, so many people get tempted to draw conclusions solely based on its statistics. Yet, the actual data might be conveying a totally different story.

This is also evident from the image below:



All nine datasets have approx. zero correlation between the two variables. However, the summary statistic, Pearson correlation in this case, gives no clue about what's inside the data because it is always zero.

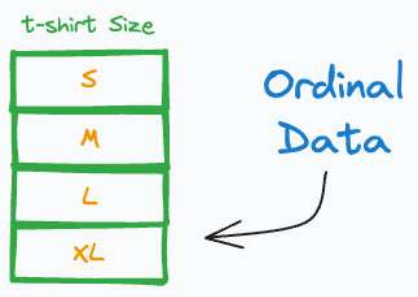
In fact, this is not just about Pearson correlation but applies to all summary statistics. The idea is that whenever you generate any summary statistic, you lose essential information.

Thus, the importance of looking at the data cannot be stressed enough. It saves us from drawing wrong conclusions, which we could have made otherwise by looking at the statistics alone.

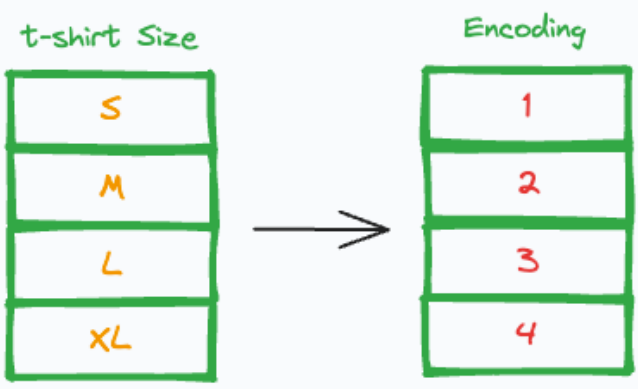
Correlation with Ordinal Categorical Data

Imagine you have an ordinal categorical feature. You want to measure its correlation with other continuous features.

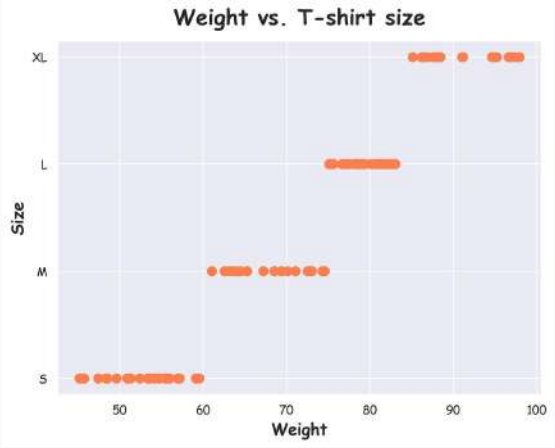
Ordinal feature is categorical data with a natural ordering in categories.



Before proceeding with the correlation analysis, you will encode the feature, which is a fair thing to do.



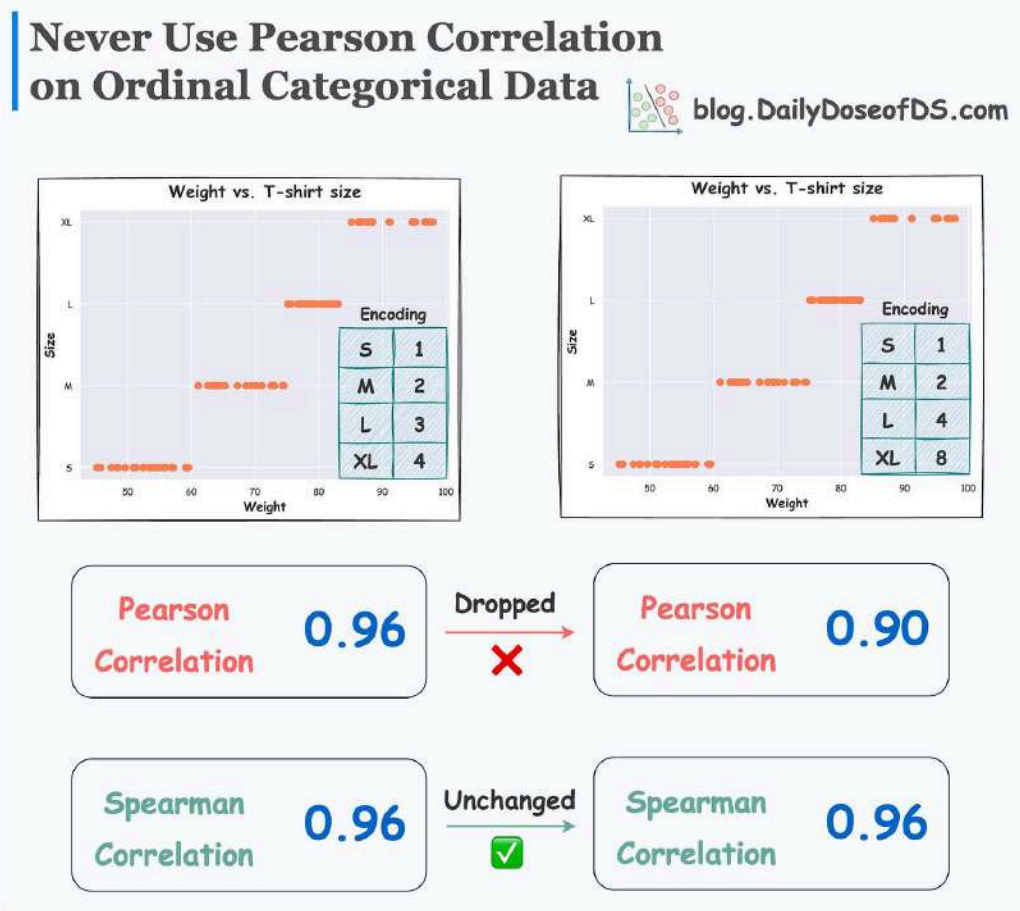
Yet, unknown to many, the choice of encoding can largely affect the correlation results. For instance, consider the dataset below:



Here, we have:

- An ordinal categorical feature: t-shirt size (S, M, L, XL).
- A continuous feature: weight.

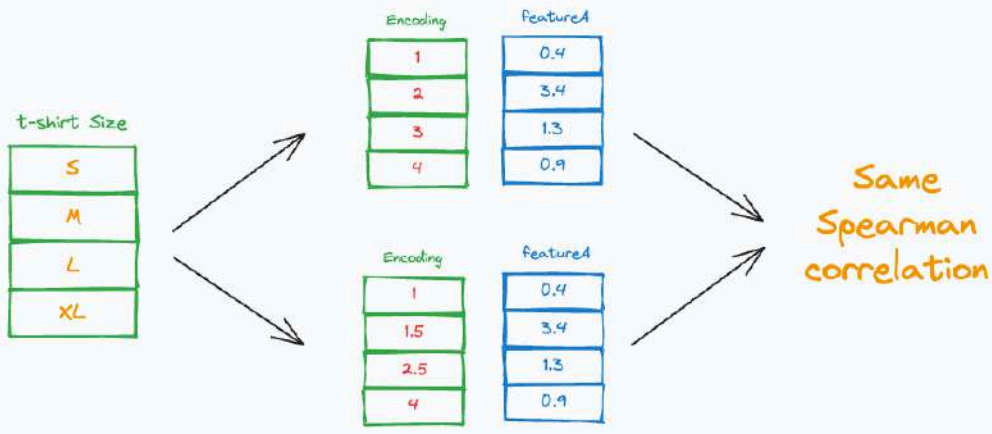
If we look at it graphically, there does exist a monotonic relationship between the two features. However, as depicted below, altering the categorical encoding affects the Pearson correlation.



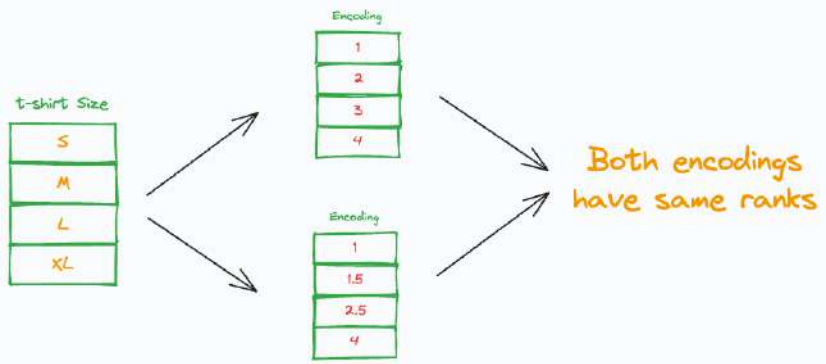
- In the left plot, we have the following encoding → S(1), M(2), L(3) and XL(4).
- In the right plot, we have the following encoding → S(1), M(2), L(4) and XL(8).

It is clear that the Spearman correlation is a better alternative to assess the monotonicity between ordinal and continuous features.

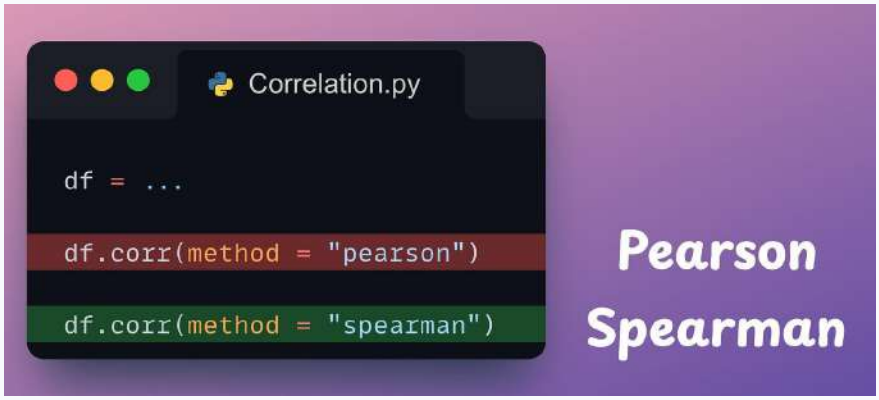
It always remains the same, irrespective of the choice of categorical encoding. This is because the Spearman correlation is rank-based.



It operates on the ranks of the data, which makes it more suitable for such cases of correlation analysis.



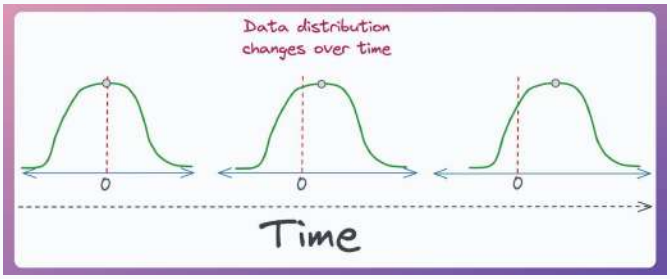
Using Spearman correlation is pretty simple as well. For instance, if you are using Pandas, just specify the desired correlation measure as follows:



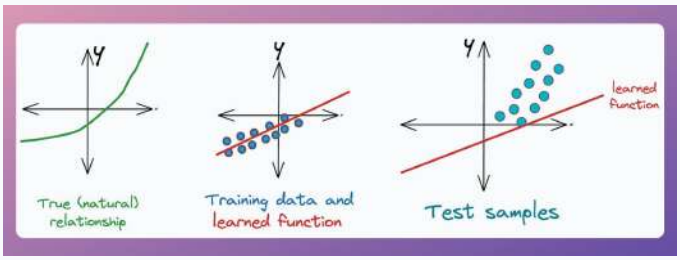
Drift

Multivariate Covariate Shift

Almost all real-world ML models gradually degrade in performance due to covariate shift.

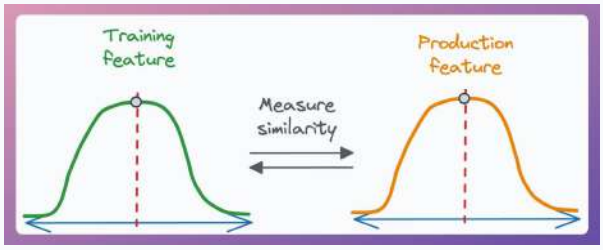


For starters, covariate shift happens when the distribution of features changes over time, but the true (natural) relationship between input and output remains the same.



It is a serious problem because we trained the model on one distribution, but it is being used to predict on another distribution in production. Thus, it is critical to detect covariate shift early so that models continue to work well.

Issues with detection methods

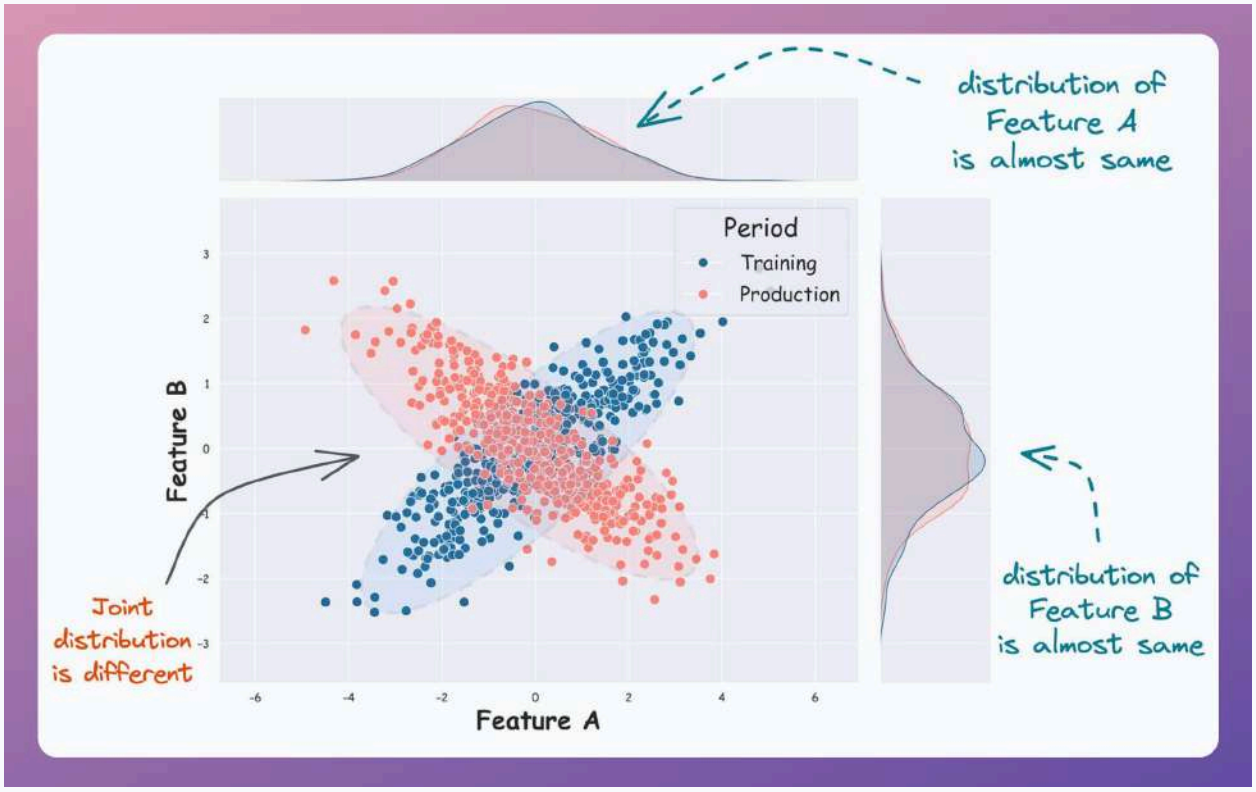
One of the most common and intuitive ways to detect covariate shift is by simply comparing the feature distribution in training data to that in production.	

This could be done in many ways, such as:

- Compare their summary statistics — mean, median, etc.
- Inspect visually using distribution plots.
- Perform hypothesis testing.
- Measure distances between training/production distributions using Bhattacharyya distance, KS test, etc.

While these approaches are often effective, the biggest problem is that they work on a single feature at a time. But, in real life, we may observe multivariate covariate shift (MCS) as well. It happens when:

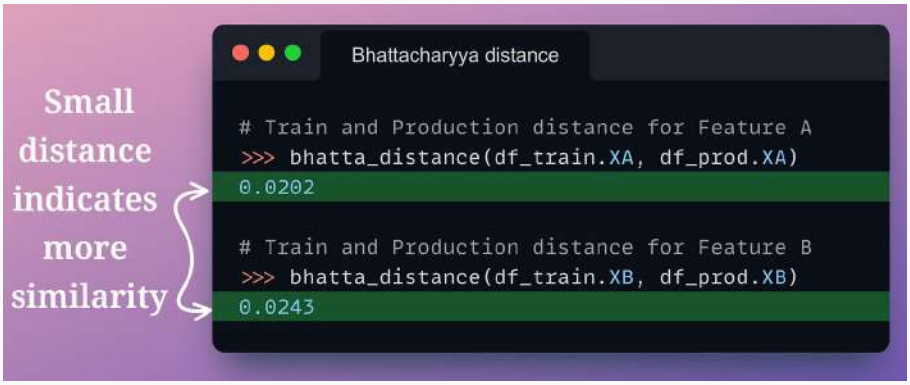
- The distribution of individual distributions remains the same:
    - $P(X1)$ and $P(X2)$ individually remain the same.
- But their joint distribution changes:
    - $P(X1, X2)$ changes.



- From the KDE plots on the top and the right, it is clear that the distribution of both features (covariates) is almost the same.

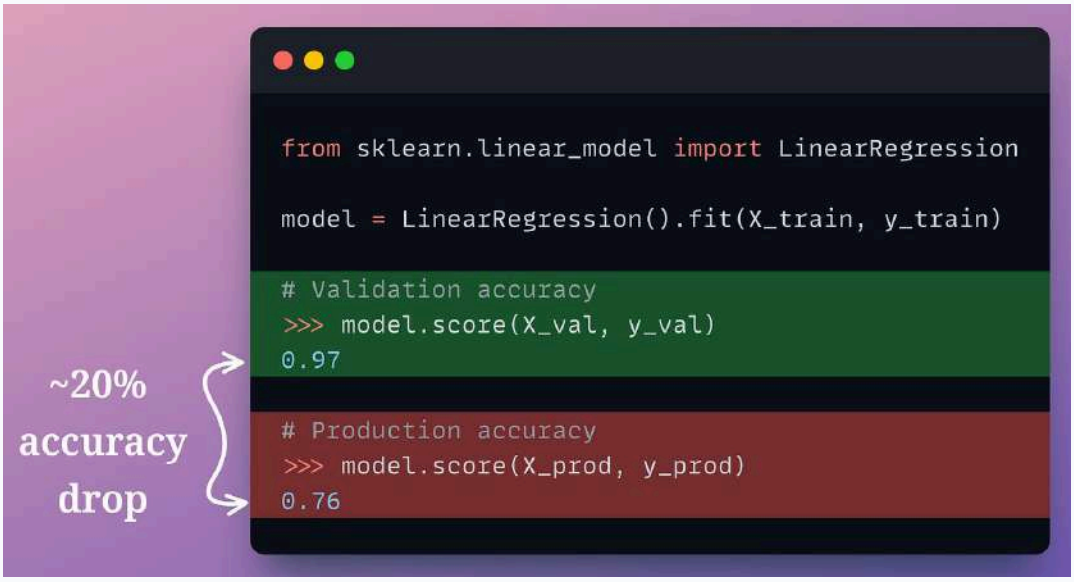
- But, the scatter plot reveals that their joint distribution in training (Blue) differs from that in production (Red).

And it is easy to guess that the univariate covariate shift detection methods discussed above will produce misleading results. For instance, as demonstrated below, measuring the Bhattacharyya distance between a training and production feature gives a very low distance value, indicating high similarity:



In fact, even though the individual feature distribution is the same, we can confirm experimentally that this will result in a drop in model performance:

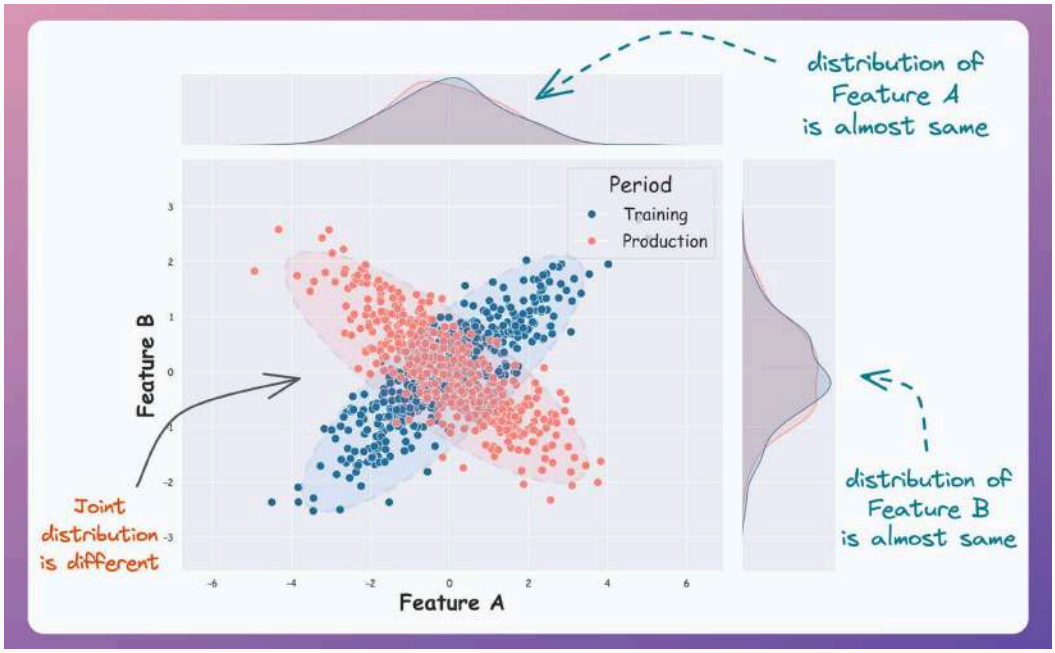
Let's say the true output `y = 2*Feature_A + 3*Feature_B + noise`.



In most cases, the true output predictions on production data are not immediately available. In the above demonstration, just assume that we have already gathered the true predictions somehow.

As depicted above, the model performance drops by 20% in production, which is huge. Here, we may completely rule out the possibility of covariate shift if we think that covariate shift can never be multivariate in nature.

Of course, in the figure below, it was easy to identify multivariate covariate shift because we are only looking at two features.



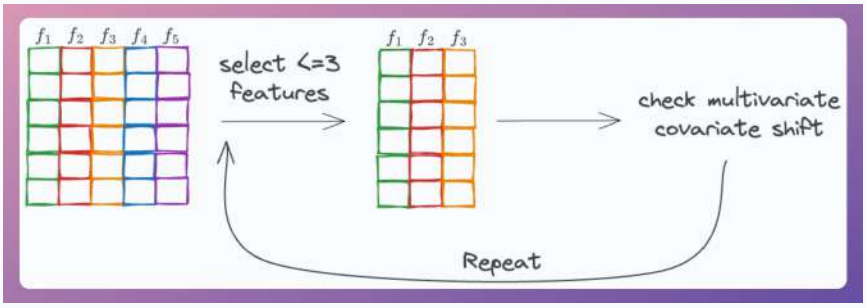
But multivariate covariate shift can happen with more than two features as well. Unlike the bivariate case above, visual inspection will not be possible for higher dimensions.

Detecting multivariate covariate shift

To begin, it is important to understand that multivariate covariate shift is a big problem, and there is no direct (or single) approach to handle this. Below, I will share a couple of ideas that I often use myself and have seen others use as well to handle multivariate covariate shift.

Idea #1: MCS is rare

As long as we are checking two (or even three) features at a time, visual inspection can be used to detect covariate shift. So here, at times, many simply ignore the possibility of any covariate shift beyond three features.



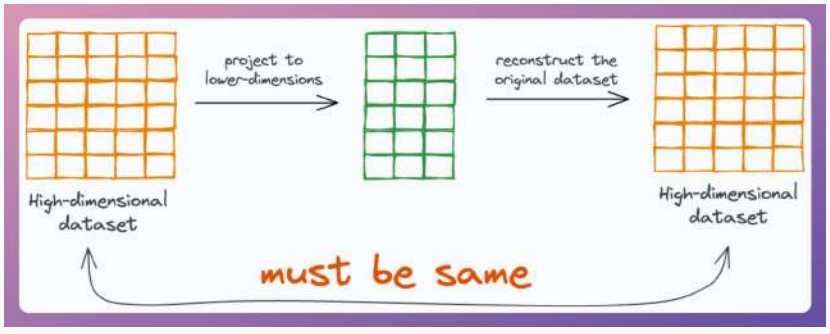
The rationale is that beyond three features, it is pretty unlikely that:

- P(X1), P(X2), P(X3), P(X4),... → all of them individually will almost remain the same.
- But their joint distribution P(X1, X2, X3, X4,...) will change.

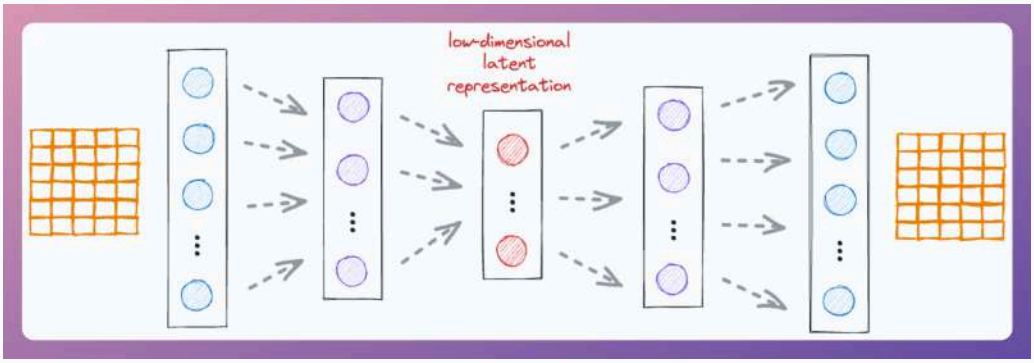
Thus, it might be fair to limit multivariate covariate shift analysis to just one, two, and three features at a time. But of course, this may not be always true, which brings us to another idea:

Idea #2: Data reconstruction

This is another cool and practical idea that I used in one of my projects. Data reconstruction, as the name suggests, revolves around learning a mapping that projects the data to low dimensions and then reconstructs the original data from the low dimensions.



This is precisely what Autoencoders do. They are a class of neural networks that learn to encode data in a lower-dimensional space, and then decode it back to the original data space.

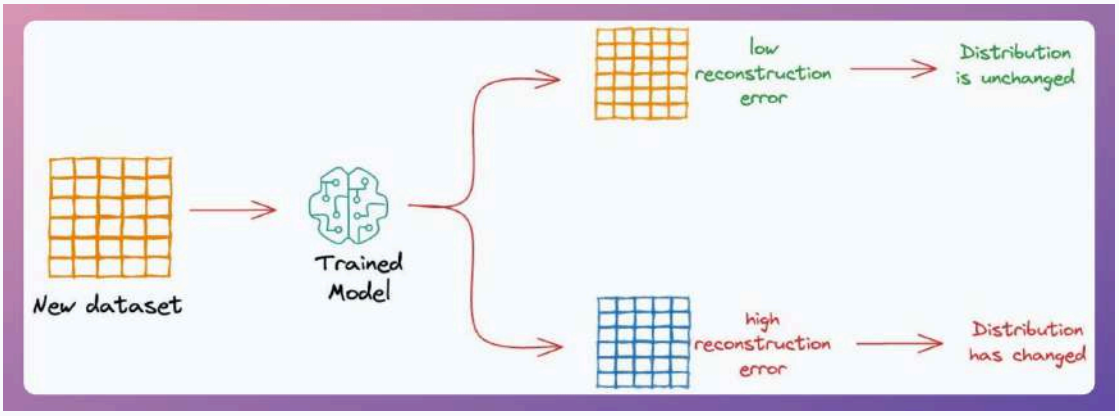


The objective is to minimize the data reconstruction error. It's like asking the model to learn a mapping that:

- Takes some data.
- Projects the data to low dimensions.
- And then gives us the exact input data back.

So here's what we can do:

- Train an Autoencoder on the original training dataset. This will give us the weights for the neural network model that reconstructs the dataset.
- Use this model on new data to check multivariate covariate shift:
    - If the reconstruction loss is high, the distribution has changed.
    - If the reconstruction loss is low, the distribution is almost the same.



One of the best things about this approach is that it does not need a labeled dataset. Essentially, Autoencoders aim to reconstruct the dataset without labels.

This is quite useful in real-world models because, as we discussed yesterday, in most cases, the true output predictions on production data are never immediately available. Instead, they always take some time.

For instance, I remember when I was working on a transactional fraud detection model at Mastercard, a card holder's issuer bank may take upto 45-50 days to send the fraud label for the transactions that went through Mastercard's network.

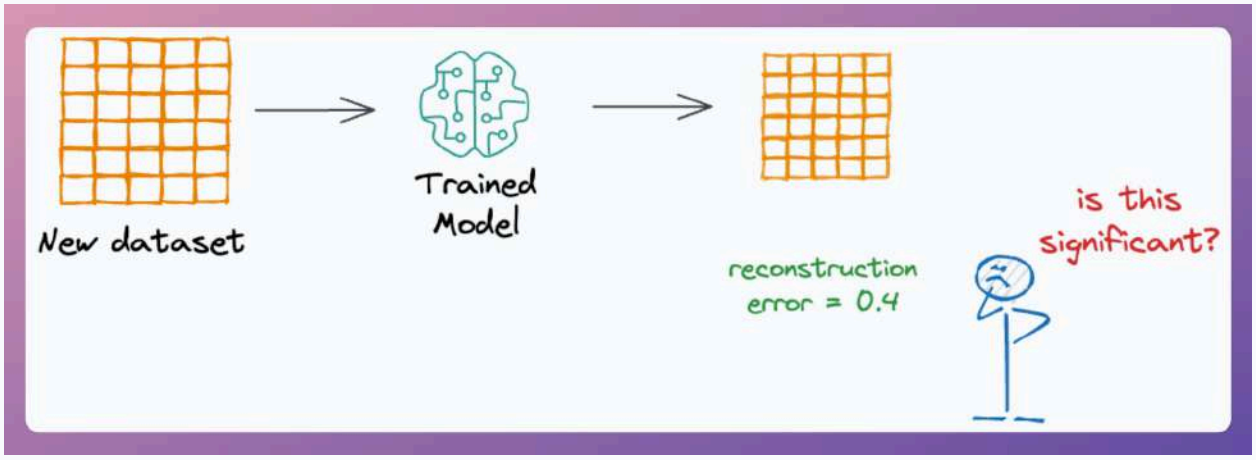
This is a lot of time, isn't it?

But using Autoencoders, we can still check data reconstruction errors on the unlabeled data. In fact, we don't necessarily have to use Autoencoders. It was just an example here.

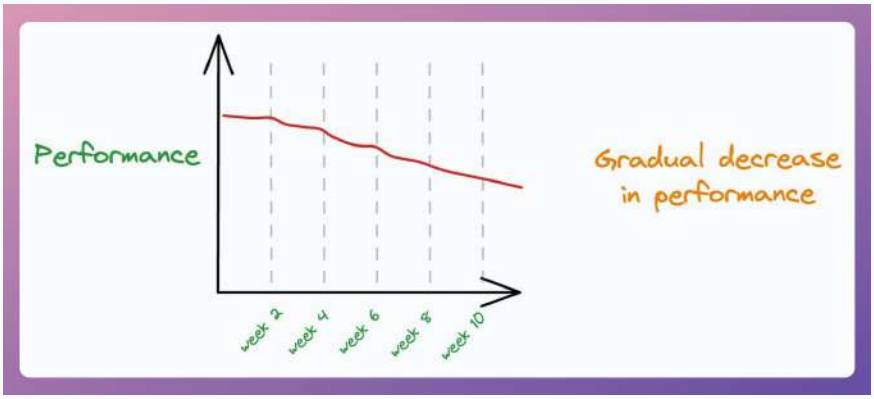
Other data reconstruction techniques can also be used, such as PCA is one of them. Nonetheless, when using any data reconstruction approach, it is important to take care of one thing. Earlier, we discussed that:

- If the reconstruction loss is high, the distribution has changed.
- If the reconstruction loss is low, the distribution is almost the same.

But interpreting reconstruction loss can be pretty subjective, and it also needs some context.

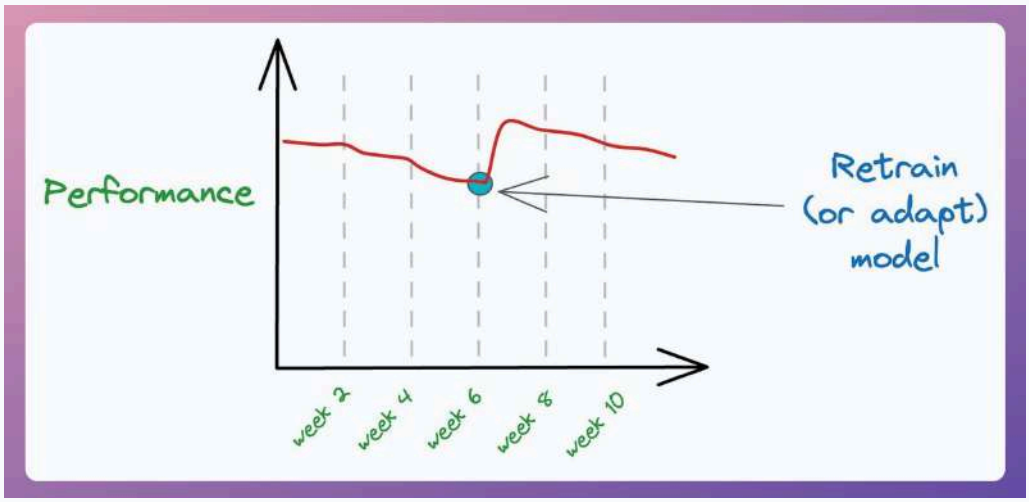


For instance, if the reconstruction loss is 0.4 (say), how do we determine whether this is significant? Contrary to what many think, covariate shift happens gradually in ML models. In other words, it is unlikely that our model is performing pretty well one day, and the next day, it starts underperforming DUE TO COVARIATE SHIFT.



Of course, abrupt degradation is possible but it is rarely due to covariate shift. Most of the time, it happens due to other issues like - May be due to some compliance issue, the team has stopped collecting a feature. As a result, your model no longer has access to a feature you trained it with.Or may be there were some updates to the inference logic which was no thoroughly tested.and more.

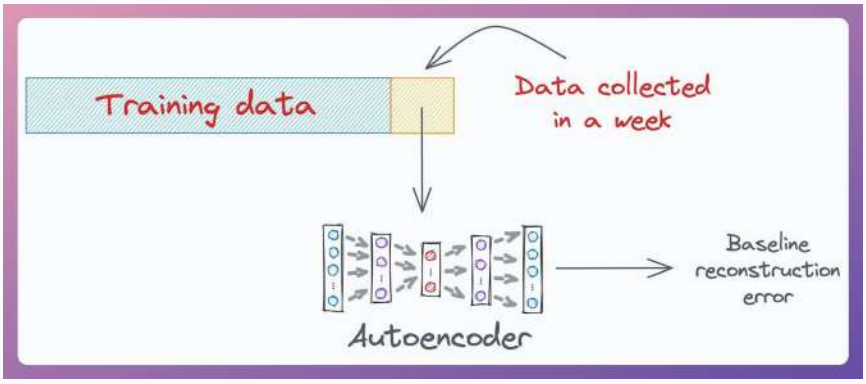
But performance degradation happens gradually due to covariate shift. And in that phase of performance degradation, we must decide to update the model at some point to adjust for distributional changes.



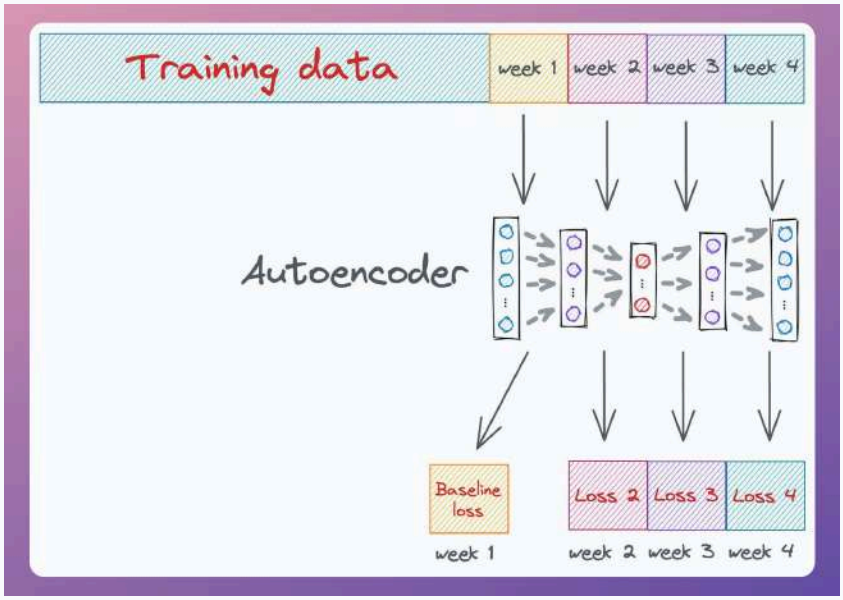
While data reconstruction loss is helpful in deciding model updates, it conveys very little meaning alone. Simply put, we need a baseline reconstruction loss value to compare our future reconstruction losses.

So here's what we can do.

Let's say we decided to review our model every week. After training the model on the gathered data:



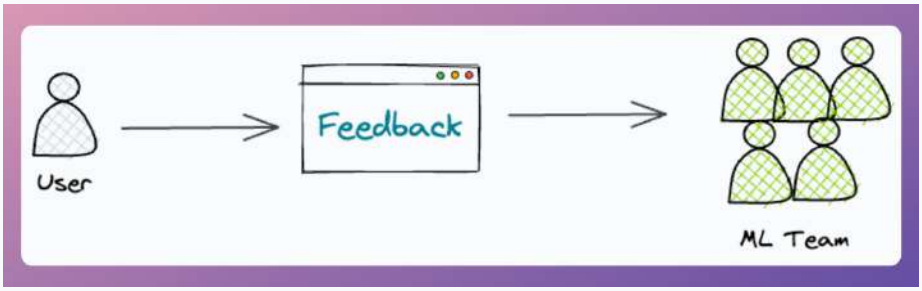
- We determine our baseline reconstruction score using the new data gathered over the first week in the post-training phase.
- We use this baseline score to compare future losses of every subsequent week.



A high difference between reconstruction loss and the baseline score will indicate a covariate shift.

Simple and intuitive, right? The above reconstruction metric, coupled with the model performance, becomes a reliable way to determine if covariate shift has stepped in. Of course, as discussed above, true labels might not be immediately available at times.

Thus, determining performance in production can be difficult. To counter this, all ML teams consistently try to gather feedback from the end user to learn if the model did well or not.



This is especially seen in recommendation systems:

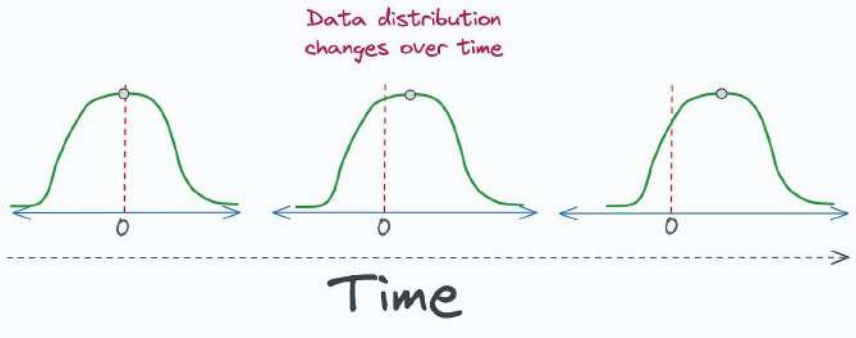
- Was this advertisement relevant to you?
- Was this new video relevant to you?
- And more.

These things are a part of the model logging phase, where we track model performance in production. This helps us decide whether we should update our model or not.

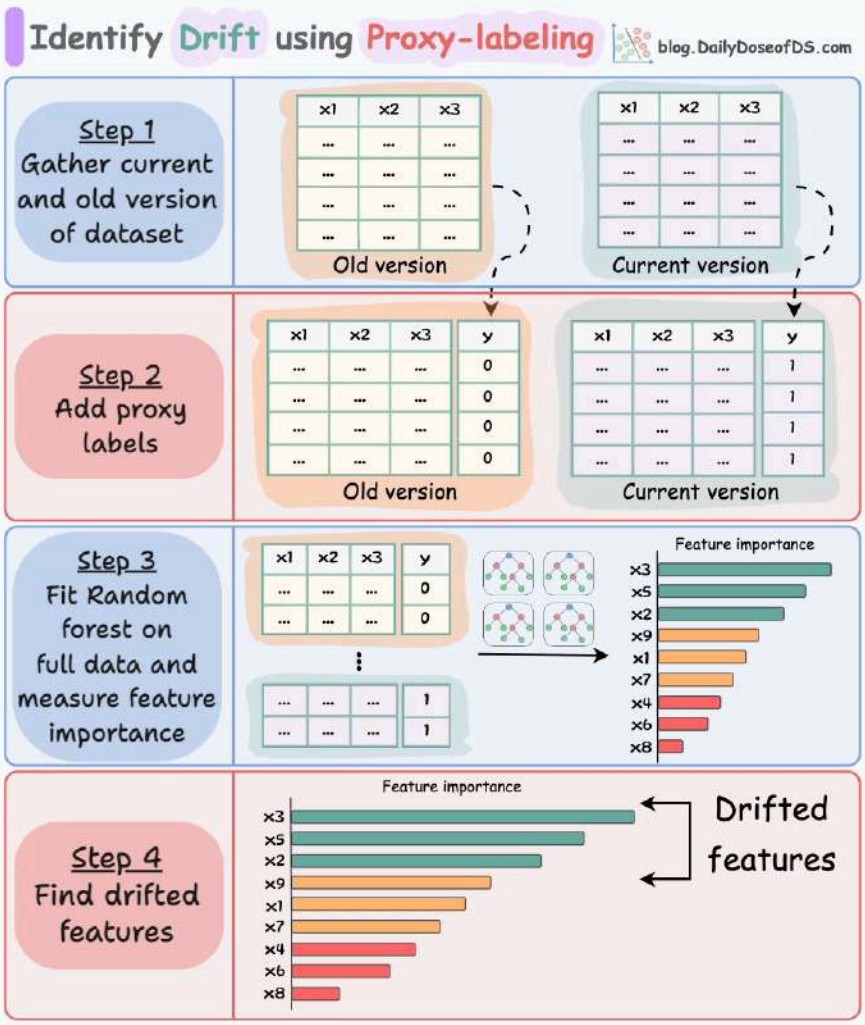


Using Proxy-Labelling to Identify Drift

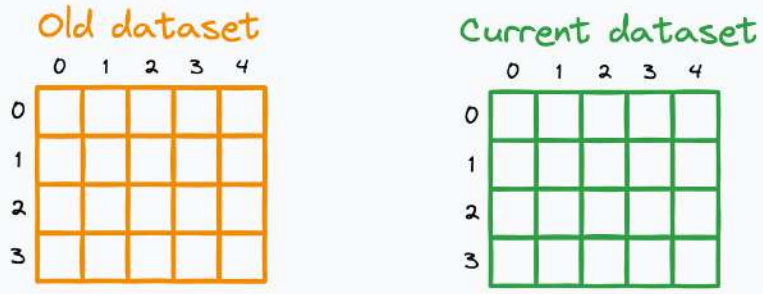
Almost all real-world ML models gradually degrade in performance due to a drift in feature distribution:



It is a serious problem because we trained the model on one distribution, but it is being used to generate predictions on another distribution in production. Thus, it is critical to detect drift early so that models continue to work well. The image depicts an intuitive technique I often use to determine which features are drifting in my dataset.



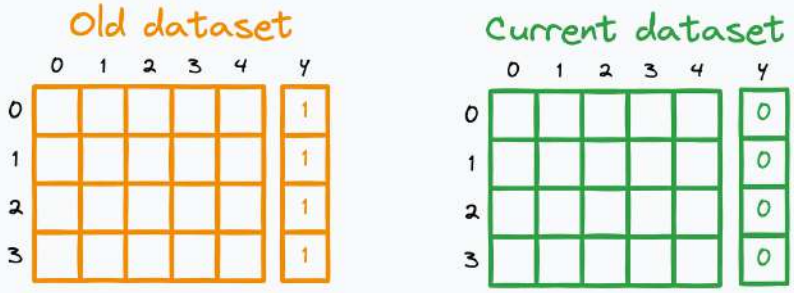
Consider we have two versions of the dataset — the old version (one on which the model was trained) and the current version (one on which the model is generating predictions):



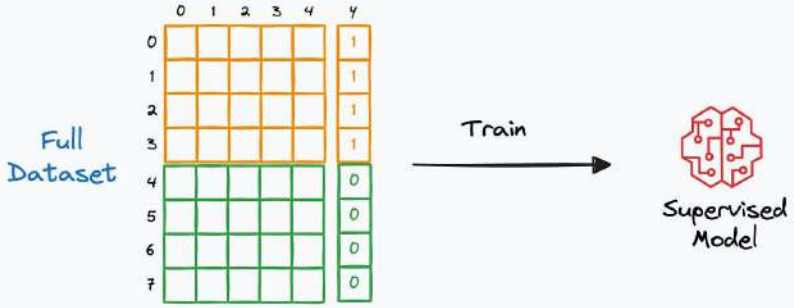
The core idea is to assess if there are any distributional dissimilarities between these two versions.

So here's what we do:

- Append a `label=1` column to the old dataset.
- Append a `label=0` column to the current dataset.



Now, merge these two datasets and train a supervised learning classification model on the combined dataset:



The choice of the classification model could be a bit arbitrary, but it should be ensured that it is possible to reliably determine feature importance.

Thus, I personally prefer a random forest classifier because it has an inherent mechanism to determine feature importance:

```
from sklearn.ensemble import RandomForestClassifier

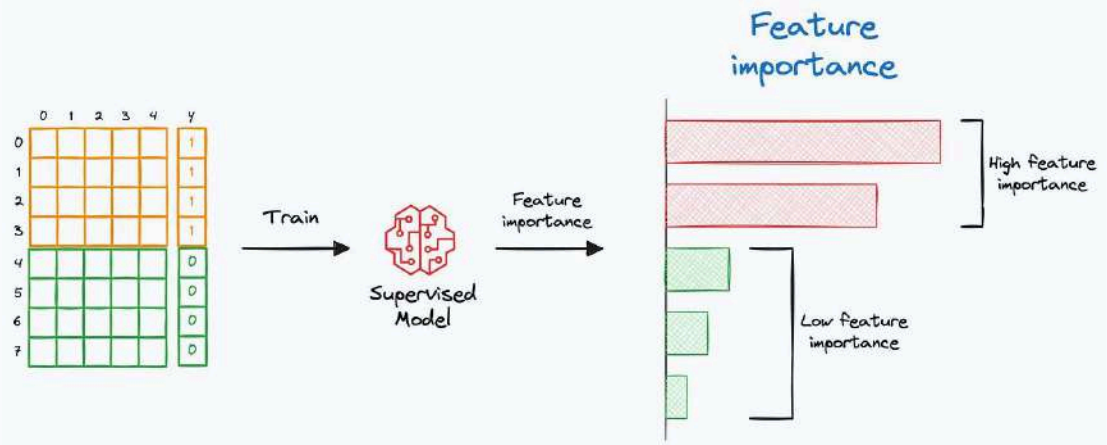
model = RandomForestClassifier().fit(X, y)

>>> model.feature_importances_
array([0.39, 0.25, 0.12, 0.10, 0.11, 0.13])
```

That said, it is not necessary to use a random forest.

Techniques like shuffle feature importance (which we discussed above) on top of a classification model can be used as well.

If the feature importance values suggest that there are features with high feature importance, this means that those features are drifting.



Why?

This is because if some features can reliably distinguish between the two versions of the dataset, then it is pretty likely that their distribution corresponding to `label=1` and `label=0` (conditional distribution) are varying.

- If there are distributional differences, the model will capture them.
- If there are no distributional differences, the model will struggle to distinguish between the classes.

This idea makes intuitive sense as well.

At this point, one question that many have is that...

Why can't we just monitor the model accuracy to determine drift?

Of course, we can do that as long as we have the true labels for the current version or have a way to compare the model performance on the old version and the new version.

But in many cases, the true output predictions on production data are never immediately available.

Instead, they always take some time.

For instance, when I was working on a transactional fraud detection model at Mastercard, a cardholder's issuer bank may take up to 45-50 days to send the fraud label for the transactions that went through Mastercard's network.

This is a lot of time, isn't it?

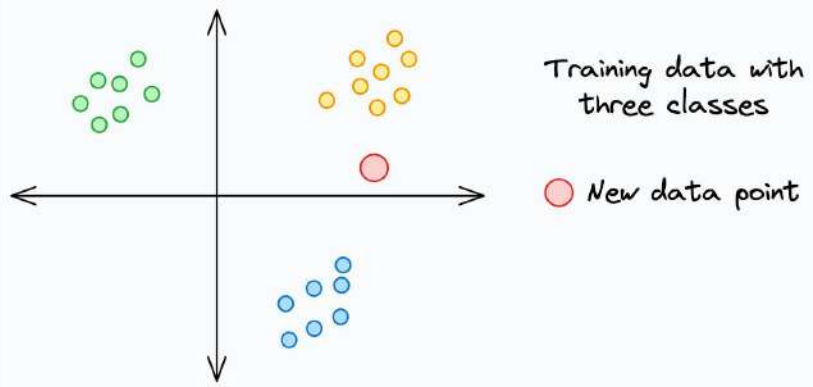
Thus, one must rely on some relevant feedback from the system to determine whether the model's performance is dropping or not.

Using this "proxy-labeling technique" discussed above is something I have found to be immensely useful.

kNN

Using kNNs on Imbalanced Datasets

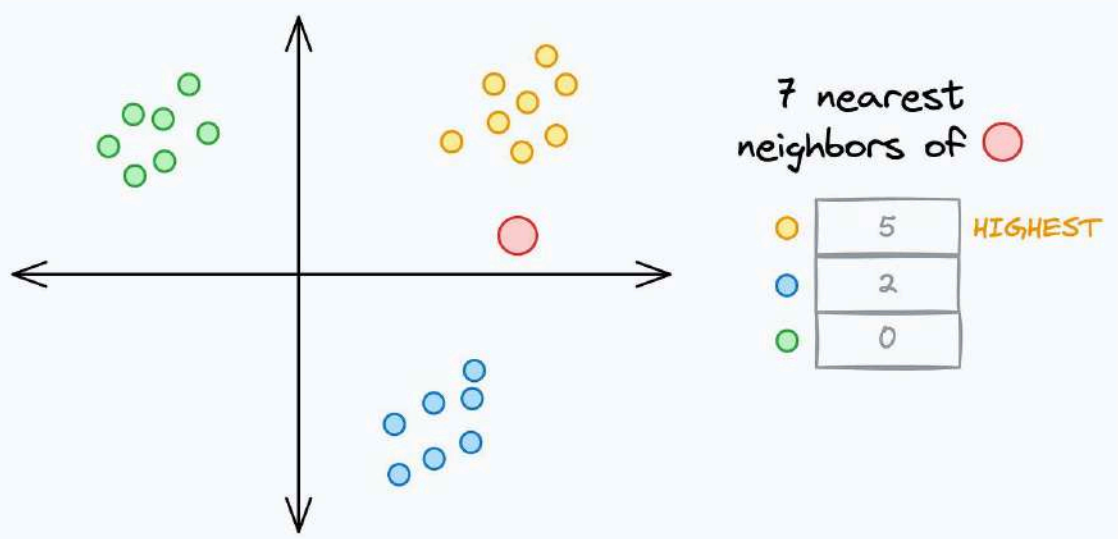
One of the things that always makes me a bit cautious and skeptical when using kNN is its HIGH sensitivity to the parameter k. To understand better, consider this dummy 2D dataset below. The red data point is a test instance we intend to generate a prediction for using kNN.



Say we set the value of $k=7$. The prediction for the red instance is generated in two steps:

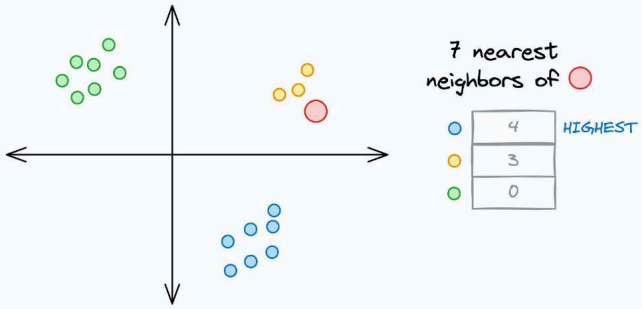
- First, we count the 7 nearest neighbors of the red data point.
- Next, we assign it to the class with the highest count among those 7 nearest neighbors.

This is depicted below:



The problem is that step 2 is entirely based on the notion of class contribution — the class that maximally contributes to the k nearest neighbors is assigned to the data point. But this can miserably fail at times, especially when we have a class with few samples.

For instance, as shown in this image, with k=7, the red data point can NEVER be assigned to the yellow class, no matter how close it is to that cluster.

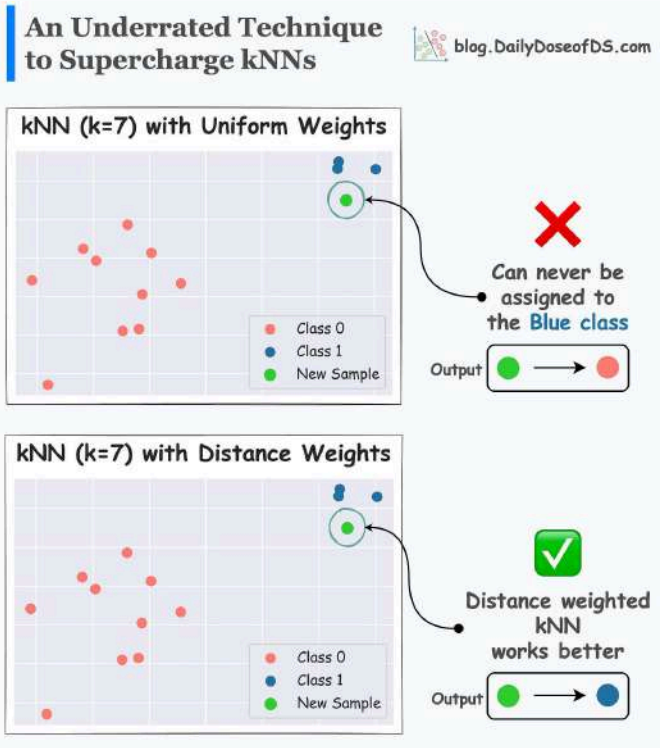


While it is easy to tweak the hyperparameter k visually in the above demo, this approach is infeasible in high-dimensional datasets. There are two ways to address this.

Solution #1: Used distance-weighed kNN

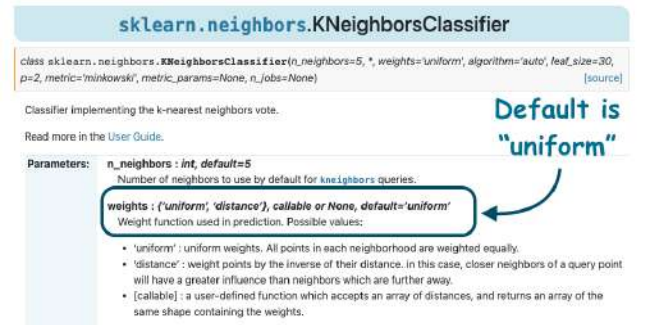
Distance-weighted kNNs are a much more robust alternative to traditional kNNs. As the name suggests, in step 2, they consider the distance to the nearest neighbor.

As a result, the closer a specific neighbor is, the more impact it will have on the final prediction. Its effectiveness is evident from this image.



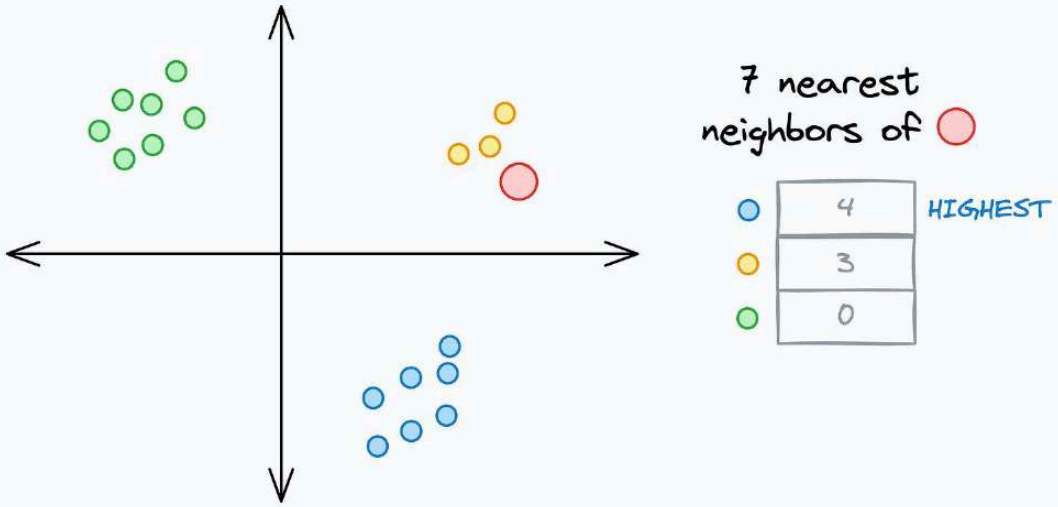
- In the first plot, traditional kNN (with k=7) can never predict the blue class.
- In the second plot, distance-weighted kNN is found to be more robust in its prediction.

As per my observation, a distance-weighted kNN typically works better than a traditional kNN. Yet, this may go unnoticed because, by default, the kNN implementation of sklearn considers "uniform" weighting.



Solution #2: Dynamically update the hyperparameter k

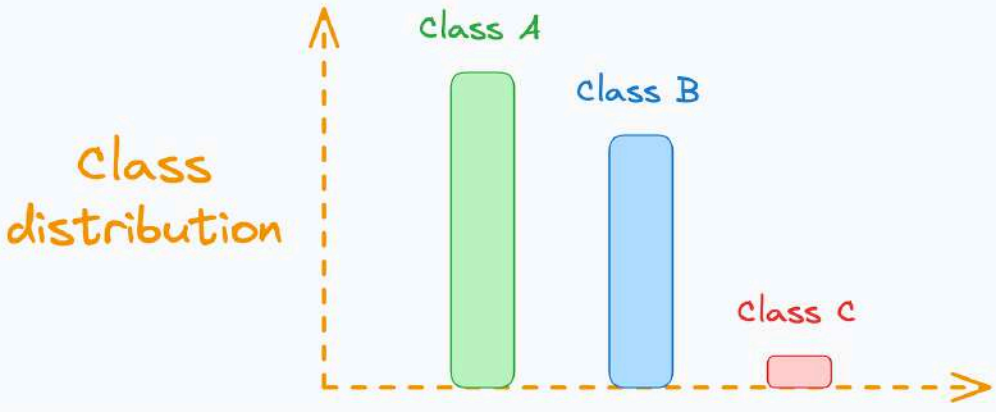
Recall the above demo again:



Here, one may argue that we must refrain from setting the hyperparameter k to any value greater than the minimum number of samples that belong to a class in the dataset.

Of course, I agree with this to an extent.

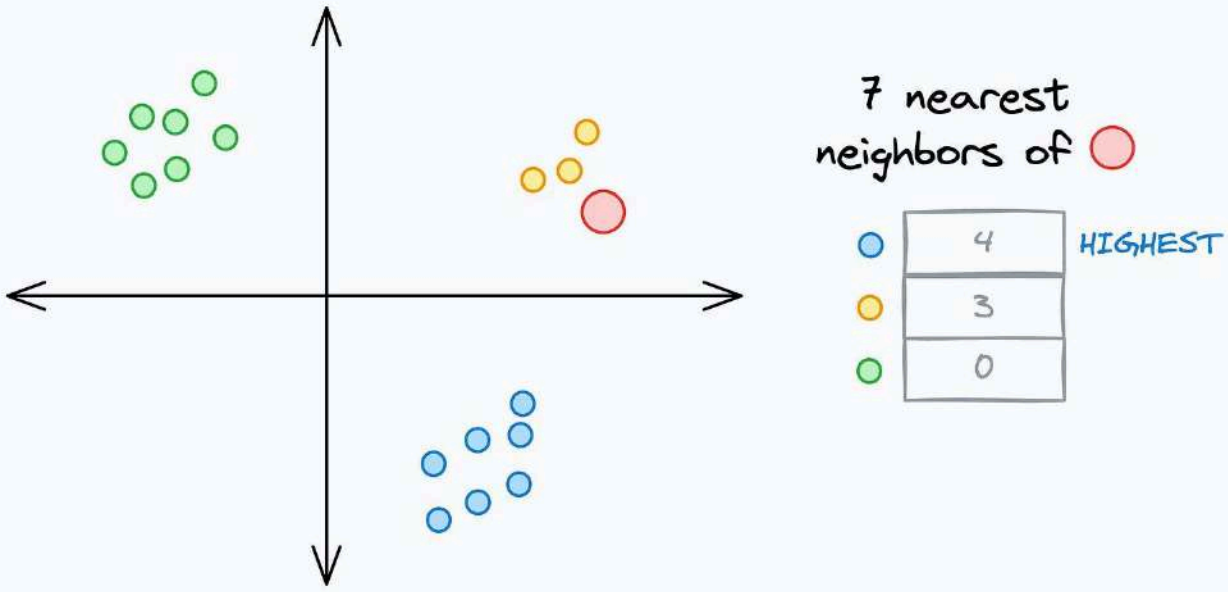
But let me tell you the downside of doing that. Setting a very low value of k can be highly problematic in the case of extremely imbalanced datasets.



To give you more perspective, I have personally used kNN on datasets that had merely one or two instances for a particular class in the training set. And I

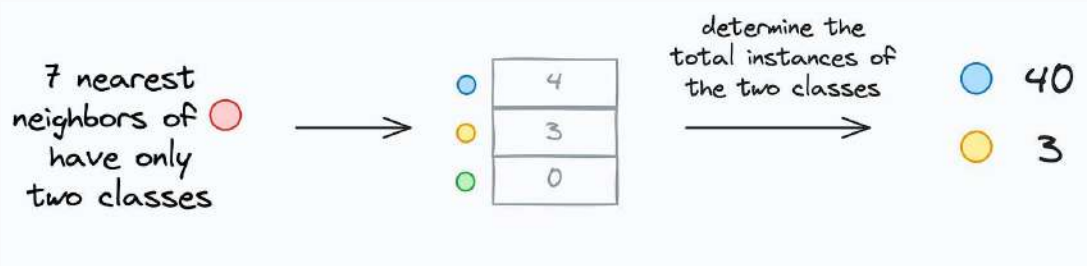
discovered that setting a low of k (say, 1 or 2) led to suboptimal performance because the model was not as holistically evaluating the nearest neighbor patterns as it was when a large value of k was used.

In other words, setting a relatively larger value of k typically gives more informed predictions than using lower values. But we just discussed above that if we set a large value of k, the majority class can dominate the classification result:



To address this, I found dynamically updating the hyperparameter k to be much more effective. More specifically, there are three steps in this approach. For every test instance:

1. Begin with a standard value of k as we usually would and find the k nearest neighbors.
2. Next, update the value of the k as follows:
    a. For all unique classes that appear in the k nearest neighbor, find the total number of training instances they have.



b. Update the value of k to:

update k value

$$k' = min(k, 40, 3)$$

3. Now perform majority voting only on the first k' neighbors only.

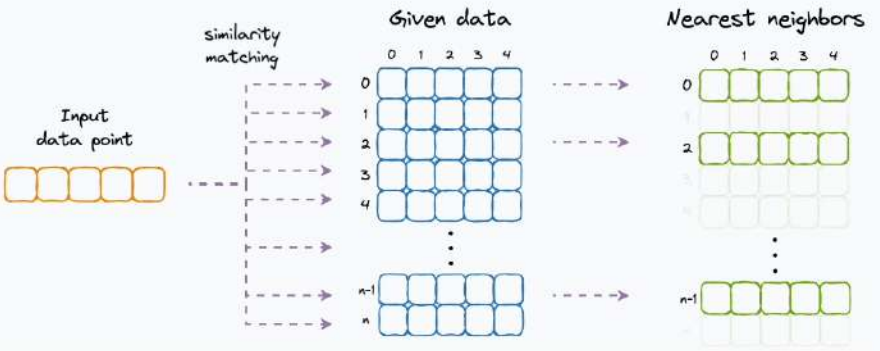
This makes an intuitive sense as well:

- If a minority class appears in the top k nearest neighbor, we must reduce the value of k so that the majority class does not dominate.
- If a minority class DOES NOT appears in the top k nearest neighbor, we will likely not update the value of k and proceed with a holistic classification.

The only shortcoming is that you wouldn't find this approach in any open-source implementations. In fact, in my projects as well, I had to write a custom implementation, so take that into account.

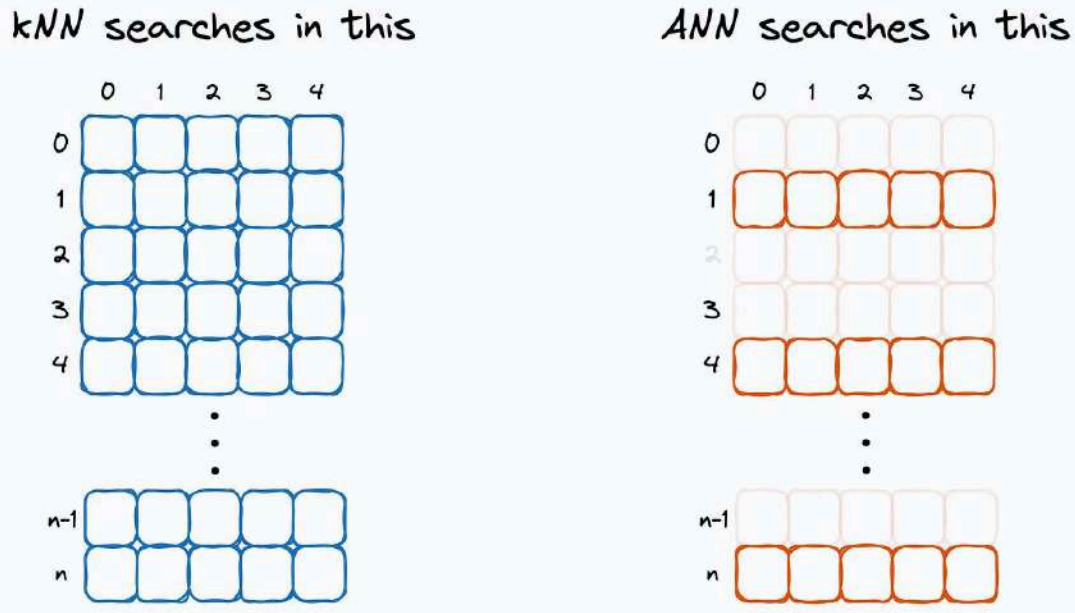
Approximate Nearest Neighbor Search Using Inverted File Index

One of the biggest issues with nearest neighbor search using kNN is that it performs an exhaustive search.



In other words, the query data point must be matched across all data points to find the nearest neighbor(s). This is highly inefficient, especially when we have many data points and a near-real-time response is necessary.

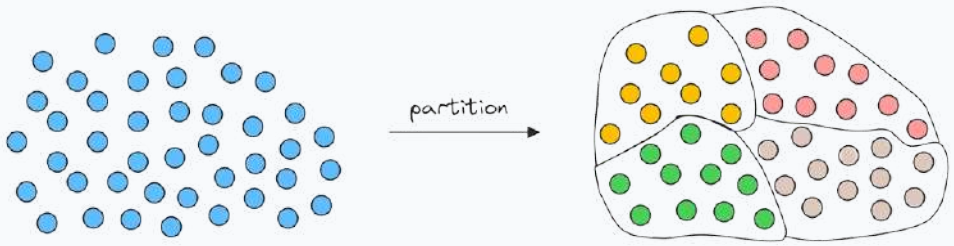
That is why approximate nearest neighbor search algorithms are becoming increasingly popular. The core idea is to narrow down the search space using indexing techniques, thereby improving the overall run-time performance.



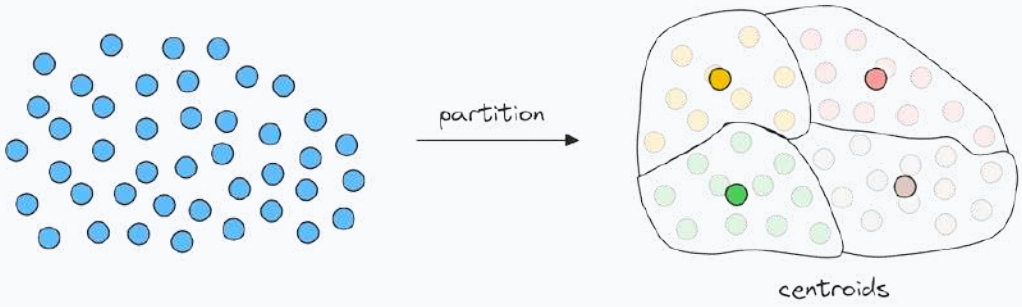
Inverted File Index (IVF) is possibly one of the simplest and most intuitive techniques here, which you can immediately start using.

Here's how it works

Given a set of data points in a high-dimensional space, the idea is to organize them into different partitions, typically using clustering algorithms like k-means.



As a result, each partition has a corresponding centroid, and every data point gets associated with only one partition corresponding to its nearest centroid.



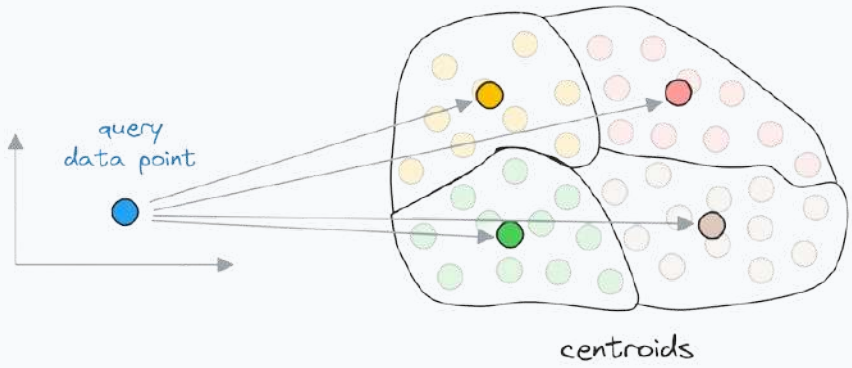
Also, every centroid maintains information about all the data points that belong to its partition.



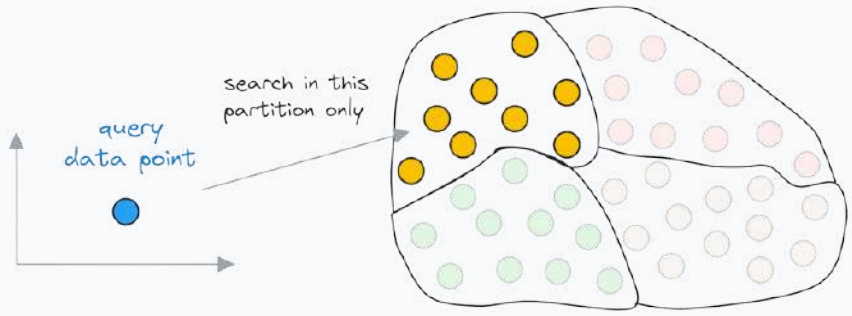
Indexing done!

Here's how we search.

When searching for the nearest neighbor(s) to the query data point, instead of searching across the entire dataset, we first find the closest centroid to the query:



Once we find the nearest centroid, the nearest neighbor is searched in only those data points that belong to the closest partition found:



Let's see how the run-time complexity stands in comparison to traditional kNN. Consider the following:

- There are N data points
- Each data point is D dimensional
- We create K partitions.
- Lastly, for simplicity, let's assume that each partition gets equal data points.

In kNN, the query data point is matched to all N data points, which makes the time complexity → O(ND).

In IVF, however, there are two steps:

1. Match to all centroids → O(KD).
2. Find the nearest neighbor in the nearest partition → O(ND/K).

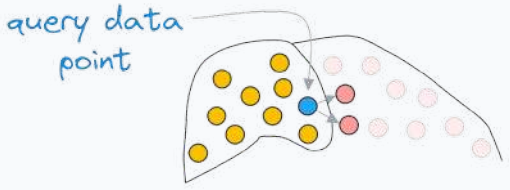
The final time complexity comes out to be the following:

$$O(KD + \frac{ND}{K})$$

...which is significantly lower than that of kNN. To get some perspective, assume we have 10M data points. The search complexity of kNN will be proportional to 10M. But with IVF, say we divide the data into 100 centroids, and each partition gets roughly 100k data points.

Thus, the time complexity comes out to be proportional to 100 + 100k = 100100, which is nearly 100 times faster.

Of course, it is essential to note that if some data points are actually close to the input data point but still happen to be in the neighboring partition, we will miss them during the nearest neighbor search, as shown in this image.



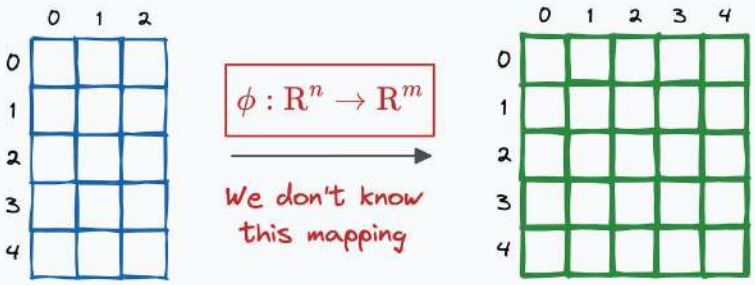
But this accuracy tradeoff is something we willingly accept for better run-time performance, which is precisely why these techniques are called "approximate nearest neighbors search."

Kernels

Why is Kernel Trick Called a "Trick"?

So many ML algorithms use kernels for robust modeling, like SVM, KernelPCA, etc.

In a gist, a kernel function lets us compute dot products in some other feature space (mostly high-dimensional) without even knowing the mapping from the current space to the other space.



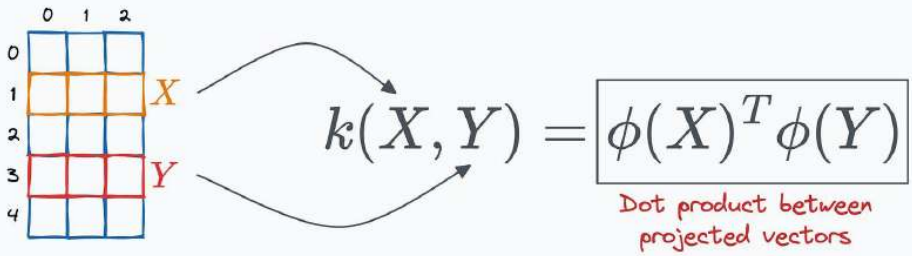
But how does that even happen?

Let's understand!

The objective

Firstly, it is important to note that the kernel provides a way to compute the dot product between two vectors, X and Y, in some high-dimensional space without projecting the vectors to that space.

This is depicted below, where the output of the kernel function is expected to be the same as the dot product between projected vectors:



The key advantage is that the kernel function is applied to the vectors in the original feature space.

However, that equals the dot product between the two vectors when projected into a higher-dimensional (yet unknown) space.

If that is a bit confusing, let me give an example.

A motivating example

Let's assume the following polynomial kernel function:

Polynomial kernel

$$k(X, Y) = (1 + X^T Y)^2$$

For simplicity, let's say both X and Y are two-dimensional vectors:



$$X = (x_1, x_2)$$



$$Y = (y_1, y_2)$$

Simplifying the kernel expression above, we get the following:

$$\begin{aligned} k(X, Y) &= (1 + X^T Y)^2 \\ &= (1 + x_1 y_1 + x_2 y_2)^2 \end{aligned}$$

Expanding the square term, we get:

$$k(X, Y) = (1 + x_1 y_1 + x_2 y_2)^2$$

expand the square term

$$= 1 + x_1^2 y_1^2 + x_2^2 y_2^2 + 2x_1 y_1 + 2x_2 y_2 + 2x_1 y_1 x_2 y_2$$

Now notice the final expression:

$$k(X,Y) = 1 + x_1^2y_1^2 + x_2^2y_2^2 + 2x_1y_1 + 2x_2y_2 + 2x_1y_1x_2y_2$$

The above expression is the dot product between the following 6-dimensional vectors:

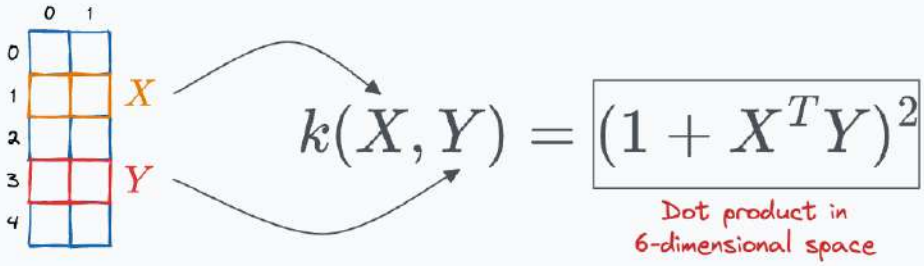
$$X' = (1, x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2)$$
$$Y' = (1, y_1^2, y_2^2, \sqrt{2}y_1, \sqrt{2}y_2, \sqrt{2}y_1y_2)$$

Thus, our projection function comes out to be:

Projection function

$$\phi(X) = (1, x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2)$$

This shows that the kernel function we chose earlier computes the dot product in a 6-dimensional space without explicitly visiting that space.

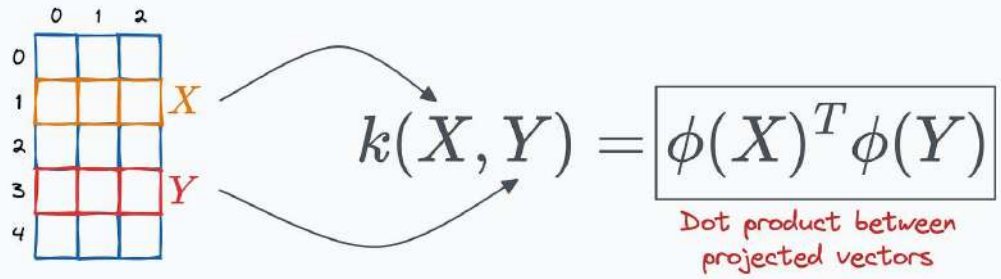


And that is the primary reason why we also call it the "kernel trick."

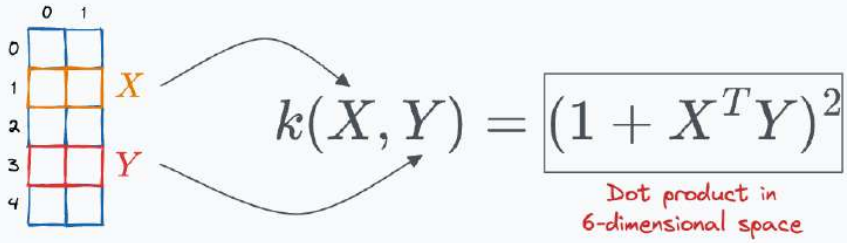
More specifically, it's framed as a "trick" since it allows us to operate in high-dimensional spaces without explicitly computing the coordinates of the data in that space.

The Mathematics Behind RBF Kernel

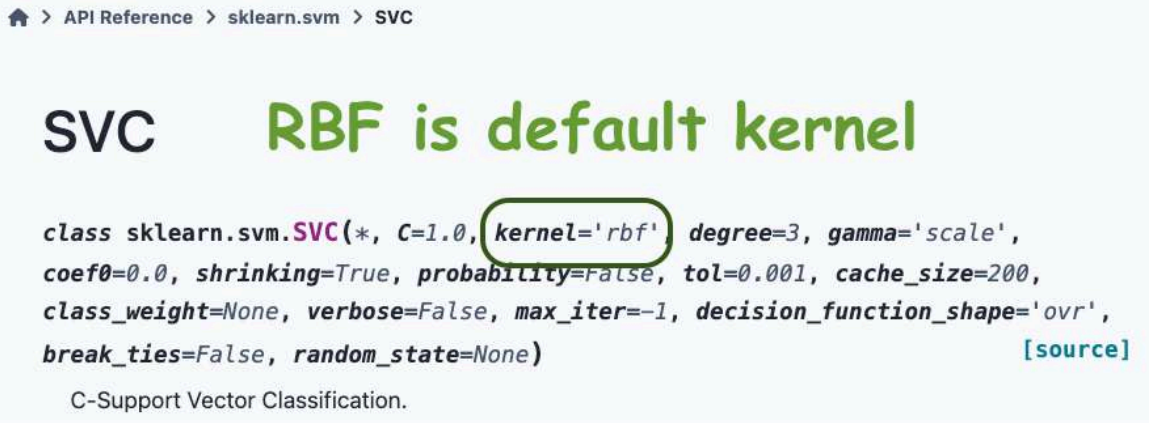
As we saw above, a kernel provides a way to compute the dot product between two vectors, X and Y, in some high-dimensional space without projecting the vectors to that space.



In that post, we looked at the polynomial kernel and saw that it computes the dot product of a 2-dimensional vector in a 6-dimensional space without explicitly visiting that space.



Next, let's talk about the RBF kernel, another insanely powerful kernel, which is also the default kernel in a support vector classifier class implemented by sklearn:



To begin, the mathematical expression of the RBF kernel is depicted below (and consider that we have just a 1-dimensional feature vector):

$$k(x, y) = e^{\left(-\gamma \|x-y\|^2\right)}$$

You may remember from high school mathematics that the exponential function is defined as follows:

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots$$

Expanding the square term in the RBF kernel expression, we get:

$$\begin{aligned} k(x, y) &= e^{\left(-\gamma \|x-y\|^2\right)} \\ &= e^{\left(-\gamma (x^2+y^2-2xy)\right)} \end{aligned}$$

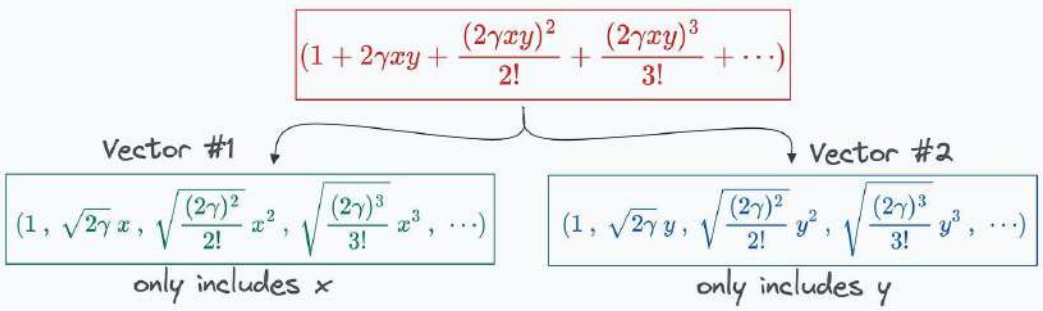
Distributing the gamma term and expanding the exponential term using the exponent rule, we get:

$$\begin{aligned} k(x, y) &= e^{\left(-\gamma (x^2+y^2-2xy)\right)} \\ &= e^{-\gamma (x^2+y^2)+2\gamma xy} \\ &= e^{-\gamma (x^2+y^2)} \cdot e^{2\gamma xy} \end{aligned}$$

Next, we apply the exponential expansion to the last term and get the following:

$$\begin{aligned} k(x, y) &= e^{-\gamma (x^2+y^2)} \cdot e^{2\gamma xy} \\ &= e^{-\gamma (x^2+y^2)} \cdot \left(1 + 2\gamma xy + \frac{(2\gamma xy)^2}{2!} + \frac{(2\gamma xy)^3}{3!} + \cdots\right) \end{aligned}$$

Notice closely that the exponential expansion above can be rewritten as the dot product between the following two vectors:

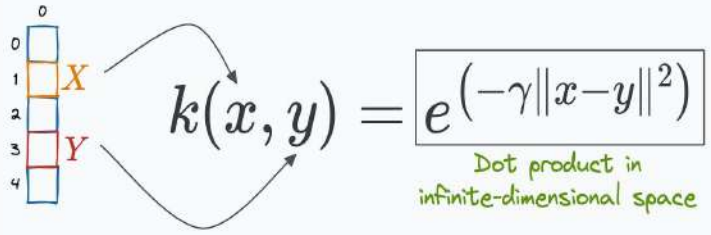


And there we get our projection function:

$$\phi(x) = (1\,,\,\sqrt{2\gamma}\,x\,,\,\sqrt{\frac{(2\gamma)^2}{2!}}\,x^2\,,\,\sqrt{\frac{(2\gamma)^3}{3!}}\,x^3\,,\,\cdots)$$

It is evident that this function maps the 1-dimensional input to an infinite-dimensional feature space.

This shows that the RBF kernel function we chose earlier computes the dot product in an infinite-dimensional space without explicitly visiting that space.



This is why the RBF kernel is considered so powerful, allowing it to easily model highly complex decision boundaries.

Here, I want to remind you that even though the kernel is equivalent to the dot product between two infinite-dimensional vectors, we NEVER compute that dot product, so the computation complexity is never compromised.

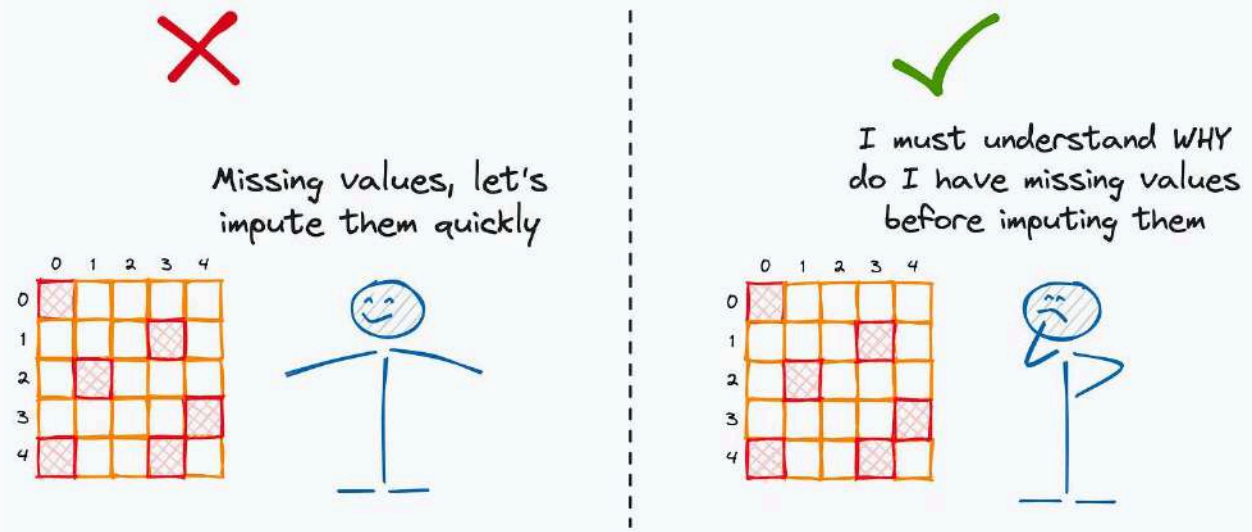
That is why the kernel trick is called a "trick." In other words, it allows us to operate in high-dimensional spaces without explicitly computing the coordinates of the data in that space.

Missing Data

3 Types of Missing Values

After seeing missing values in a dataset, most people jump directly into imputing them. But as counterintuitive as it may sound, the first step towards imputing missing data should NEVER be imputation.

Instead, the first step must be to understand the reason behind data missingness. This is because the data imputation strategy largely depends on WHY data is missing.



More specifically, missingness could be of three types:

- Missing completely at random (MCAR).
- Missing at random (MAR).
- Missing not at random (MNAR).

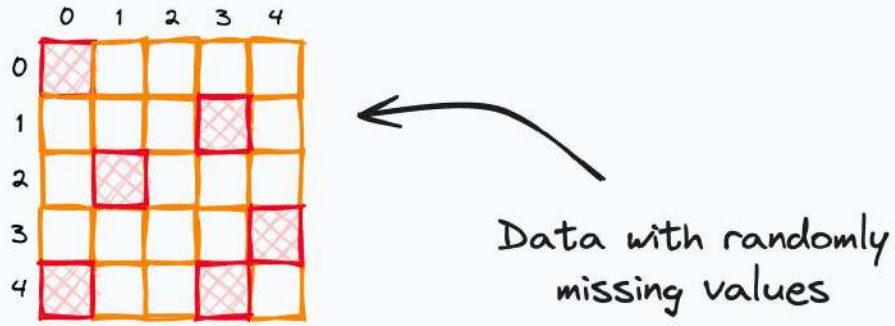
Only when we determine the reason can we proceed with the appropriate imputation techniques.

What are MCAR, MAR, and MNAR?

Let's understand them in this chapter in detail.

1) Missing completely at random (MCAR)

MCAR is a situation in which the data is genuinely missing at random and has no relation to any observed or unobserved variables. In other words, the missing data points follow no discernable pattern.



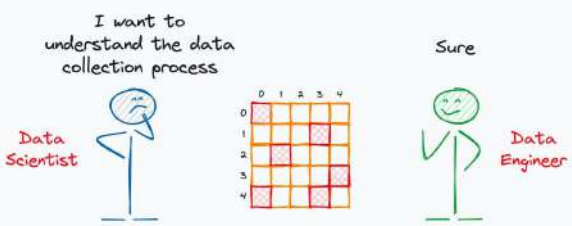
For instance, in survey responses with missing values, assuming MCAR would mean that some participants may have unintentionally skipped to answer any random question.

However, in my experience, MCAR has mostly been an unrealistic assumption for missingness, i.e., data is usually not MCAR for most real-world datasets with missing values. This is because, in real-world scenarios, the occurrence of missing data often tends to be influenced by some factor, either observed or unobserved.

Human behavior, survey administration, or any external events can be motivating factors for missing values to appear in a dataset. For instance, participants may selectively omit sensitive information, or certain groups may be more prone to non-response, causing missingness.

That is why assuming MCAR for missing data is not a sensible assumption unless you know the end-to-end data collection process and/or have domain expertise in.

Here, it becomes essential for data scientists to talk to data engineers and understand the data collection process. In fact, this is not just about MCAR but applicable to the other two situations as well, which we shall discuss shortly.

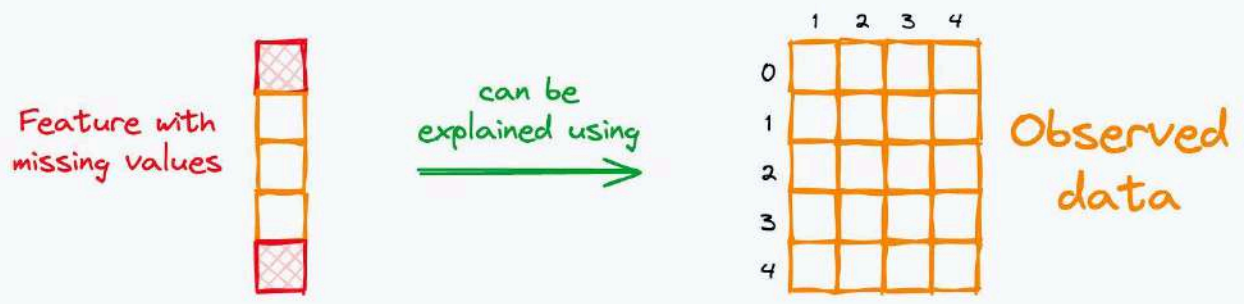


Understanding the data collection process will NEVER hurt.

That said, nothing stops us from assuming that missing values are MCAR if that appears to be a fair thing to do based on the analysis and/or input from the domain experts and/or after understanding the data collection mechanism. You can proceed with the simplest univariate imputation techniques.

2) Missing at random (MAR)

MAR is a situation in which the missingness of one feature can be explained by other observed features in the dataset.

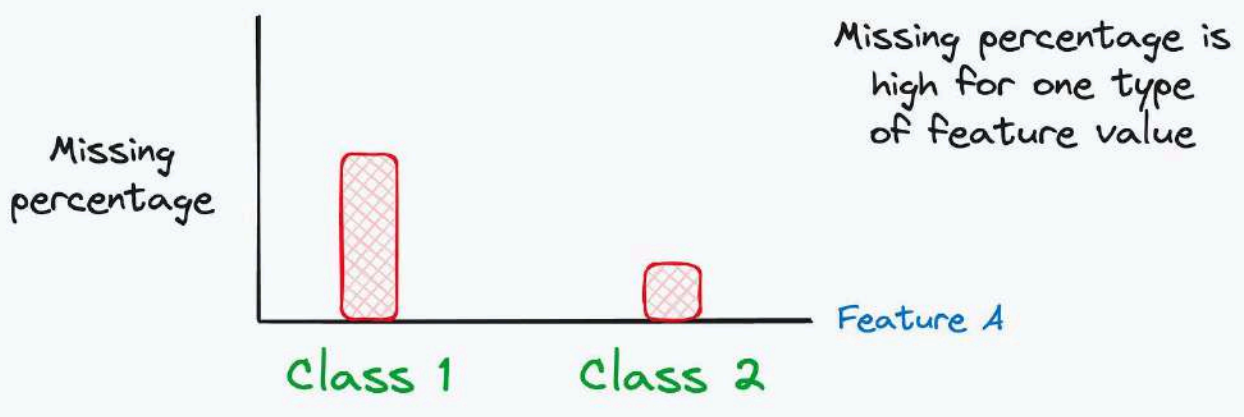


In contrast to MCAR, MAR is more practically observed.

In this case, the missingness can be accounted for through appropriate statistical methods with reasonable accuracy.

Thus, even though the data is missing, its occurrence can still be (somewhat) estimated based on the information available in the dataset.

A common way to determine MAR is by conditioning on another observed features and noticing any increase in the probability of missingness.



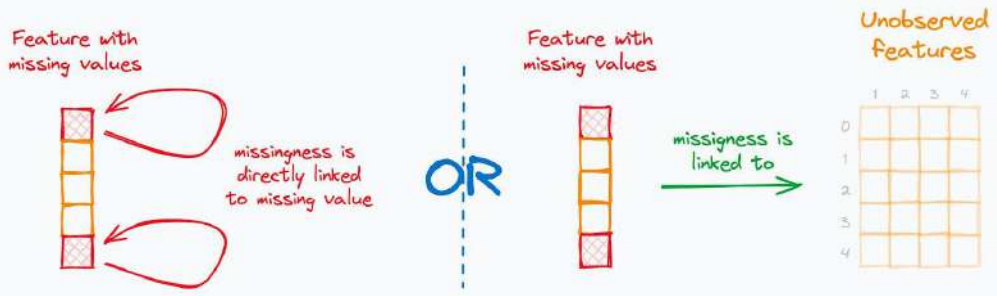
For instance, in an academic performance survey, students with higher grades might be less likely to disclose information about the number of hours they study (due to competition, maybe).

This is where techniques like kNN imputation, Miss Forest, etc., are quite effective. They use other observed features to impute the missing feature.

If you want to learn about these techniques, the chapter is precisely about this topic.

3) Missing not at random (MNAR)

MNAR is the most complicated situation of all three. In MNAR, missingness is either attributed to the missing value itself or the feature(s) that we didn't collect data for.



In other words, within MNAR, there is a definite pattern in missing variables. However, the pattern is unrelated to any observed feature(s). This is different from MCAR where there is no definite pattern.

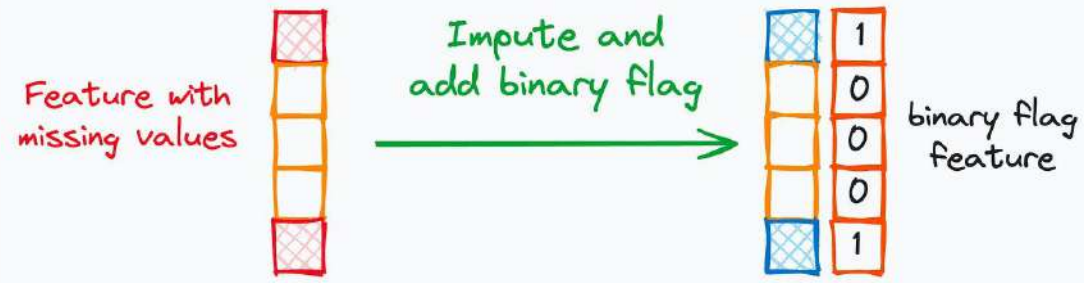
For instance, in a health survey, participants with very high stress levels might consciously choose not to disclose their stress level due to stigma, or fear of judgment. As a result, the missing data on stress level is not random; it is influenced by the stress level itself.

So, in a way, the higher the stress level, the less likely one will disclose it, and the more likely the value will be missing from the collected dataset. Thus, the missingness is directly dependent on the very variable that is missing in the first place. That's tricky, isn't it?

There's not much we can do to address this, except for collecting more data/features. What's more, domain expertise become extremely important to smartly tackle MNAR and improve the data collection process.

At times, I have also preferred proceeding with typical imputation techniques (used in MCAR and MAR) because further data collection can be infeasible in most cases. But as discussed above, there is a definite missingness pattern in MNAR, which can be important. But direct imputation will discard that information.

One way to retain that missingness pattern is by adding a binary feature, indicating whether the feature was imputed.

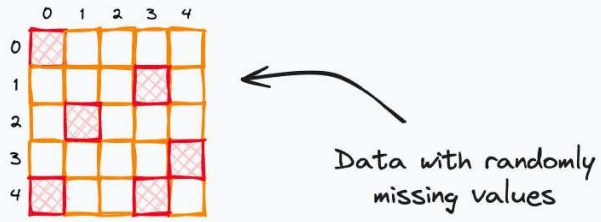


This way, the ML algorithm can still access and learn the missing data patterns.

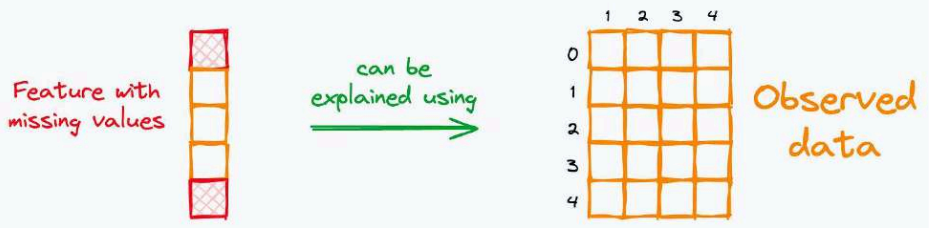
MissForest and kNN Imputation

The imputation strategy for missing data largely depends on the type of missingness, which can be of three types:

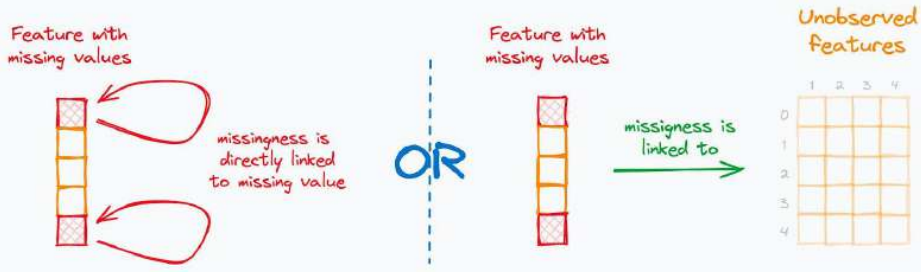
- Missing completely at random (MCAR): Data is genuinely missing at random and has no relation to any observed or unobserved variables.



- Missing at random (MAR): The missingness of one feature can be explained by other observed features in the dataset.



- Missing not at random (MNAR): Missingness is either attributed to the missing value itself or the feature(s) that we didn't collect data for.



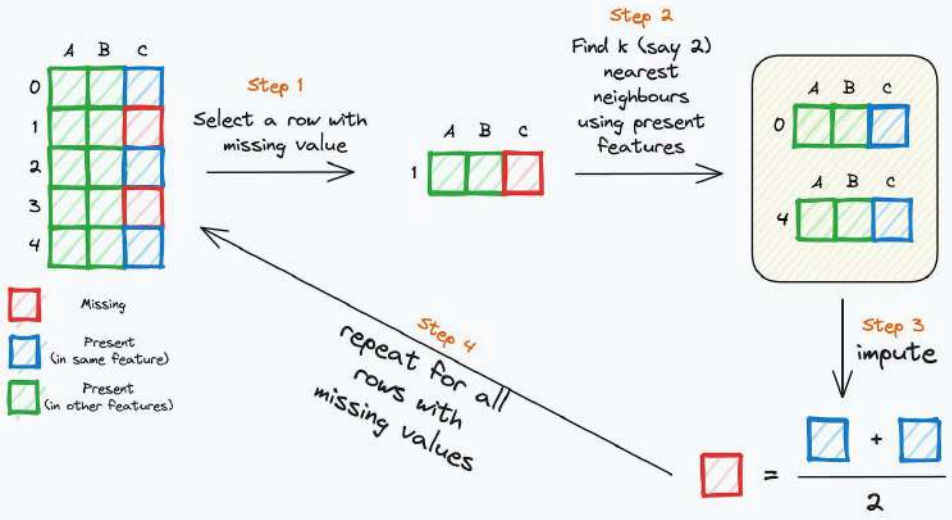
I have observed missing at random (MAR) appearing relatively much more in practical situations than the other two, so in this chapter, I want to share two imputation strategies I typically prefer to use in such a case.

#1) kNN imputation

As the name suggests, it imputes missing values using the k-nearest neighbors algorithm.

More specifically, missing features are imputed by running a kNN on non-missing feature values.

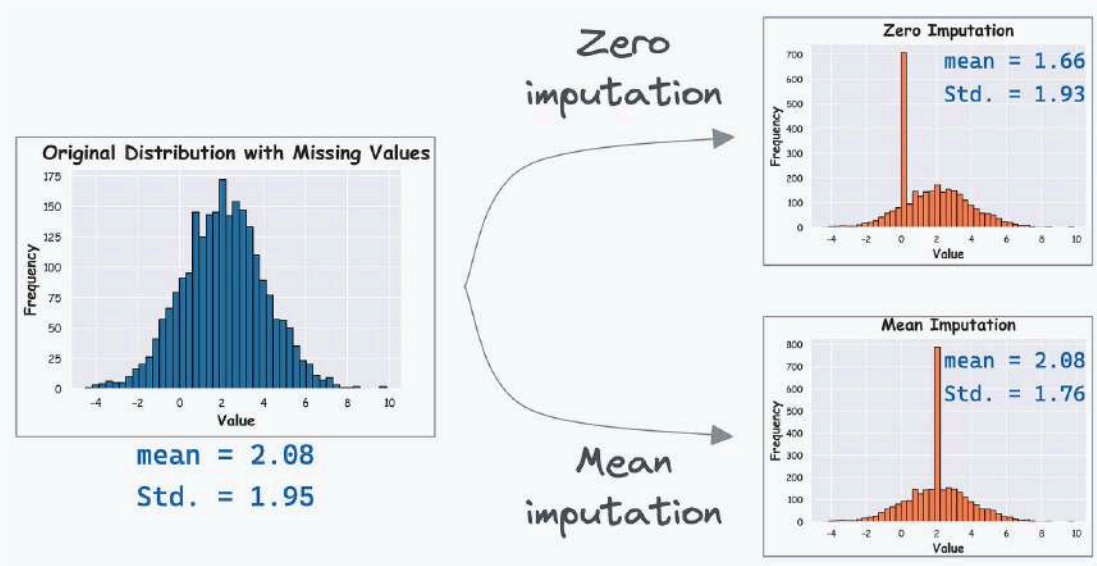
The following image depicts how it works:



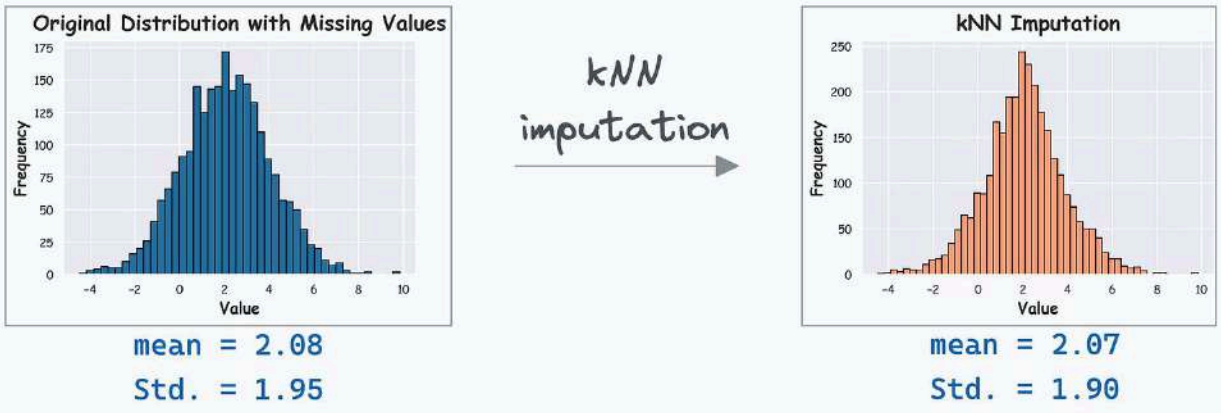
- Step 1: Select a row (r) with a missing value.
- Step 2: Find its k nearest neighbors using the non-missing feature values.
- Step 3: Impute the missing feature of the row (r) using the corresponding non-missing values of k nearest neighbor rows.
- Step 4: Repeat for all rows with missing values.

Its effectiveness over mean/zero imputation is evident from the demo below.

On the left, we have a feature distribution with missing values. Assuming we have already validated that the data was missing at random (MAR), using mean/zero alters the summary statistics and distribution, as depicted below.



However, as depicted below, kNN imputer appears to be more reliable, and it preserves the summary statistics:

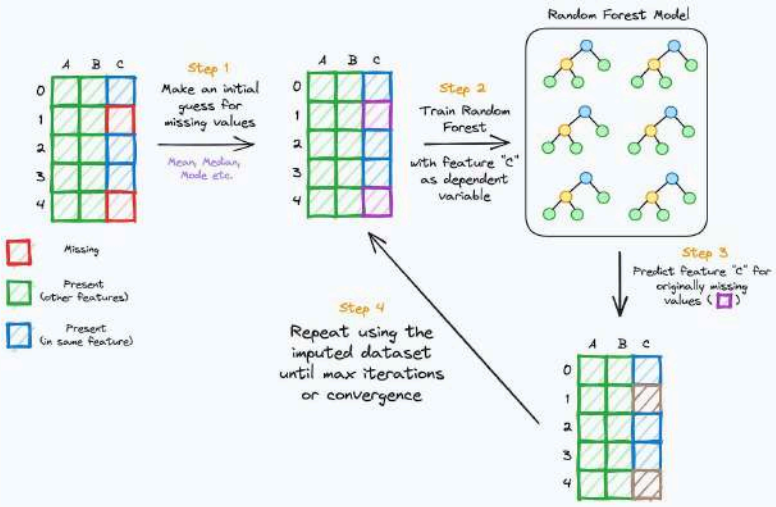


#2) MissForest

Some major issues with kNN imputation are:

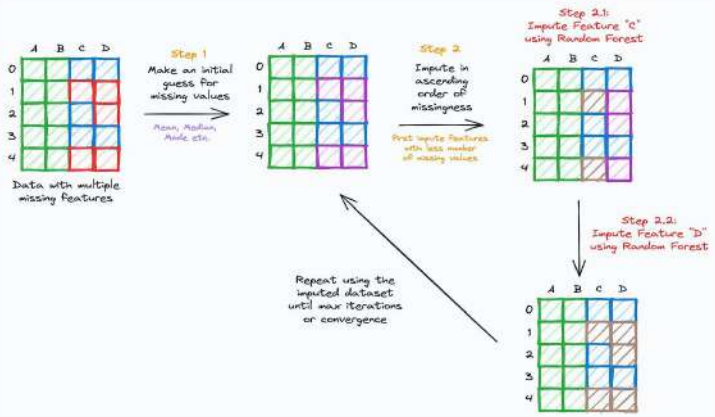
1. It has a high run-time for imputation — especially for high-dimensional datasets.
2. It raises issues with distance calculation in the case of categorical non-missing features.
3. It requires feature scaling, etc.

MissForest imputer is another reliable choice for missing value imputation when data is missing at random (MAR). As the name suggests, it imputes missing values using the Random Forest algorithm.



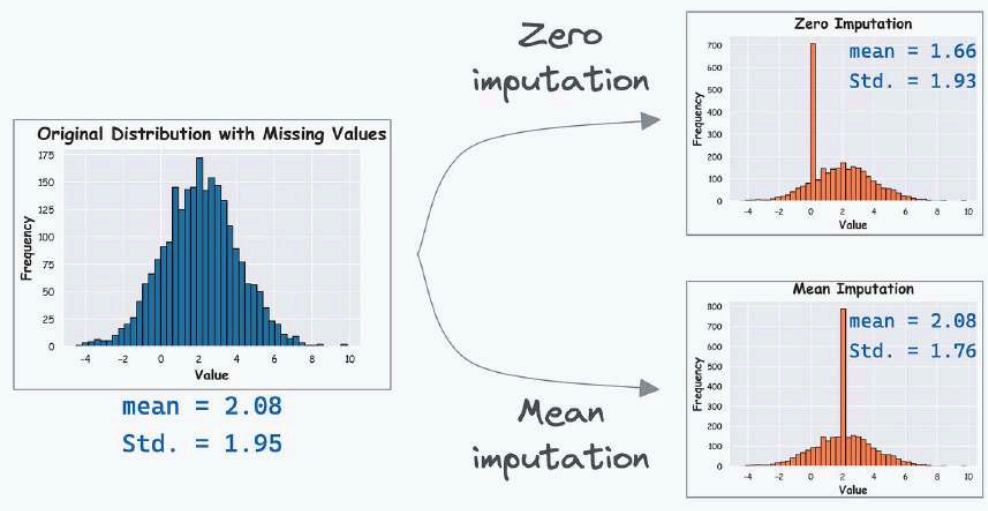
1. To begin, impute the missing feature with a random guess — Mean, Median, etc.
2. Model the missing feature using Random Forest.
3. Impute ONLY originally missing values using Random Forest's prediction.
4. Back to Step 2. Use the imputed dataset from Step 3 to train the next Random Forest model.
5. Repeat until convergence (or max iterations).

In case of multiple missing features, the idea (somewhat) stays the same:

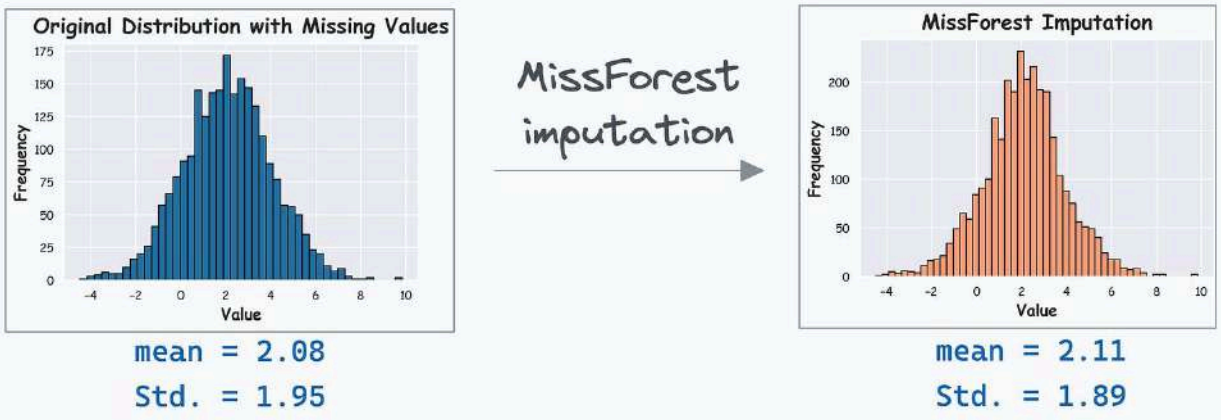


Impute features sequentially in increasing order missingness — features with fewer missing values are imputed first.

Like before, on the left, we have a feature distribution with missing values. Again, assuming we have already validated that the data was missing at random (MAR), using mean/zero alters the summary statistics and distribution:



However, as depicted below, MissForest appears more reliable and preserves the summary statistics.

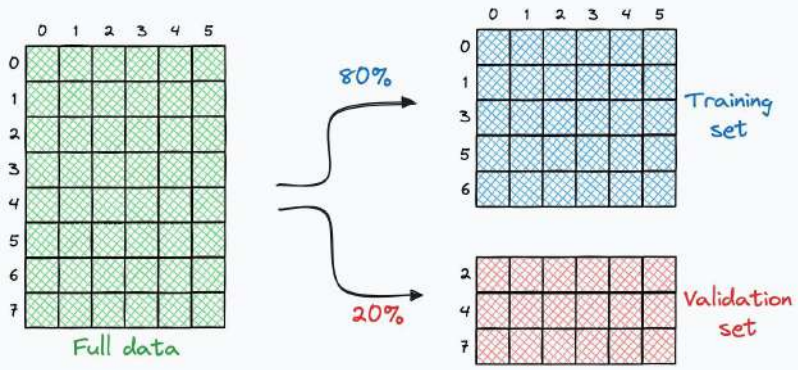


kNN imputation and MissForest Jupyter notebook: https://bit.ly/3RW3i8k.

Pitfalls and Misconceptions

When is Random Splitting Fatal for ML Models?

One thing we hear in almost all introductory ML lessons is to split the given data RANDOMLY into train and validation sets.



Note: Ignore the test set for now.

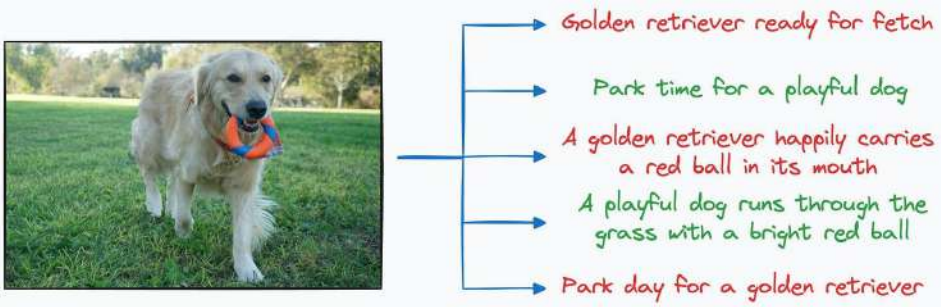
Random splitting makes sense because it ensures that the data is divided without any bias.

However, I have come across many situations where random splitting is fatal for model building. Yet, many people don't realize it.

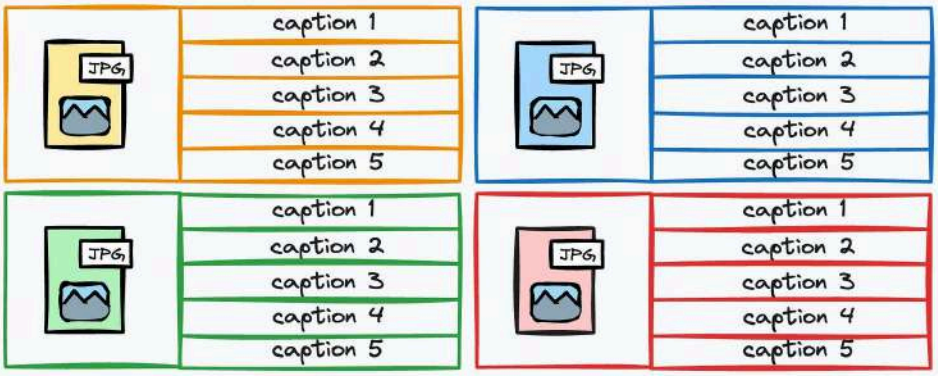
And I am not talking about temporal datasets here.

Scenario

Consider you are building a model that generates captions for images.

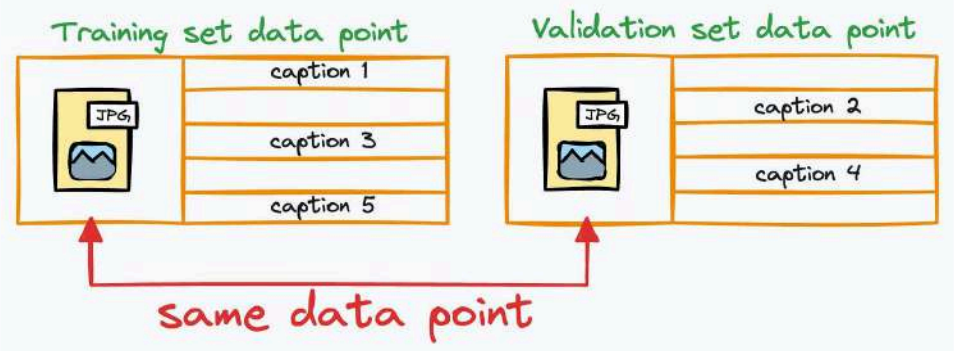


Due to the inherent nature of language, every image can have many different captions:



Now, you might realize what would happen if we randomly split this dataset into train and validation sets.

During the random split, the same data point (image) will be available in the train and validation sets.



This is a typical example of data leakage, which results in high overfitting!

This type of leakage is also known as group leakage.

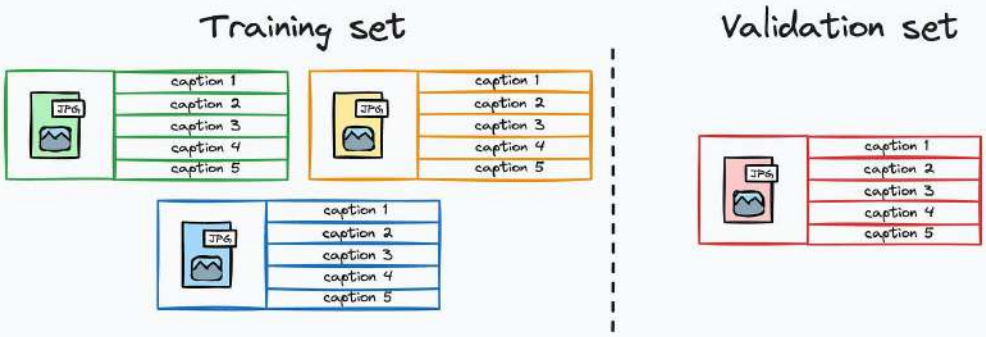
Solution

From the above discussion, it is clear that random splitting is the cause of the problem.

Group shuffle split helps us solve this.

There are two steps:

- Group all training instances corresponding to one image (or features that may result in leakage, any other grouping criteria, etc.).
- After grouping, the whole group must be randomly sent to either the training set or the validation set.



This will prevent the group leakage we witnessed earlier and prevent overfitting.

One thing to note here is that in the above example, all features in the dataset, i.e., the image pixels, contributed to the grouping criteria.

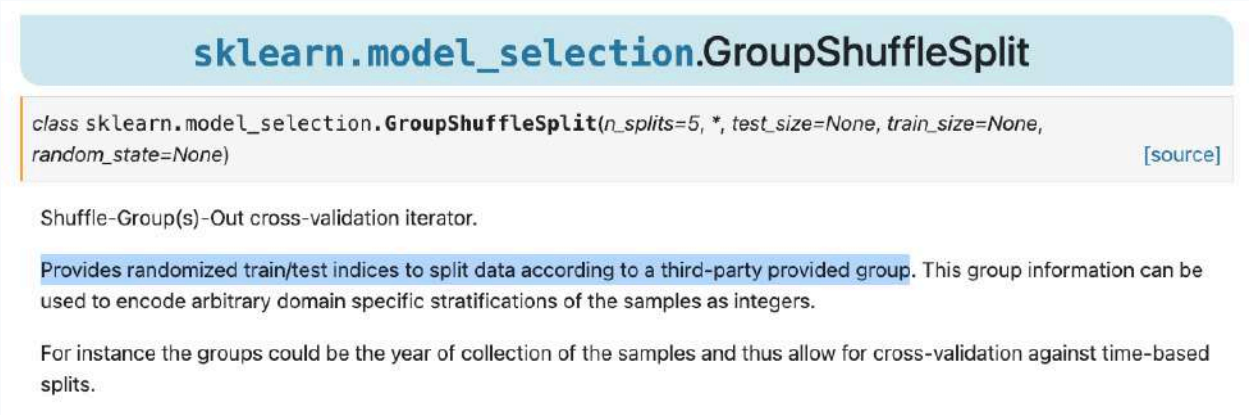
But more generally speaking, there could only be a subset of features that must be grouped together for data splitting.

For instance, consider a dataset containing medical imaging data. Each sample consists of multiple images (e.g., different views of the same patient's body part), and the model is intended to detect the severity of a disease.

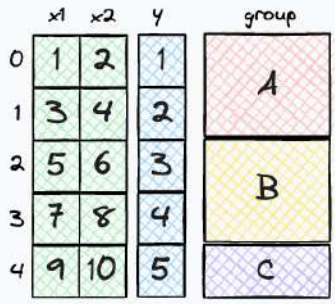
In this case, it is crucial to group all images corresponding to the same patient together and then perform data splitting. Otherwise, it will result in data leakage and the model will not generalize well to new patients.

Demo

If you use Sklearn, the GroupShuffleSplit implements this idea.



Consider we have the following dataset:



- x1 and x2 are the features.
- y is the target variable.
- group denotes the grouping criteria.

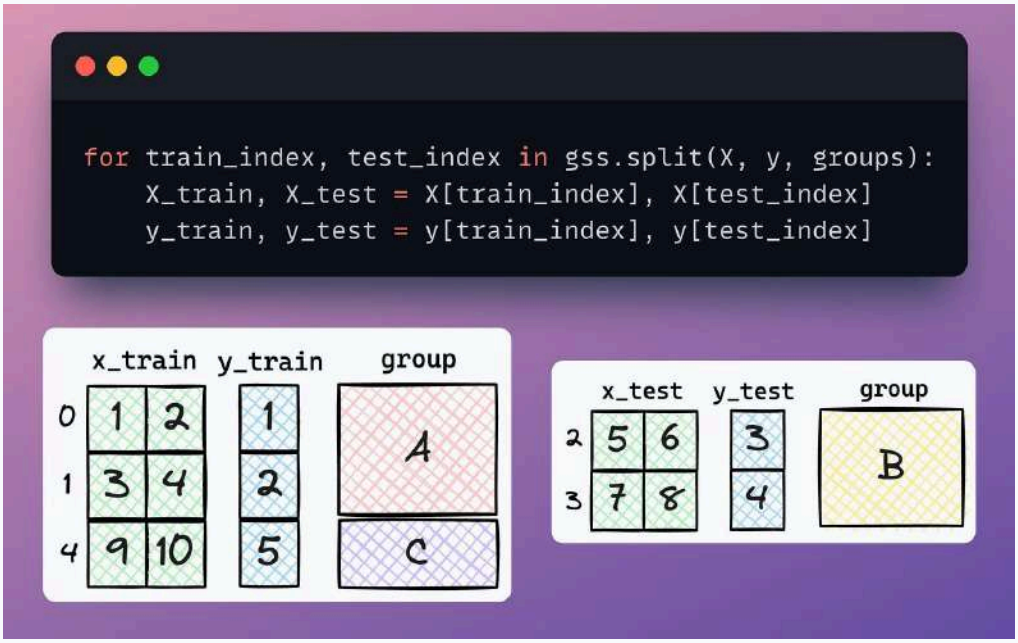
First, we import the GroupShuffleSplit from sklearn and instantiate the object:



The split() method of this object lets us perform group splitting:



This returns a generator, and we can unpack it to get the following output:



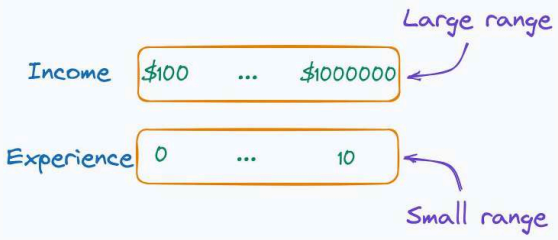
As demonstrated above:

- The data points in groups "A" and "C" are together in the training set.

The data points in group "B" are together in the validation/test set.

Feature Scaling is NOT Always Necessary

Feature scaling is commonly used to improve the performance and stability of ML models. This is because it scales the data to a standard range. This prevents a specific feature from having a strong influence on the model's output.



For instance, in the image above, the scale of income could massively impact the overall prediction. Scaling both features to the same range can mitigate this and improve the model's performance. I am sure you already know this, so we won't get into more detail here. However, have you ever wondered the following:

Is feature scaling always necessary when our dataset's features have a diverse range?

While feature scaling is often crucial, we often overlook whether it is even needed or not. This is because many ML algorithms are unaffected by scale.

This is evident from the image, which depicts the test accuracy of some classification algorithms with and without feature scaling.

Logistic regression (trained using SGD), SVM Classifier, MLP, and kNN do better with feature scaling.

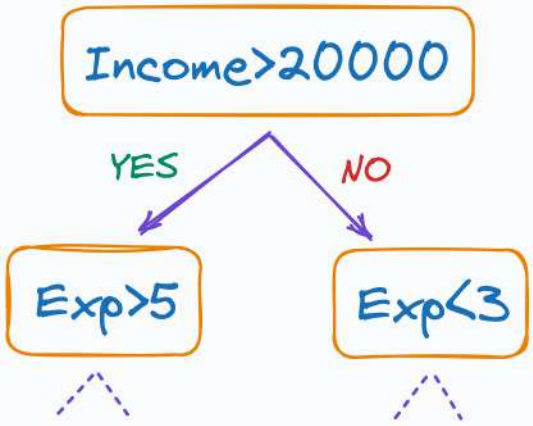
Decision trees, Random forests, Naive bayes, and Gradient boosting are unaffected by scale.

Feature Scaling is NOT Always Necessary

blog.DailyDoseofDS.c

Algorithm	Test Set Classification Performance: Without Feature Scaling	With Feature Scaling	Scaling Required?	
Logistic Regression	0.53	0.70	YES	Better performance with feature scaling
Support Vector Classifier	0.72	0.94	YES	
MLP Classifier	0.73	0.89	YES	
kNN Classifier	0.66	0.93	YES	
Decision Tree	0.83	0.83	NO	Same performance irrespective of feature scaling
Random Forest	0.91	0.91	NO	
Gradient Boosting	0.86	0.86	NO	
Naive Bayes	0.75	0.75	NO	

To understand better, consider a decision tree. It splits the data based on thresholds determined solely by the feature values, regardless of their scale.



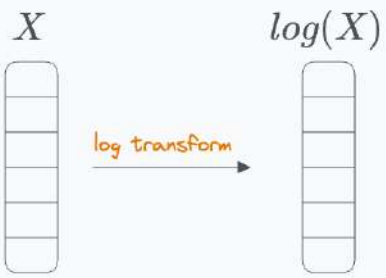
As a result, its performance is unaffected by the scale. This makes intuitive sense as well.

Thus, the takeaway is that when we do feature scaling, it's important to understand not just the nature of our data but also the algorithm we intend to use. We may never need feature scaling if the algorithm is insensitive to the scale of the data.

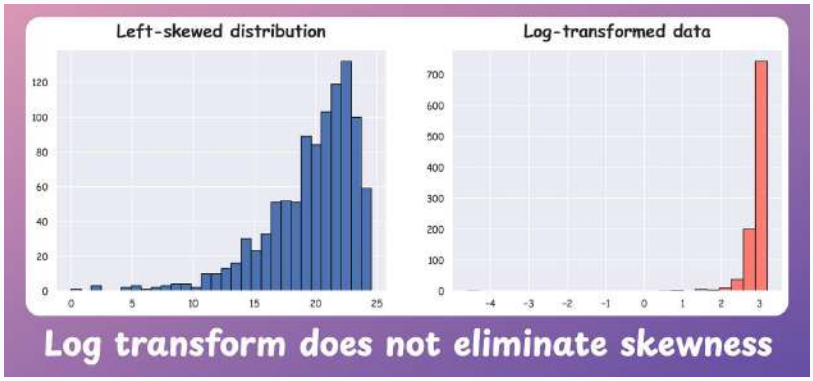
A Misconception About Log Transform

Log transform is commonly used to eliminate skewness in data. Yet, it is not always the ideal solution for eliminating skewness. It is important to note that log transform:

- DOES NOT eliminate left-skewness.
- Only works for right-skewness, that too when the values are small and positive.

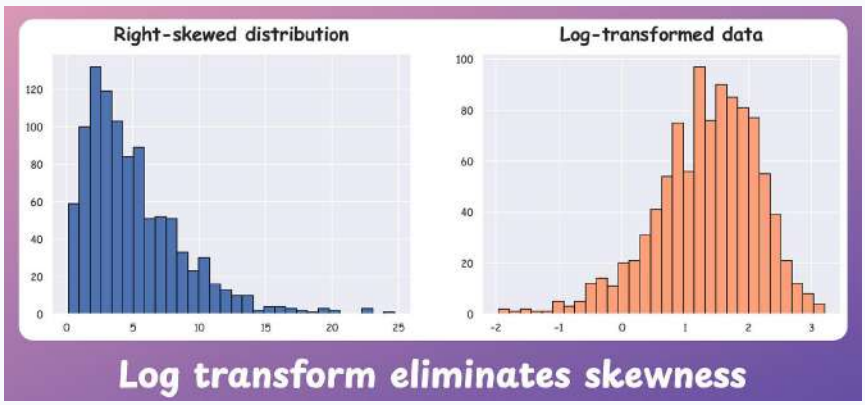


To understand better, consider the left-skewed distribution and its log transform below:



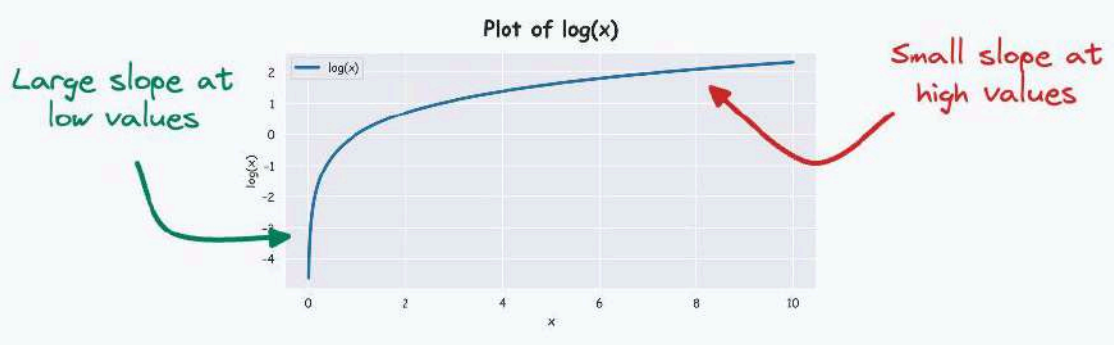
It is clear that log transform did not eliminate skewness.

However, now consider a right-skewed distribution:

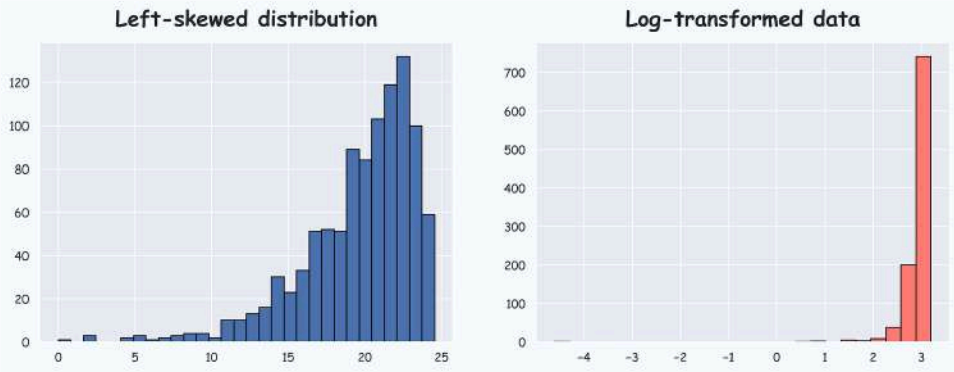


This time, it eliminates the skewness.

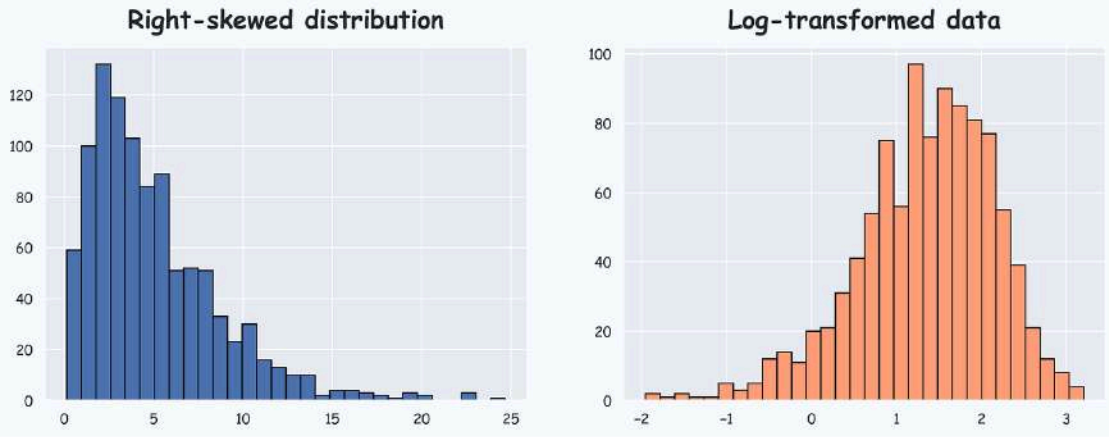
This happens because the log function grows faster for lower values. Thus, it stretches out the lower values more than the higher values.



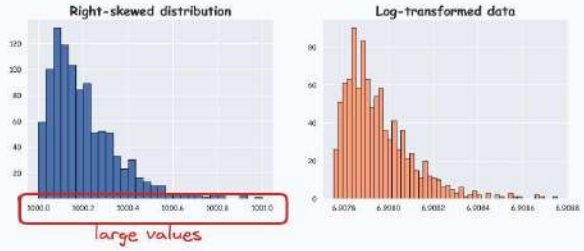
More specifically, in the case of left-skewness, the tail exists to the left, which gets stretched more than the majority of the probability mass that exists to the right. Thus, skewness isn't affected much.



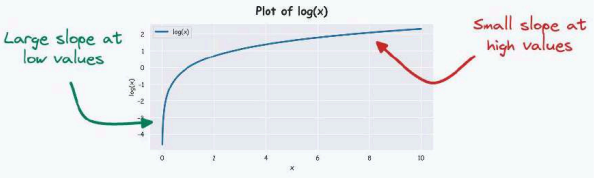
However, in the case of right-skewness, the majority of probability mass exists to the left, which gets stretched out more.



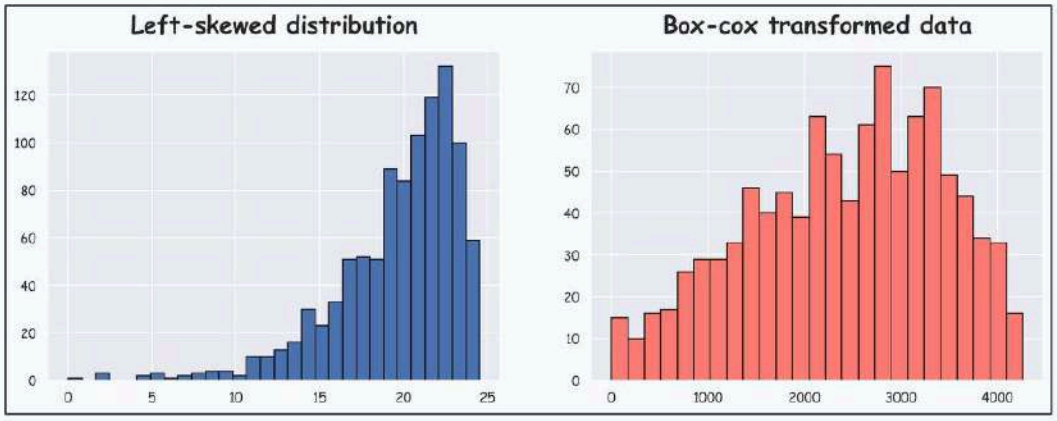
That said, it is important to note that even if we have a right-skewed distribution, log transform will not be effective if the values are large.

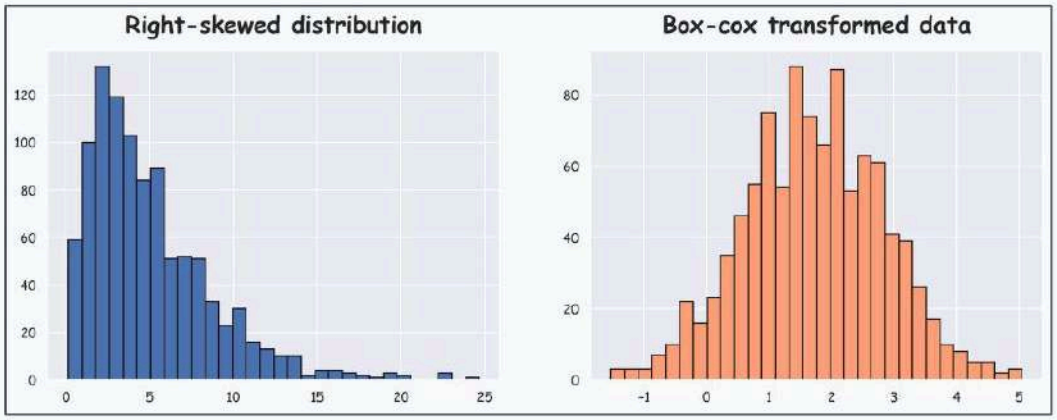


This happens because the stretch effect of the log function diminishes at large values:



I have often found the box-cox transform to be quite effective at eliminating both right-skewness and left-skewness, as depicted below:





The True Purpose of Feature Scaling and Standardization

Feature scaling and standardization are two common ways to alter a feature's range. For instance:

- MinMaxScaler changes the range of a feature to [0,1]:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

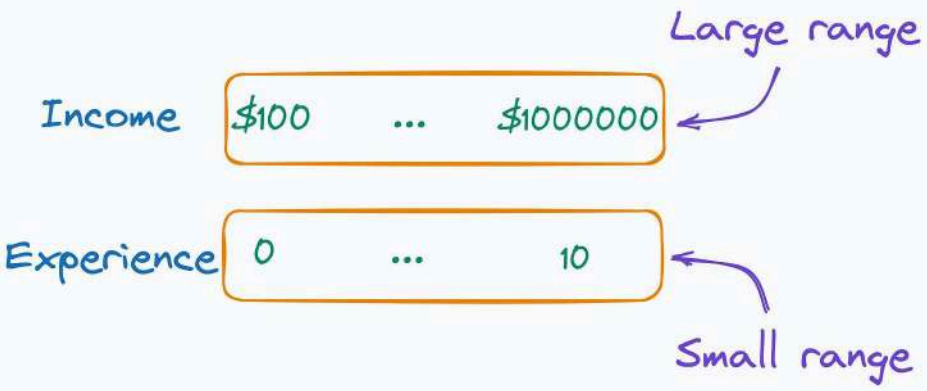
- Standardization makes a feature's mean zero and standard deviation one:

$$z = \frac{x - \mu}{\sigma}$$

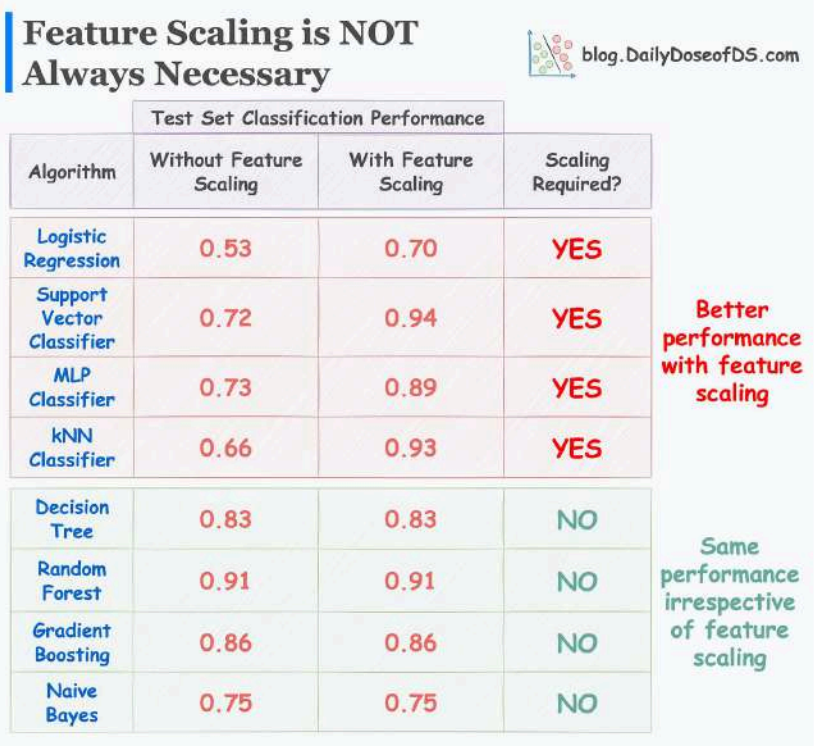
As you may already know, these operations are necessary because:

- They prevent a specific feature from strongly influencing the model's output.
- They ensure that the model is more robust to wide variations in the data.

For instance, in the image below, the scale of "income" could massively impact the overall prediction.



Scaling (or standardizing) the data can mitigate this and improve the model's performance. We also learned this in the feature scaling chapter:

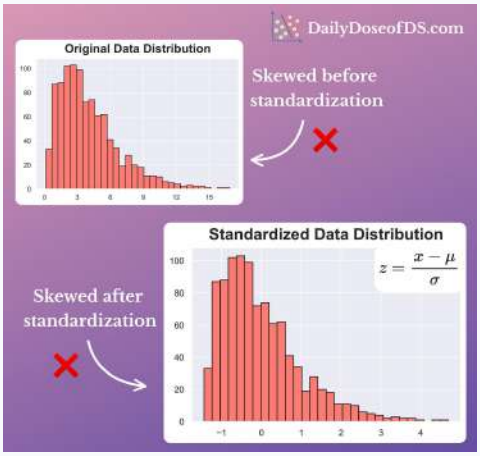


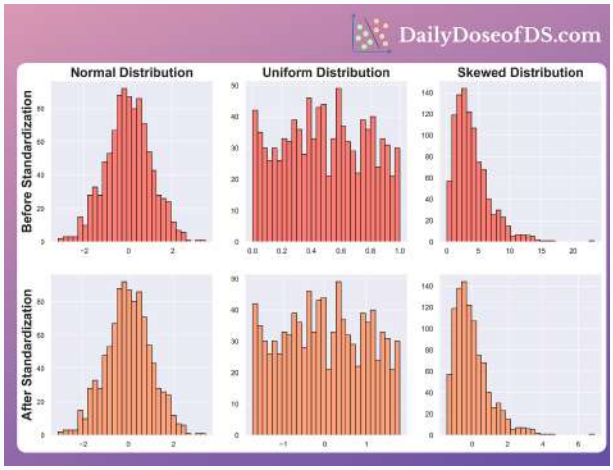
As depicted above, feature scaling is necessary for the better performance of many ML models. So while the importance of feature scaling and standardization is pretty clear and well-known, I have seen many people misinterpreting them as techniques to eliminate skewness.

But contrary to this common belief, feature scaling and standardization NEVER change the underlying distribution. Instead, they just alter the range of values. Thus, after scaling (or standardization):

- Normal distribution → stays Normal
- Uniform distribution → stays Uniform
- Skewed distribution → stays Skewed
- and so on…

We can also verify this from the two illustrations below:





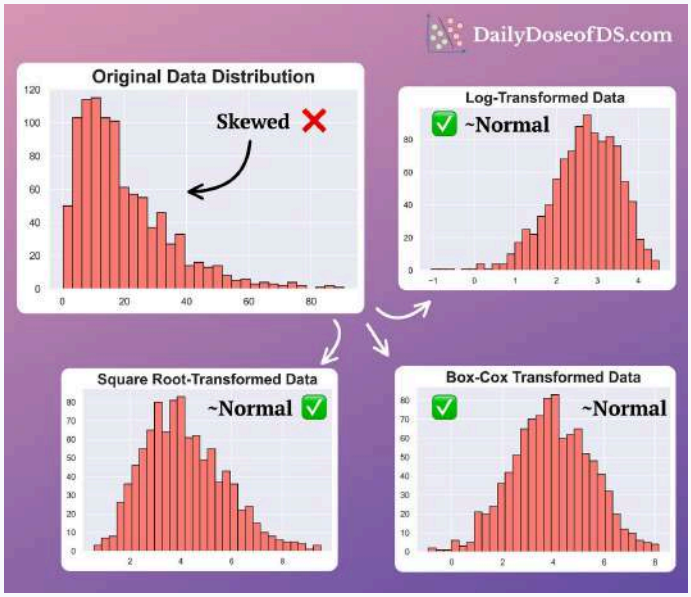
It is clear that scaling and standardization have no effect on the underlying distribution. Thus, always remember that if you intend to eliminate skewness, scaling/standardization will never help. Try feature transformations instead. There are many of them, but the most commonly used transformations are:

- Log transform
- Sqrt transform
- Box-cox transform

Their effectiveness is evident from the image below:

As depicted in this image, applying these operations transforms the skewed data into a (somewhat) normally distributed variable.

Before I conclude, please note that while log transform is commonly used to eliminate data skewness, it is not always the ideal solution.



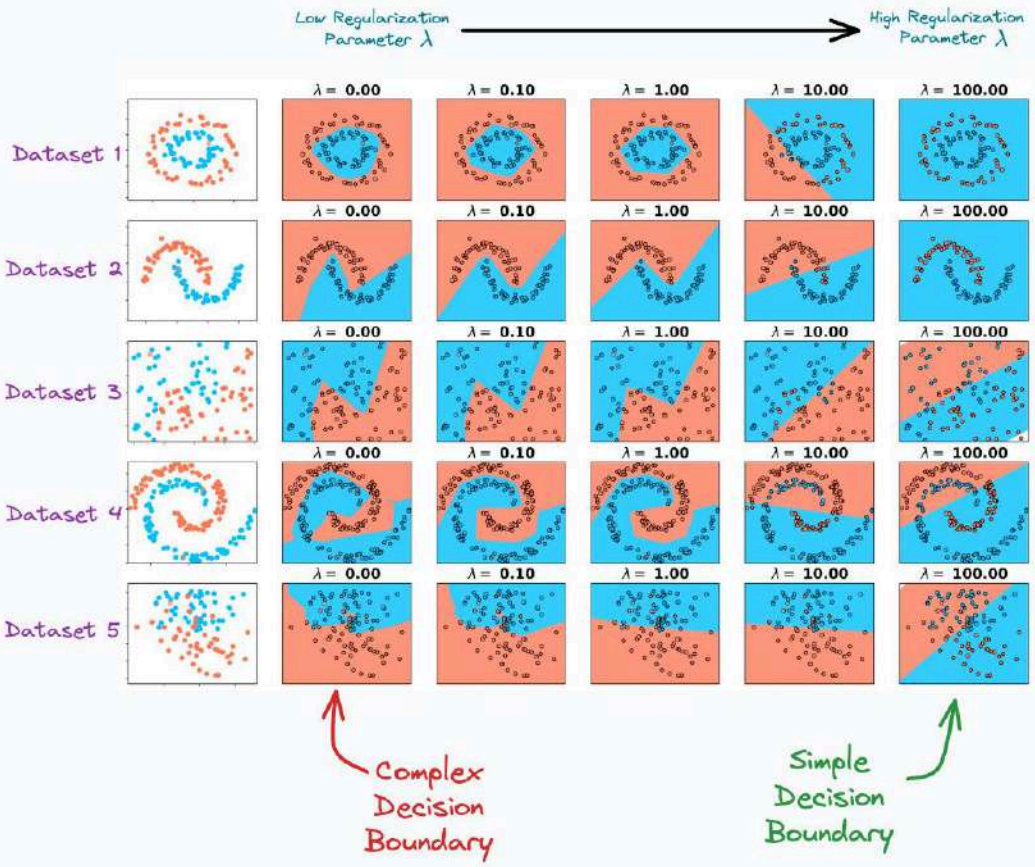
L2 Regularization is Not Just Used for Regularization

Almost every tutorial/course/blog mentioning L2 regularization I have seen talks about just one thing:

L2 regularization is a machine learning technique that avoids overfitting by introducing a penalty term into the model's loss function based on the squares of the model's parameters.

In classification tasks, for instance, increasing the effect of regularization will produce simpler decision boundaries.

Of course, the above statements are indeed correct, and I am not denying that. In fact, we can also verify this from the diagram below:

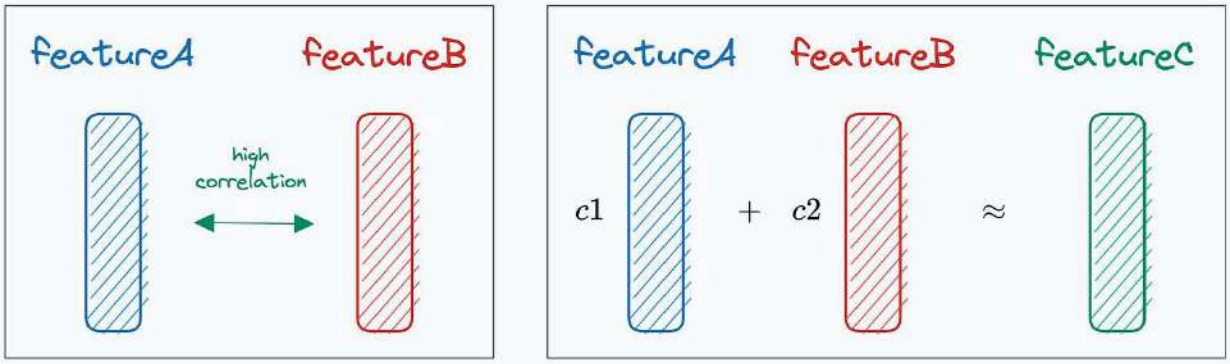


In the image above, as we move to the right, the regularization parameter increases, and the model creates a simpler decision boundary on all 5 datasets.

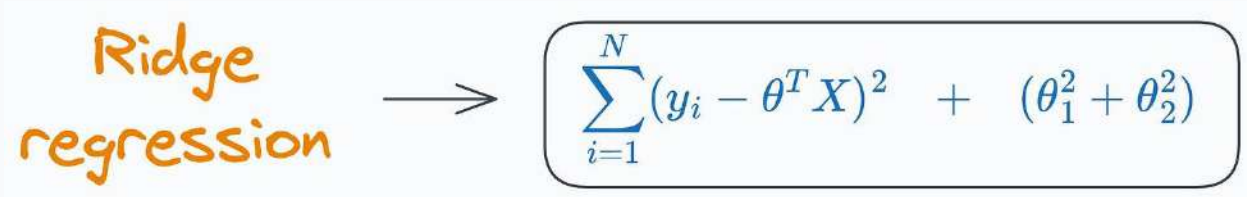
Coming back to the topic...

However, what disappoints me the most is that most resources don't point out that L2 regularization is a great remedy for multicollinearity.

Multicollinearity arises when two (or more) features are highly correlated OR two (or more) features can predict another feature:



When we use L2 regularization in linear regression, the algorithm is also called Ridge regression.



But how does L2 regularization eliminate multicollinearity?

In this chapter, let's build a demonstrative intuition into this topic, which will also explain why "ridge regression" is called "ridge regression."

Dummy dataset

For demonstration purposes, consider this dummy dataset of two features:

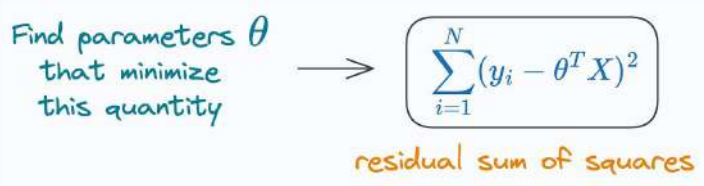


As shown above, we have intentionally made featureB highly correlated with featureA. This gives us a dummy dataset to work with.

Going ahead, we shall be ignoring any intercept term for simplicity.

Linear regression without L2 penalty

During regression modeling, the goal is to determine those specific parameters (θ_1, θ_2), which minimizes the residual sum of squares (RSS):

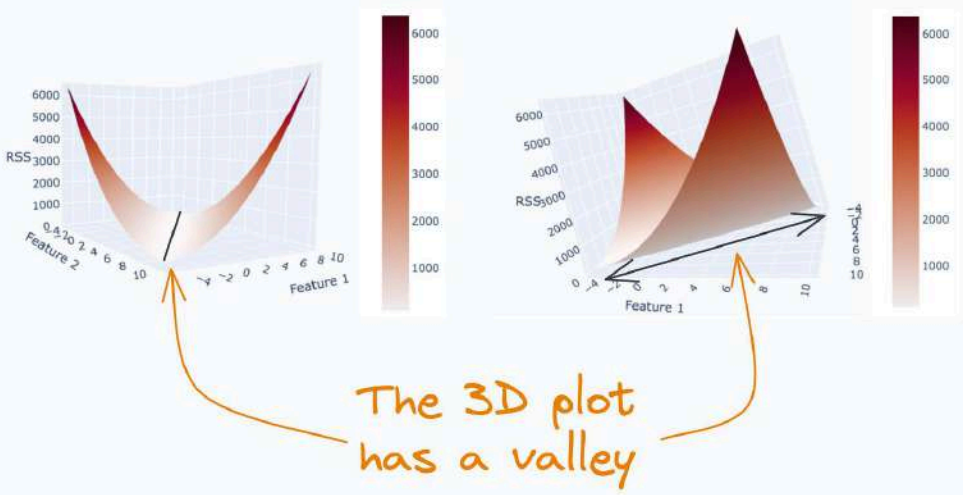


So how about we do the following:

- We shall plot the RSS value for many different combinations of (θ_1, θ_2) parameters. This will create a 3D plot:
  - x-axis → θ_1
  - y-axis → θ_2
  - z-axis → RSS value
- Next, we shall visually assess this plot to locate those specific parameters (θ_1, θ_2) that minimize the RSS value.

Let's do this.

Without the L2 penalty, we get the following plot (it's the same plot but viewed from different angles):



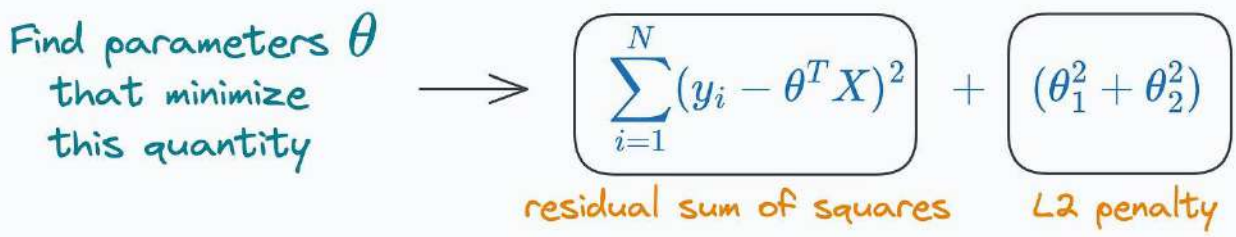
Did you notice something?

The 3D plot has a valley. There are multiple combinations of parameter values (θ_1, θ_2) for which RSS is minimum.

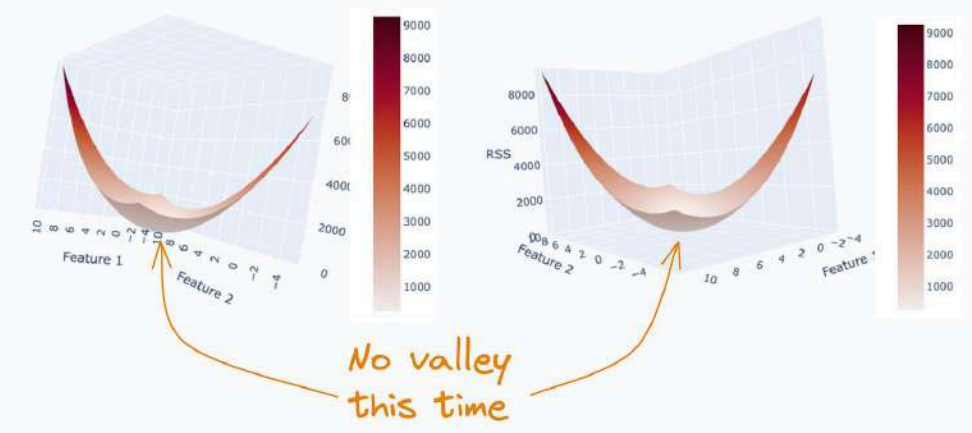
Thus, obtaining a single value for the parameters (θ_1, θ_2) that minimize the RSS is impossible.

Linear regression with L2 penalty

When using an L2 penalty, the goal is to minimize the following:



Creating the same plot again as we did above, we get the following:



Did you notice something different this time?

As depicted above, using L2 regularization removes the valley we saw earlier and provides a global minima to the RSS error.

Now, obtaining a single value for the parameters (θ_1, θ_2) that minimizes the RSS is possible.

Out of nowhere, L2 regularization helped us eliminate multicollinearity.

Why the name "ridge regression"?

In fact, this is where "ridge regression" also gets its name from — it eliminates the ridge in the likelihood function when the L2 penalty is used.

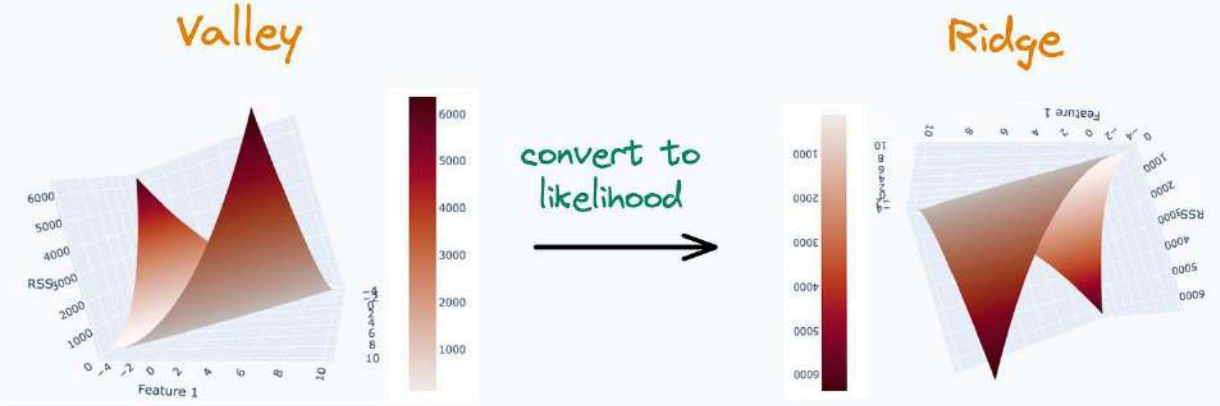


Of course, in the demonstrations we discussed earlier, we noticed a valley, not a ridge.

However, in that case, we were considering the residual sum of error — something which is minimized to obtain the optimal parameters.

Thus, the error function will obviously result in a valley.

If we were to use likelihood instead — something which is maximized, it would (somewhat) invert the graph upside down and result in a ridge instead:



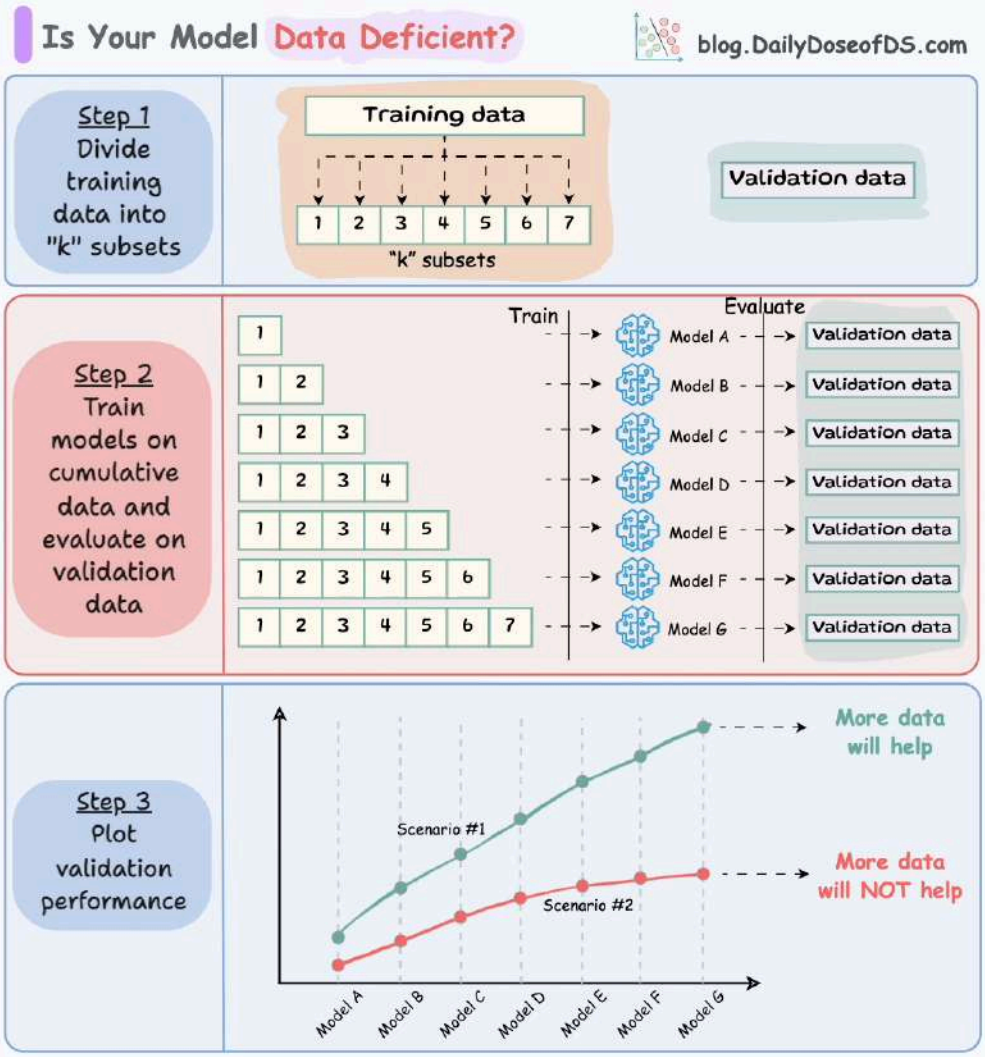
Apparently, while naming the algorithm, the likelihood function was considered.

And that is why it was named "ridge regression."

When I first learned about this some years back, I literally had no idea that such deep thought went into naming ridge regression.

Miscellaneous

Is Your Model Data Deficient?

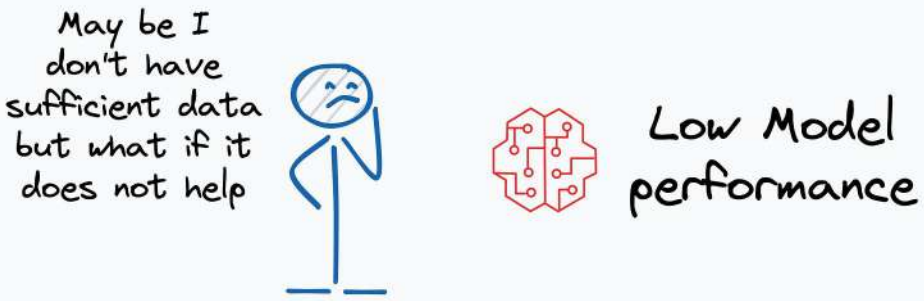


During model development, many people find themselves in situations where, no matter how much they try, the model performance barely improves:

- Feature engineering is giving marginal improvement.
- Trying different models does not produce satisfactory results either.
- and more...

This is usually (not always) an indicator that the model is data deficient.

In other words, we don't have enough data to work with.



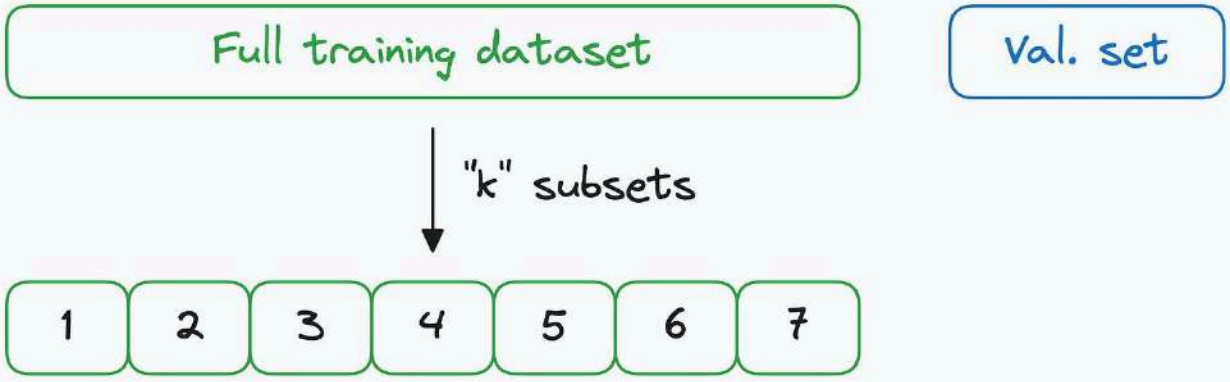
However, gathering new data can be a time-consuming and tedious process. So before venturing into that direction, it would be good to get some insights about whether new data will help. Here's a trick I have often used to determine this.

Data subsetting and Model building

Let's say this is your full training and validation set:

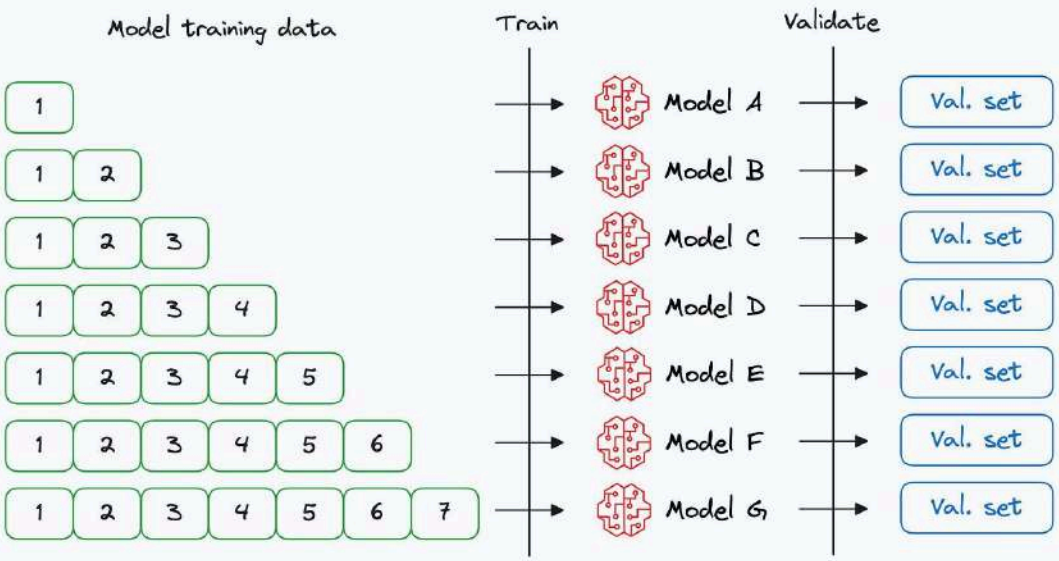


Divide the training dataset into "k" equal parts. The validation set remains as is.



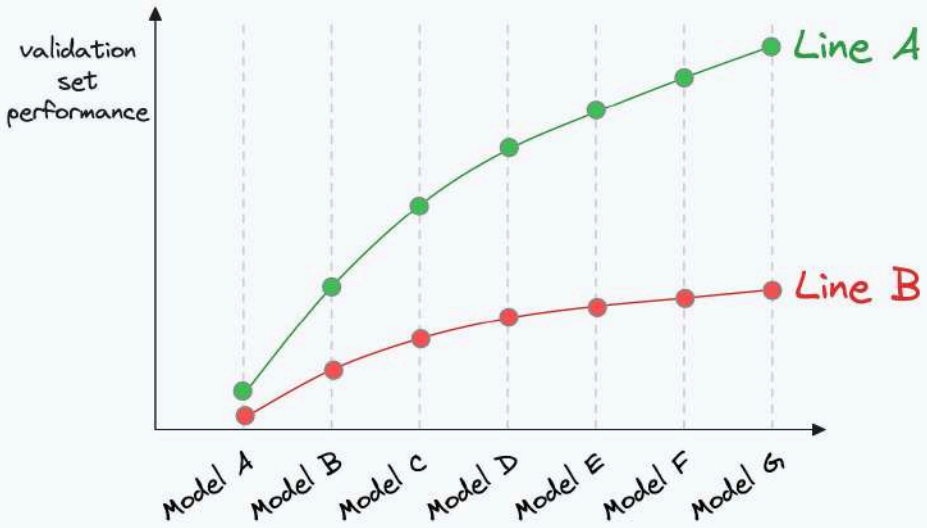
"k" does not have to be super large. Any number between 7 to 12 is fine depending on how much data you have. If there's plenty of data, setting a low value in this range is recommended (you will understand why shortly).

Next, train models cumulatively on the above subsets and measure their performance on the validation set:



- Train a model on the first subset and evaluate the validation set.
- Train a model on the first two subsets and evaluate the validation set.
- Train a model on the first three subsets and evaluate the validation set.
- And so on...

Plotting the validation performance of these models (in order of increasing training data) is likely to produce two types of lines:



- Line A conveys that adding more data is likely to increase model performance.
- Line B conveys that the model performance has already saturated. Adding more data will most likely not result in any considerable gains.

Now, you might also understand why I mentioned this above: "If there's plenty of data, setting a low value in this range is recommended."

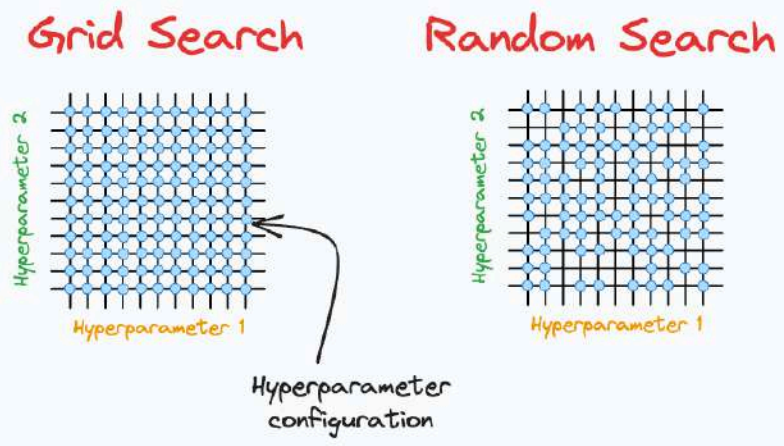
- Because we train multiple models, setting a high value of "k" means more subsets, which in turn means more models.

This way, you can determine whether the model is data deficient and whether gathering data will be helpful or not.

Bayesian Optimization

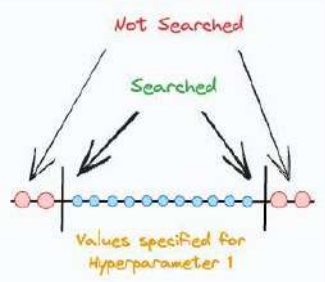
Hyperparameter tuning is a tedious and time-consuming task in training ML models.

Typically, we use two common approaches for this – Grid search and Random search.



But they have many limitations. For instance:

- Grid search performs an exhaustive search over all combinations. This is computationally expensive.
- Grid search and random search are restricted to the specified hyperparameter range. Yet, the ideal hyperparameter may exist outside that range.



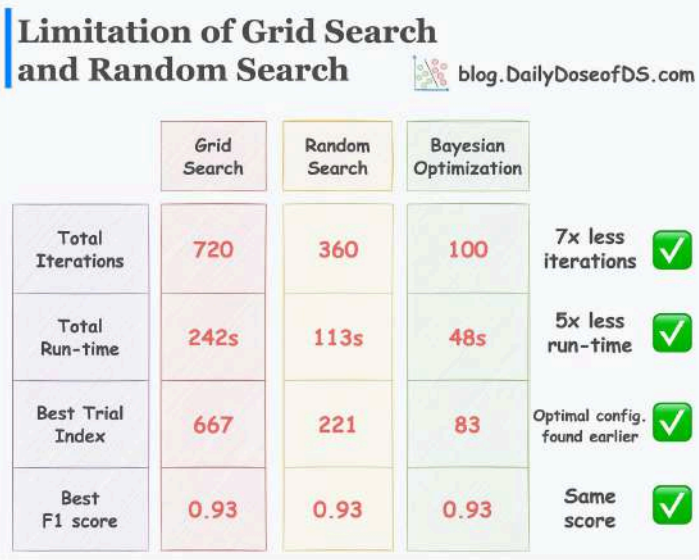
- They can ONLY perform discrete searches, even if the hyperparameter is continuous.

To this end, Bayesian Optimization is a highly underappreciated yet immensely powerful approach for tuning hyperparameters.

It uses Bayesian statistics to estimate the distribution of the best hyperparameters. Here's how it differs from Grid search and Random Search:

Both Grid search and Random Search evaluate every hyperparameter configuration independently. Thus, they iteratively explore all hyperparameter configurations to find the most optimal one.

However, Bayesian Optimization takes informed steps based on the results of the previous hyperparameter configurations. This lets it confidently discard non-optimal configurations. Consequently, the model converges to an optimal set of hyperparameters much faster. The efficacy of Bayesian Optimization is evident from the image.



Bayesian optimization leads the model to the same F1 score but:

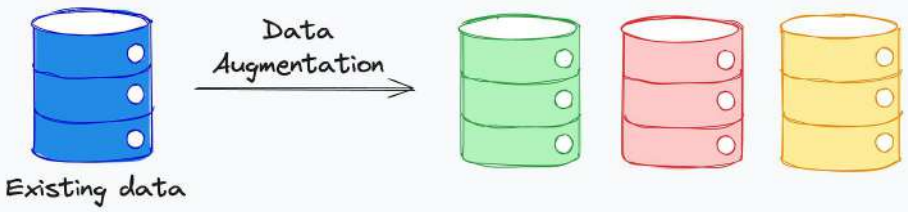
- it takes 7x fewer iterations
- it executes 5x faster
- it reaches the optimal configuration earlier

Pretty cool, isn't it?

Train and Test-time Data Augmentation

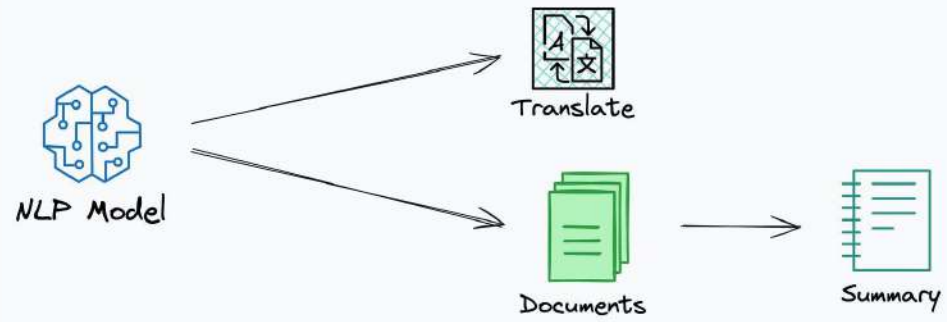
Data augmentation strategies are typically used during training time.

The idea is to use some clever techniques to create more data from existing data, which is especially useful when you don't have much data to begin with:



Let me give you an example.

These days, language-related ML models have become quite advanced and general-purpose. The same model can translate, summarize, identify speech tags (nouns, adjectives, etc.), and much more.



But earlier, models used to be task-specific (we have them now as well, but they are fewer than we used to have before).

- A dedicated model that would translate.
- A dedicated model that would summarize, etc.

In one particular use case, I was building a named entity recognition (NER) model, and the objective was to identify named entities.

An example is shown below:



I had minimal data — around 8-10k labeled sentences. The dataset was the CoNLL 2003 NER dataset if you know it.

Here's how I approached data augmentation in this case.

Observation: In NER, the factual correctness of the sentences does not matter.

Revisiting the above example, it would not have mattered if I had the following sentence in the training data:



The sentence is factually incorrect, of course, but that does not matter.

The only thing that matters to the model is that the output labels (named entity tags in this case) must be correct.

So using this observation, I created many more sentences by replacing the named entities in an existing sentence with other named entities in the whole dataset:



For such substitutions, I could have used named entities from outside. However, it was important to establish a fair comparison with other approaches.

This technique (along with a couple more architectural tweaks) resulted in state-of-the-art performance.

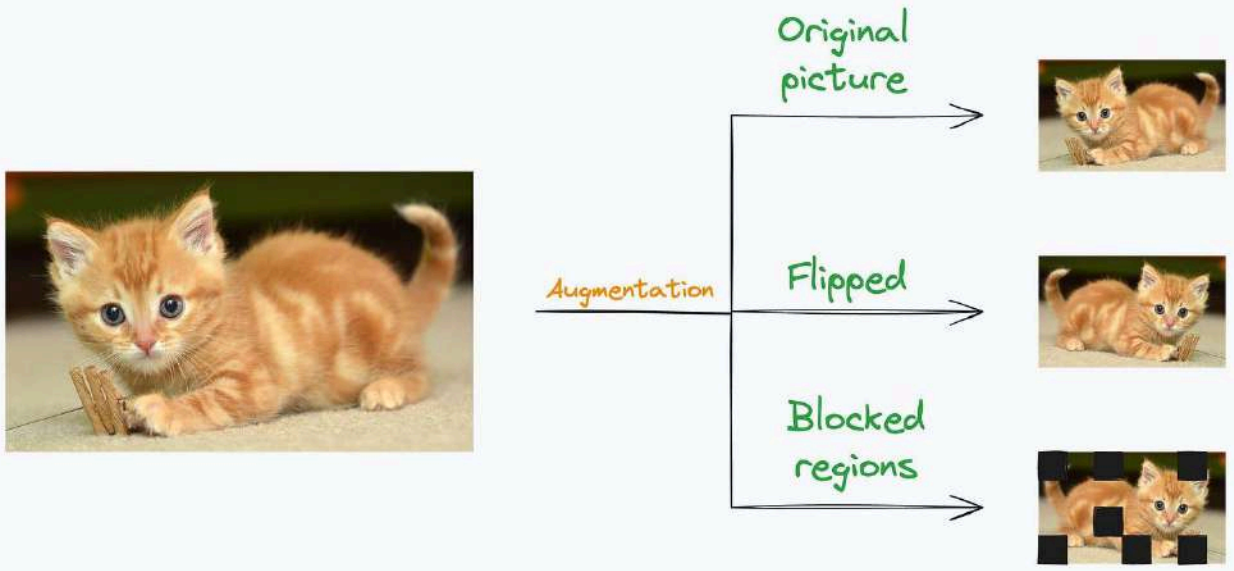
Moving on...

The above discussion was about training data augmentation.

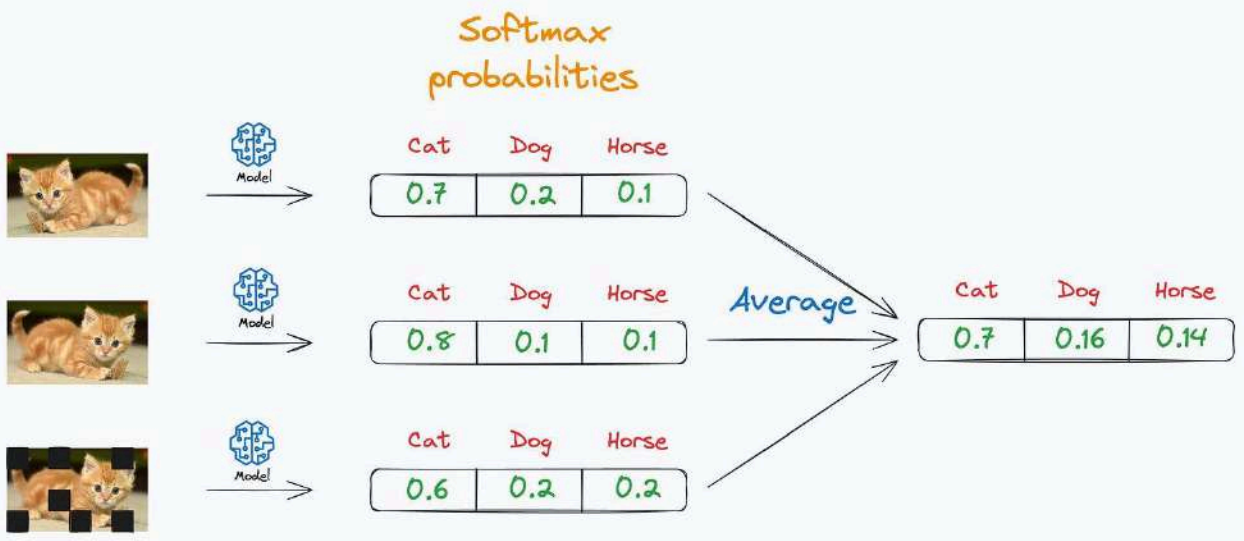
But there's also test-time augmentation.

Test Time Augmentation (TTA) is when we apply data augmentation during testing.

More specifically, instead of showing just one test example to the model, we show multiple versions of the test example by applying different operations.



The model makes probability predictions for every version of the test example, which are then averaged to generate the final prediction:



Essentially, TTA creates an ensemble of predictions by considering multiple augmented versions of the same input, which leads to a more robust final prediction.

In fact, in this paper (https://arxiv.org/html/2402.06892v1), the authors proved that the average model error with TTA never exceeds the average error of the original model, which is great.

As you may have guessed, the only catch is that it increases the inference time.

- Data augmentation takes some time.
- Generating multiple predictions increases the overall prediction run-time.

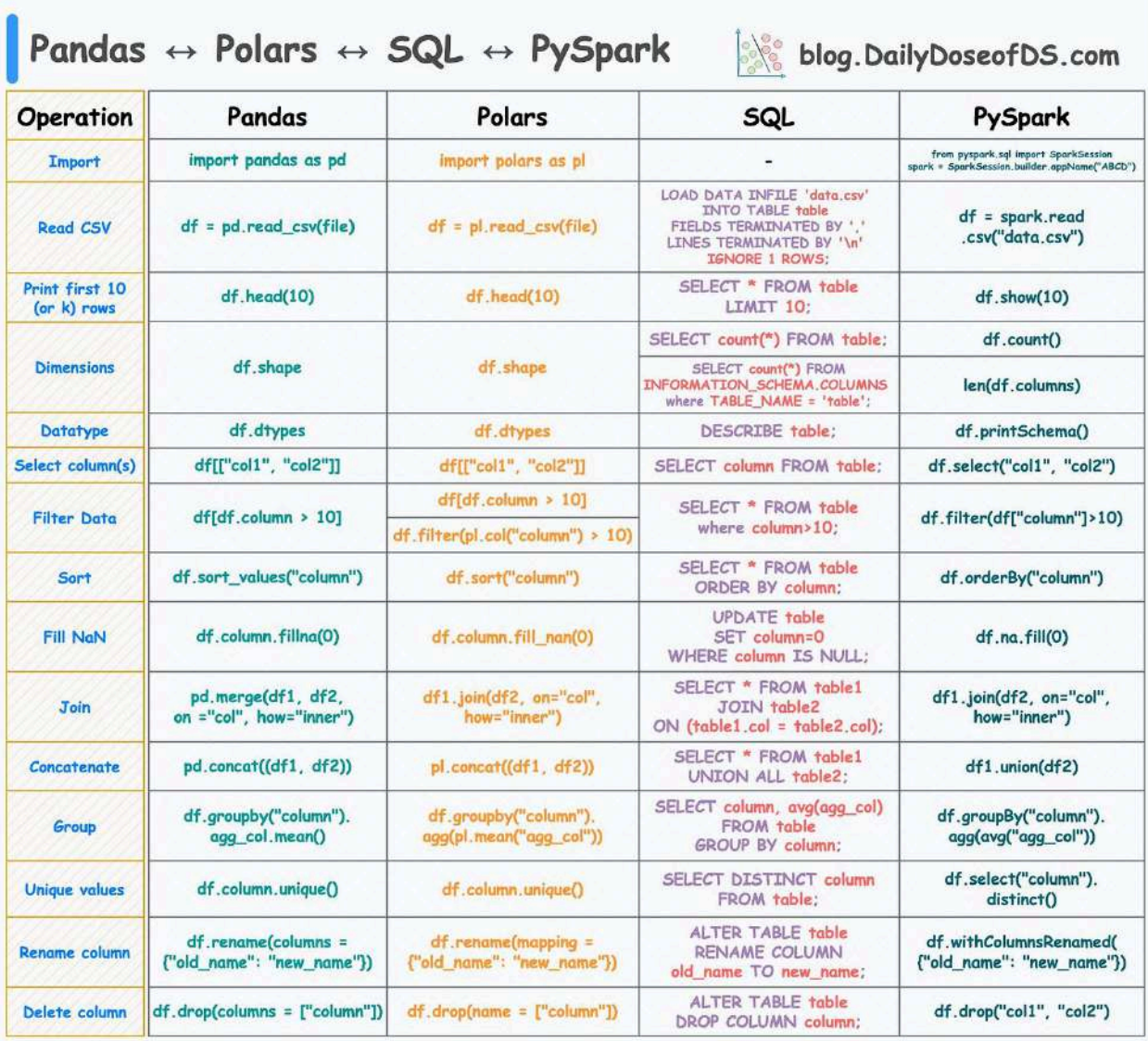
So, when a low inference time is important to you, think twice about TTA.

To summarize, if you can compromise a bit on inference time, TTA can be a powerful way to improve predictions from an existing model without having to engineer a better model.

Data Analysis

15 Pandas ⇔ Polars ⇔ SQL ⇔ PySpark Translations

The following visual depicts the 15 most common tabular operations in Pandas and their corresponding translations in SQL, Polars, and PySpark.

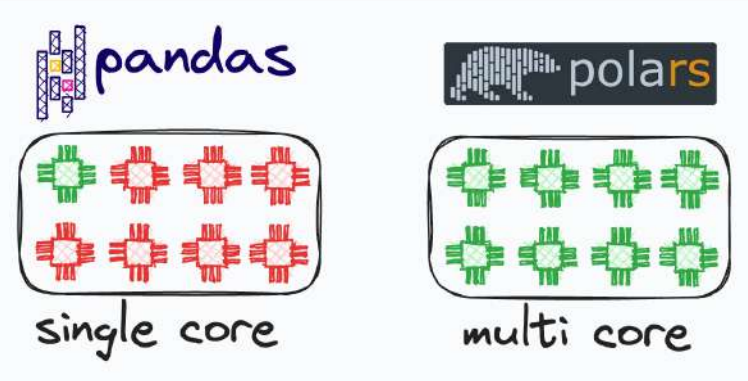


Operation	Pandas	Polars	SQL	PySpark
Import	import pandas as pd	import polars as pl	-	from pyspark.sql import SparkSession spark = SparkSession.builder.appName("ABCD")
Read CSV	df = pd.read_csv(file)	df = pl.read_csv(file)	LOAD DATA INFILE 'data.csv' INTO TABLE table FIELDS TERMINATED BY ',' LINES TERMINATED BY '\n' IGNORE 1 ROWS;	df = spark.read .csv("data.csv")
Print first 10 (or k) rows	df.head(10)	df.head(10)	SELECT * FROM table LIMIT 10;	df.show(10)
Dimensions	df.shape	df.shape	SELECT count(*) FROM table;	df.count()
			SELECT count(*) FROM INFORMATION_SCHEMA.COLUMNS where TABLE_NAME = 'table';	len(df.columns)
Datatype	df.dtypes	df.dtypes	DESCRIBE table;	df.printSchema()
Select column(s)	df[["col1", "col2"]]	df[["col1", "col2"]]	SELECT column FROM table;	df.select("col1", "col2")
Filter Data	df[df.column > 10]	df[df.column > 10] df.filter(pl.col("column") > 10)	SELECT * FROM table where column>10;	df.filter(df["column"]>10)
Sort	df.sort_values("column")	df.sort("column")	SELECT * FROM table ORDER BY column;	df.orderBy("column")
Fill NaN	df.column.fillna(0)	df.column.fill_nan(0)	UPDATE table SET column=0 WHERE column IS NULL;	df.na.fill(0)
Join	pd.merge(df1, df2, on ="col", how="inner")	df1.join(df2, on="col", how="inner")	SELECT * FROM table1 JOIN table2 ON (table1.col = table2.col);	df1.join(df2, on="col", how="inner")
Concatenate	pd.concat((df1, df2))	pl.concat((df1, df2))	SELECT * FROM table1 UNION ALL table2;	df1.union(df2)
Group	df.groupby("column"). agg_col.mean()	df.groupby("column"). agg(pl.mean("agg_col"))	SELECT column, avg(agg_col) FROM table GROUP BY column;	df.groupBy("column"). agg(avg("agg_col"))
Unique values	df.column.unique()	df.column.unique()	SELECT DISTINCT column FROM table;	df.select("column"). distinct()
Rename column	df.rename(columns = {"old_name": "new_name"})	df.rename(mapping = {"old_name": "new_name"})	ALTER TABLE table RENAME COLUMN old_name TO new_name;	df.withColumnsRenamed({"old_name": "new_name"})
Delete column	df.drop(columns = ["column"])	df.drop(name = ["column"])	ALTER TABLE table DROP COLUMN column;	df.drop("col1", "col2")

While the motivation for Pandas and SQL is clear and well-known, let me tell you why you should care about Polars and PySpark.

Why Polars?

Pandas has many limitations, which Polars addresses, such as:



- Pandas always adheres to single-core computation → Polars is multi-core.
- Pandas offers no lazy execution → Polars does.
- Pandas creates bulky DataFrames → Polars' DFs are lightweight.
- Pandas is slow on large datasets → Polars is remarkably efficient.

In fact, if we look at the run-time comparison on some common operations, it's clear that Polars is much more efficient than Pandas.

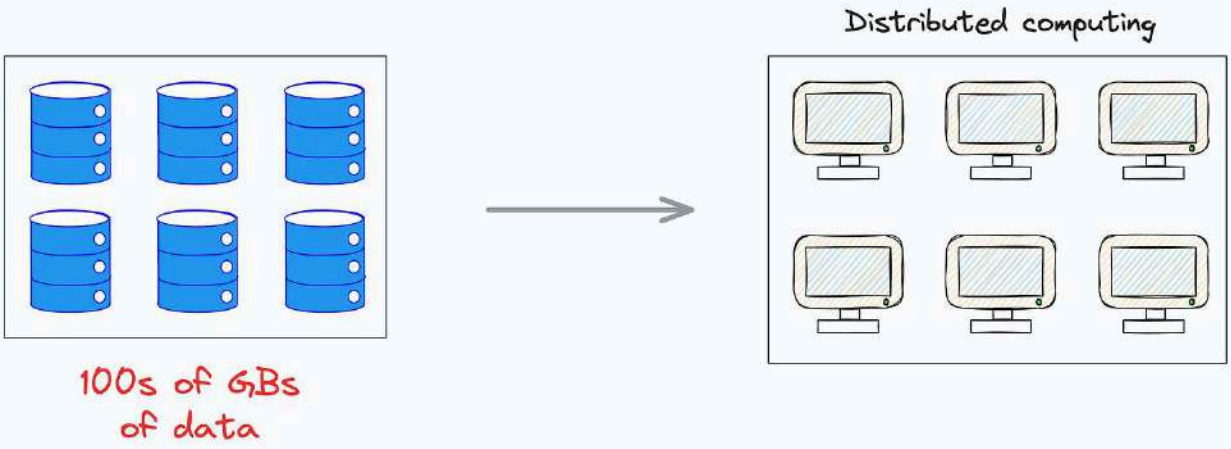
DailyDoseofDS.com

Method	Pandas	Polars	*Records: 4M
Memory Usage	1.7 GBs	0.63 GBs	Polars consumes 3x Less memory
Read CSV	4.52 s	0.72 s	Polars is 6x Faster
Save to CSV	15.3 s	3.25 s	Polars is 5x Faster
Selecting Columns	36.5 ms	2.2 µs	Polars is 16500x Faster
Filtering	130 ms	90 ms	Polars is 1.5x Faster
Grouping	217 ms	56 ms	Polars is 4x Faster
Sorting	1.1 s	1.04s	Similar Performance

Why Spark?

While tabular data space is mainly dominated by Pandas and Sklearn, one can hardly expect any benefit from them beyond some GBs of data due to their single-node processing.

A more practical solution is to use distributed computing instead — a framework that disperses the data across many small computers.



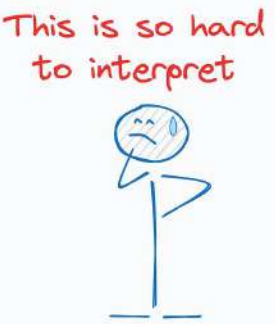
Spark is among the best technologies used to quickly and efficiently analyze, process, and train models on big datasets.

That is why most data science roles at big tech demand proficiency in Spark. It's that important

2 Alternatives to Pandas' Describe

Probably the first (or second) thing I do when I load any Pandas or Polars DataFrame is describe it, using the df.describe() method. However, I always find its output to be pretty naive and almost of no use. In other words, it hardly highlights any key information about the data.

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

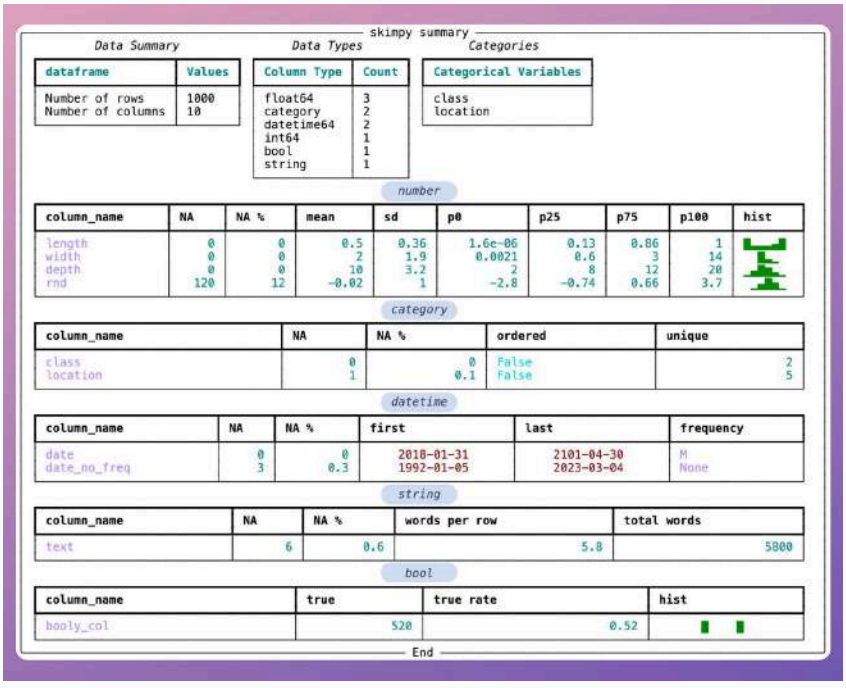


But some time back, I came across two pretty cool libraries that IMMENSELY supercharge this DataFrame summary. Since then, I don't think I have ever used the describe() method.

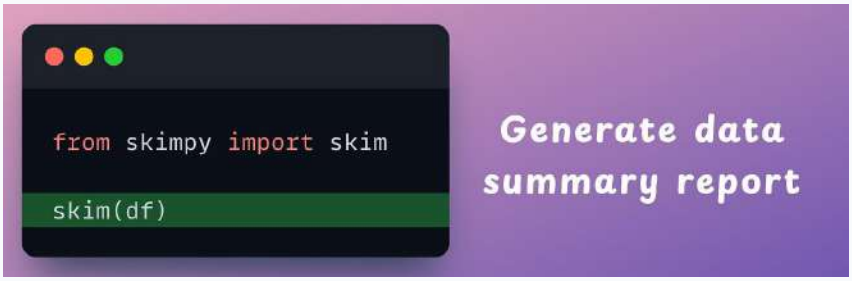
#1) Skimpy

It is a Jupyter-based tool that provides a standardized and comprehensive data summary.

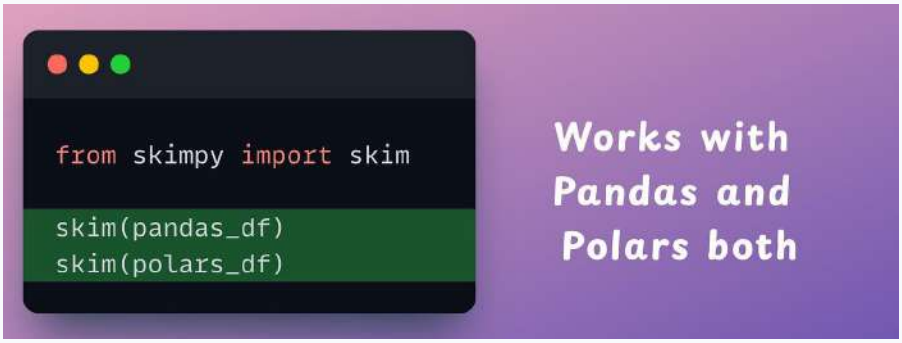
This includes data shape, column data types, column summary statistics, distribution charts, missing stats, etc.:



What's more, the summary is grouped by datatypes for faster analysis. This is the code to use Skimpy:

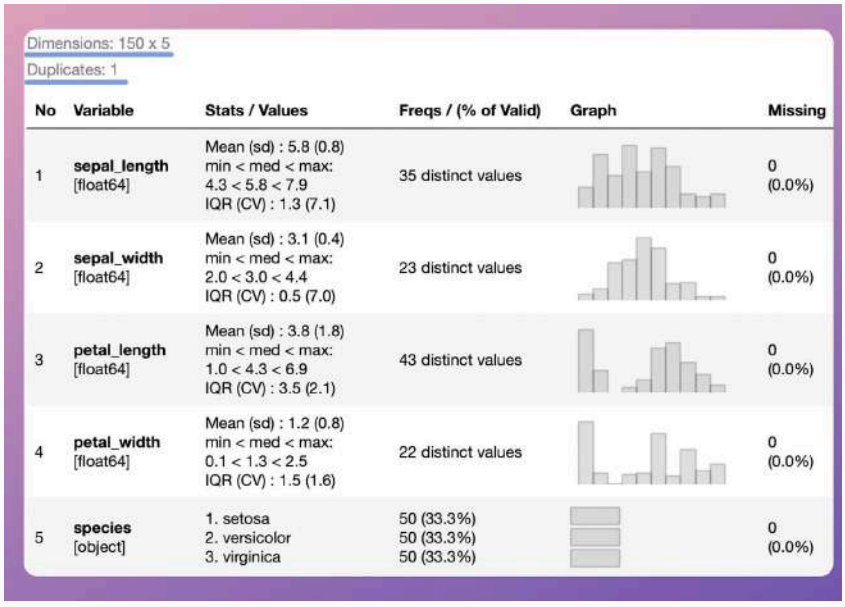


One thing I really love about Skimpy is that it works seamlessly with Polars, which I have started using more often than I use Pandas these days.



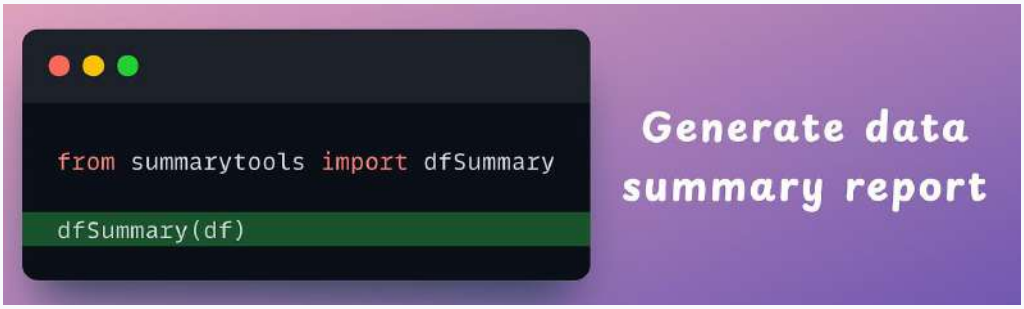
#2) SummaryTools

The second one is SummaryTools, which does almost the exact same thing as Skimpy, i.e., it generates a standardized report:


Dimensions: 150 x 5
Duplicates: 1

No	Variable	Stats / Values	Freqs / (% of Valid)	Graph	Missing
1	sepal_length [float64]	Mean (sd) : 5.8 (0.8) min < med < max: 4.3 < 5.8 < 7.9 IQR (CV) : 1.3 (7.1)	35 distinct values		0 (0.0%)
2	sepal_width [float64]	Mean (sd) : 3.1 (0.4) min < med < max: 2.0 < 3.0 < 4.4 IQR (CV) : 0.5 (7.0)	23 distinct values		0 (0.0%)
3	petal_length [float64]	Mean (sd) : 3.8 (1.8) min < med < max: 1.0 < 4.3 < 6.9 IQR (CV) : 3.5 (2.1)	43 distinct values		0 (0.0%)
4	petal_width [float64]	Mean (sd) : 1.2 (0.8) min < med < max: 0.1 < 1.3 < 2.5 IQR (CV) : 1.5 (1.6)	22 distinct values		0 (0.0%)
5	species [object]	1. setosa 2. versicolor 3. virginica	50 (33.3%) 50 (33.3%) 50 (33.3%)		0 (0.0%)

This is the code to use SummaryTools:

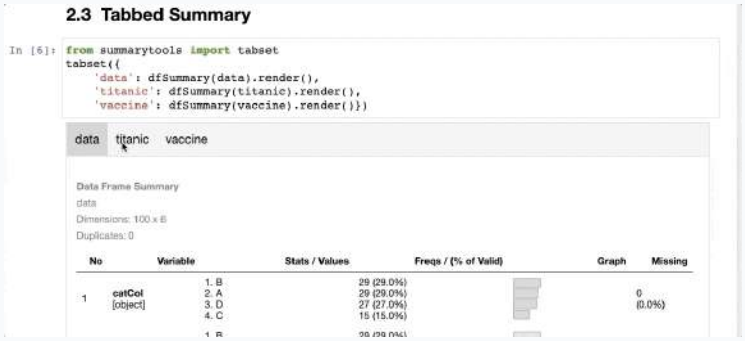


Two pretty cool things about SummaryTools are that it can create:

1. A collapsible summary of the dataset, as illustrated below:



2. A tabbed summary of the dataset, as shown below:



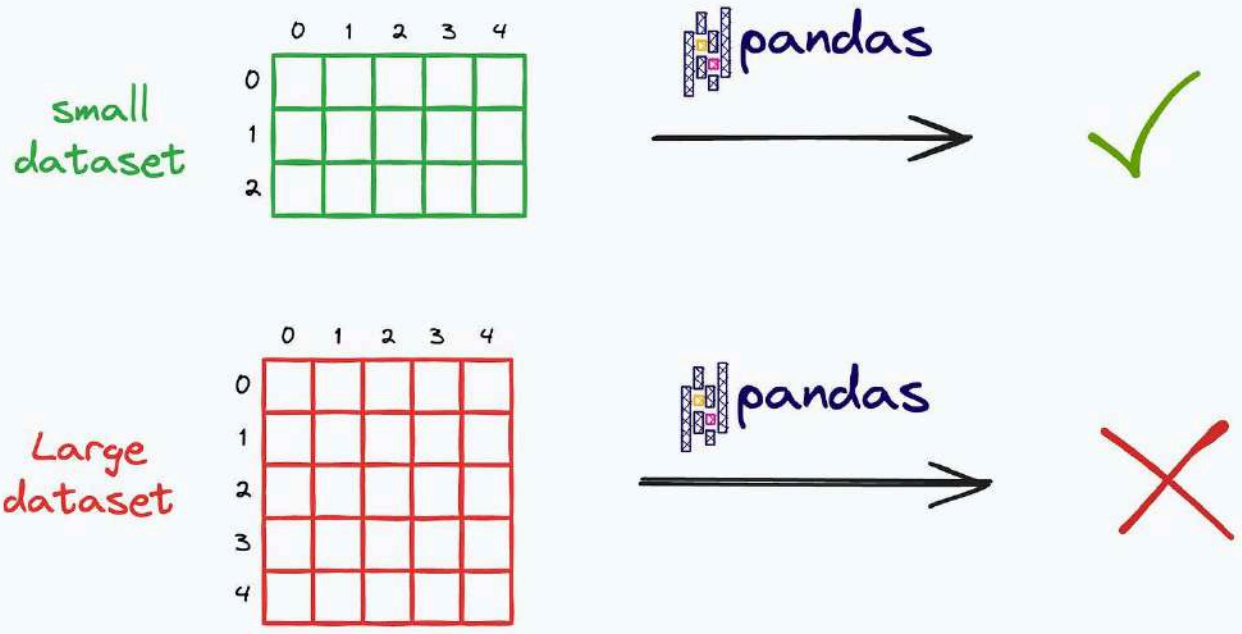
The only thing I don't like about SummaryTools is that it is not compatible with Polars (yet). Nonetheless, I find both of them extremely promising for understanding my dataset with more granularity than Pandas' `describe()` method.

Try by downloading this Jupyter notebook: https://bit.ly/45Rheq1.

Accelerate Pandas with GPU Using RAPIDS cuDF

Two of the biggest problems with Pandas is that:

- It always adheres to a single-core computation on a CPU.
- It creates bulky DataFrames.



While many libraries (Polars, for instance) do address these limitations, they are still limited to CPU-driven computations.

NVIDIA's RAPIDS cuDF library allows Pandas users to supercharge their Pandas workflow with GPUs.

How to use it?

Within a GPU runtime, do the following :

- Load the extension: `%load_ext cudf.pandas`
- Import Pandas: `import pandas as pd`

Done! Use Pandas' methods as you usually would.

Just loading the extension provides immense speedups.



As per NVIDIA's official release, this can be as fast as 150x. In my personal experimentation, however, I mostly observed it to range between 50-70x, which is still pretty good.

The good thing is that the extension accelerates most Pandas' methods. Yet, if needed, it can automatically fall back to the CPU.

How does it work?

Whenever `cudf.pandas` is enabled, the import pandas as pd statement does not import the original Pandas library which we use all the time. Instead, it imports another library that contains GPU-accelerated implementations of all Pandas methods.

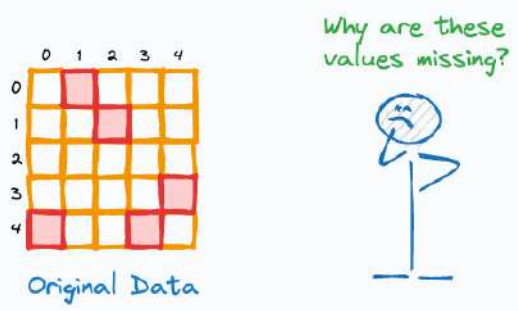


This alternative implementation preserves the entire syntax of Pandas. So if you know Pandas, you already know how to use cuDF's Pandas.

You can find the code here: https://bit.ly/4cOAPZW.

Missing Data Analysis with Heatmaps

Real-world datasets almost always have missing values. In most cases, it is unknown to us beforehand why values are missing.



But it's good to know that there could be multiple reasons for missing values:

- Missing Completely at Random (MCAR): The value is genuinely missing by itself and has no relation to that or any other observation.
- Missing at Random (MAR): Data is missing due to another observed variable. For instance, we may observe that the percentage of missing values differs significantly based on other variables.
- Missing NOT at Random (MNAR): This one is tricky. MNAR occurs when there is a definite pattern in the missing variable. However, it is unrelated to any feature we can observe in our data. In fact, this may depend on an unobserved feature.

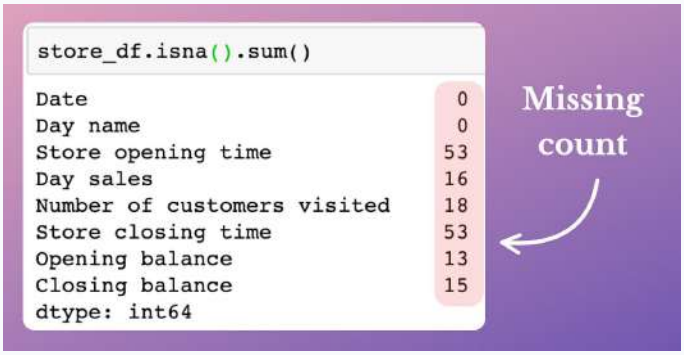
And identifying the reason for missingness can be extremely useful for further analysis, imputation, and modeling.

Consider we have a daily sales dataset of a store that has the following information:

	Date	Day name	Store opening time	Day sales	Number of customers visited	Store closing time	Opening balance	Closing balance
0	2023-01-02	Monday	09:00 AM	845.912507	102	08:00 PM	1431.060983	2276.973489
1	2023-01-03	Tuesday	08:00 AM	710.915666	77	08:00 PM	1696.911885	2407.827551
2	2023-01-04	Wednesday	10:00 AM	977.149863	133	06:00 PM	1247.242954	2224.392818
3	2023-01-05	Thursday	08:00 AM	690.447059	85	07:00 PM	1354.447274	2044.894333
4	2023-01-06	Friday	10:00 AM	NaN	43	07:00 PM	1264.830272	1785.676885
5	2023-01-07	Saturday	09:00 AM	108.363621	119	06:00 PM	1808.420181	1916.783802
6	2023-01-08	Sunday	NaN	0	0	NaN	1916.783802	1916.783802

- Day and Date
- Store opening and closing time
- Number of customers
- Total sales
- Account balance at open and close time

The reason for missing values is unknown to us. Here, when doing EDA, many folks compute the column-wise missing frequency as follows:



The above table just highlights the number of missing values in each column.

More specifically, we get to know that:

- Missing values are relatively high in two columns compared to others.
- Missing values in the opening and closing time columns are the same (53).

That's the only info it provides. However, the problem with this approach is that it hides many important details about missing values, such as:

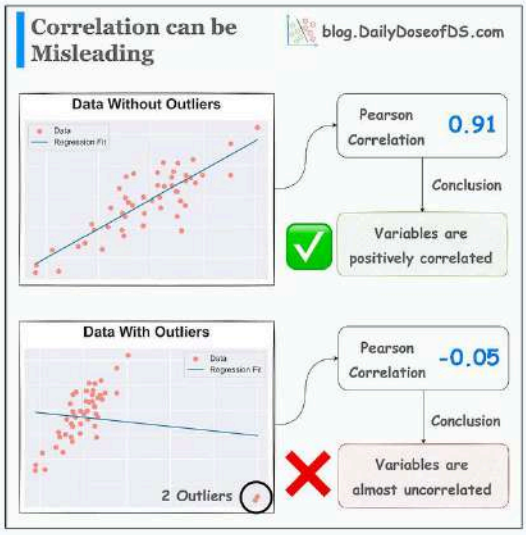
- Their specific location in the dataset.
- Periodicity of missing values (if any).
- Missing value correlation across columns, etc.

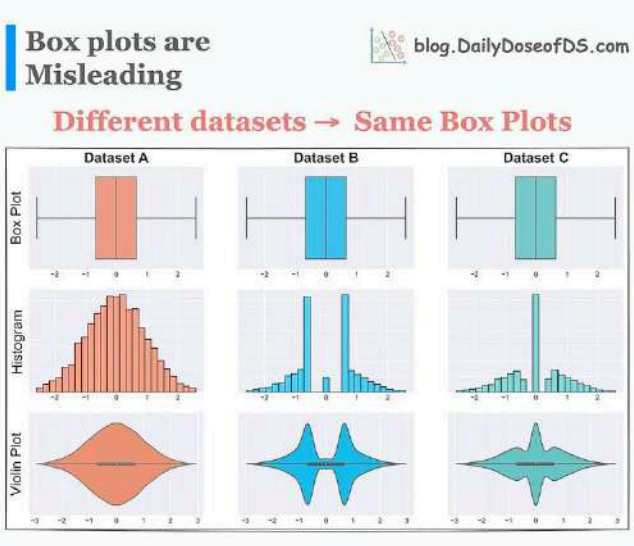
...which can be extremely useful to understand the reason for missingness.

To put it another way, the above table is more like summary statistics, which rarely depict the true picture.

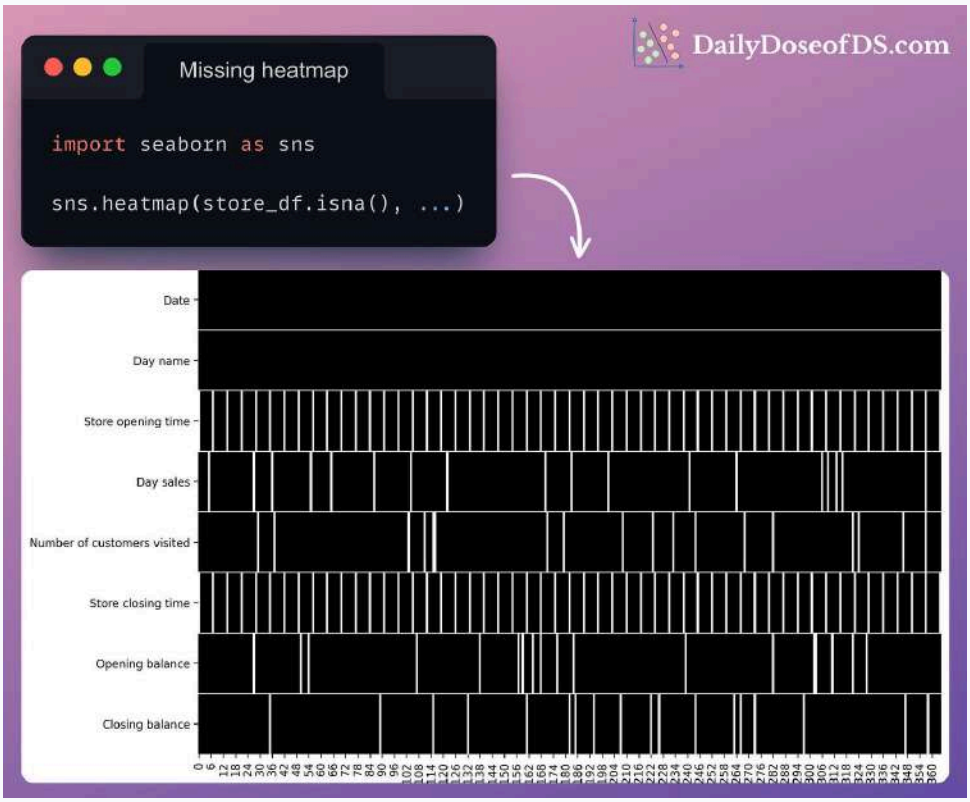
Why?

We discussed this in the statistics section of this book.





But here's how I often enrich my missing value analysis with heatmaps. Compare the missing value table we discussed above with the following heatmap of missing values:

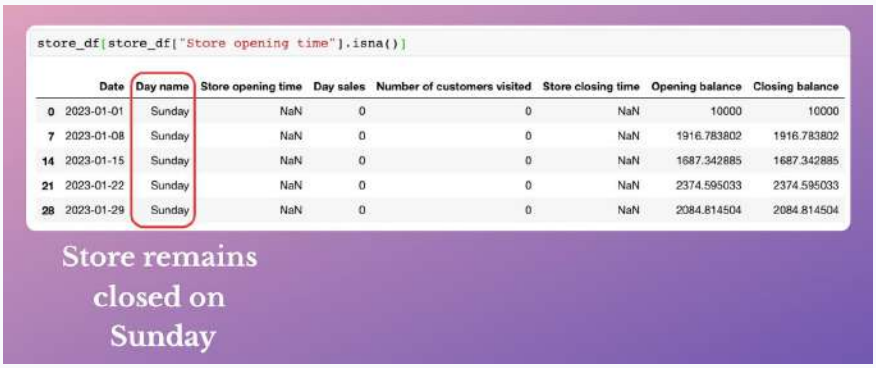


The white vertical lines depict the location of missing values in a specific column.

Now, it is immediately clear that:

- Values are periodically missing in the opening and closing time columns.
- Missing values are correlated in the opening and closing time columns.
- The missing values in other columns appear to be (not necessarily though) missing completely at random.

Further analysis of the opening time lets us discover that the store always remains closed on Sundays:



	Date	Day name	Store opening time	Day sales	Number of customers visited	Store closing time	Opening balance	Closing balance
0	2023-01-01	Sunday	NaN	0	0	NaN	10000	10000
7	2023-01-08	Sunday	NaN	0	0	NaN	1916.783802	1916.783802
14	2023-01-15	Sunday	NaN	0	0	NaN	1687.342885	1687.342885
21	2023-01-22	Sunday	NaN	0	0	NaN	2374.595033	2374.595033
28	2023-01-29	Sunday	NaN	0	0	NaN	2084.814504	2084.814504

Now, we know why the opening and closing times are missing in our dataset. This information can be beneficial during its imputation.

This specific situation is "Missing at Random (MAR)." Essentially, as we saw above, the missingness is driven by the value of another observed column.

As we know the reason, we can use either the kNN imputation or the MissForest techniques to impute these values, which we discussed in the ML section of this book (MissForest and kNN Imputation).

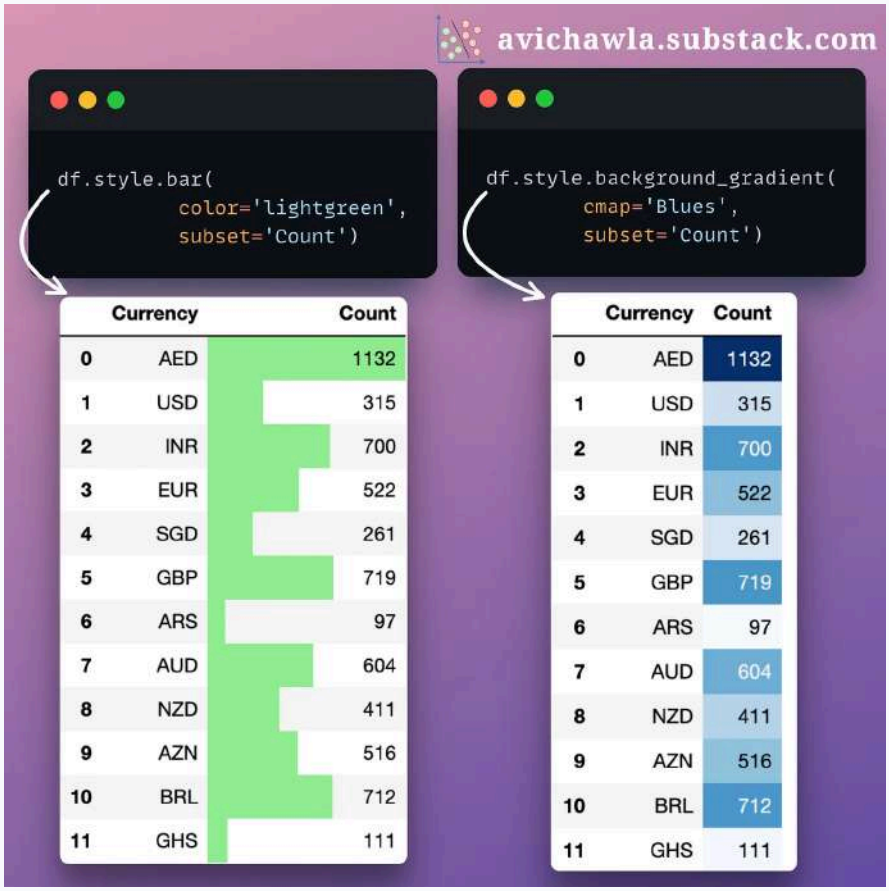
DataFrame Styling

Instead of previewing raw DataFrames, styling can make data analysis much easier and faster.

Here's how.

Jupyter is a web-based IDE. Anything you print is rendered using HTML and CSS.

This means you can style your output in many different ways.



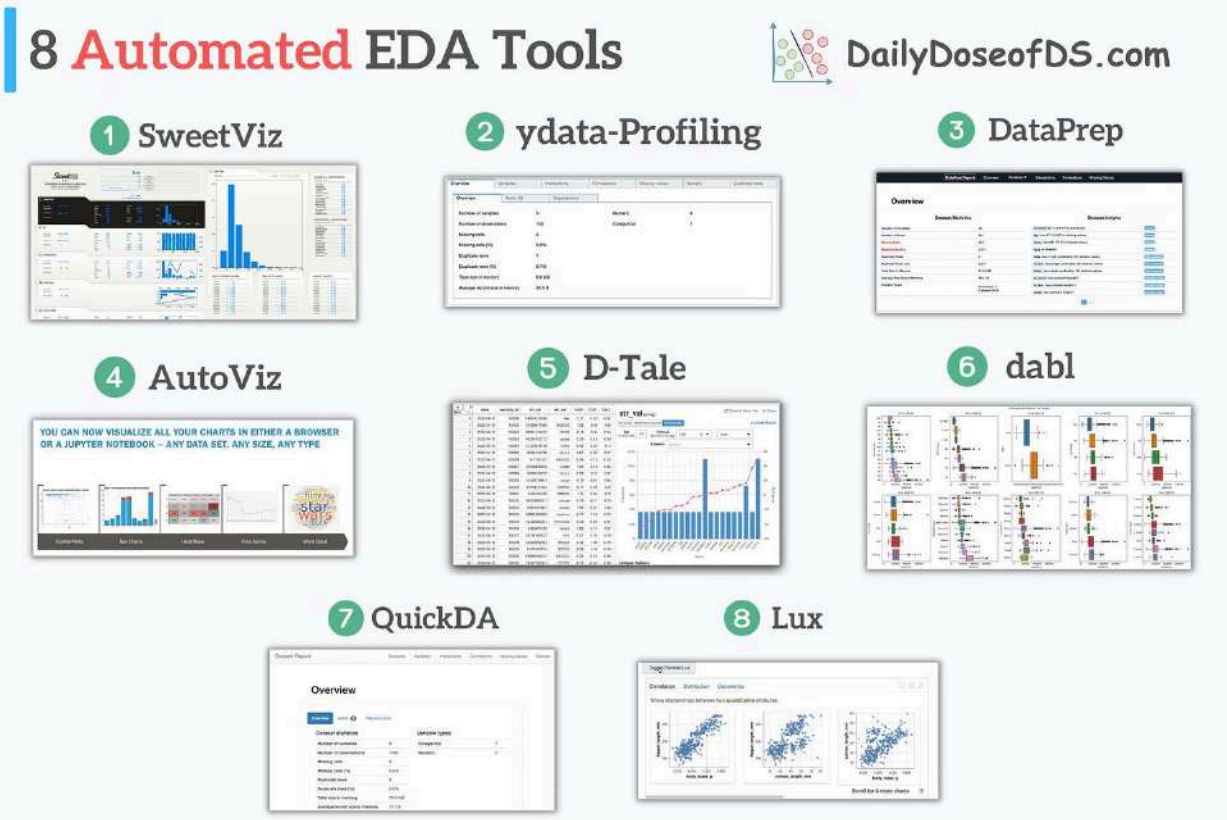
To style Pandas DataFrames, use its Styling API (**df.style**).

As a result, the DataFrame is rendered with the specified styling.

Pandas documentation page on Styling: https://bit.ly/3zDmxgE.

8 Automated EDA Tools

Below are 8 powerful EDA tools that automate many redundant EDA steps and help you profile your data quickly.



Please note that these tools are not the ultimate EDA alternatives that will answer all your questions about the dataset.

But given that the preliminary EDA steps in almost all projects are the same — plotting the response variable, checking imbalance, running correlation analysis, missing value analysis, and more, these tools pretty well automate these steps in my opinion.

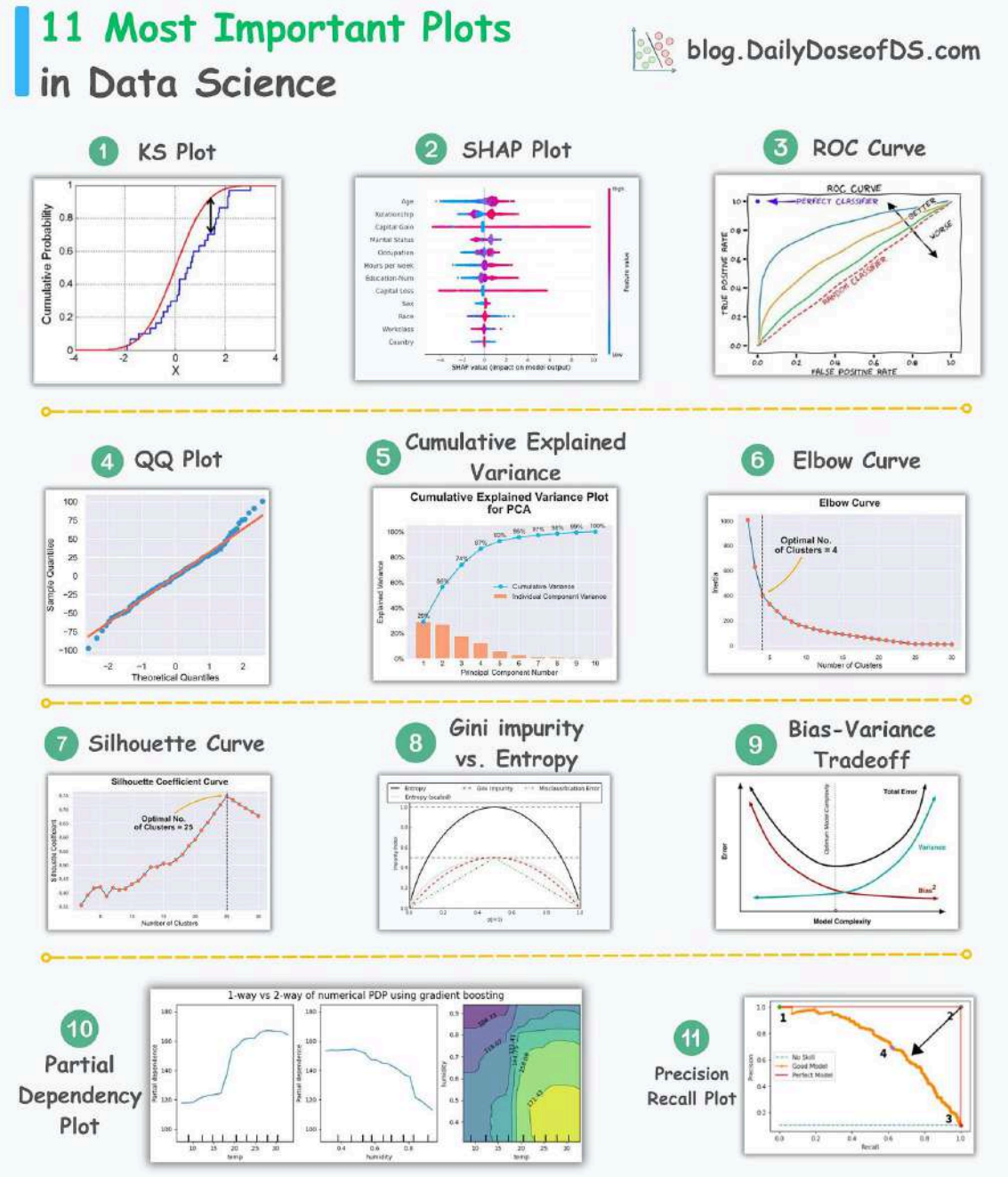
Also, at times, manual EDA can be prone to human errors and one may miss out on checking a few things. Automated tools eliminate these risks and provide a standardized report across all projects.

Full summary about each tool is available here: https://bit.ly/3xZP4fK.

Data Visualisation

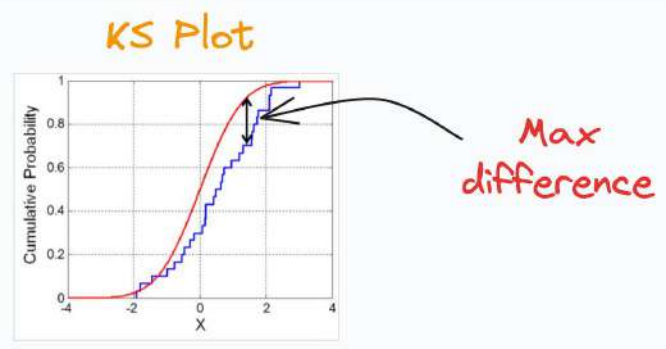
Most Important Plots in Data Science

The visual below depicts the 11 most important and must-know plots in data science:



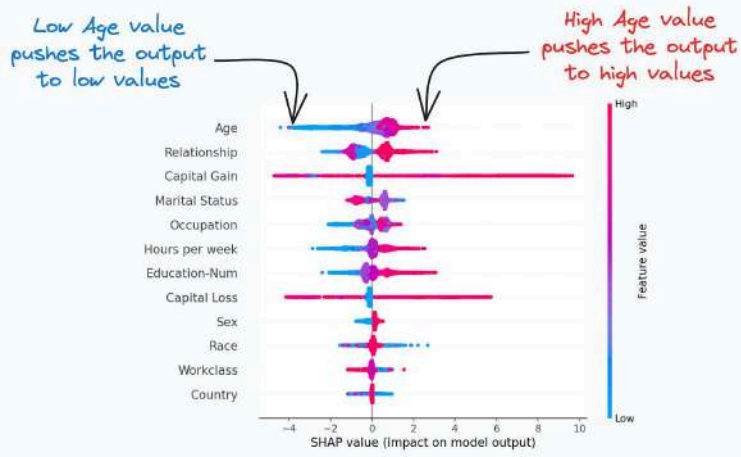
Let's understand them briefly and how they are used.

KS Plot:



- It is used to assess the distributional differences.
- The core idea is to measure the maximum distance between the cumulative distribution functions (CDF) of two distributions.
- The lower the maximum distance, the more likely they belong to the same distribution.
- Thus, instead of a "plot", it is mainly interpreted as a "statistical test" to determine distributional differences.

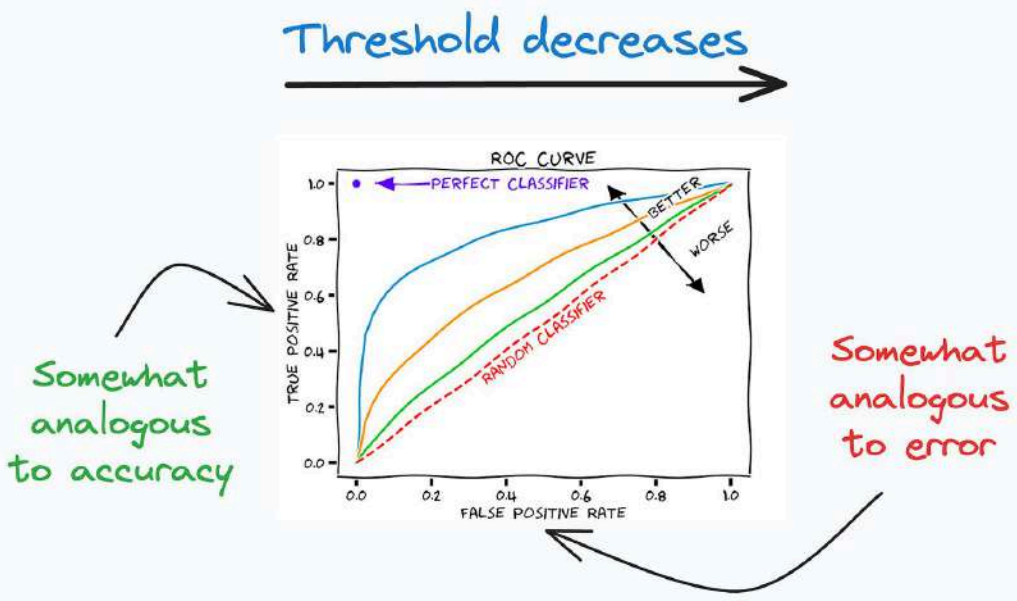
SHAP Plot:



- It summarizes feature importance to a model's predictions by considering interactions/dependencies between them.

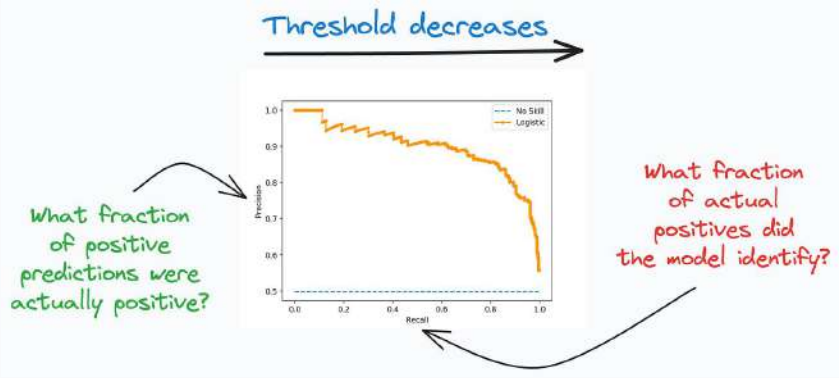
- It is useful in determining how different values (low or high) of a feature affect the overall output.

ROC Curve:



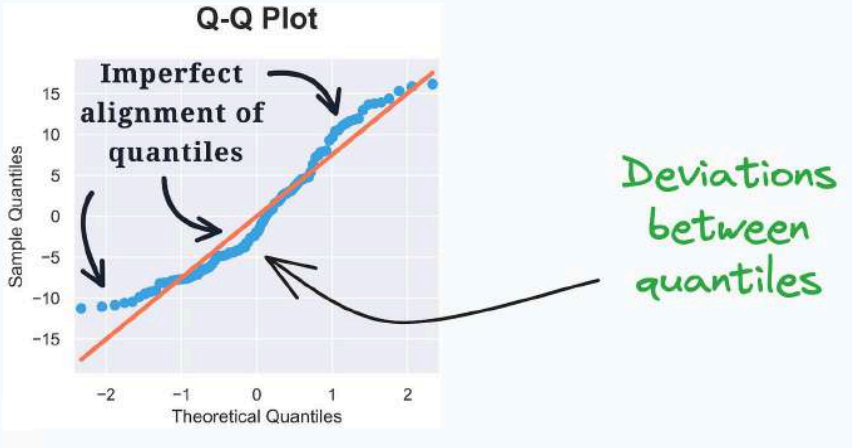
- It depicts the tradeoff between the true positive rate (good performance) and the false positive rate (bad performance) across different classification thresholds.
- The idea is to balance TPR (good performance) vs. FPR (bad performance).

Precision-Recall Curve:



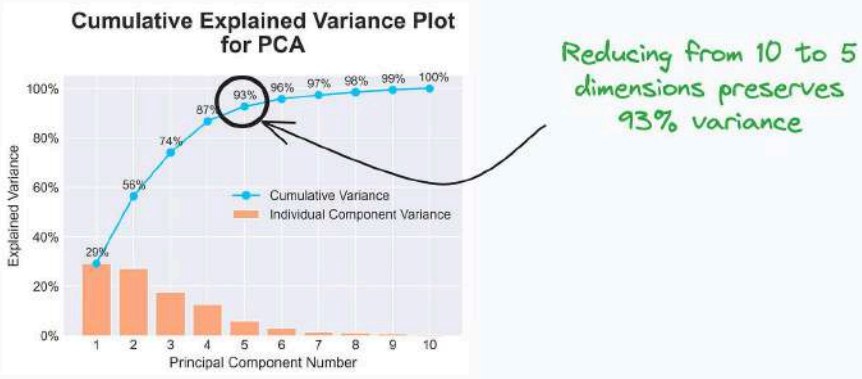
- It depicts the tradeoff between Precision and Recall across different classification thresholds.

QQ Plot:



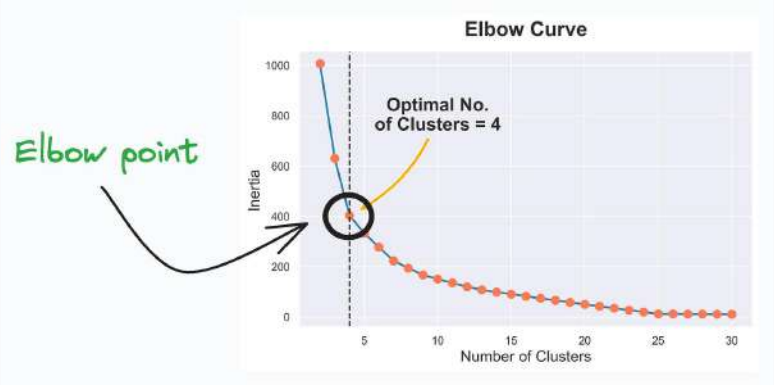
- It assesses the distributional similarity between observed data and theoretical distribution.
- It plots the quantiles of the two distributions against each other.
- Deviations from the straight line indicate a departure from the assumed distribution.

Cumulative Explained Variance Plot:



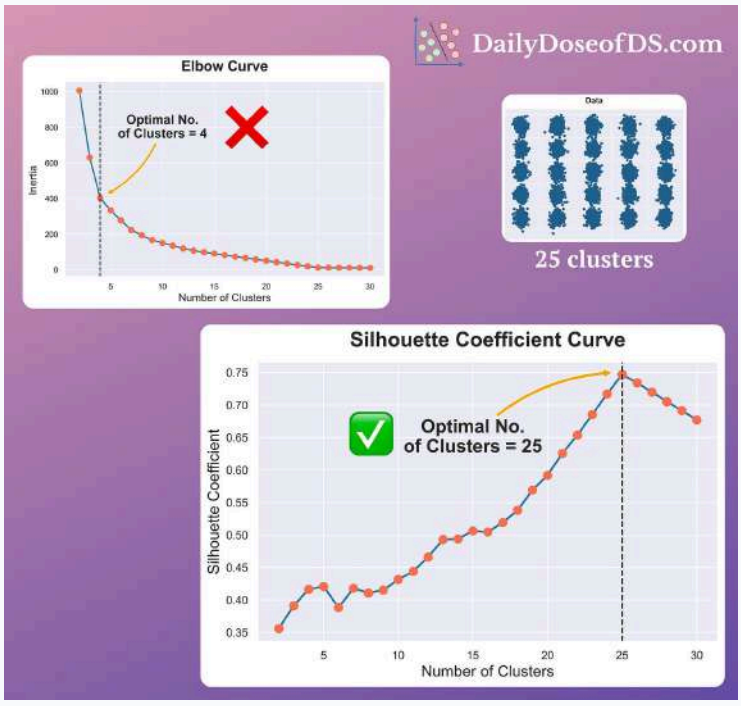
- It is useful in determining the number of dimensions we can reduce our data to while preserving max variance during PCA.

Elbow Curve:



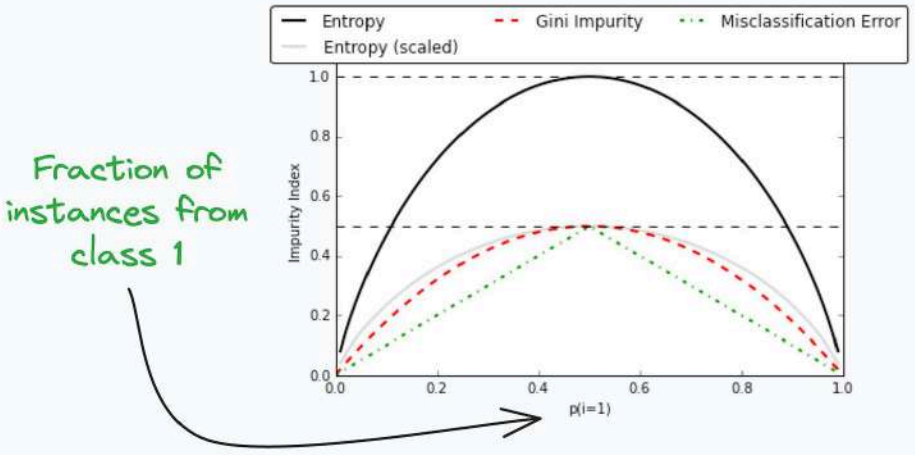
- The plot helps identify the optimal number of clusters for the k-means algorithm.
- The point of the elbow depicts the ideal number of clusters.

Silhouette Curve:



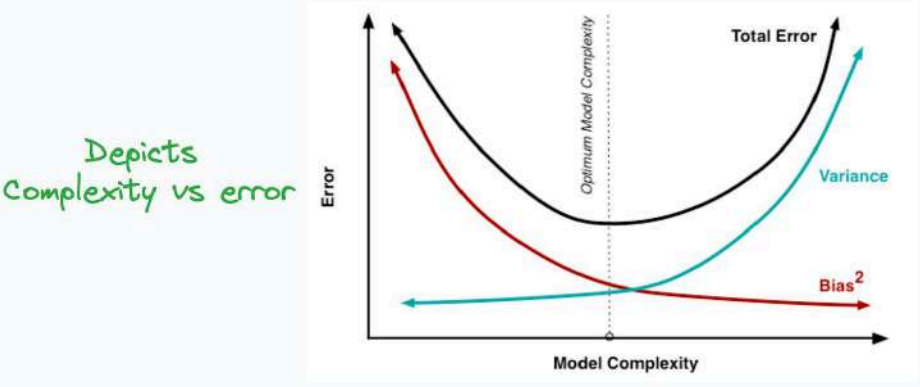
- The Elbow curve is often ineffective when you have plenty of clusters.
- Silhouette Curve is a better alternative, as depicted above.

Gini-Impurity and Entropy:



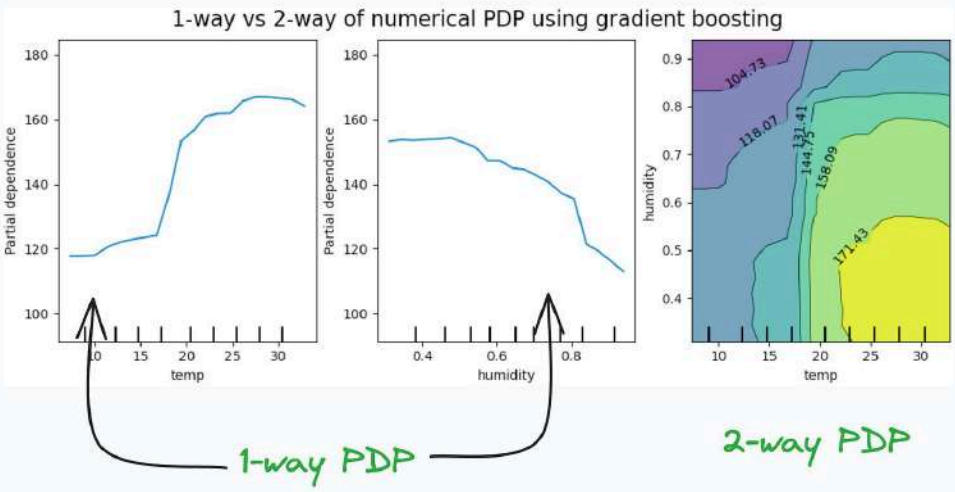
- They are used to measure the impurity or disorder of a node or split in a decision tree.
- The plot compares Gini impurity and Entropy across different splits.
- This provides insights into the tradeoff between these measures.

Bias-Variance Tradeoff:



- It's probably the most popular plot on this list.
- It is used to find the right balance between the bias and the variance of a model against complexity.

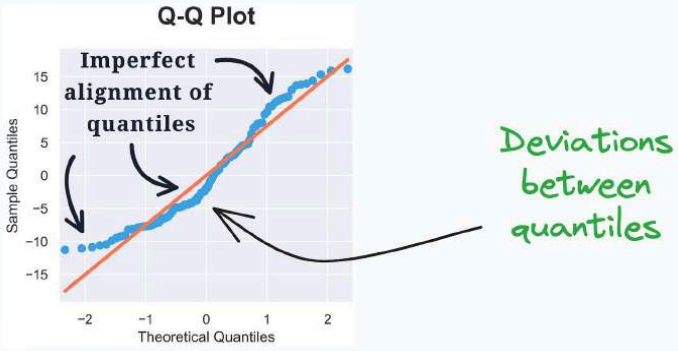
Partial Dependency Plots:



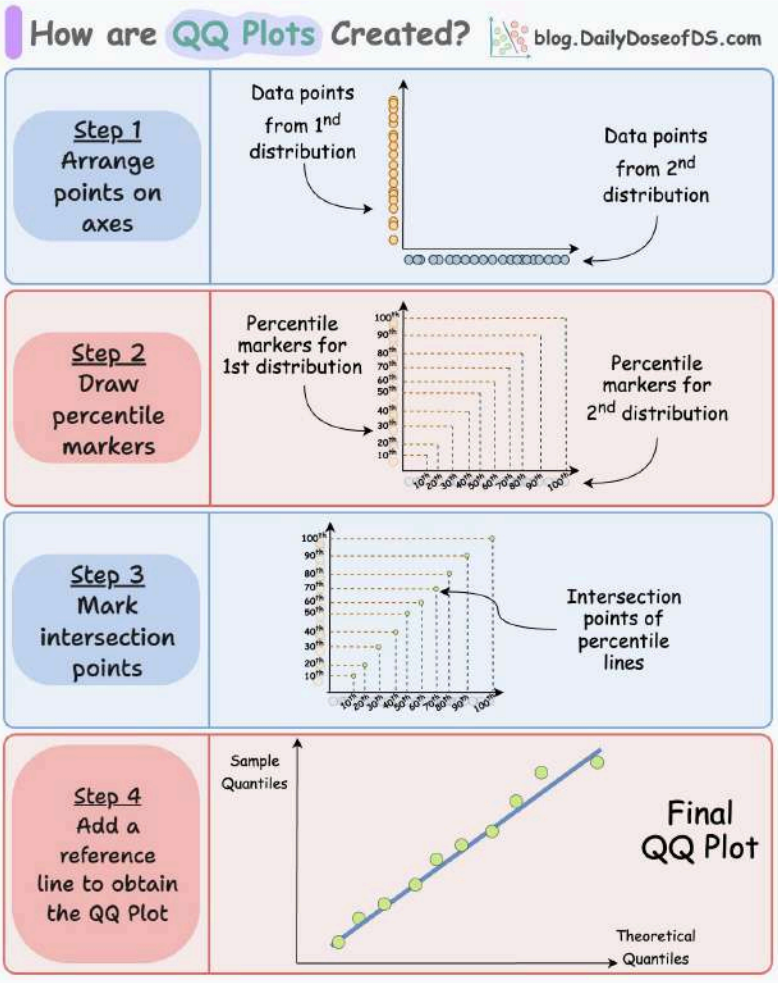
- Depicts the dependence between target and features.
- A plot between the target and one feature forms → 1-way PDP.
- A plot between the target and two feature forms → 2-way PDP.
- In the leftmost plot, an increase in temperature generally results in a higher target value.

How are QQ Plots Created?

A QQ plot is a great way to visually assess the similarity between two distributions.



It does this by plotting the quantiles of the two distributions against each other. The deviations from the straight line indicate the differences between the two distributions. The following visual depicts how it is created:

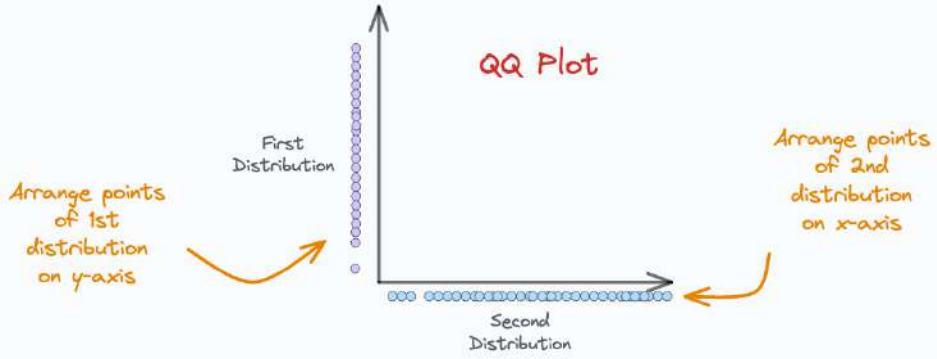


Let's discuss it in more detail.

Consider we have two distributions, D1 and D2.

Step 1) Arrange points on axes:

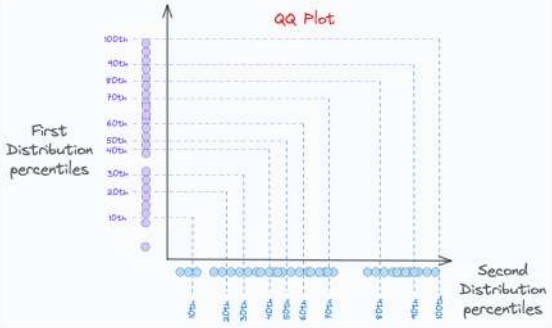
As shown below, we arrange points of D1 on the y-axis and D2 on the x-axis.



Step 2) Draw percentile lines

Next, for both distributions, we create some percentile lines.

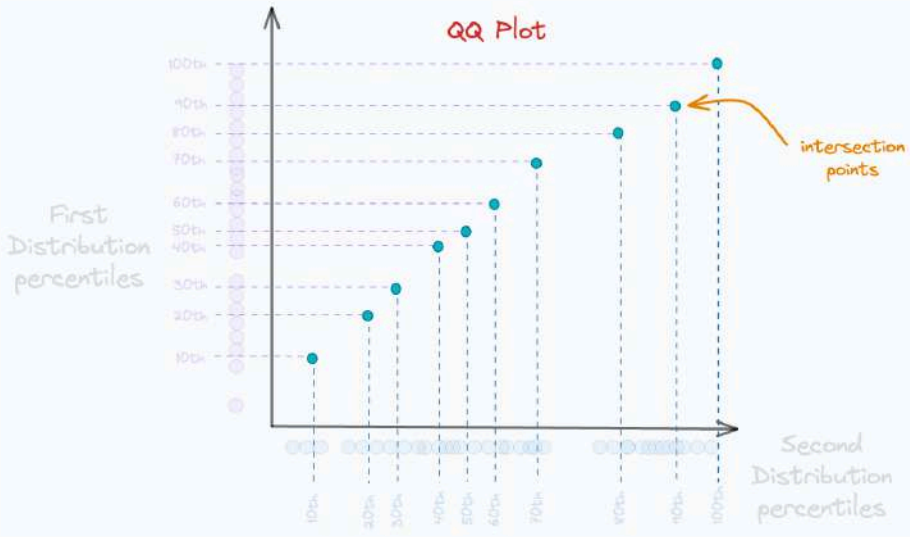
For instance, on both axes, we can mark the points of 10th percentile, 20th percentile, 30th percentile, etc., from both distributions. This is shown below:



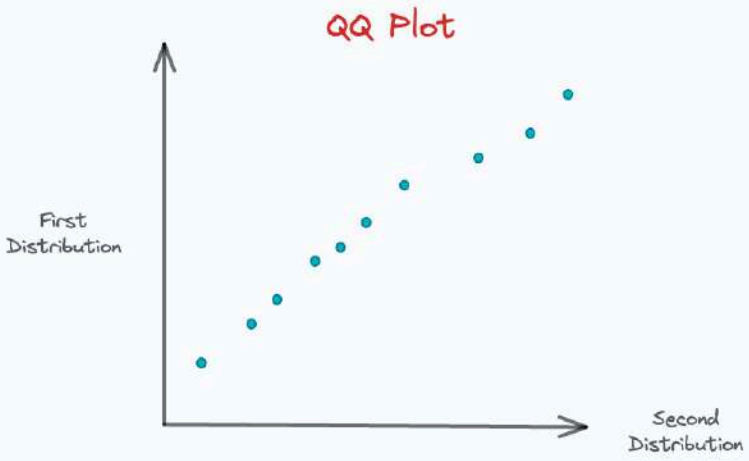
We mark the percentile locations for both distributions and intersect the corresponding lines.

- 10th percentile of D1 is intersected with 10th percentile of D2.
- 20th percentile of D1 is intersected with 20th percentile of D2.
- and so on.

The intersection points of these percentile lines gives us the points we typically see in a QQ plot:



Now, we can get rid of the percentile marker lines.



In a gist, the above plot gives us the location where the corresponding percentiles of the two distributions match.

Step 3) Add the reference line

Finally, we must add a reference line to determine the deviations between the two distributions.

There are many ways to do this.

For instance:

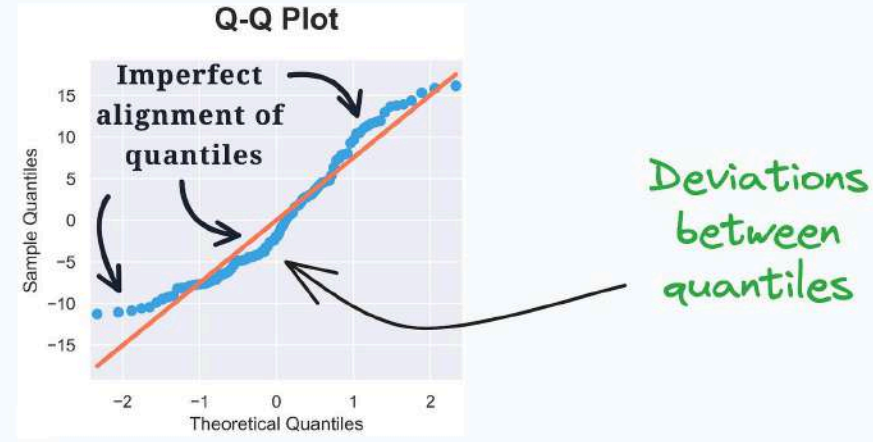
- The line connecting the 25th and 75th percentiles of both distributions can be considered as a reference line.
- The regression fit on the above scatter plot can be considered as a reference line.

Typically, the line connecting the 25-75th percentile is preferred because the regression fit can be influenced by outliers.

After adding the reference line, we get our QQ plot:



The deviations from this reference line indicate that the two distributions differ from each other. In other words, the deviations mean that the corresponding percentiles do not align.



This becomes an indicator of distributional dissimilarities.

And, of course, the more percentiles we plot, the better and more accurate the QQ plot will be.

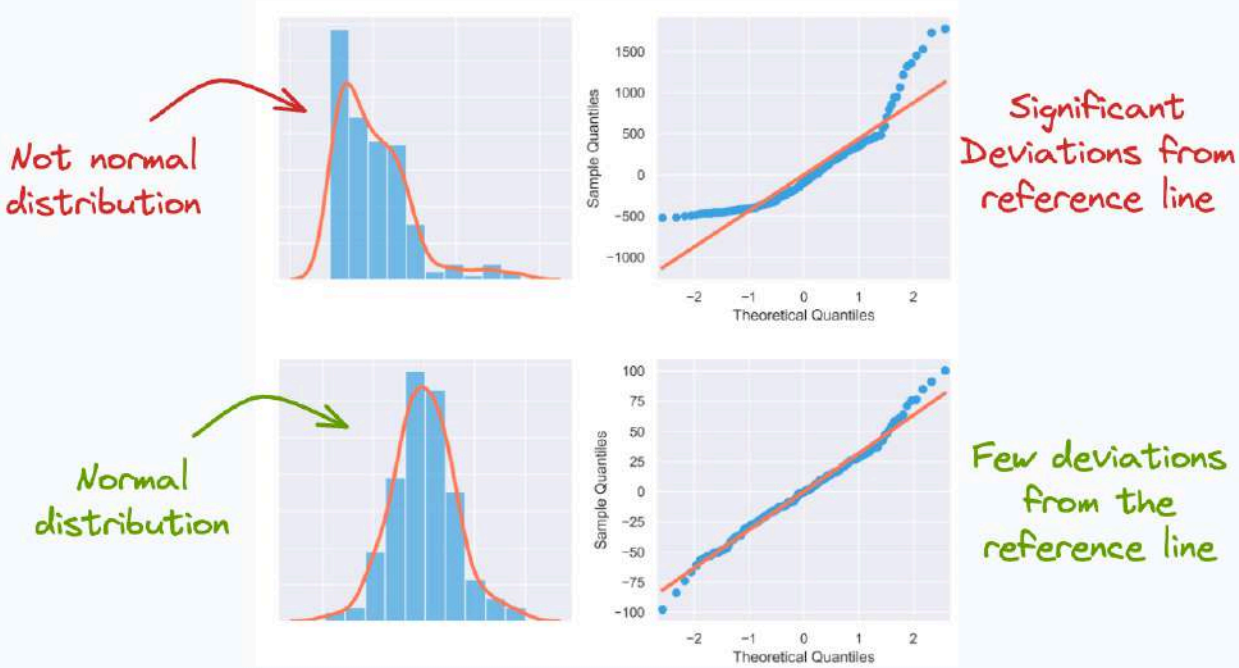
There are many applications of the QQ plot.

For instance, say we have an observed distribution and want to determine if it resembles a normal distribution.

We can use a QQ plot for this:

- D1: The observed distribution
- D2: Normal distribution.

If the percentile points lie closer to the reference line, this would mean that the observed distribution is more like a normal distribution. This is depicted below:



8 Elegant Alternatives to Traditional Plots

Scatter plots, bar plots, line plots, box plots, and heatmaps are the most frequently used plots for data visualization. Although they are simple and known to almost everyone, I believe they are not the right choice to cover every possible scenario.

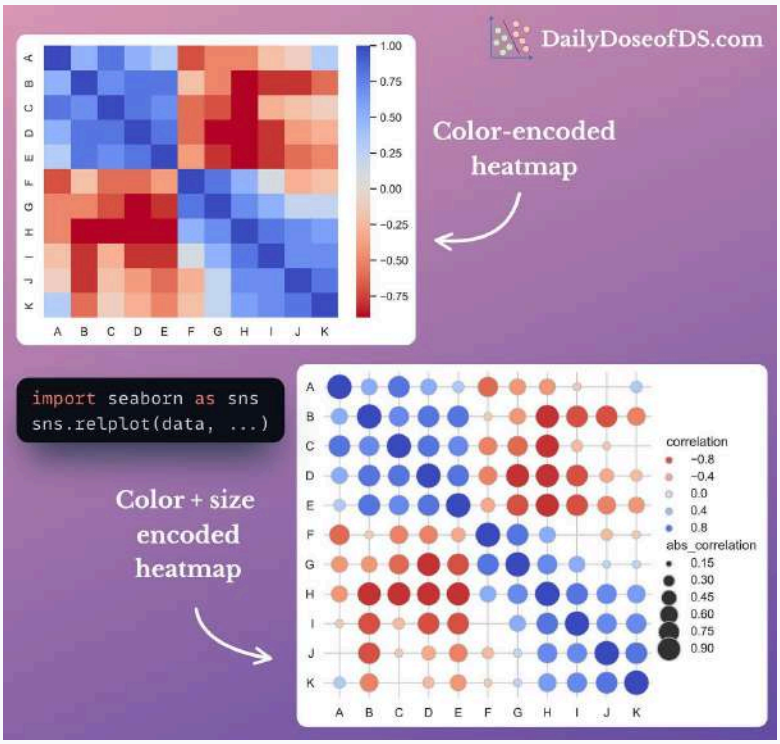
Instead, many other plots originate from these standard plots that can be much more suitable, if used appropriately. In this chapter, let's discuss a few alternatives to these popular plots. We will also understand specific situations where they can be more useful over standard plots.

This chapter is not intended to discourage the use of these traditional plots. They will always have their place.

Instead, it is to highlight specific situations where they can replaced with better plotting ideas.

#1) Size-encoded heatmaps

A traditional heatmap represents the values using a color scale. Yet, mapping the cell color to exact numbers is still challenging. Embedding a size component to heatmaps can be extremely helpful in such cases. In essence, the bigger the size, the higher the absolute value



This is especially useful to make heatmaps cleaner, as many values nearer to zero will immediately shrink.

#2) Waterfall charts

To visualize the change in value over time, a line (or bar) plot may not always be an apt choice.

This is because a line plot (or bar plot) depicts the actual values in the chart. Thus, it is difficult to visually estimate the scale and direction of incremental changes.

Instead, you can use a waterfall chart. It elegantly depicts these rolling differences, as depicted in this image.

Here, the start and final values are represented by the first and last bars.



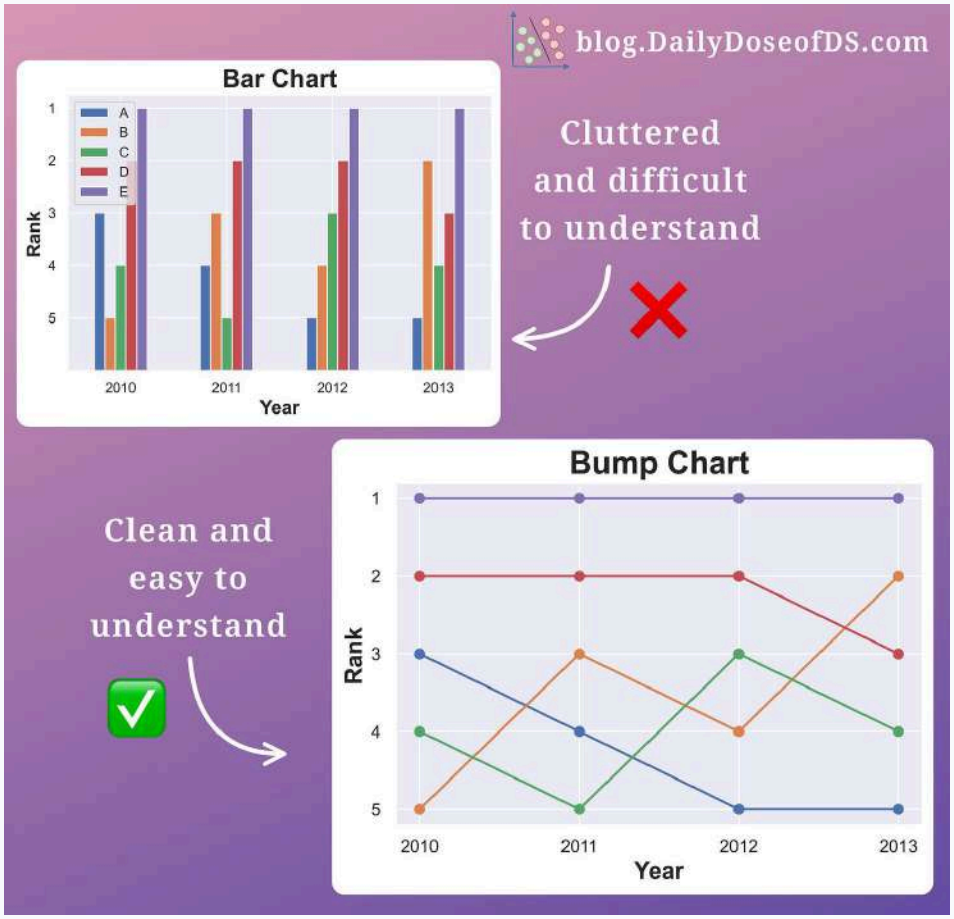
Also, the consecutive changes are automatically color-coded, making them easier to interpret.

#3) Bump charts

When visualizing the change in rank over time of multiple categories, using a bar chart may not be appropriate.

This is because bar charts quickly become cluttered with many categories.

Instead, try Bump Charts. They are specifically used to visualize the rank of different items over time.



Comparing the bar chart and bump chart above, it is far easier to interpret the change in rank with a bump chart rather than a bar chart.

#4) Raincloud Plots

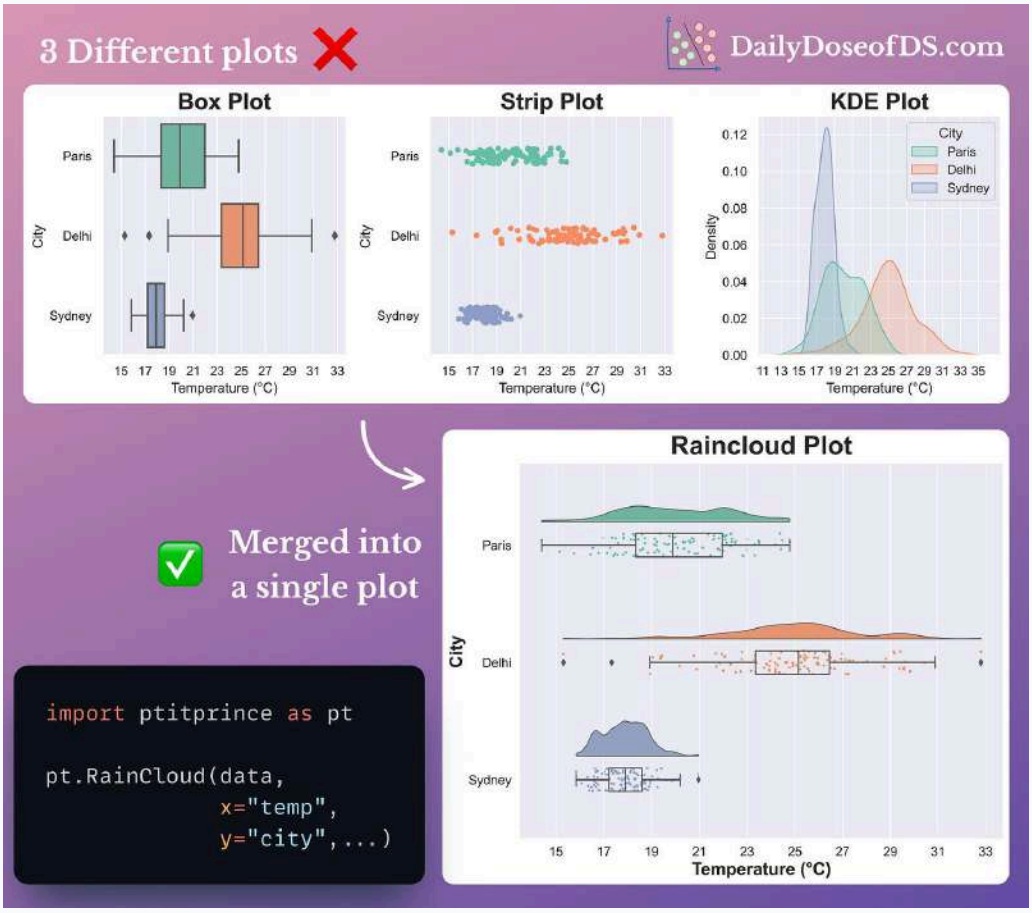
Visualizing data distributions using box plots and histograms can be misleading at times. This is because:

- It is possible to get the same box plot with entirely different data.
- Altering the number of bins changes the shape of a histogram.

Thus, to avoid misleading conclusions, it is always recommended to plot the data distribution as precisely as possible. Raincloud plots provide a concise way to combine and visualize three different types of plots together.

These include:

- Box plots for data statistics.
- Strip plots for data overview.
- KDE plots for the probability distribution of data.



With Raincloud plots, you can:

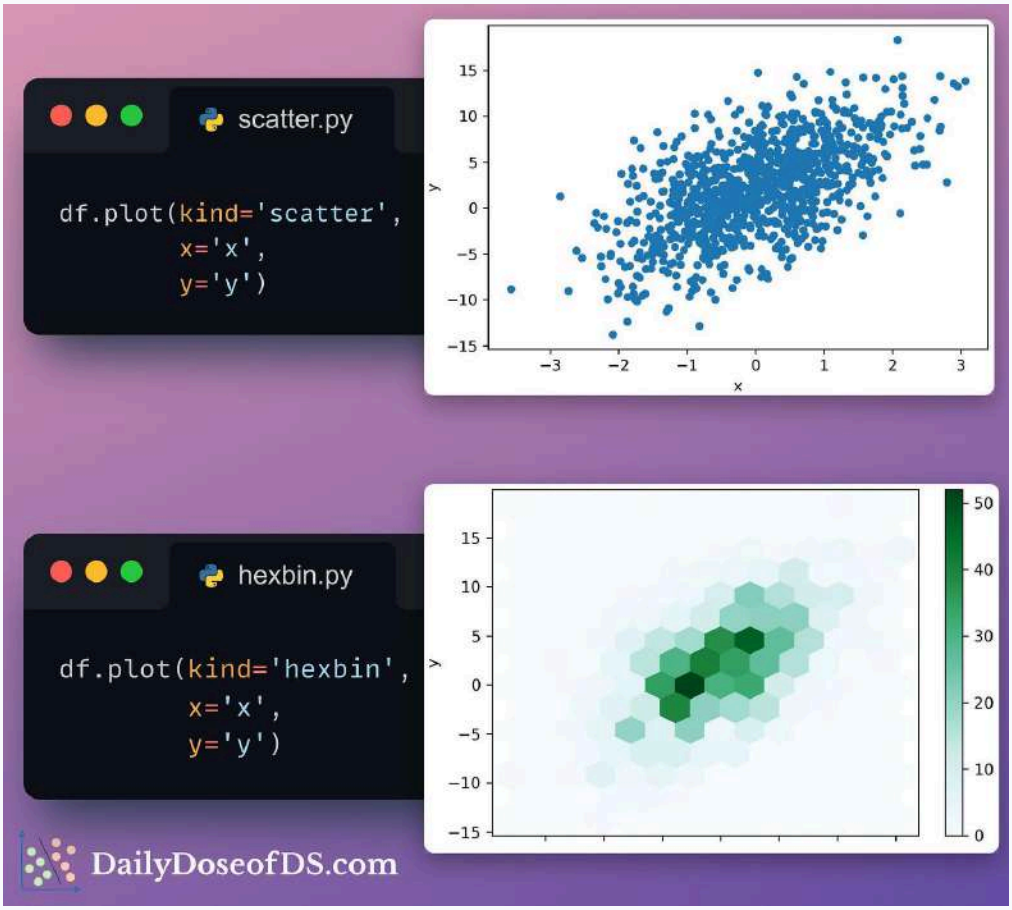
- Combine multiple plots to prevent incorrect/misleading conclusions
- Reduce clutter and enhance clarity
- Improve comparisons between groups
- Capture different aspects of the data through a single plot

#5-6) Hexbin and Density Plots

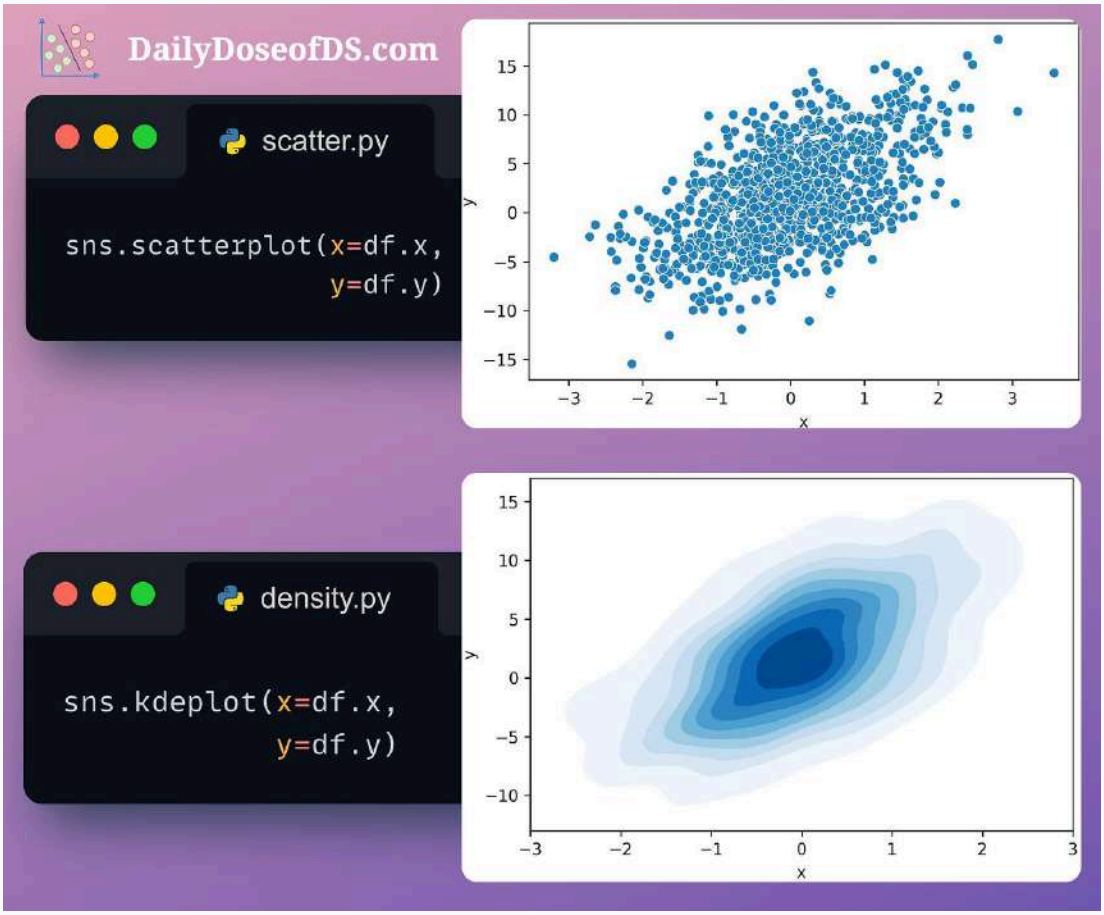
Scatter plots can get too dense to interpret when you have thousands of data points.

Instead, you can replace them with Hexbin plots.

Hexbin plots bin the area of a chart into hexagonal regions. Each region is assigned a color intensity based on the method of aggregation used (the number of points, for instance).



Another choice is a density plot, which illustrates the distribution of points in a two-dimensional space.



A contour is created by connecting points of equal density. In other words, a single contour line depicts an equal density of data points.

#7-8) Bubble charts and Dot plots

As discussed above, bar plots quickly get messy and cluttered as the number of categories increases.

A bubble plot is often a better alternative in such cases.

They are like scatter plots but:

- with one categorical axis
- and one continuous axis



As depicted above:

- It is difficult to interpret the bar plot because it has too many bars packed into a small space,
- But size-encoded bubbles make it pretty easy to visualize the change over time.

Another alternative to bar plots in such situations is dot plots.



Both dot plots and bubble charts are based on the idea that, at times, when we have a bar plot with many bars, we're often not paying attention to the individual bar lengths.

Instead, we mostly consider the individual endpoints that denote the total value.

These plots precisely help us depict that while also eliminating the long bars of little to no use.

Interactive Controls

While using Jupyter, one often finds themselves in situations where they repeatedly modify a cell and re-rerun it. This makes data exploration irreproducible and time-consuming. What's more, the notebook also gets messy and cluttered.

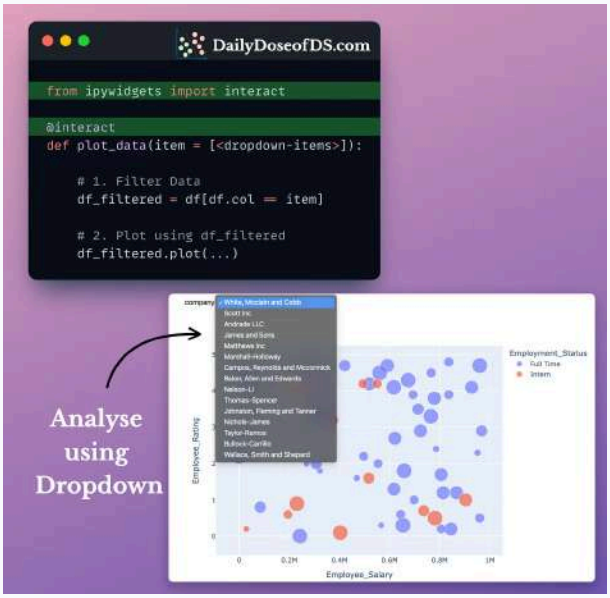
To address this, one of the things I actively leverage in my Jupyter notebooks is interactive controls using `IPywidgets`.

A single decorator (`@interact`) allows us to add:

- sliders
- dropdowns
- text fields, and more.

As a result, one can:

- explore the data interactively
- speed-up data exploration
- avoid repetitive cell modifications and executions
- organize the data analysis.

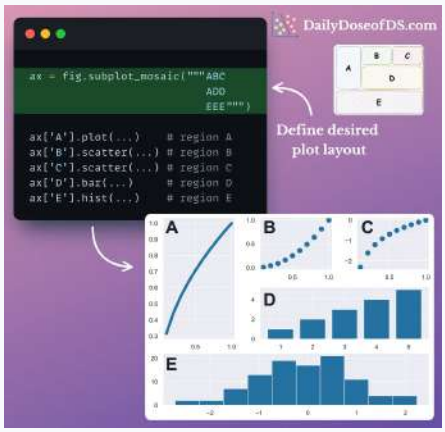


A Jupyter Notebook is available here to get started: https://bit.ly/3XZtsea.

Mosaic Plots

We mostly use `plt.subplots()` method to create subplots using Matplotlib. But this, at times, gets pretty tedious and cumbersome. For instance, it offers limited flexibility to create a custom layout, it is prone to indexing errors, and more.

Instead, use the `plt.subplot_mosaic()` method. Here, you can create a plot of any desired layout by defining the plot structure as a string.

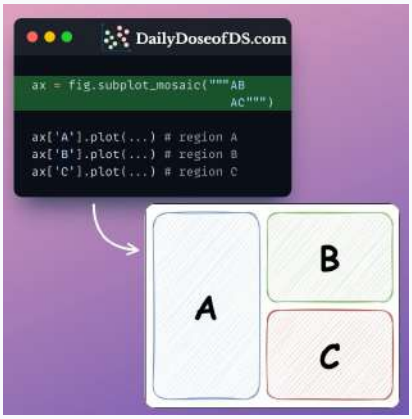


For instance, the string layout:

- AB
- AC

...will create three subplots, wherein:

- subplot "A" spans 1st column
- subplot "B" spans top half of the 2nd column
- subplot "C" spans the bottom half of 2nd column



Next, create a subplot in a specific region by indexing the axes dictionary with its subplot key ("A", "B", or "C").



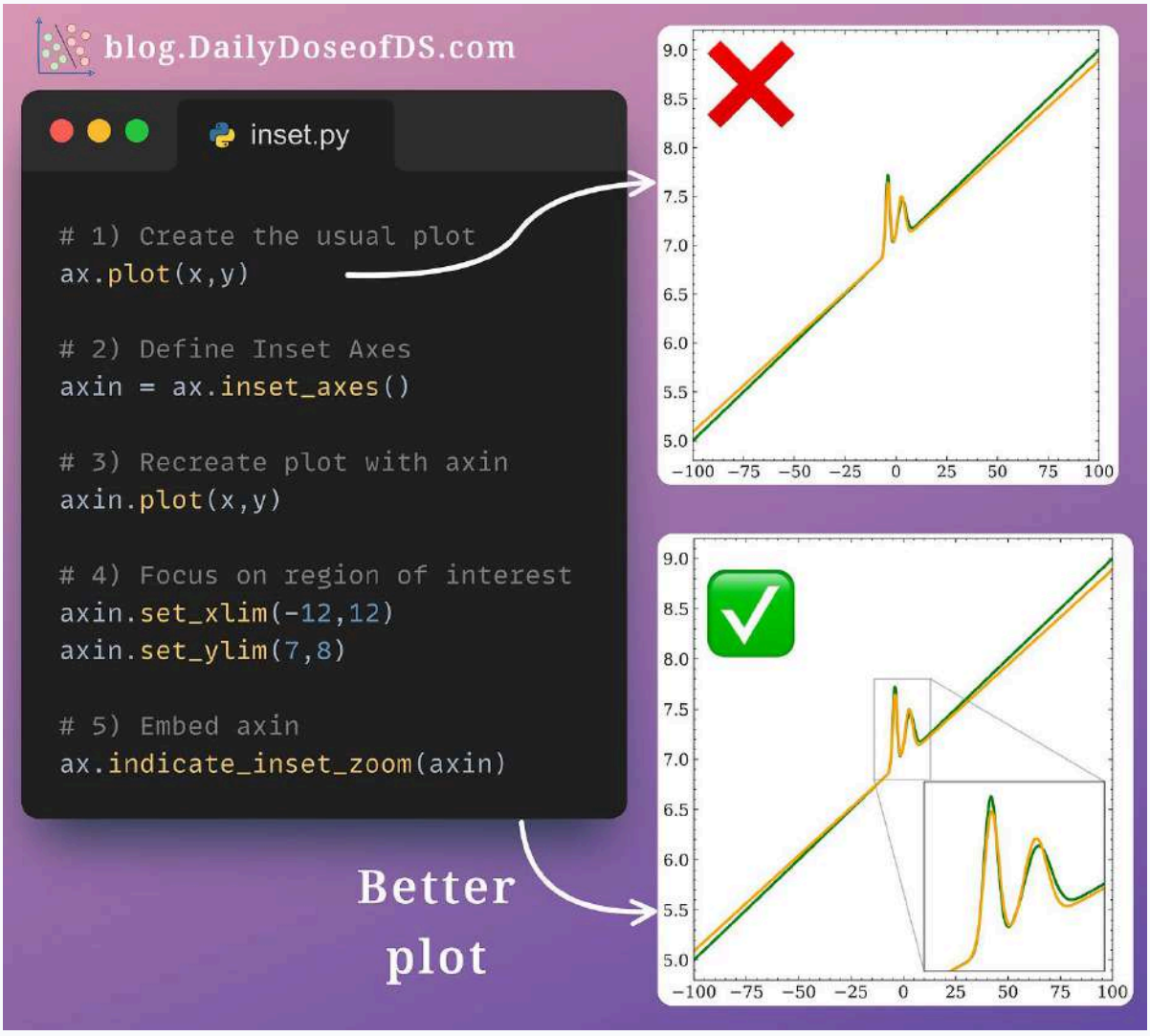
Enrich Matplotlib Plots with Inset Axis and Annotations

While creating data visualizations, there are often certain parts that are particularly important.

Yet, they may not be immediately obvious to the viewer.

A good data storyteller always ensures that the plot guides the viewer's attention to these key areas.

One great way is to zoom in on specific regions of interest in a plot, as depicted below.



In contrast to the usual plot, the other plot guides the viewer's attention to a specific area of interest.

Such efforts always ensure that the plot communicates what we intend it to depict — even if the plot's creator is not present at that time.

In matplotlib, we can do so using indicate_inset_zoom(). It adds an indicator box, which can be zoomed-in for better clarity.

The embedded plot is treated like any other matplotlib plot. Thus, we can add axis labels to it, if needed.

Another great way to provide extra info is by adding text annotations to a plot, as depicted below:



Such efforts always ensure that the plot indeed communicates what we intend it to depict — even if the plot's creator is not present at that time.

In matplotlib, you can use **annotate**(), as depicted below:



It adds explanatory texts to your plot, which lets you guide a viewer's attention to specific areas and aid their understanding.

Jupyter notebook for plot annotations is available here: https://bit.ly/4bzQ5Ja.

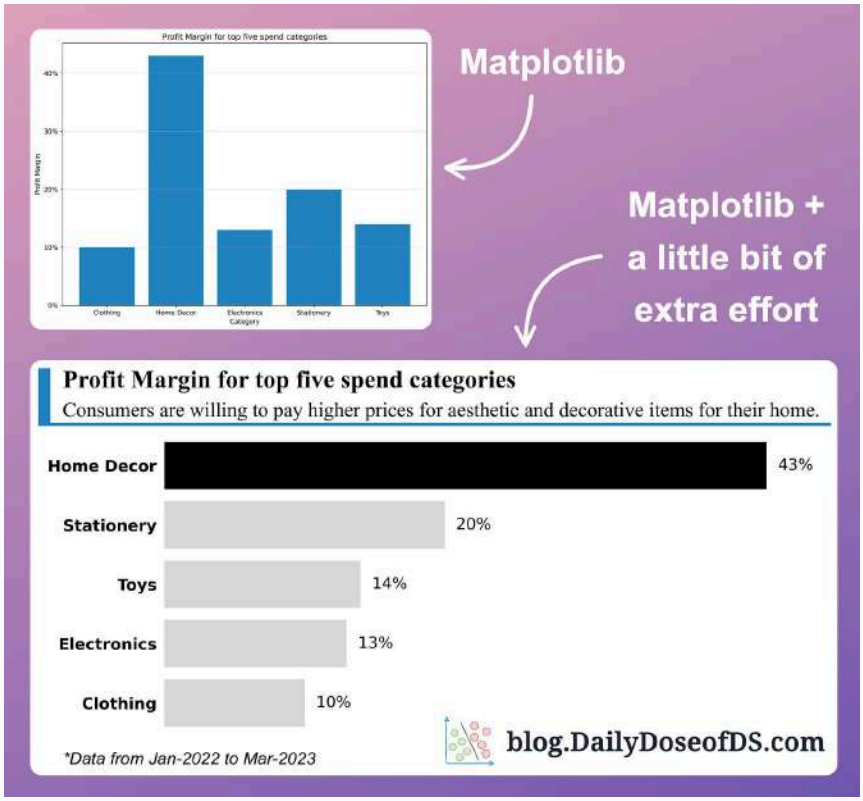
Professionalize Matplotlib Plots

Context

I have been using matplotlib for many years now. Based on that experience, I believe that one of the best yet underrated potentials of matplotlib is the amount of customizability it offers.

But being unaware of that, most matplotlib users use it as a naive plotting utility with almost zero customization. And as the default plots never appear "appealing", they resort to other libraries, Plotly, for instance, to create elegant plots.

Yet, I believe that in 90-95% of cases, you would NEVER need to look beyond matplotlib. It can do much more than what most users think. For instance, consider the two plots in this image.



Yes! Both plots were created using matplotlib.

But some custom formatting makes the second plot much more elegant, informative, appealing, and easy to follow.

- The title and subtitle significantly aid the story.

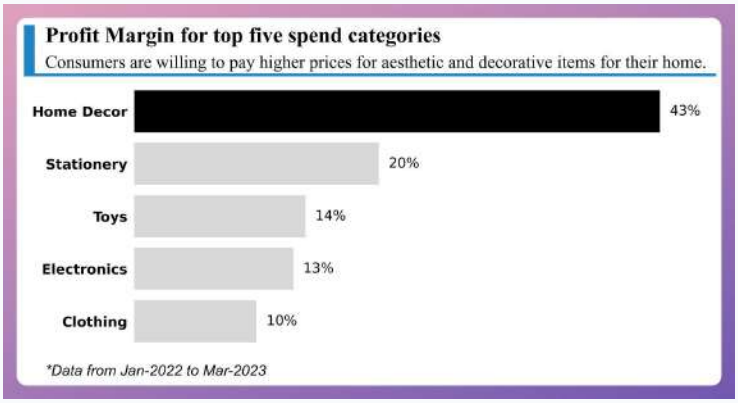
- Also, the footnote offers extra important information, which is nowhere to be seen in the basic plot.
- Lastly, the bold bar immediately draws the viewer's attention and conveys the purchase category's importance.

Thus, in my opinion, the overwhelming potential for customization makes matplotlib far more capable than what most users think.

Conclusion

One of the things I always ensure towards being a good storyteller in my data science projects is that my plot must demand minimal effort from the viewer. Thus, I never shy away from putting in that extra effort.

This has been especially true for professional environments. At times, it is also good to ensure that our visualizations convey the right story, even if they are viewed in our absence. The below plot is a classic example of that.

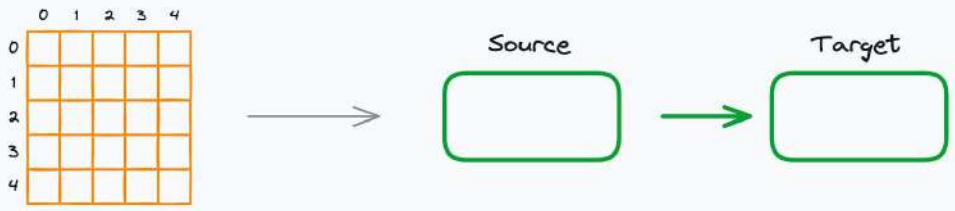


In this entire chapter, I never discussed what that plot is about — somewhat indicating my absence. Yet, by staring at this plot for a few seconds, you can quickly figure out what I intended to highlight here, can't you?

You can download the code notebook for this post here: https://bit.ly/3zv8pWG.

Sankey Diagrams

Many tabular data analysis tasks can be interpreted as a flow between several source and target entities.



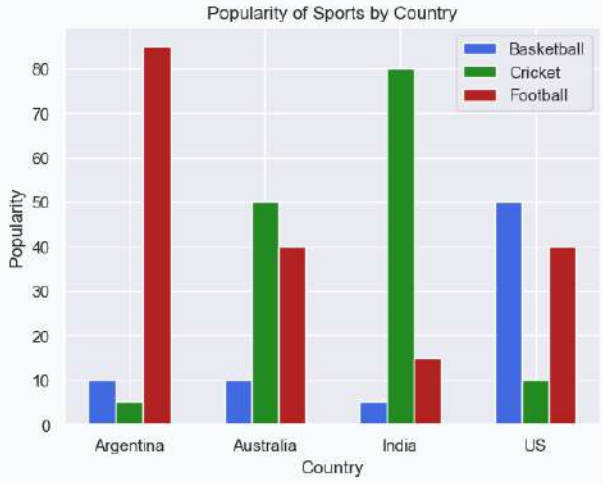
For instance, consider we have a sports popularity dataset, which lists the popularity index of a sport in a country.

In this dataset:

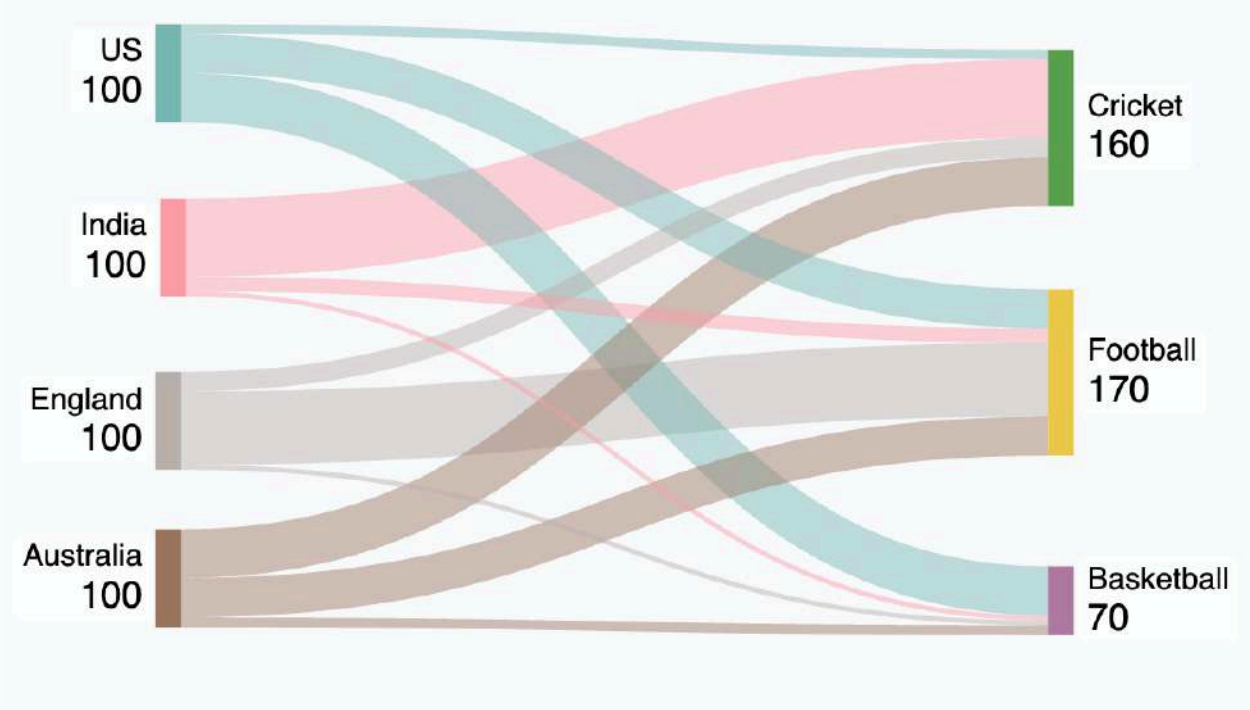
- Countries are entities.
- Sports are entities.
- Information flowing between them is the popularity value.

	Country	Sport	Popularity
0	US	Cricket	10
1	US	Football	40
2	US	Basketball	50
3	India	Cricket	80
4	India	Football	15
5	India	Basketball	5
6	Australia	Cricket	50
7	Australia	Football	40
8	Australia	Basketball	10

While typical plots, like grouped bar plots, could be used here to understand the popularity distribution of country-wise sports, as shown below:



...in my opinion, however, Sankey diagrams stand out as a pretty cool and elegant alternative to represent such flow datasets:



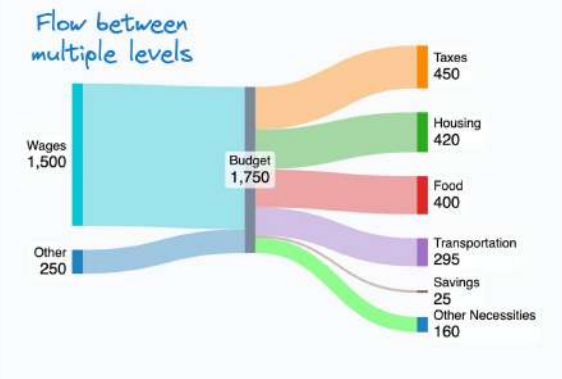
Their links are represented with arcs whose width is proportional to the value of the flow. This immensely simplifies the data analysis process. For instance, from the Sankey diagram above, one can quickly infer that:

- The most popular sport in India is Cricket.
- Basketball and Football are almost equally popular in the US.
- Basketball is hardly popular in India and England.
- England's most popular sport is Football.
- Cricket and Football are almost equally popular in Australia.
- Overall, Football is the most popular sport in this dataset.
- and many many more.

Imagine doing that by looking at the tabular data or a grouped bar chart.

- This process will be time-consuming.
- You may miss out on a few insights.
- The grouped bar chart can appear pretty cluttered and messy at times.

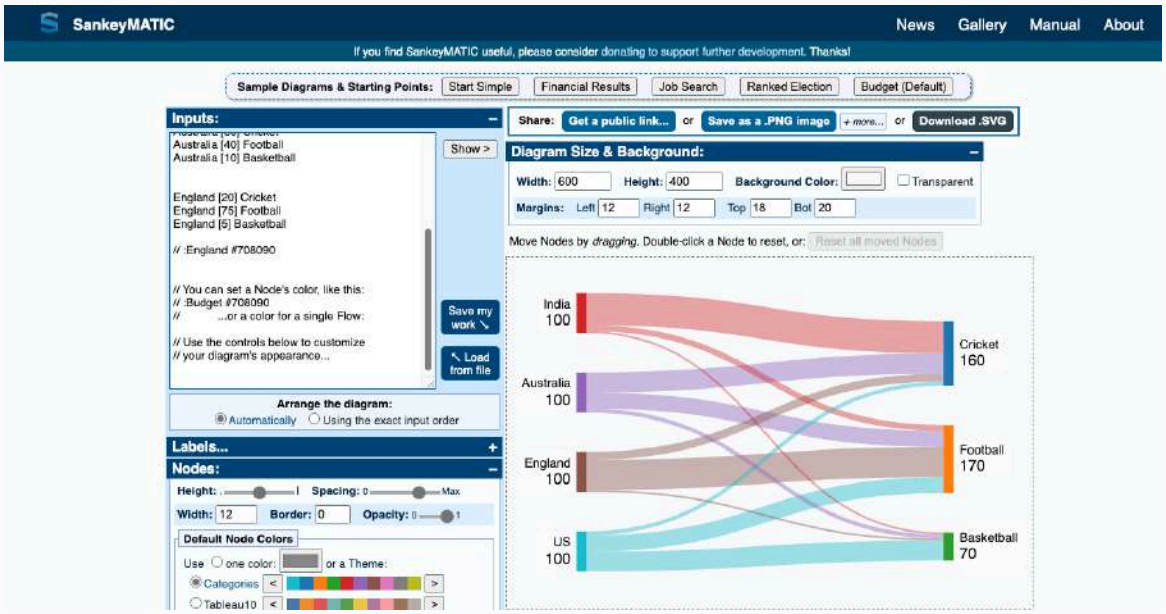
Of course, Sankey diagrams can have multiple levels as well, as shown below:



To determine when to use Sankey diagrams, see if the data involves any kind of flow of resources, energy, or information flow between multiple stages or entities. If yes, Sankey diagrams could be pretty valuable.

There are multiple ways to create Sankey diagrams:

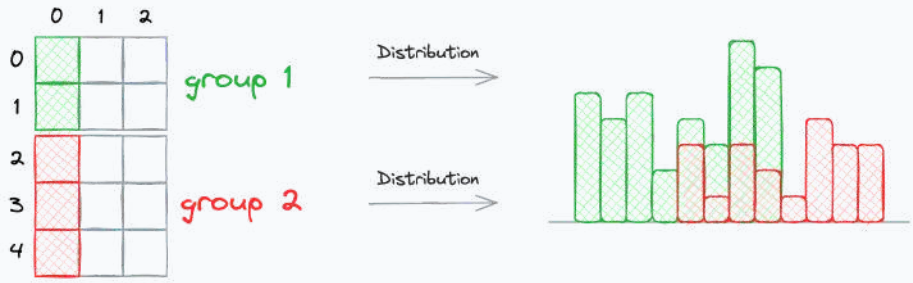
- To generate them programmatically, you may use the ipysankeywidget library: https://github.com/ricklupton/ipysankeywidget.
- If you prefer GUI, SankeyMATIC is a pretty cool, and easy-to-use tool to create Sankey diagrams which I often use: https://bit.ly/3WdMAnx.



Ridgeline Plots

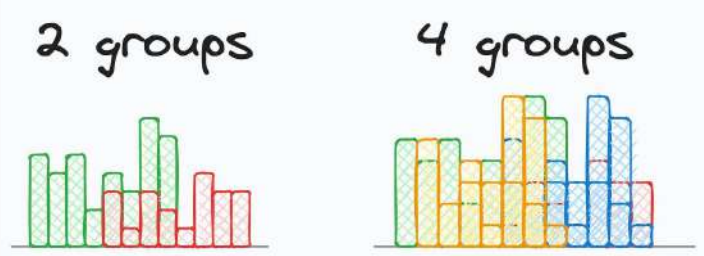
Understanding the distributional differences of distinct groups in a variable is quite useful in uncovering insights around:

- behavioral disparities,
- feature engineering,
- predictive modeling, and more.



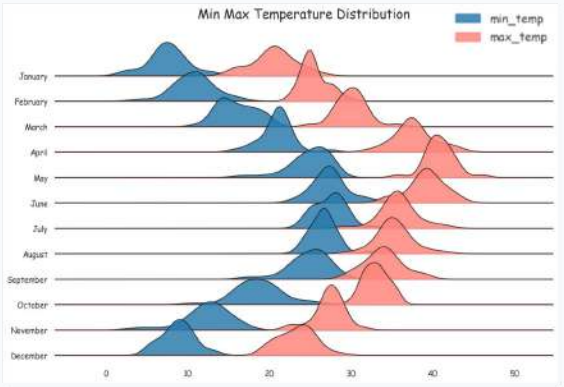
But in such situations, many data scientists tend to create group-level distribution plots (histograms or density plots) on a single axis and compare them.

While this is (somewhat) okay when there are limited groups, in the presence of many groups, it can create cluttered plots, which may not reveal many insights about distributional differences:

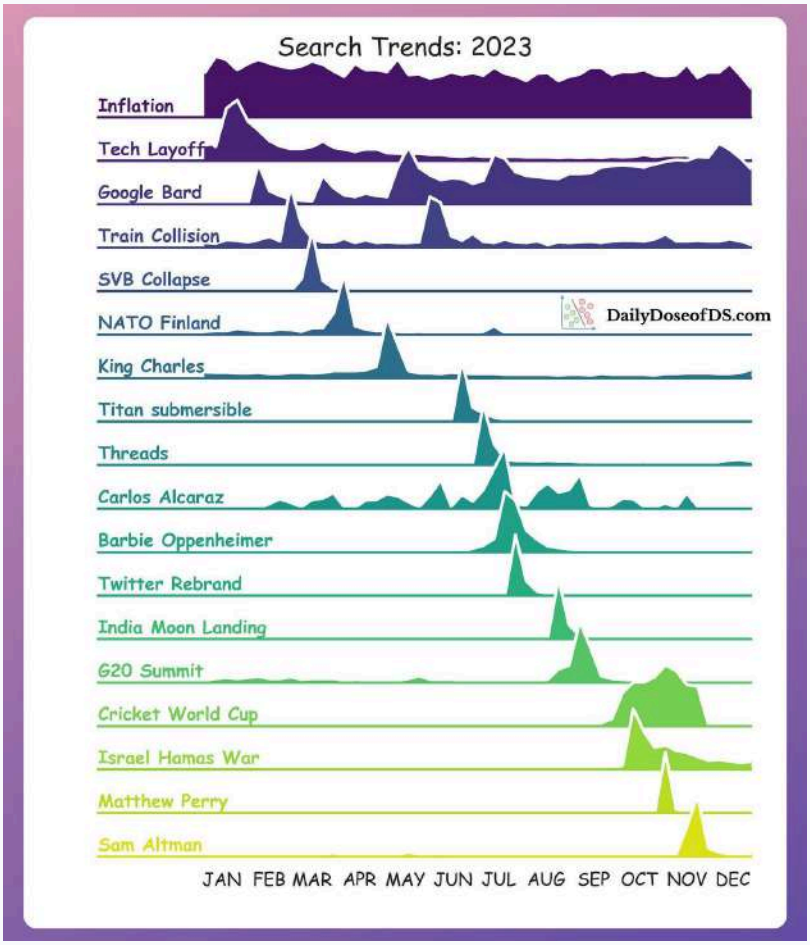


Ridgeline plots (shown below) are a pretty compact and elegant way to visualize the distribution of different variables (or categories of a variable).

More specifically, the vertical stacking on a common axis provides an easy comparison between groups and reveals many insights into the shape and variation of the distributions, which otherwise would be difficult to understand.



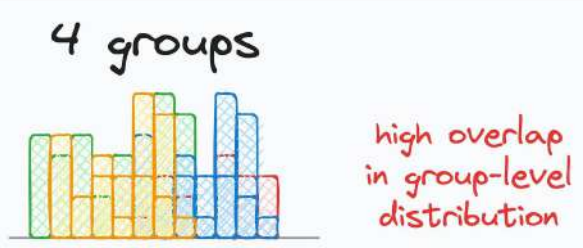
This allows us to compare the distributions of multiple groups side by side and understand how they differ. The image below is another classic example of Ridgeline plots. It depicts the search interest across various events that happened in 2023, and it's so easy to visualize.



While Seaborn provides a way to create Ridgeline plots, I have often found the Joypy library to be pretty useful and easy to use.

When to consider Ridgeline plot?

Typically, creating a Ridgeline plot makes sense when the variable has anything above 3-4 groups. This is to avoid the overlap that might appear when visualizing them in a single plot:



Also, Ridgelines plots are relatively more useful when there is a clear pattern and/or ranking on the continuous variable plotted between groups like:

- monotonically increasing,
- monotonically decreasing,
- increasing then decreasing,
- decreasing then increasing, etc.

That is why the order in which you vertically stack the distribution of groups becomes quite important. For instance, consider the above "Search trends" plot again but with a random arrangement of groups:



I don't think I have to ask you which one is easier to visualize and understand the flow of events in 2023. So these were some points that will help you determine whether a Ridgeline plot will be a good fit for visualizing your data.

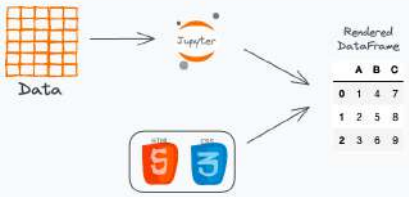
I created this notebook for you to get started with Ridgeline plots using JoyPy: https://bit.ly/3xMUxXj.

Sparkline Plots

When doing any data analysis task in Jupyter, we mostly create standalone charts and visualizations. Of course, there's nothing wrong, but in this chapter, let me introduce you to another pretty cool and elegant way I often use to create visualizations in Jupyter Notebook.

Background

Whenever we display a DataFrame in Jupyter, it is rendered using HTML and CSS.



This means we can format its output just like any other web page. Moreover, when we create any plot in Jupyter, it is rendered as an image.

However, the same image can also be rendered within an HTML image tag in Jupyter.

And magic happens when we combine the above two ideas — rendering plots inside a DataFrame, as depicted below:



Formally, these compact plots are called Sparklines, and they provide a pretty elegant way to visualize data without taking up too much space.

Unlike traditional plots, we typically do not create axis ticks and labels in Sparklines.

One of the coolest ways I prefer using Sparklines is by adding them to a DataFrame's cell, as depicted in the image above.

A Jupyter Notebook on creating Sparkline plots is available here: https://bit.ly/4eVhVmf.

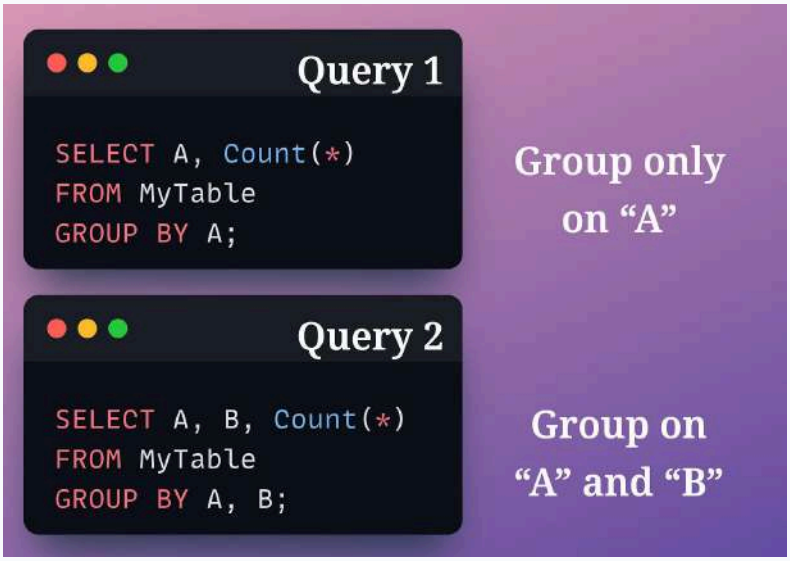
In this notebook, I have provided the step-by-step procedure to embed Sparklines in a DataFrame.

SQL

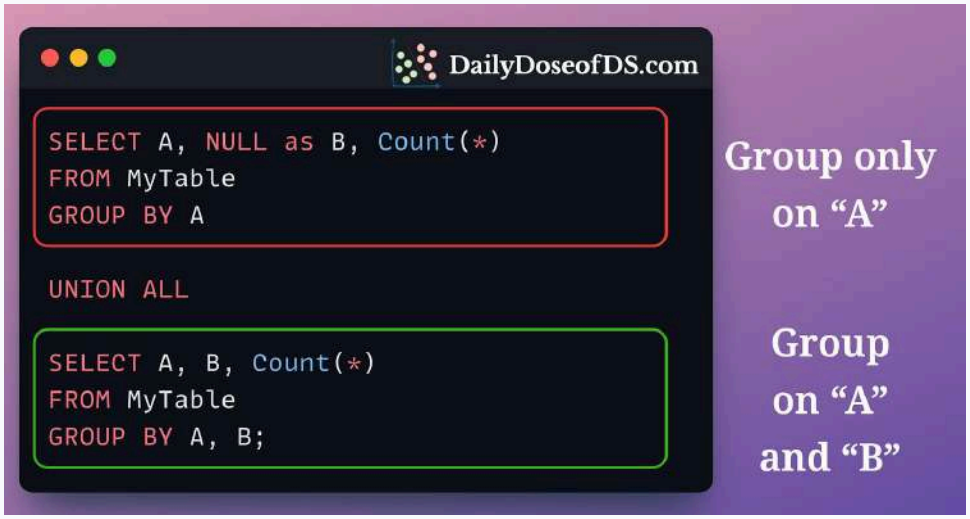
Grouping Sets, Rollup and Cube in SQL

A typical GroupBy query aggregates on just one set of columns. For instance:

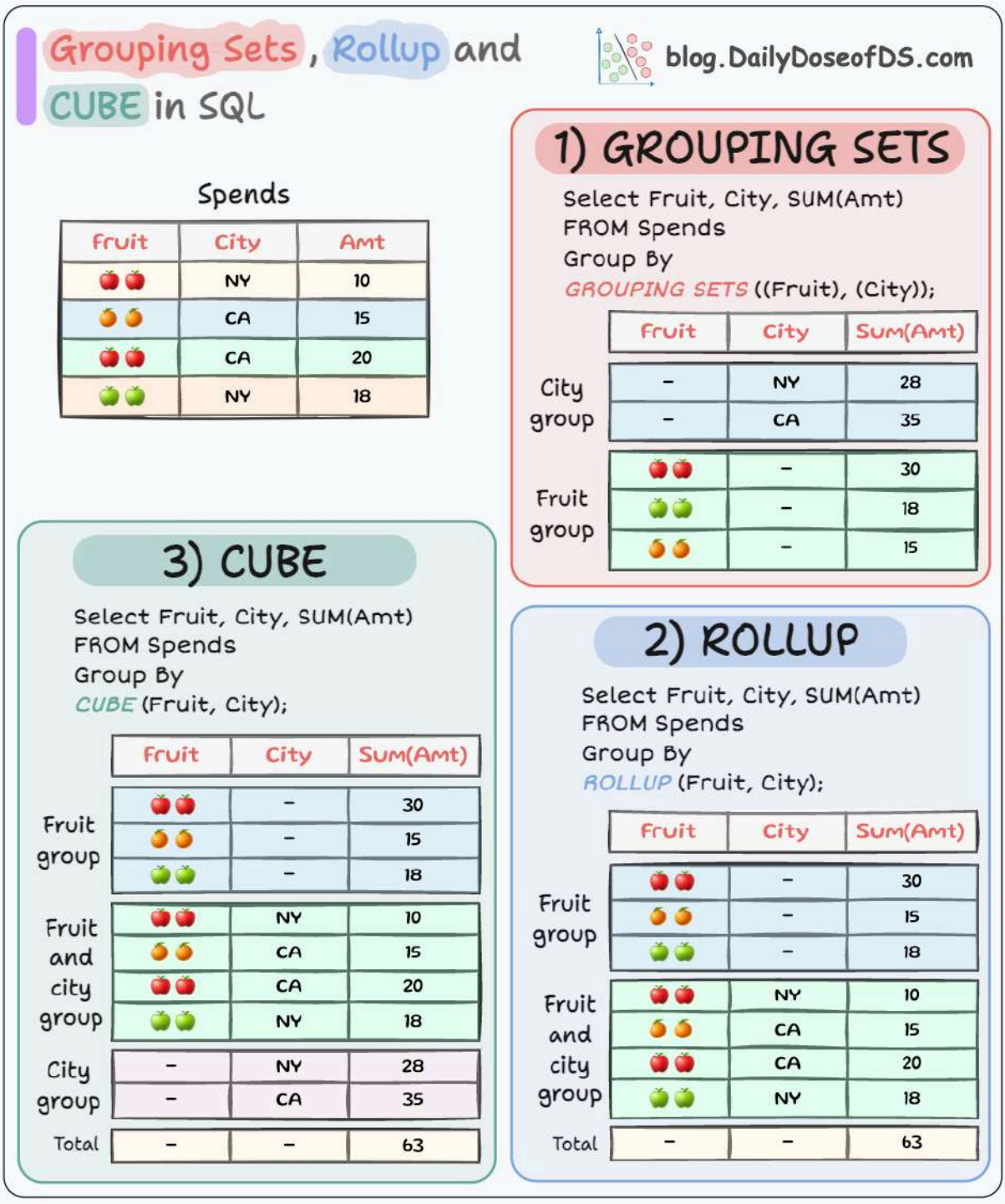
- Grouping data on column "A" will require one query.
- Grouping data on columns "A" and "B" will require a separate query.



Next, if the two outputs must be gathered in a single table, we use UNION or UNION ALL (as needed).

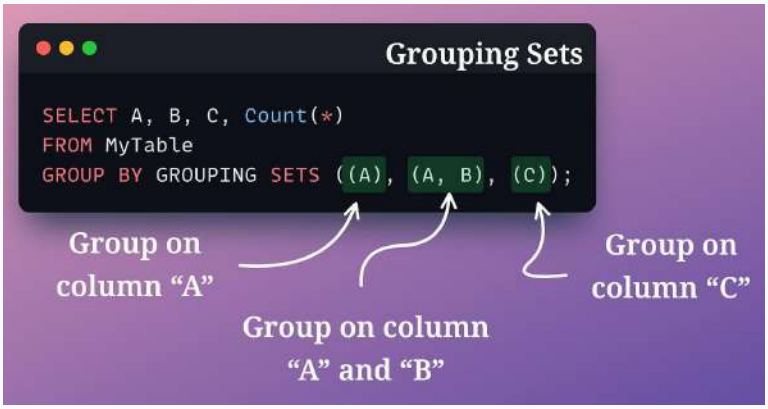


But this is not efficient as it scans the same table twice. Instead, there are three ways to run multiple aggregations on the same table by scanning the table just once. This makes our query much more efficient. These are Grouping Sets, Rollup, and Cube, and their usage has been depicted in the image below:



#1) Grouping sets

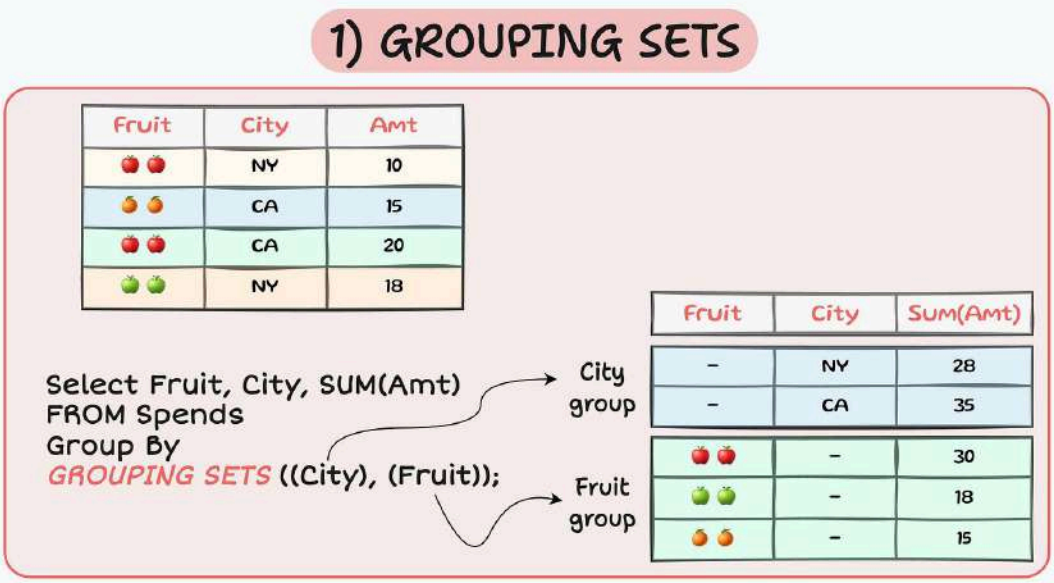
The GROUPING SETS clause allows us to define multiple groupings in a single query. Each grouping set defines a combination of columns by which the data is grouped.



Given the above query, here are the group aggregations that will be created:

- (A): Aggregated by A, counting all rows across all other columns.
- (A, B): Aggregated by (A, B), counting all rows across all other columns.
- (C): Aggregated by C, counting all rows across all other columns.

A demonstration is shown below:



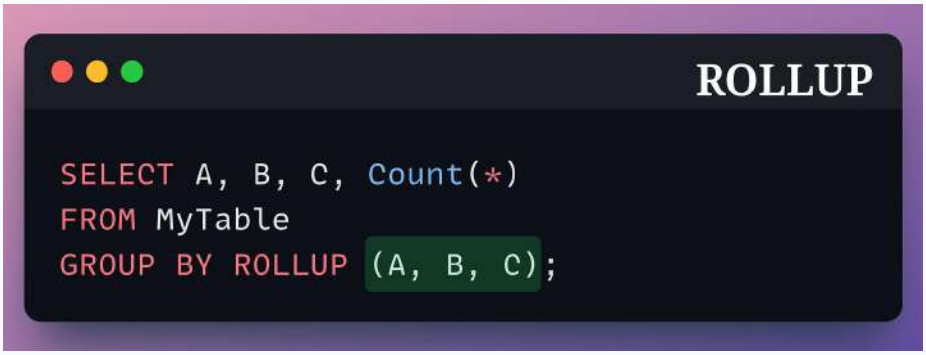
The above query:

- Generates an aggregation on City column.
- Generates another aggregation on Fruit column.

#2) ROLLUP

ROLLUP creates a result set that includes subtotals and a grand total in addition to the regular grouped results.

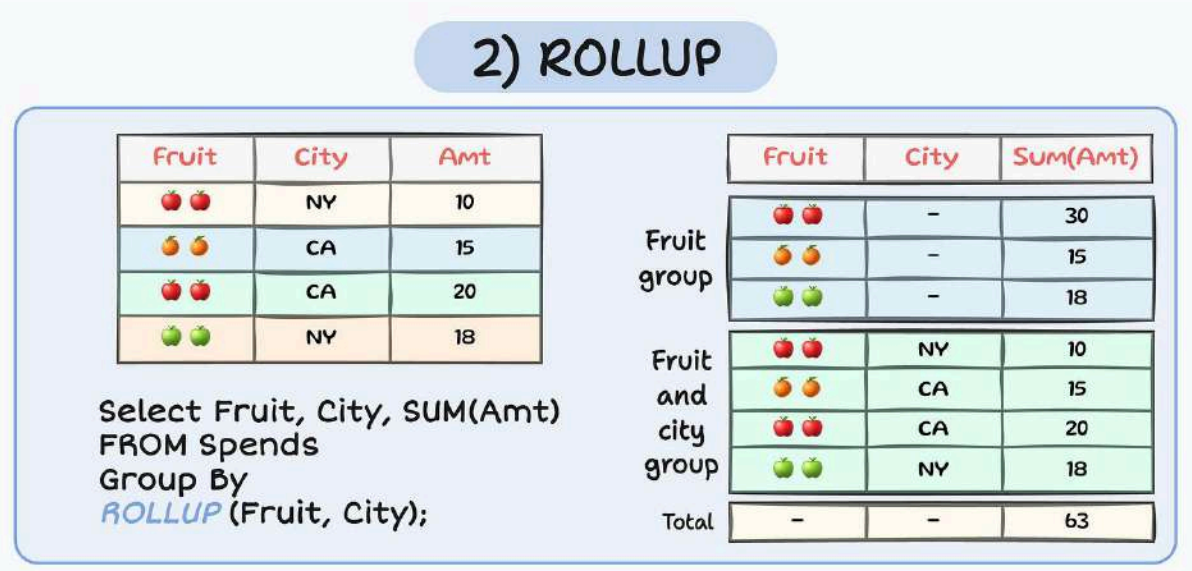
It does this by grouping the data at multiple levels of aggregation.



Given the above query, here are the group aggregations that will be created:

- (A): Subtotal for each A, aggregated across all B and C.
- (A, B): Subtotal for each A and B combination, aggregated across all C.
- (A, B, C): Regular group by all three columns.
- (): Grand total, aggregated across all A, B, and C.

A demonstration is shown below:



The above query:

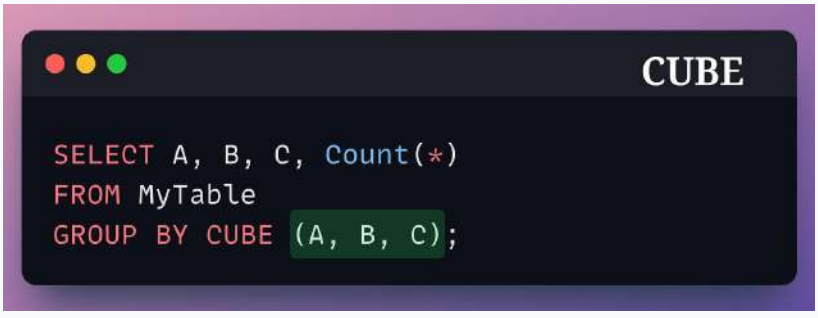
- Generates an aggregation on Fruit column.
- Generates another aggregation on (Fruit, City) column.
- Generates a grand total.

Unlike Grouping Sets, the order is important in ROLLUP.

More specifically, ROLLUP (A, B) will not be the same as ROLLUP (B, A).

#3) CUBE

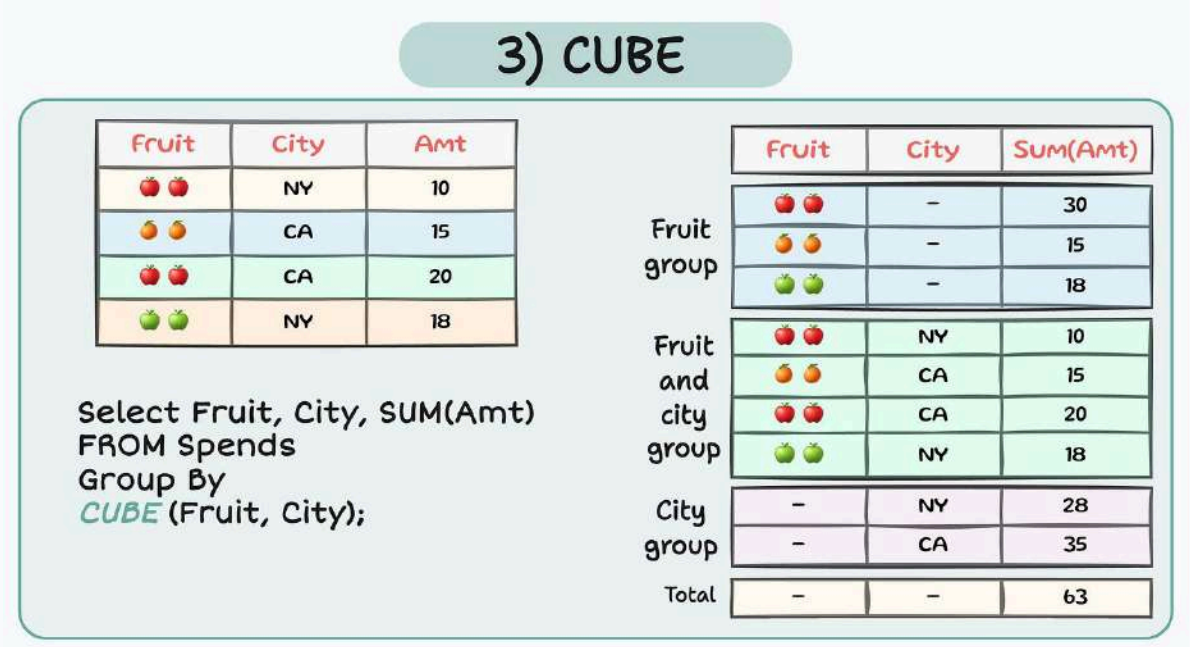
Finally, CUBE creates a result set that includes all possible combinations of aggregations for the specified columns.



Given the above query, here are the group aggregations that will be created:

- (A, B, C): Regular group by all three columns.
- (A, B): Subtotal for each A and B combination, aggregated across all C.
- (A, C): Subtotal for each A and C combination, aggregated across all B.
- (B, C): Subtotal for each B and C combination, aggregated across all A.
- (A): Subtotal for each A, aggregated across all B and C.
- (B): Subtotal for each B, aggregated across all A and C.
- (C): Subtotal for each C, aggregated across all A and B.
- (): Grand total, aggregated across all A, B, and C.

A demonstration is shown below:



The above query:

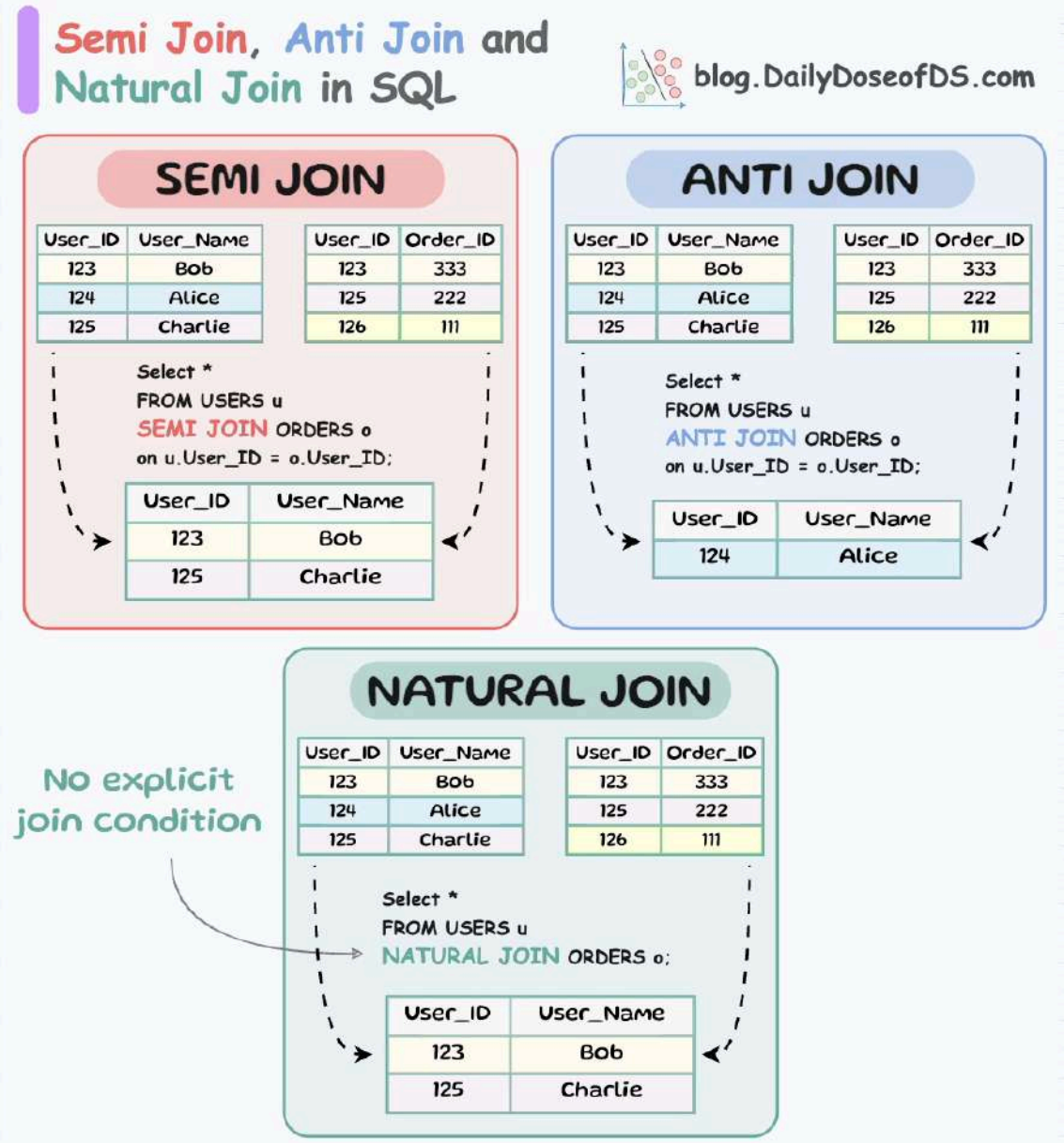
- Generates an aggregation on Fruit column.
- Generates an aggregation on City column.
- Generates another aggregation on (Fruit, City) column.
- Generates a grand total.

Try it out by downloading this Jupyter Notebook: https://bit.ly/4bx4SUX.

Semi, Anti, and Natural Joins

We all have heard of LEFT JOIN, RIGHT JOIN, INNER JOIN, and OUTER JOIN, haven't we? These four are the most prevalent types of SQL joins. But there are more.

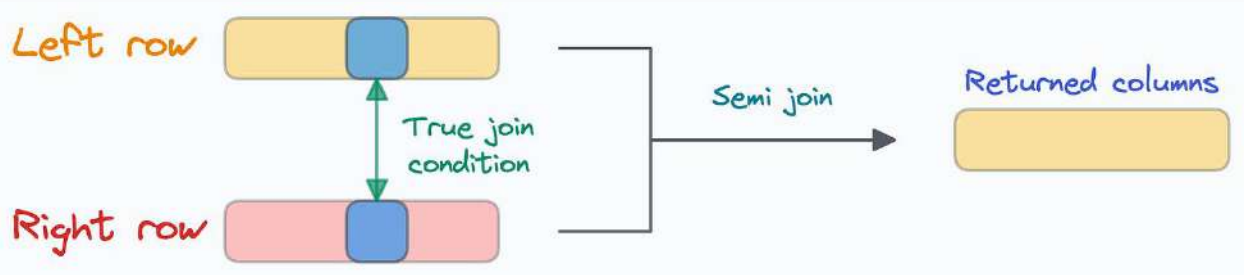
And in this chapter, I want to introduce you to three of them which I find pretty handy at times. These are – Semi Join, Anti Join and Natural Join.



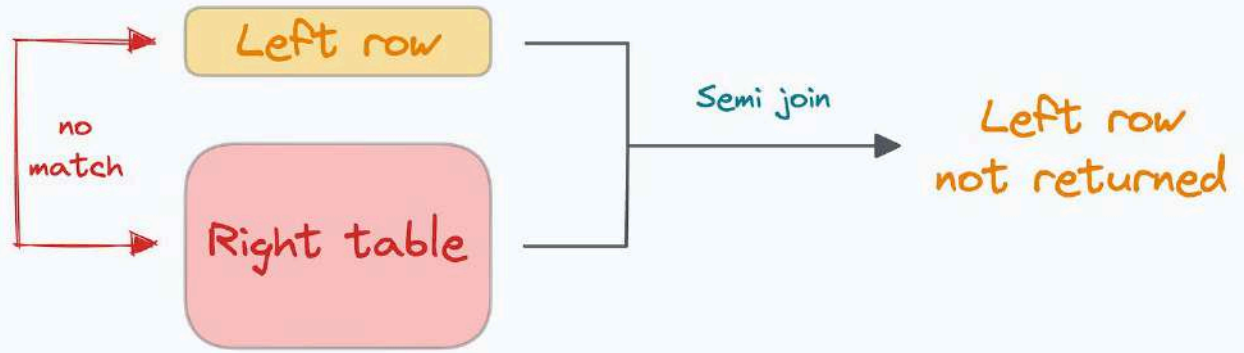
#1) Semi-Join

Semi-join appears quite similar to a left join, but there are three notable differences:

1. If the join condition between two rows is TRUE, columns from only the left table are returned. Compare this to the left join, which returns columns from both tables.

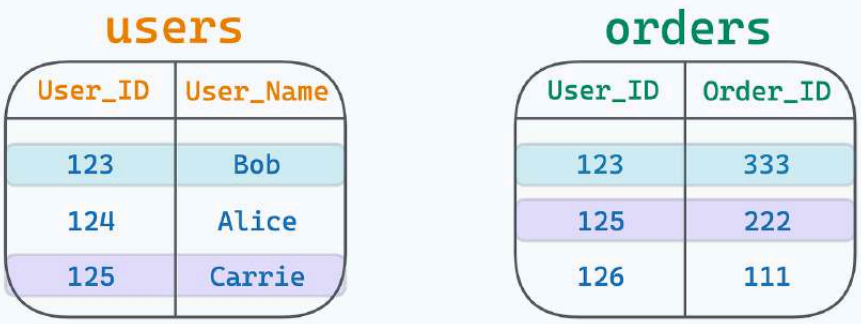


2. If a row in the left table has no match, then that row is not returned. In left join, however, all rows from the left table are returned irrespective of whether they have a match or not.

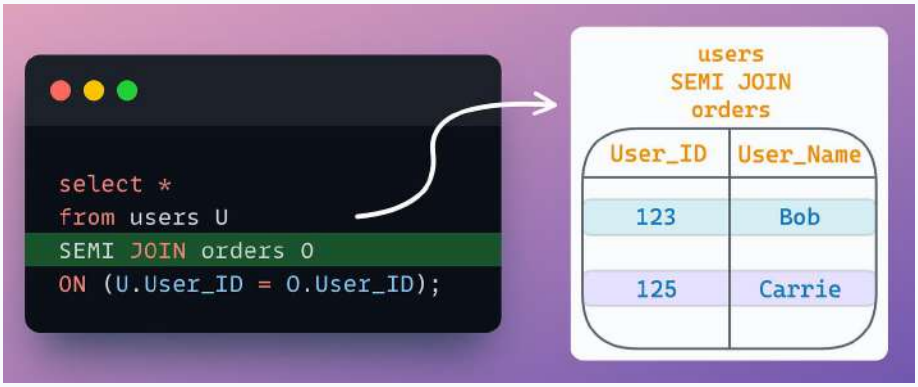


3. If a row in the left table has multiple matches, only one entry is returned. In left join, however, multiple matches are returned an equivalent number of times.

For instance, consider we have the following two tables:



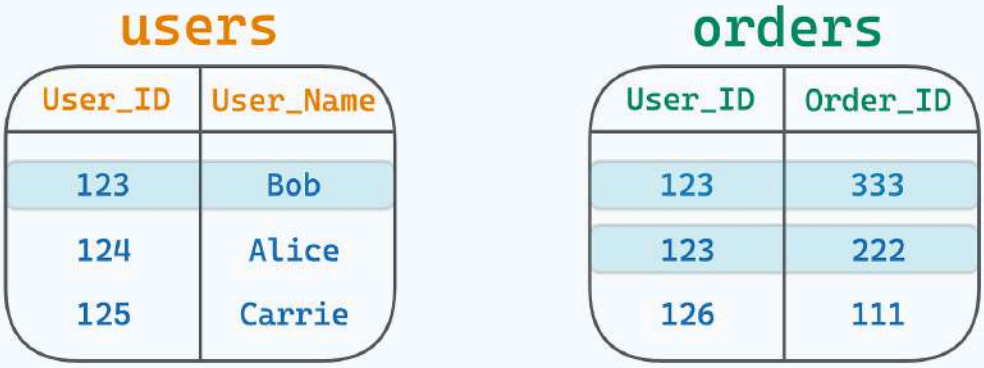
Executing a semi-join, we get the following results:



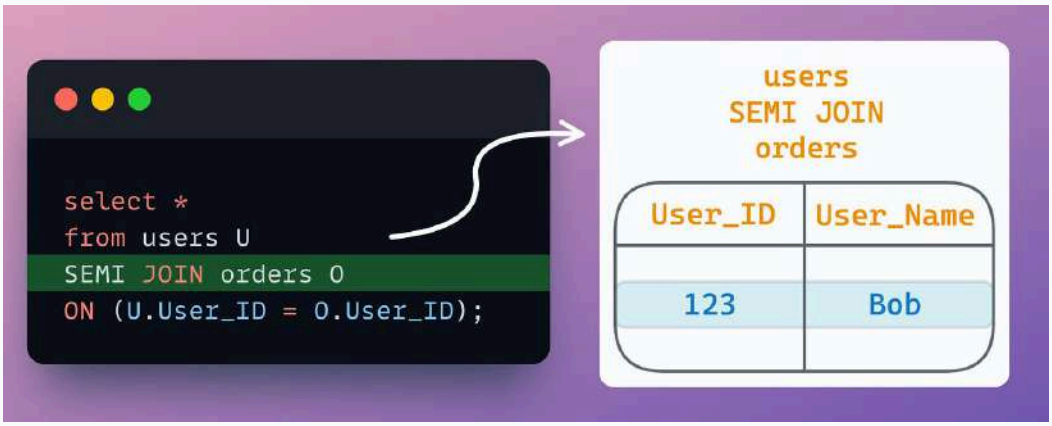
As depicted above, unlike left-join:

- It only returns columns from the left table.
- It only returns the matched rows from the left table.

If a record has multiple matches, like in this case:



...then we notice that semi-join only returns one record from the left table:



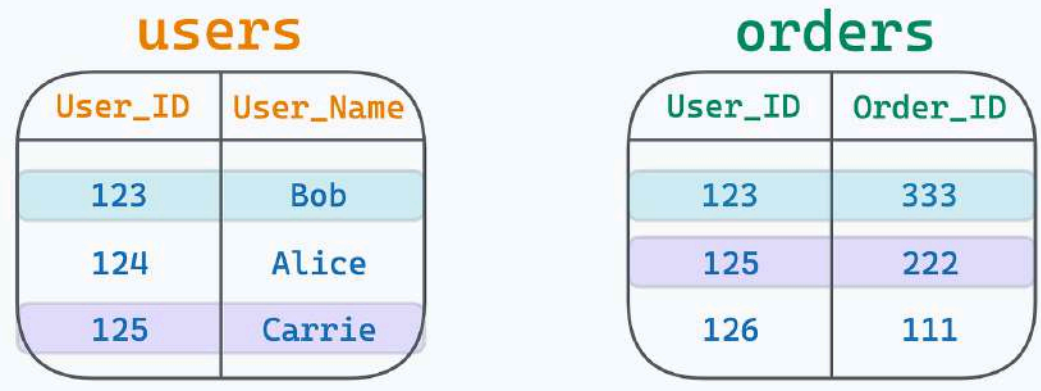
I find semi-join to be particularly useful when I only care about the existence of records in another table. Left join returns duplicates which are not of interest at that point.

#2) Anti-Join

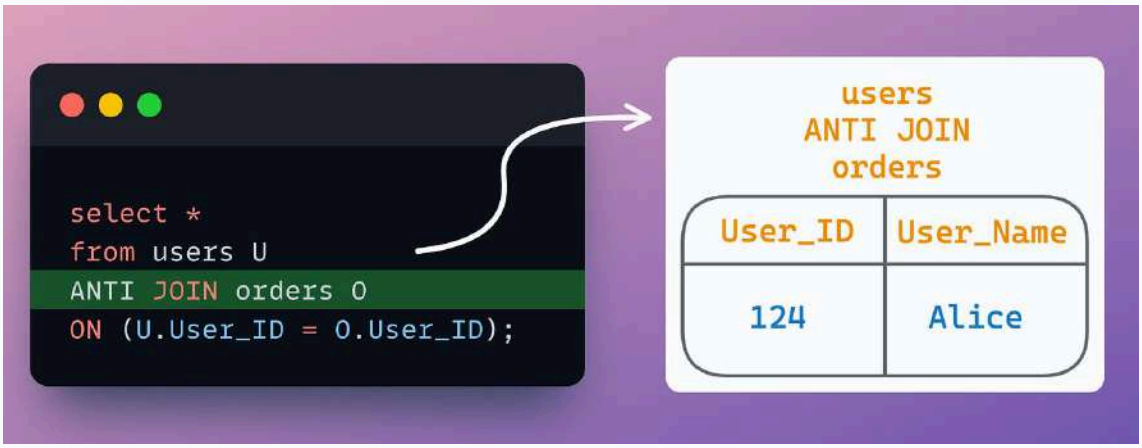
The rows discarded by the semi-join from the left table are the results of an anti-join. So, in a way, we can say that:

```
[SEMI JOIN] + [ANTI JOIN] = [LEFT TABLE]
```

Consider the above orders and users table again (one in which there were no multiple matches):



Executing an anti-join, we get the following results:



It is clear from the semi-join and anti-join results that:



Of course, the order can be different. When I say "[SEMI JOIN] + [ANTI JOIN] = [LEFT TABLE]", I mean the collection of all records.

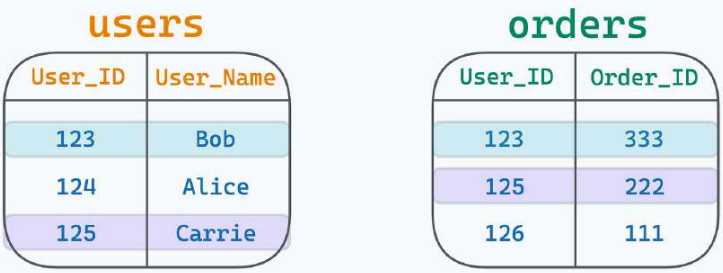
I find anti-join to be particularly useful when I wish to know which records do not exist in another table.

#3) Natural Join

This one is similar to INNER JOIN, but there's no need to specify a join condition explicitly.

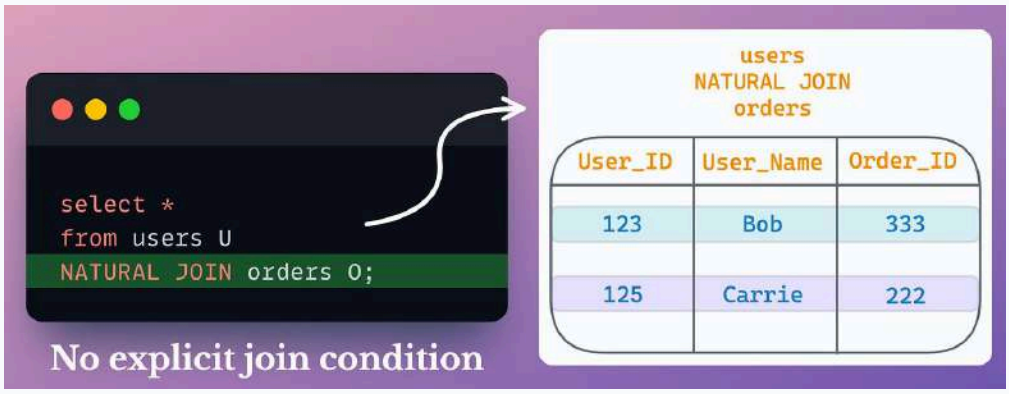
Instead, it automatically considers a join condition on ALL the matching column names.

Consider the users and orders table yet again:



Here, the User_ID column is present in both tables.

Executing a natural join, we get the following results:



As depicted above, the results are similar to what we would get with INNER JOIN.

However, we did not have to explicitly specify a JOIN condition, which, admittedly, could be good or bad.

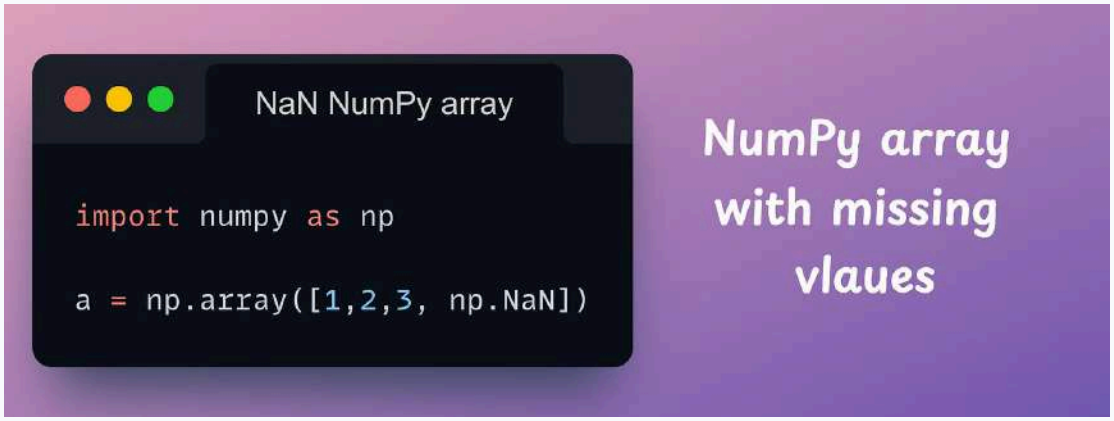
- It is good because it helps us write concise queries.
- It is bad because we are not explicit about the columns being joined.

These were three more types of SQL Joins, which I use at times to write concise and elegant SQL queries.

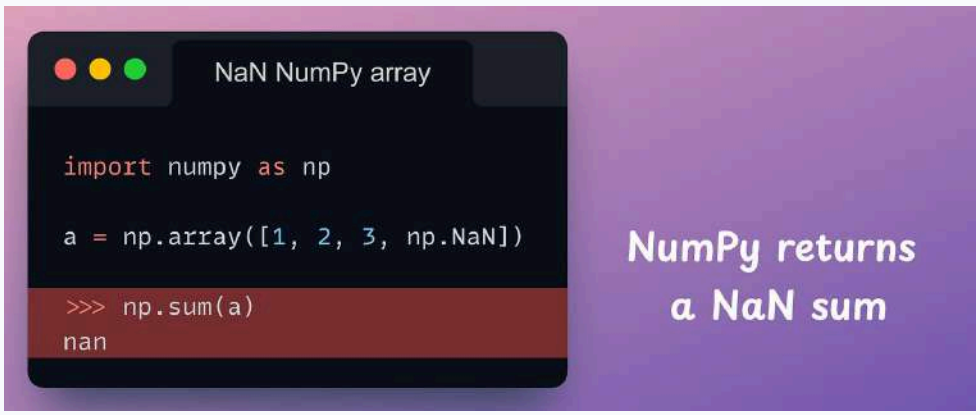
If you wish to experiment with what we discussed, download this Jupyter Notebook: https://bit.ly/4cRGMWg.

Use SQL "NOT IN" With Caution

In my experience, many unexpected errors in libraries/tools/languages can be attributed to the presence of missing values. For instance, consider a 1D NumPy array with NaN values.



When we aggregate this NumPy array to calculate, say, its sum, we get the following output:



Strange, right? Although an output of 6 may have made more sense here, NumPy produces a NaN value instead.

Anyway, this is not the topic of this chapter, but I hope you get the point

A similar silent mistake can be found in SQL as well, specifically in the usage of "NOT IN" clause, which many SQL users are not aware of.

Let's understand!

Consider we have the following tables (students and names), in our database:

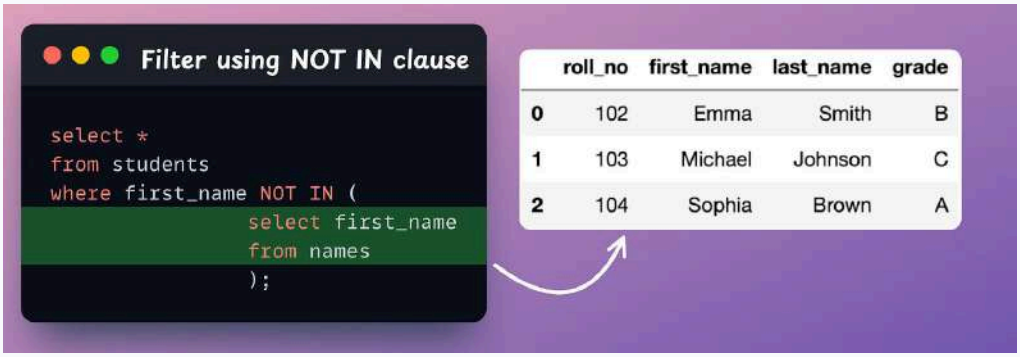
students

	roll_no	first_name	last_name	grade
0	101	John	Doe	A
1	102	Emma	Smith	B
2	103	Michael	Johnson	C
3	104	Sophia	Brown	A
4	105	Ethan	Williams	B

names

	first_name
0	John
1	Peter
2	Ethan

The task is to select records from students table where `first_name` is not in the names table. One way to do this is by using the `NOT IN` clause:



This provides the expected results as well.

Now, say our names table had a NULL value:

students

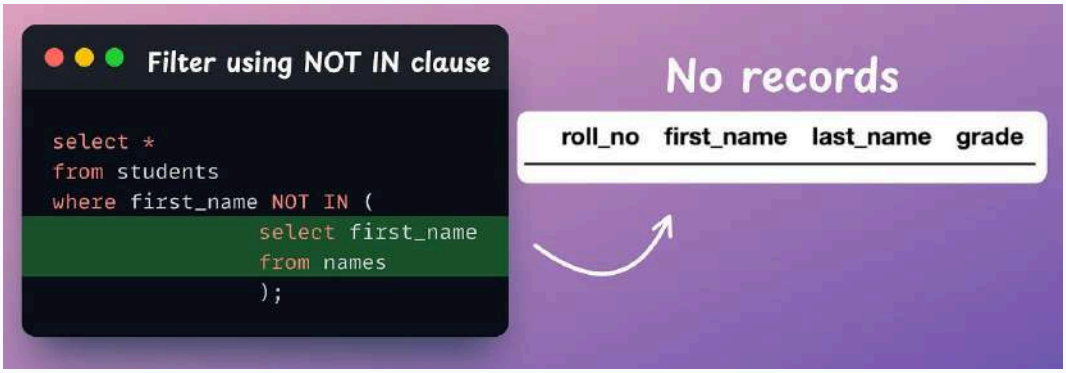
	roll_no	first_name	last_name	grade
0	101	John	Doe	A
1	102	Emma	Smith	B
2	103	Michael	Johnson	C
3	104	Sophia	Brown	A
4	105	Ethan	Williams	B

names

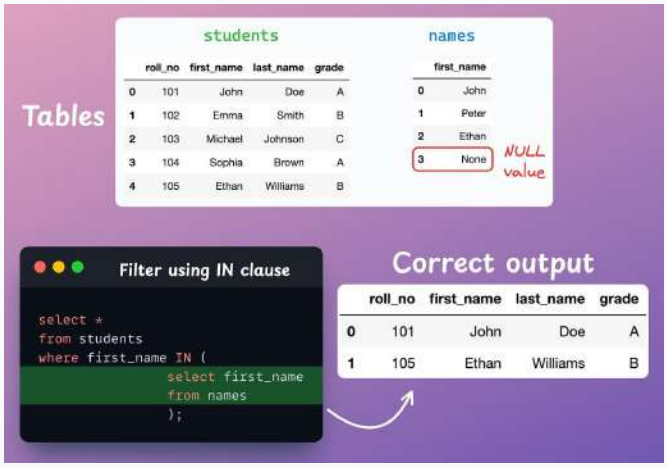
	first_name
0	John
1	Peter
2	Ethan
3	None

NULL value

If we run the above query again, we get no records this time:



On a side note, had we used the IN clause to select rows from the `students` table where `first_name` was in the names table, we notice that it works as expected:

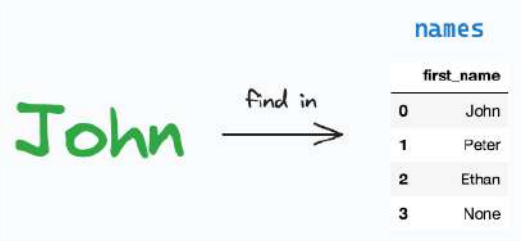


That's strange, isn't it?

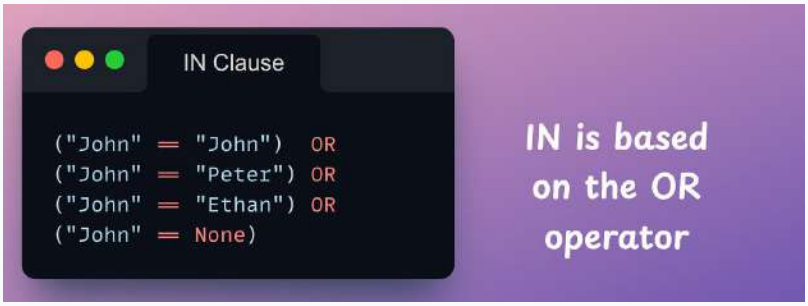
Why do we get no records with NOT IN?

The reason we get no records when we use `NOT IN` but the `IN` clause works as expected has to do with how these two clauses operate internally.

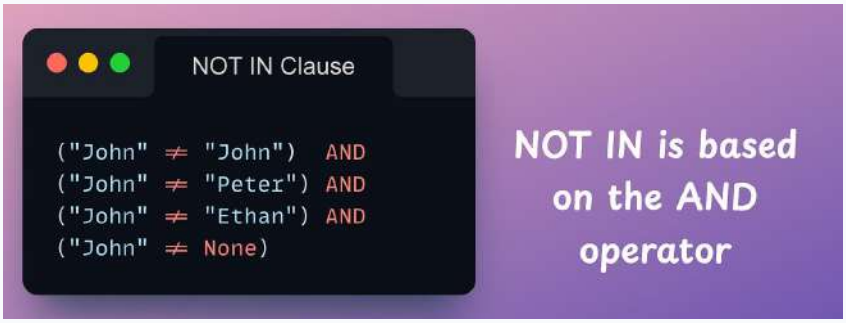
For simplicity, consider we are currently checking the first record (where first_name is "John") from the students table:



The WHERE clause needs a boolean value to determine whether a record must be filtered or not. When we use "IN", this boolean value is evaluated using the OR operator as follows:

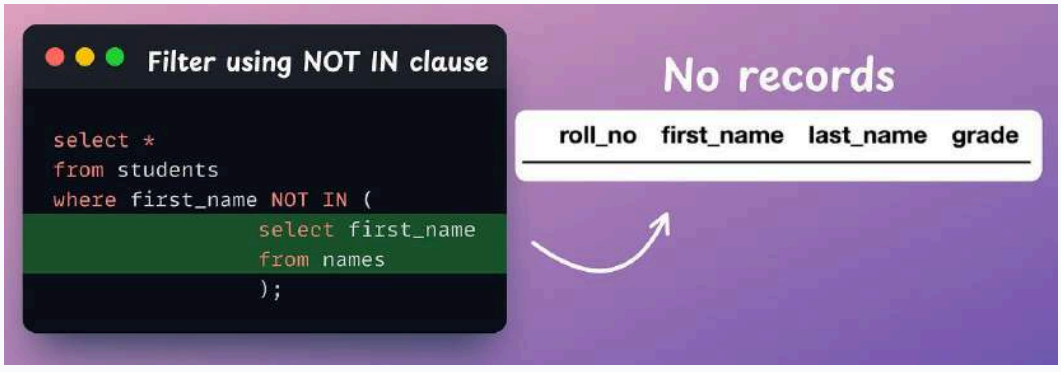


If any condition is TRUE, the row gets filtered. However, when we use "NOT IN", the boolean value is evaluated using the AND operator as follows:



For the above expression to be TRUE, all individual conditions MUST be TRUE.

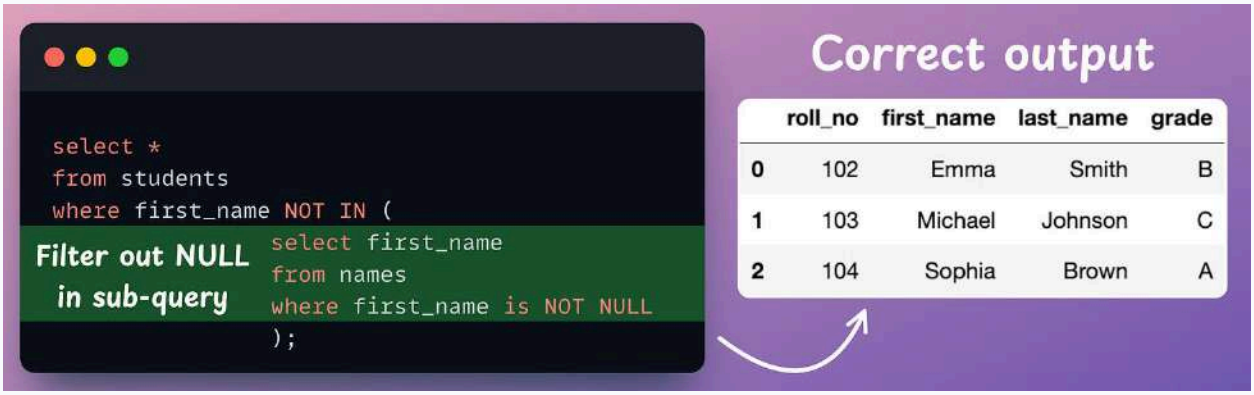
But the (JOHN != None) condition produces a conflict because, typically, this condition results in an UNKNOWN value. Thus, the entire expression evaluates to UNKNOWN — producing no records. This happens for every record in the students table. As a result, the query results in no records:



What to do instead?

There are many ways to avoid this. As our task is to select records from `students` table where `first_name` is not in the names table, we can:

- Filter out the NULL values in the sub-query:



- Use Anti Joins: Anti join returns only those rows from the left table where no match is found in the right table. This is precisely what we need in our case and it is implemented below:



At times, such mistakes can take some serious time to debug if you are not aware of them beforehand.

If you wish to experiment with what we discussed about the NOT IN clause, download this Jupyter Notebook: https://bit.ly/3xVOIXF.

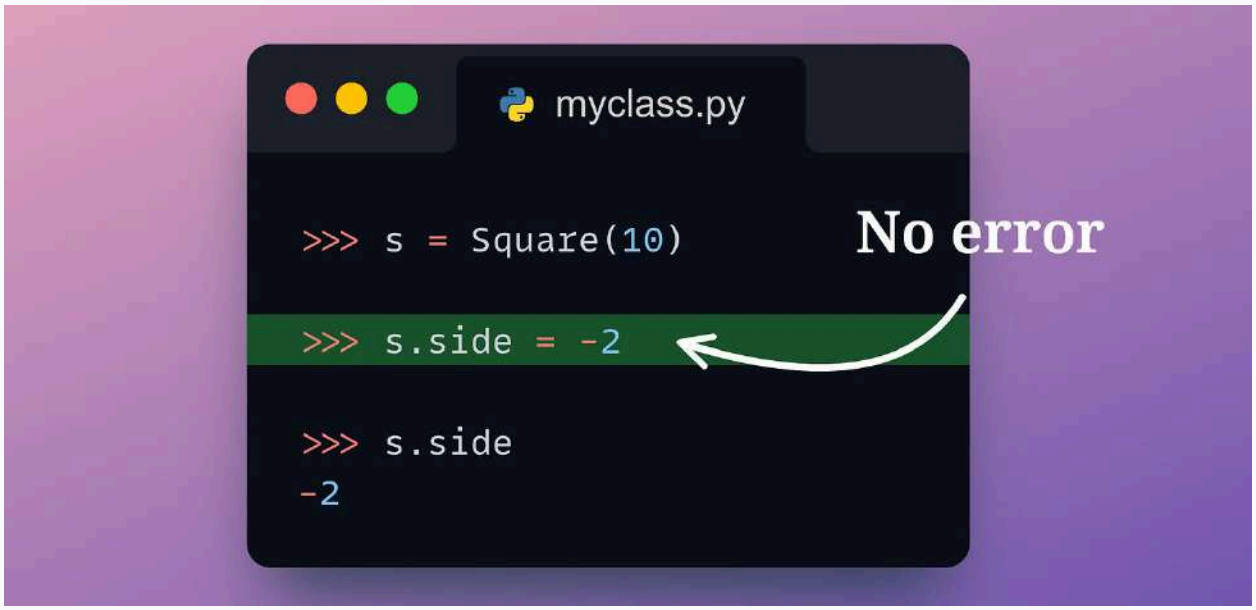
Python OOP

Getters and Setters

Dot notation provides a concise and elegant way to access and modify an object's attributes.



Yet, with dot notation, we can not validate the updates made to an attribute. This means we can assign invalid values to an instance's attributes, as shown below:



One common way to avoid this is by defining a setter (`set_side()`), which validates the assignment step.



But explicitly invoking a setter method isn't as elegant as dot notation, is it?

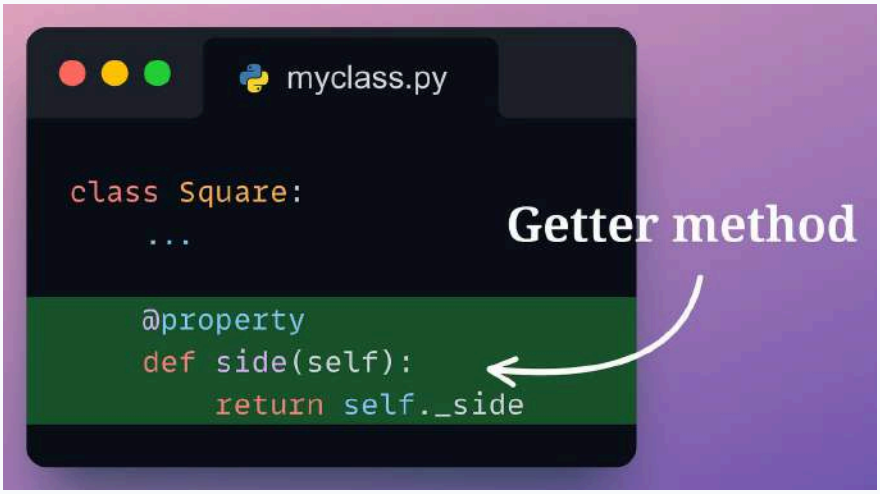
Ideally, we would want to use dot notation and still apply those validation checks. The @**property** decorator in Python can help. Here's how we can use it here.

First, define a getter as follows:

Declare a method with the attribute's name.

There's no need to specify any parameters for this method.

Decorate it with the @**property** decorator.



Next, define a setter as follows:



- Declare a method with the attribute's name.
- Specify the parameter you want to update the attribute with.
- Write the conditions as you usually would in any other setter method.
- Decorate it with the @**attribute-name.setter** decorator.

Now, you can use the dot notation while still having validation checks in place.

This approach offers both the validation and control of explicit setters and getters and the elegance of dot notations.



Descriptors in Python

In this chapter, let's continue our discussion on the above topic and discuss a limitation of the above approach. Moving on, we shall see how Descriptors in Python provide a much more elegant way of setting and getting values.

Limitations of @property decorator

Consider the above class implementation again:



The biggest issue here is that we must define a getter and setter for every instance-level attribute.

So what if our class has, say, 3 such attributes, and all must be positive?

Of course, we will have 3 getters and 3 setters, which makes the overall implementation long, messy, and redundant.



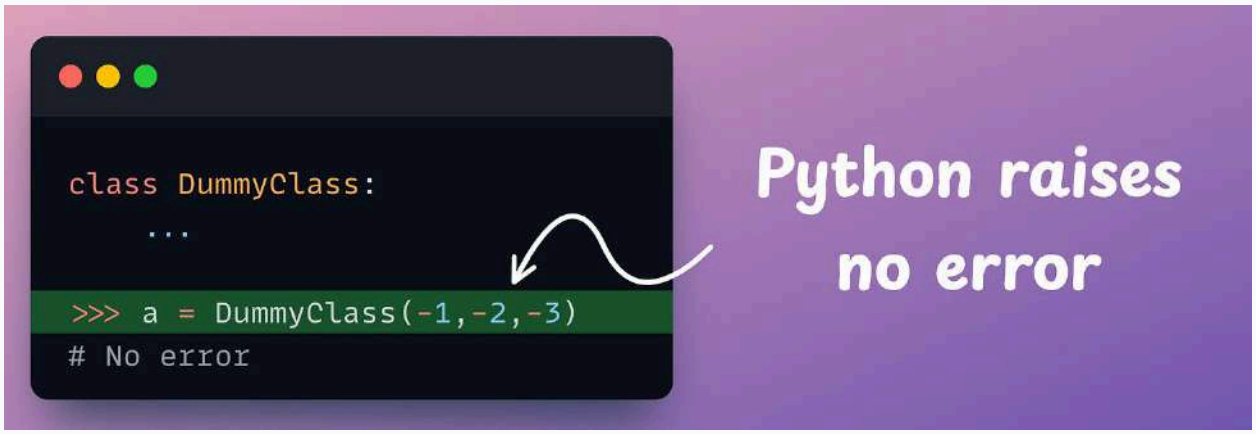
There's redundancy because every setter method will have almost the same lines of code (the if statements for validation).

Also, if you think about it, the getter methods are somewhat redundant and unnecessary too, as they just return an attribute.

If that is clear, there's one more issue with the above implementation.

Recall what I mentioned earlier: "Our class will have 3 such instance-level attributes, and all must be positive?"

See what happens when we create an object with an invalid input:

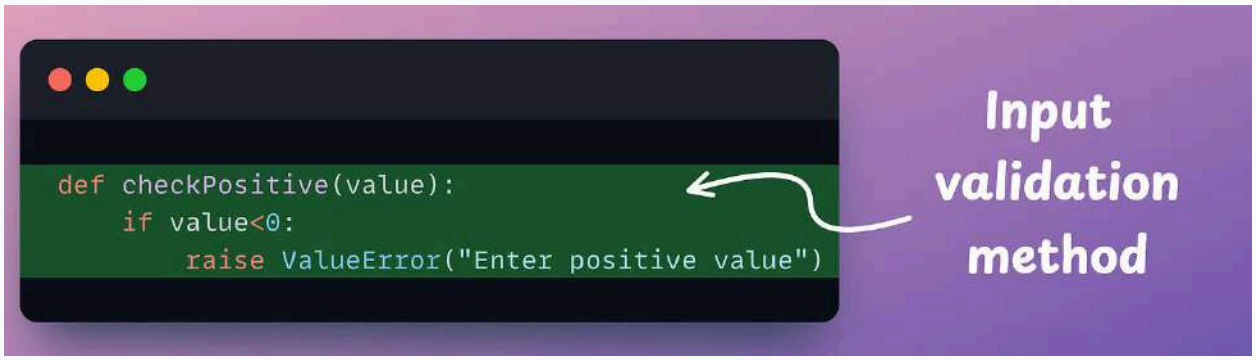


As depicted above, Python does not raise any error, when ideally, it should.

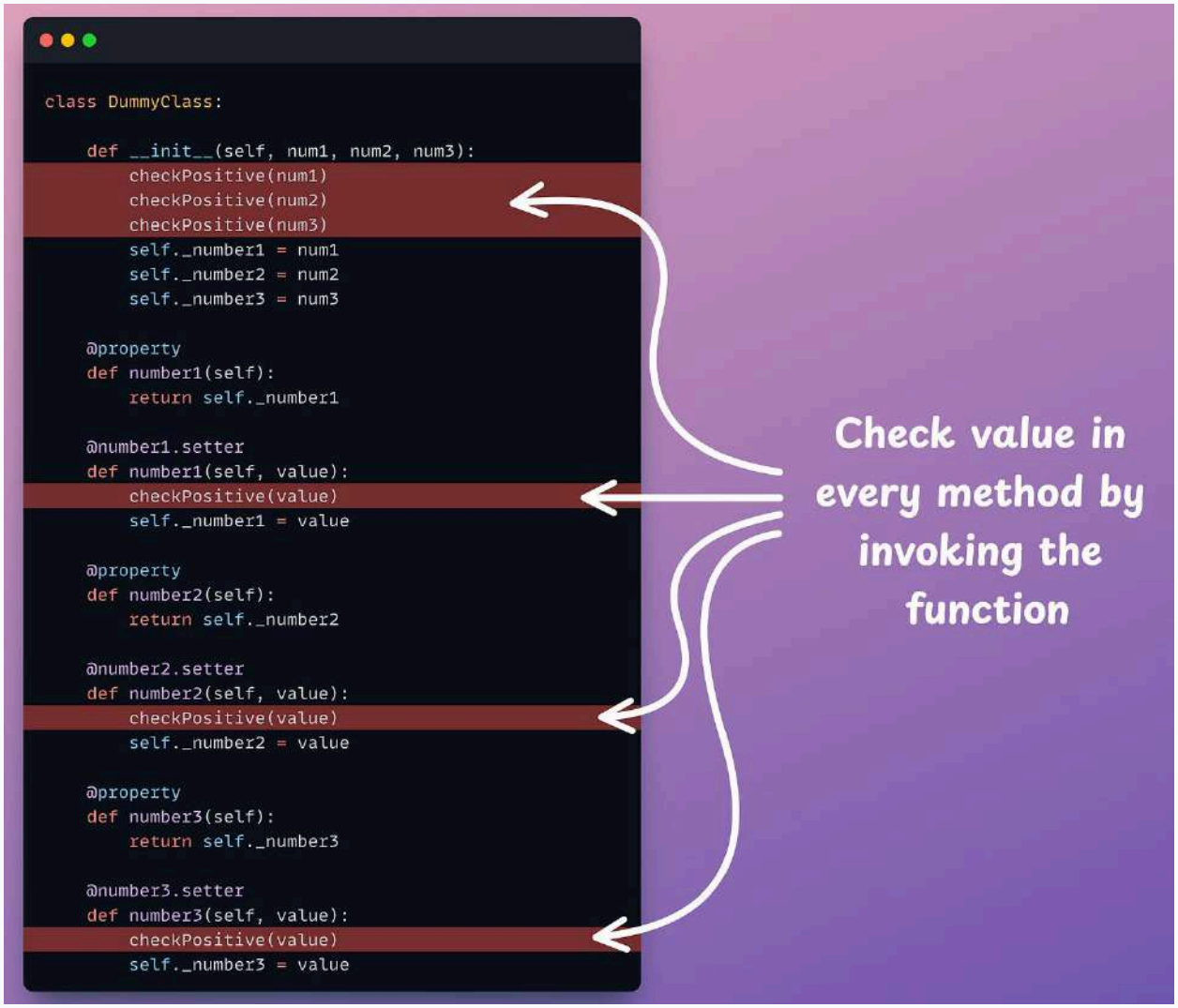
An imperfect solution

One common way programmers try to eliminate redundancy is by defining explicit validation functions.

For instance, we can define a function that just validates the value received, as demonstrated below:



Next, we can invoke this method wherever needed:



But this does not solve the problem either:

We still have explicit and redundant function calls in each setter method.

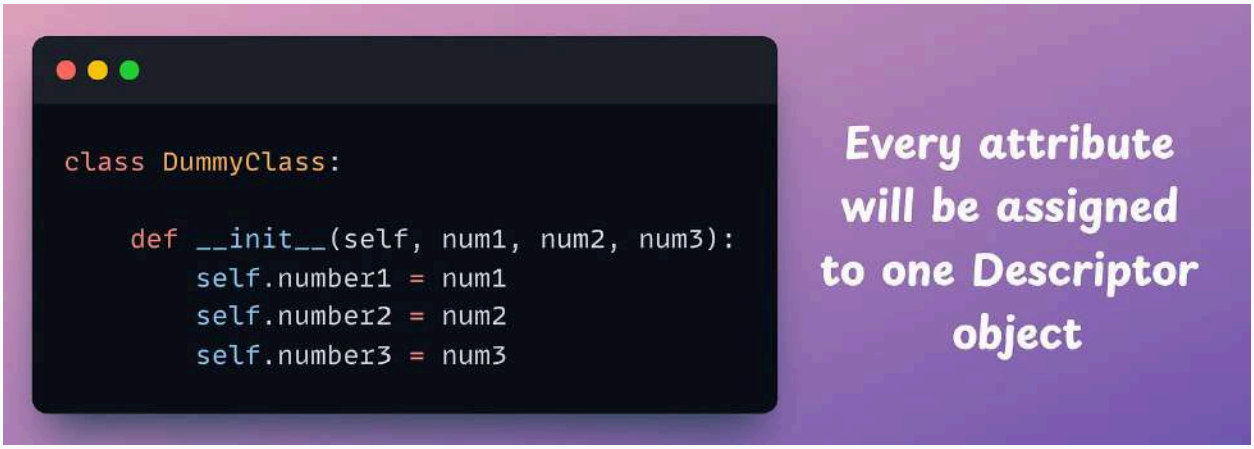
All getter methods still do the same thing and have high redundancy.

And most importantly, the __init__ method is now messed up with multiple function calls.

Descriptors

Simply put, Descriptors are objects with methods (like __get__, __set__, etc.) that are used to manage access to the attributes of another class. So, every descriptor object is assigned to only one attribute of another class.

And just to be clear, this "another class" is the class we are primarily interested in — the DummyClass we saw earlier, for instance.



- The attribute number1 → gets its own descriptor.
- The attribute number2 → gets its own descriptor.
- The attribute number3 → gets its own descriptor.

A typical Descriptor class is implemented with three methods, as shown below:



The __set__ method is called when the attribute is assigned a new value. We can define the custom checks here.

The __set_name__ method is called when the descriptor object is assigned to a class attribute. It allows the descriptor to keep track of the name of the attribute it's assigned to within the class.

The __get__ method is called when the attribute is accessed.

Also:



The instance parameter refers to the object of the desired class — DummyClass().

The owner parameter is the desired class itself — DummyClass.

The value parameter is the value being assigned to an attribute of the desired class.

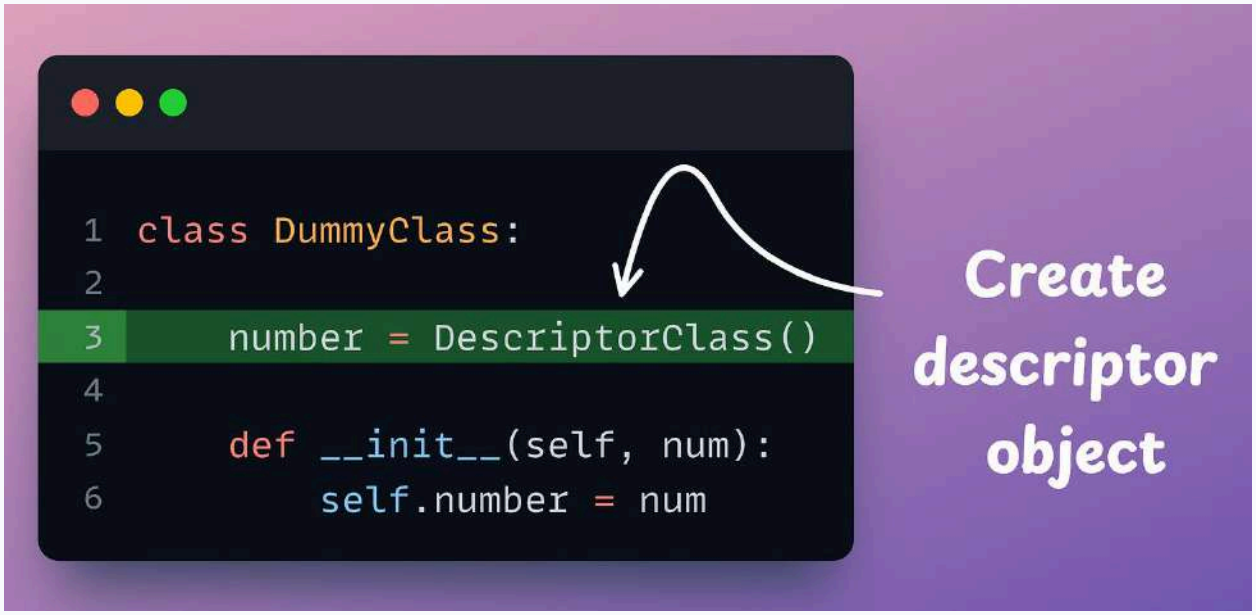
The name parameter is the name of the attribute.

If it's unclear, let me give you a simple demonstration.

Consider this Descriptor class:



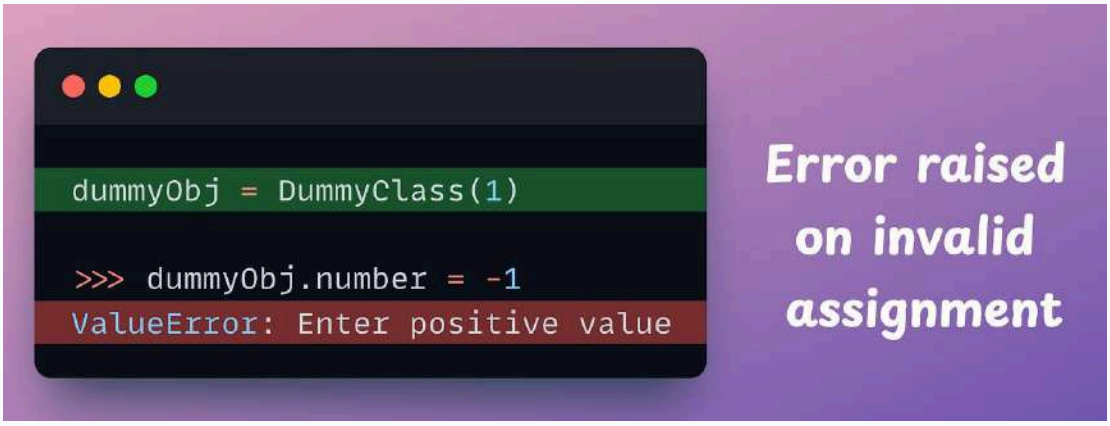
I'll explain this implementation shortly, but before that, let's consider its usage, which is demonstrated below:



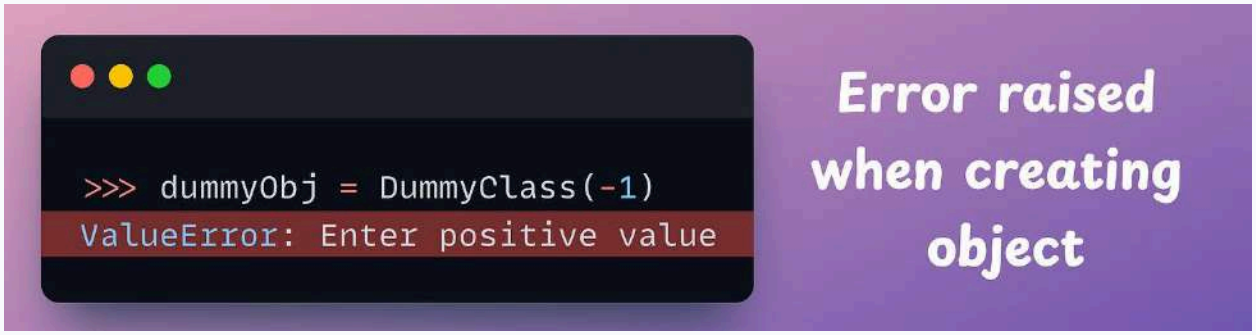
Now, let's go back to the DescriptorClass implementation:

- __set_name__(self, owner, name): This method is called when the descriptor is assigned to a class attribute (line 3). It saves the name of the attribute in the descriptor for later use.
- __set__(self, instance, value): When a value is assigned to the attribute (line 6), this method is called. It raises an error if the value is negative. Otherwise, it stores the value in the instance's dictionary under the attribute name we defined earlier.
- __get__(self, instance, owner): When the attribute is accessed, this method is called. It returns the value from the instance's dictionary.

Now, see how this solution smartly solves all the problems we discussed earlier. Let's create an object of the DummyClass:



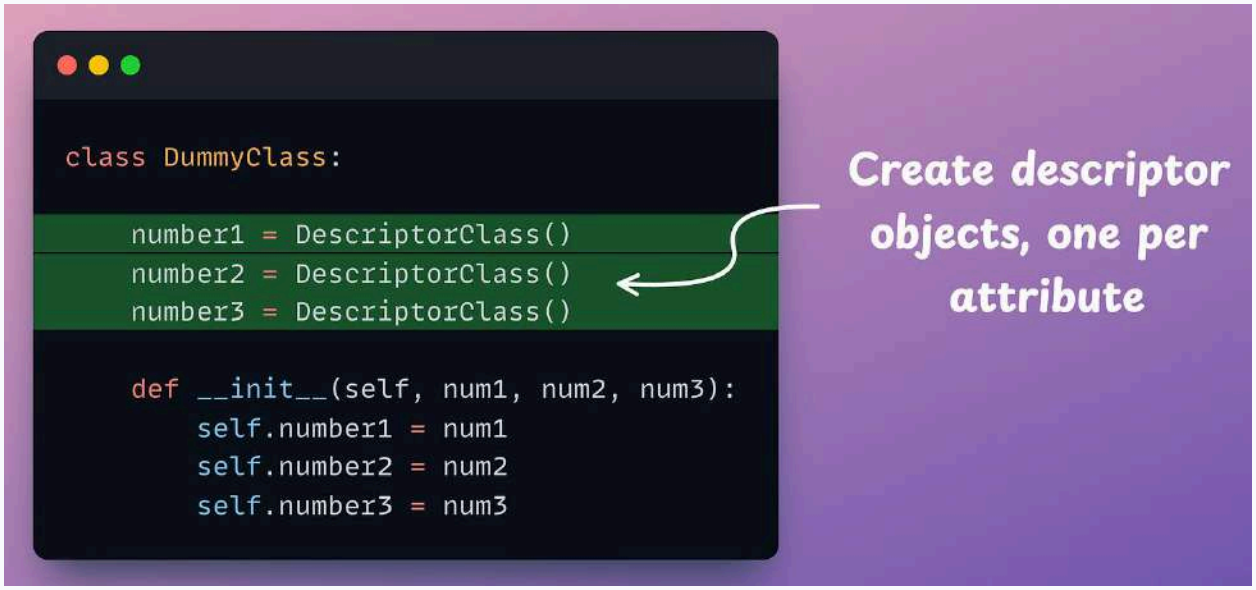
As depicted above, assigning an invalid value to the attribute raises an error. Next, let's see what happens when the attribute specified during the initialization is invalid:



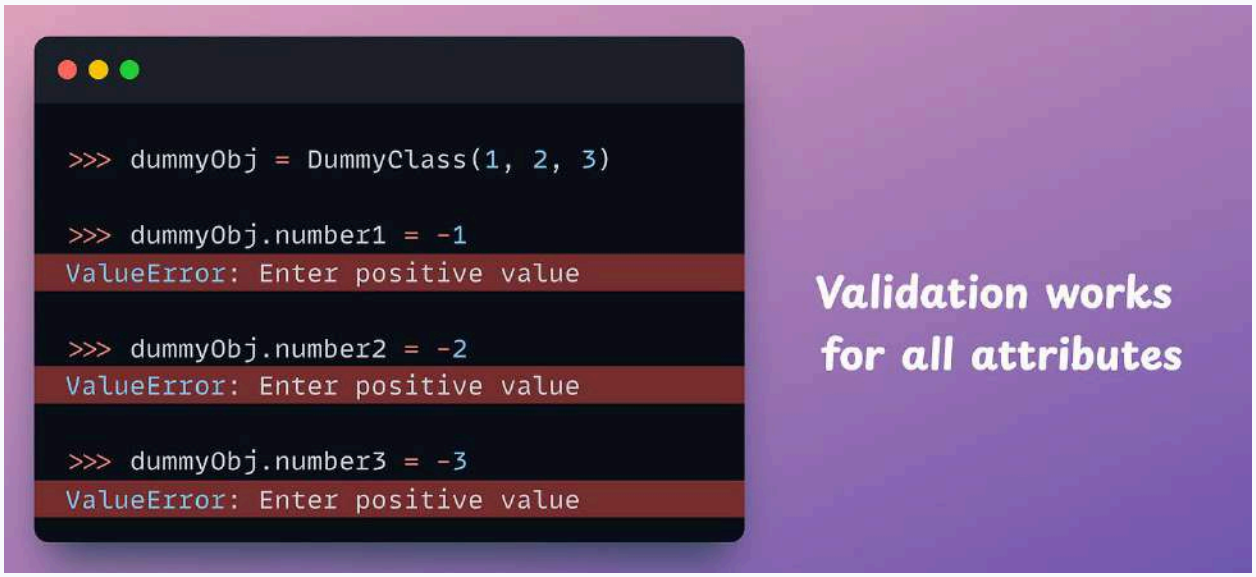
Great! It validates the initialization too.

Here, recall that we never defined any explicit checks in the __init__ method, which is super cool.

Moving on, let's define multiple attributes in the DummyClass now:



Creating an object and setting an invalid value for any of the attributes raises an error:



Works seamlessly!

Recall that we never defined multiple getters and setters for each attribute individually, like we did with the @property decorator earlier.

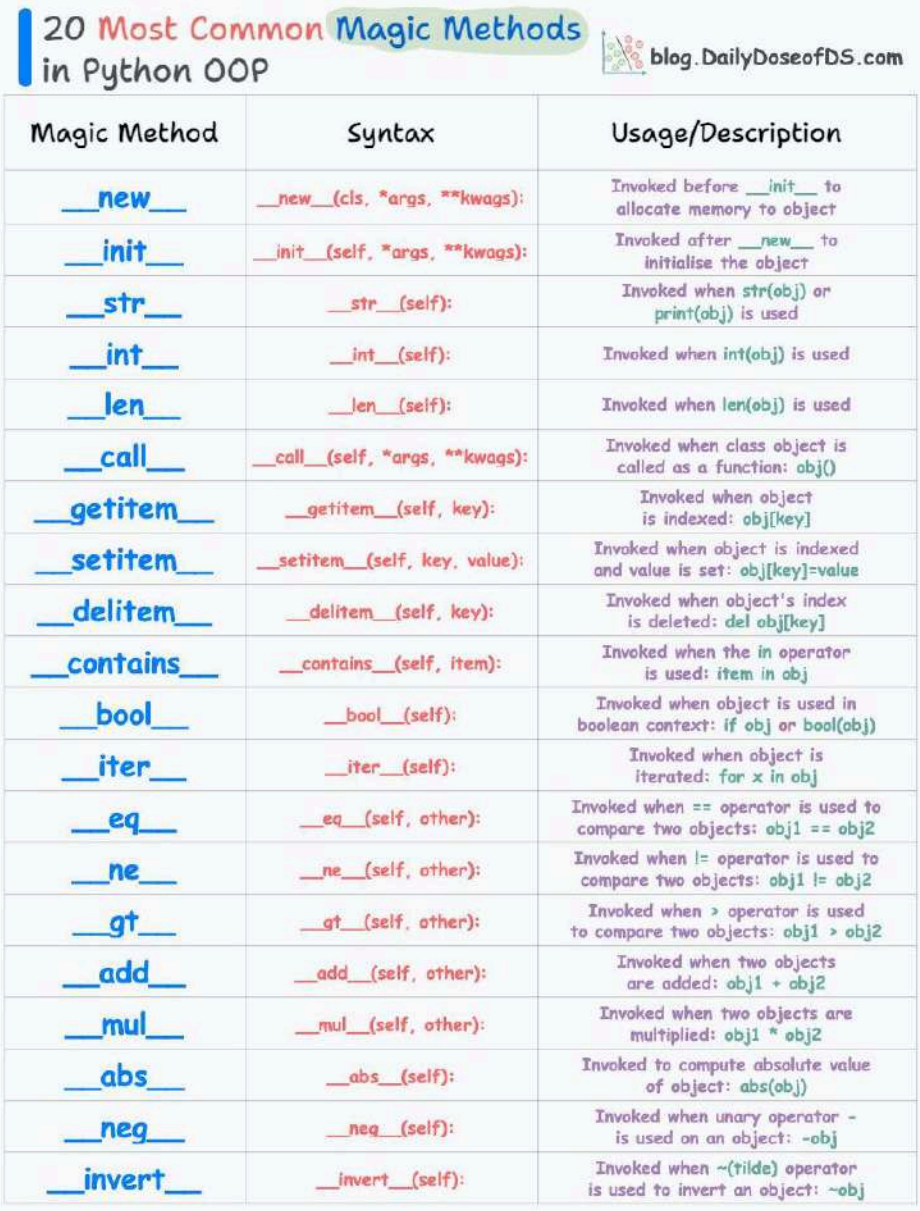
This is great, isn't it?

I find descriptors to be massively helpful in reducing work and code redundancy while also making the entire implementation much more elegant.

If you want to try them out, I prepared this notebook for you to get started: https://bit.ly/4cqsyM6.

20 Most Common Magic Methods

Here are 20 most common magic methods used in Python OOP:



Magic Method	Syntax	Usage/Description
__new__	__new__(cls, *args, **kwags):	Invoked before __init__ to allocate memory to object
__init__	__init__(self, *args, **kwags):	Invoked after __new__ to initialise the object
__str__	__str__(self):	Invoked when str(obj) or print(obj) is used
__int__	__int__(self):	Invoked when int(obj) is used
__len__	__len__(self):	Invoked when len(obj) is used
__call__	__call__(self, *args, **kwags):	Invoked when class object is called as a function: obj()
__getitem__	__getitem__(self, key):	Invoked when object is indexed: obj[key]
__setitem__	__setitem__(self, key, value):	Invoked when object is indexed and value is set: obj[key]=value
__delitem__	__delitem__(self, key):	Invoked when object's index is deleted: del obj[key]
__contains__	__contains__(self, item):	Invoked when the in operator is used: item in obj
__bool__	__bool__(self):	Invoked when object is used in boolean context: if obj or bool(obj)
__iter__	__iter__(self):	Invoked when object is iterated: for x in obj
__eq__	__eq__(self, other):	Invoked when == operator is used to compare two objects: obj1 == obj2
__ne__	__ne__(self, other):	Invoked when != operator is used to compare two objects: obj1 != obj2
__gt__	__gt__(self, other):	Invoked when > operator is used to compare two objects: obj1 > obj2
__add__	__add__(self, other):	Invoked when two objects are added: obj1 + obj2
__mul__	__mul__(self, other):	Invoked when two objects are multiplied: obj1 * obj2
__abs__	__abs__(self):	Invoked to compute absolute value of object: abs(obj)
__neg__	__neg__(self):	Invoked when unary operator - is used on an object: -obj
__invert__	__invert__(self):	Invoked when ~(tilde) operator is used to invert an object: ~obj

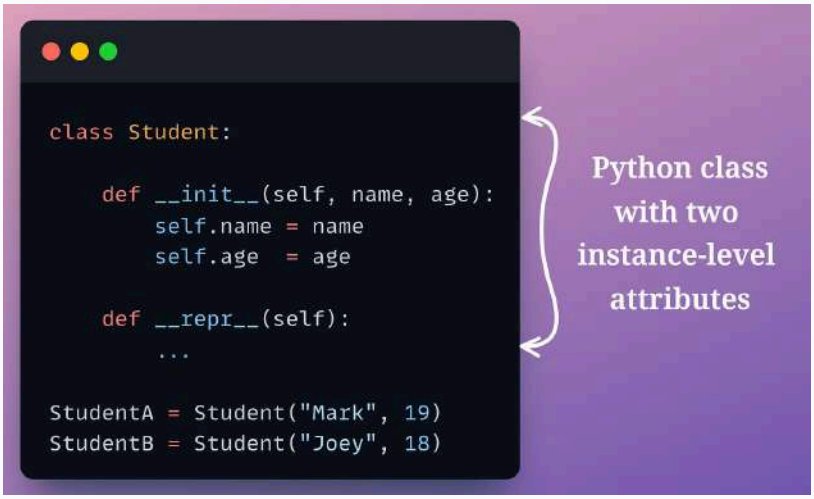
In my experience, these are possibly the only 20 magic methods you would ever need in most Python projects utilizing OOP.

Syntactically, they are prefixed and suffixed with double underscores, such as `__len__`, `__str__`, and many more. That is why they are also called "Dunder methods" — short for Double UNDERscore.

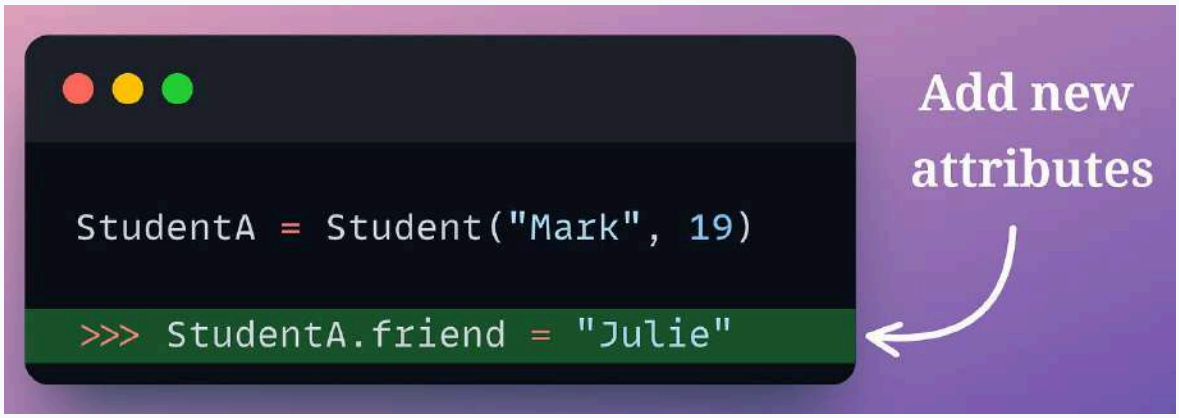
Memory Efficient Class Objects using Slots

When we define a class in Python, it is possible to dynamically add new attributes to its objects during run-time.

For instance, consider the following Python class here.

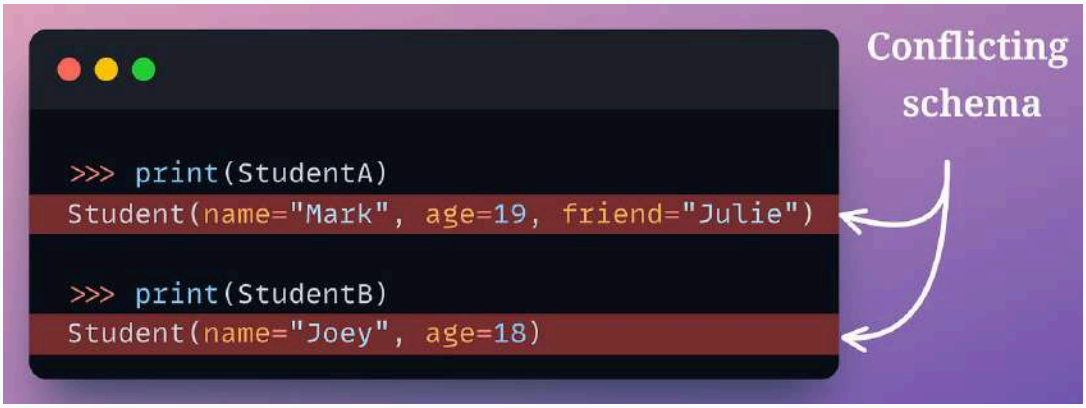


Here, it is perfectly legal to add new attributes to any object at run-time, as shown below:



However, this is not always recommended because:

- It may lead to bugs if the code assumes all class instances will always have the same attributes.
- It makes it difficult to debug code when objects keep on accumulating new attributes.
- It leads to a conflicting schema, etc.



Can we restrict this dynamicity?

Of course we can!

Defining a slotted class helps us achieve this. Simply put, it allows us to fix the instance-level attributes a class object can ever possess.

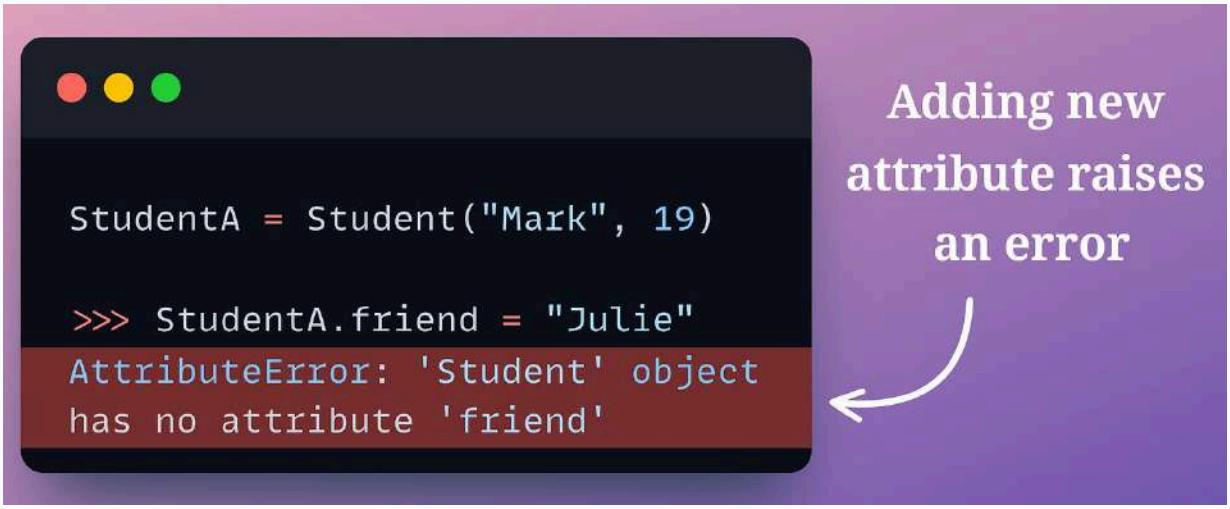
A slotted class is declared as follows:



Define a `__slots__` attribute with all attributes an object may possess.

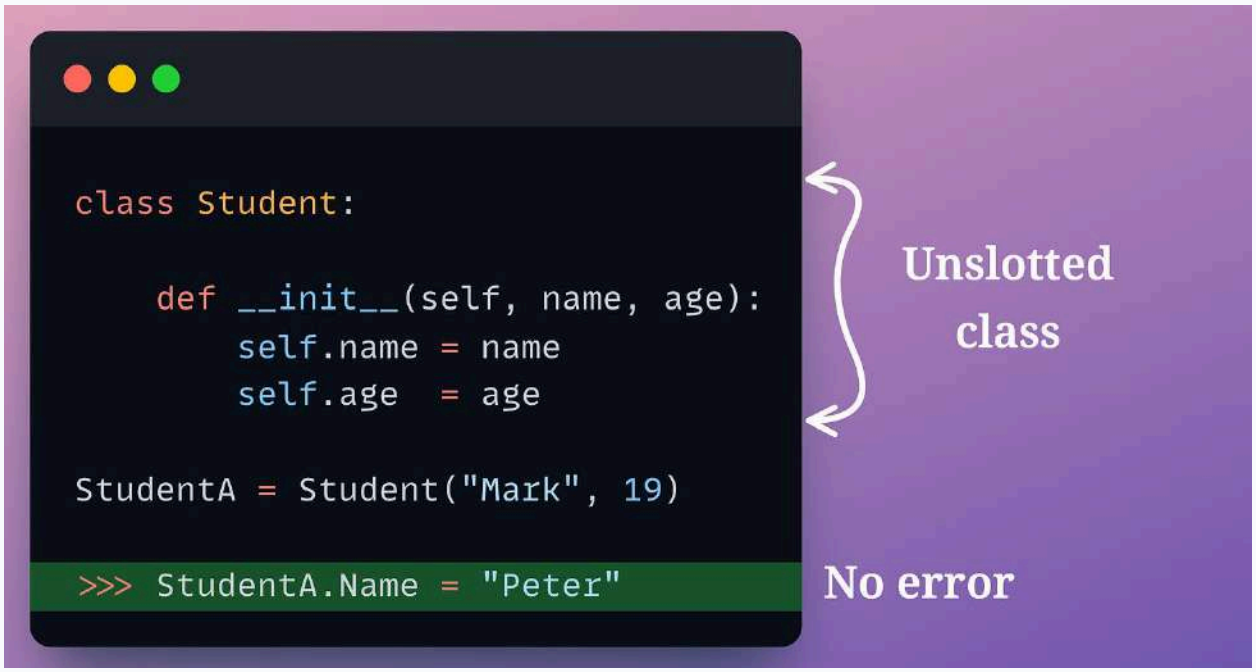
Done!

Now, if we try to add a new attribute at run-time, it raises an error:

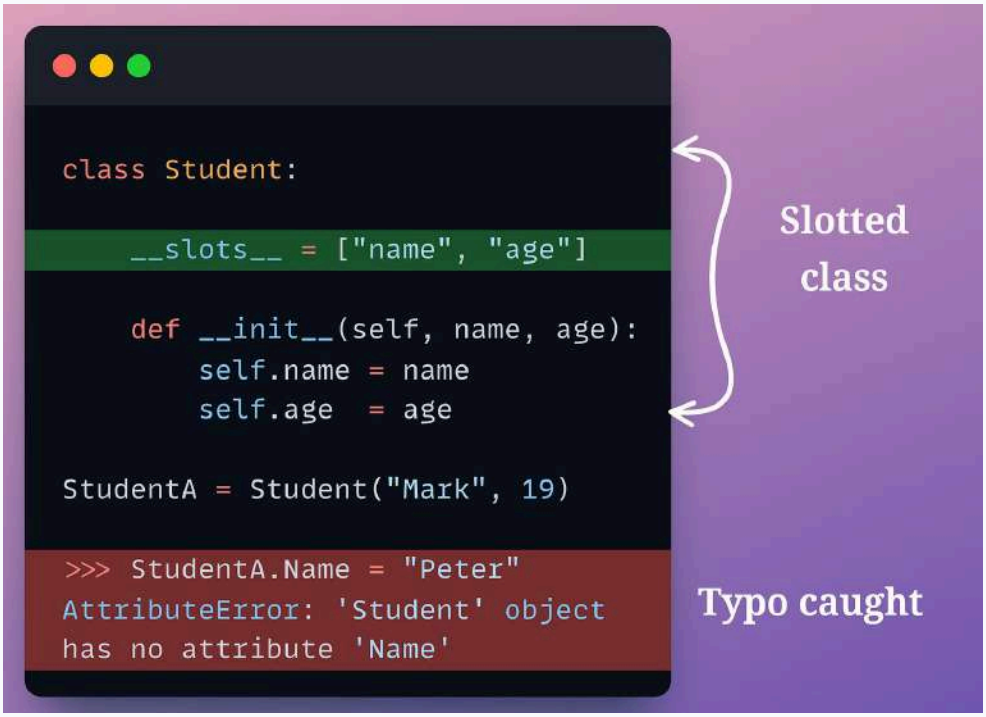


Declaring a slotted class has many advantages as well. For instance, it can help us avoid typical code typos. Say we want to change the name of `studentA`.

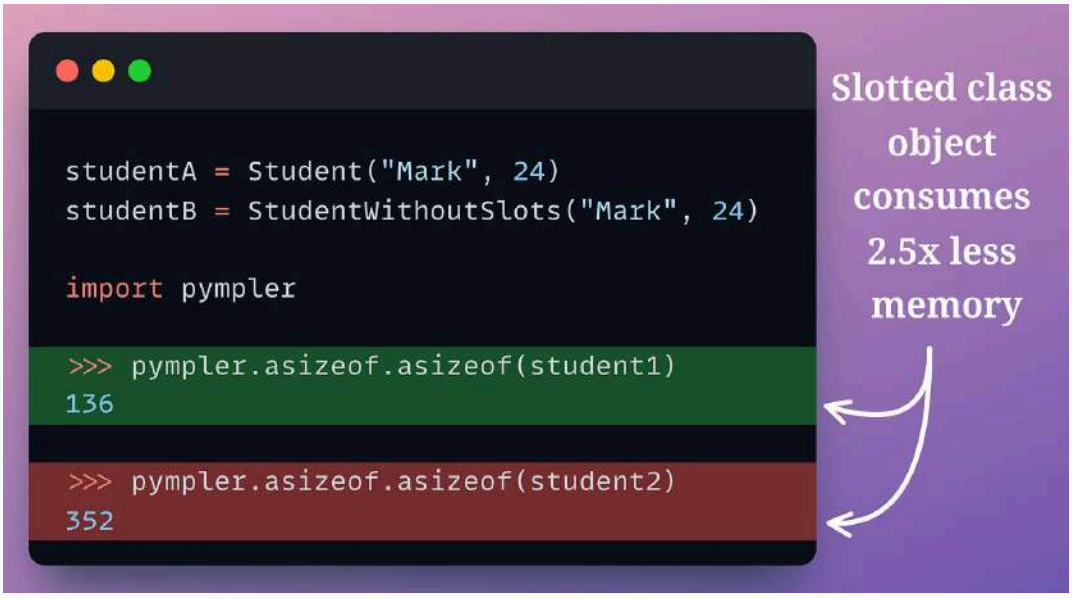
Here, instead of referring to the attribute name (with a lowercase n), we mistakenly wrote Name (with an uppercase n). A normal unslloted class will not raise an error, as shown below:



But a slotted class will catch this typo:

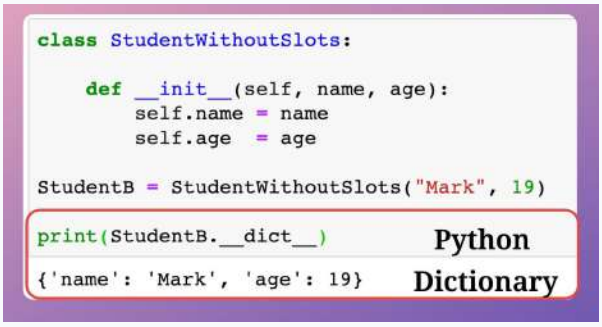


Declaring a slotted class provides memory advantages as well. As shown below, an object of the slotted class consumes 2.5x less memory than an object of a normal class:

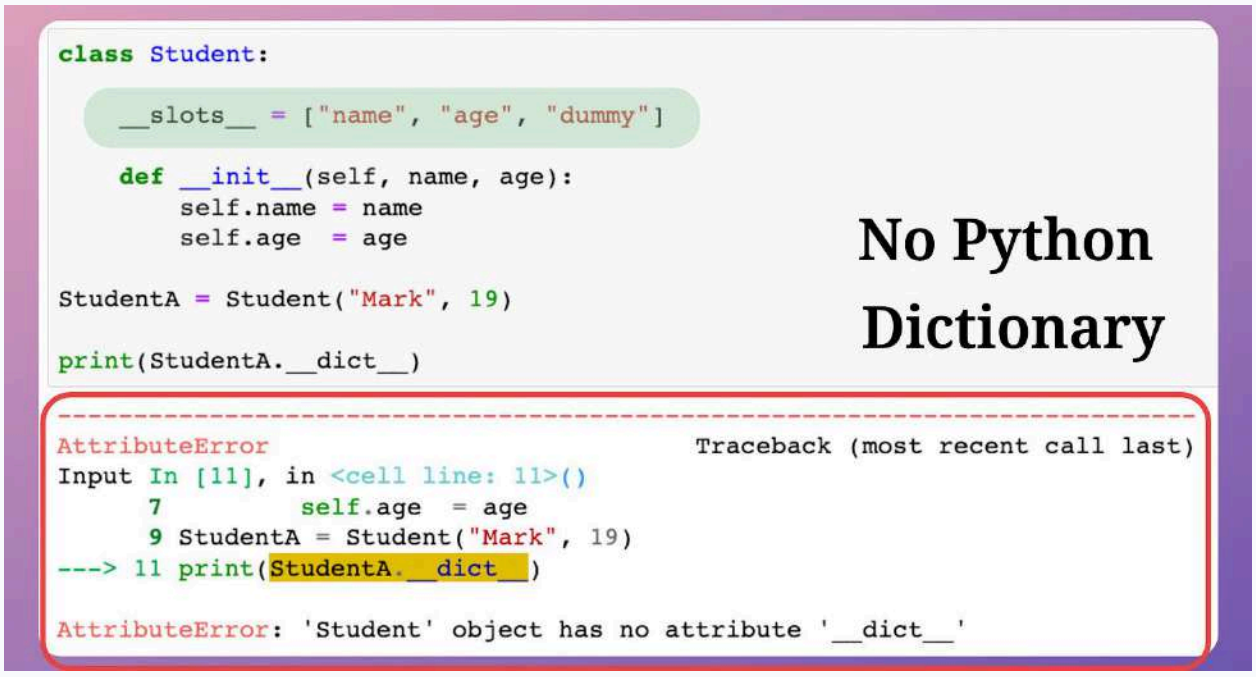


Why, you may wonder?

Typically, Python creates a dictionary for every object that maps attributes to their values. As a dictionary is mutable, this is precisely what allows us to add/delete attributes dynamically. But this introduces memory overheads as Python always tries to keep room for new attributes that may get added at some point.



In case of slotted class, however, Python can do away with this dictionary:



As a result, the object consumes less memory. As a departing note, if you don't want to dynamically add new attributes to an object, it is better to create a slotted class.

In fact, even if you know the attributes that a class object will ever possess, but some of these attributes are not available during the object's initialization, you can still declare them in the `__slots__` class attribute and assign a value to them later in the program whenever they are available.

Why Don't We Invoke model.forward() in PyTorch?

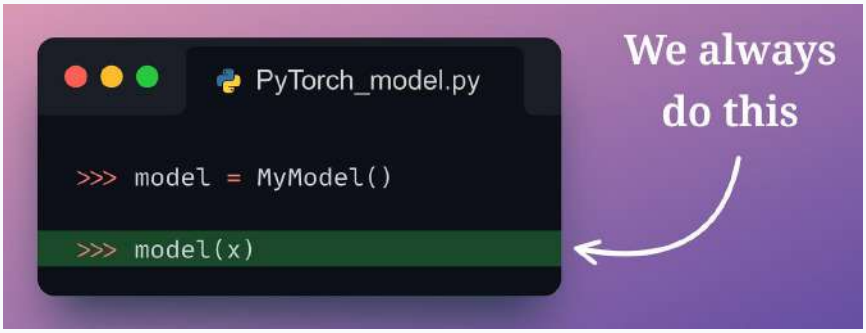
In PyTorch, the forward pass is implemented in the forward() method, as demonstrated below:



Here, have you ever wondered that when we want to run the forward pass, we rarely invoke this forward() method:



Instead, we always invoke the model (a class object) as demonstrated below, as it was a function:



We can also verify that model is a class object:



How can a class object be invoked like a function and what are we missing here?

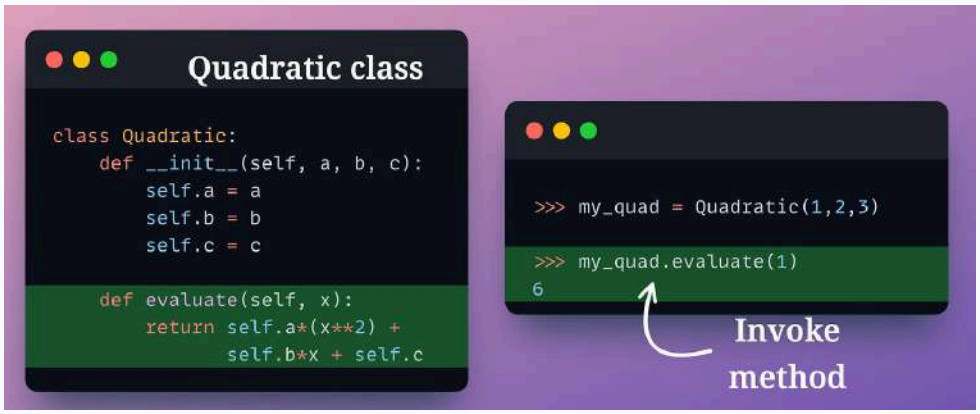
Let's understand!

A simple example

Consider we want to evaluate the following quadratic:

$$f(x)=ax2+bx+c$$

One way is to define a method that accepts the input and returns the value of the quadratic, as shown below:

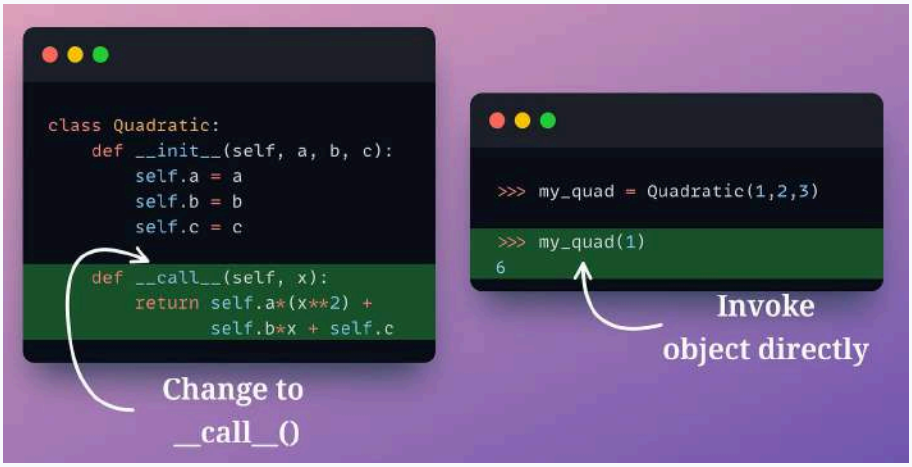


Of course, there is nothing wrong with this approach.

But there is one smart and elegant way of doing this in Python. Instead of explicitly invoking a method, we can define the `__call__()` magic method.

This magic method allows you to define the behavior of the class object when it is invoked like a function (like this: `object()`).

Let's rename the `evaluate()` method to `__call__()`.



As a result, we can now invoke the class object directly instead of explicitly invoking a method. This can have many advantages. For instance:

- It allows us to implement objects that can be used in a flexible and intuitive way.
- It allows us to use a class object in contexts where a callable object is expected — using a class object as a decorator, for instance.

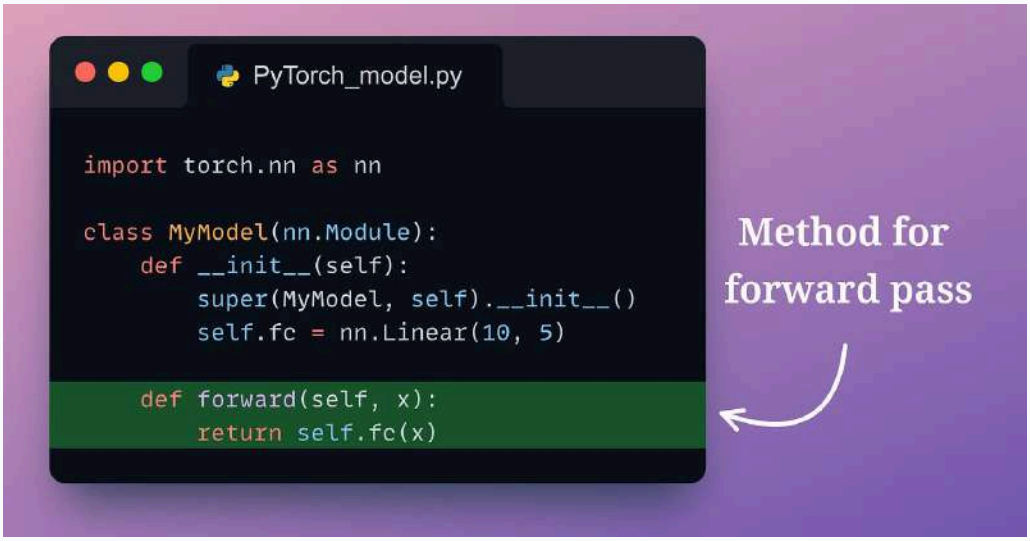
What is callable?

In Python, a callable is any object that can be called using parentheses and may return a value. For instance, a function is a callable object (one that can be called/invoked).



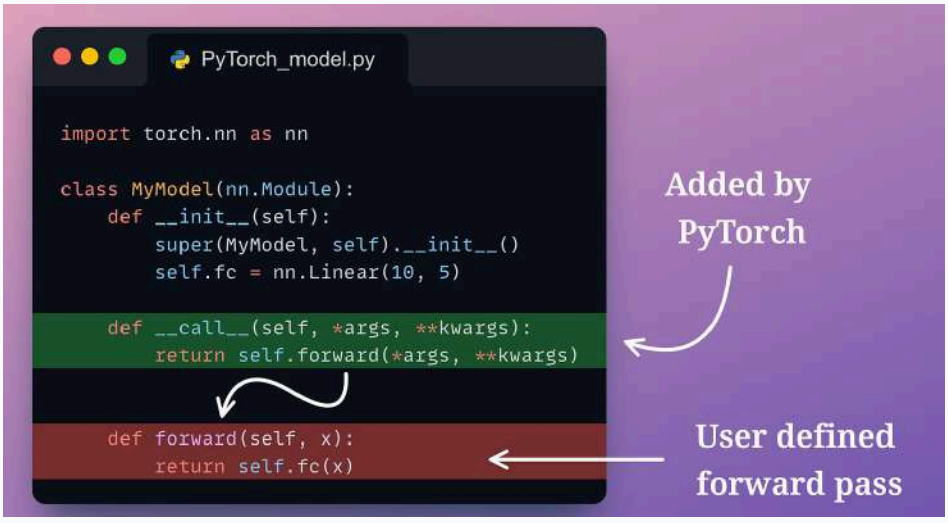
Coming back to PyTorch

This is what happens when we build deep learning models with PyTorch. For instance, consider the PyTorch class again:



As you may have already guessed, the model object can be invoked because all PyTorch classes implicitly declare the `__call__()` method themselves. Within that `__call__()` method, they invoke the user-defined forward pass.

A simplified version of this is depicted below:



- PyTorch itself adds the `__call__()` method.
- The `__call__()` method invokes the user-defined `forward()` method.

This way, Python gets to know that the model object can be invoked like a function — `model()`.

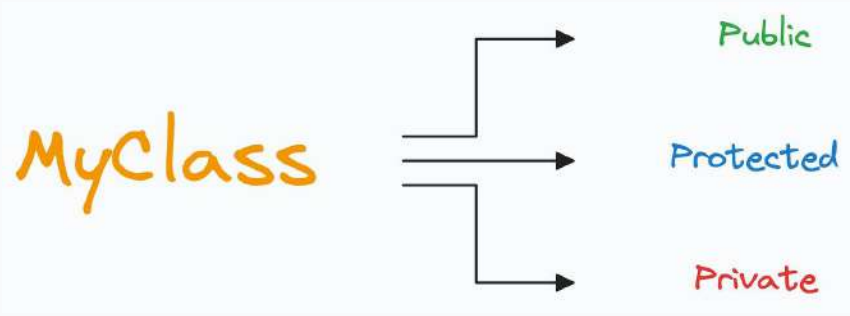
In fact, we can verify that we will get the same output no matter which way we run the forward pass:



All three ways return the same output.

True OOP Encapsulation is Missing From Python

Using access modifiers (public, protected, and private) is fundamental to encapsulation in OOP.



This is not just about Python but applicable to OOP in general. However, when we talk specifically about Python, we notice that it fails to deliver true encapsulation procedures, which one would want to leverage in their OOP code.

How?

Let's understand this in this chapter.

As you may already know, class attributes in OOP can be of three types:

- A public member is accessible everywhere inside and outside the base class, and it is inherited by all child classes.
- A protected member is accessible everywhere inside the base class and child class(es). It is not accessible outside the class.
- A private member is accessible only inside the base class.

But, with Python, there are no such strict enforcements. This is unlike many other programming languages like C++. Thus, protected members behave exactly like public members.

What's more, private members can be (somehow) accessed outside the class as well.

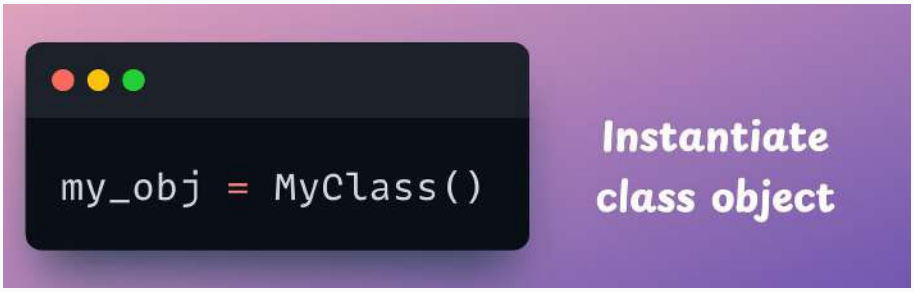
For better clarity, consider the following class implementation:



Syntactically speaking, in Python:

- A public member is declared with 0 leading underscores.
- A protected member is declared with 1 leading underscore.
- A private member is declared with 2 leading underscores.

Next, we instantiate a class object:



Moving on, as one would expect, the public attribute is accessible outside the class:



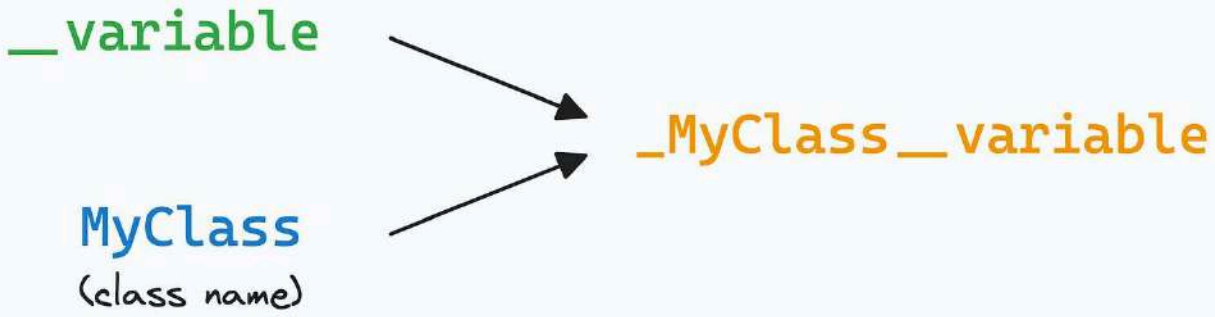
However, in a similar way, even the protected attribute is accessible outside the class, which, ideally, should not happen:



Lastly, when declaring private members, Python performs name mangling.

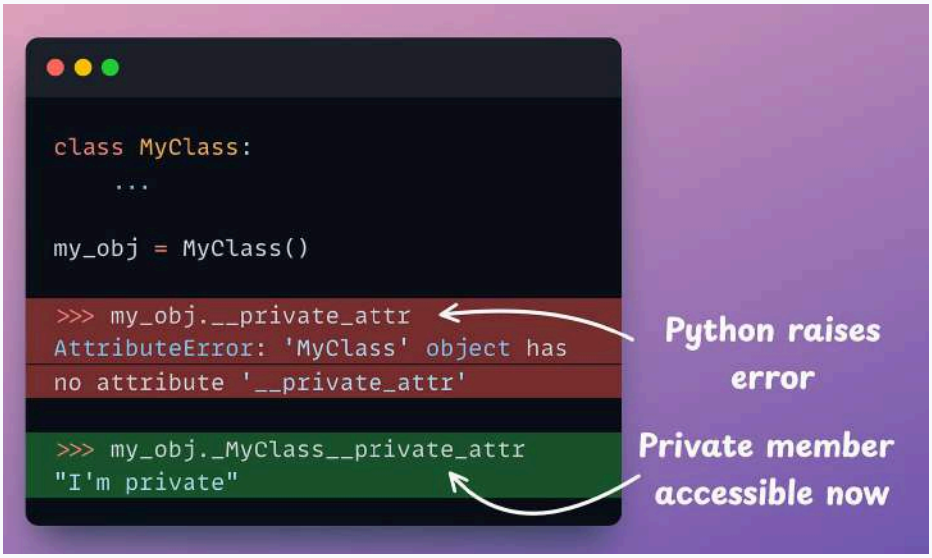
It is a technique used in programming to avoid naming conflicts between different classes.

Python performs name mangling by attaching underscore-prefixed class name (`_MyClass`) to all members with two leading underscores.



So, while the private member is not directly accessible using its original name, it can still be accessed with the modified name obtained from name mangling.

This is demonstrated below:

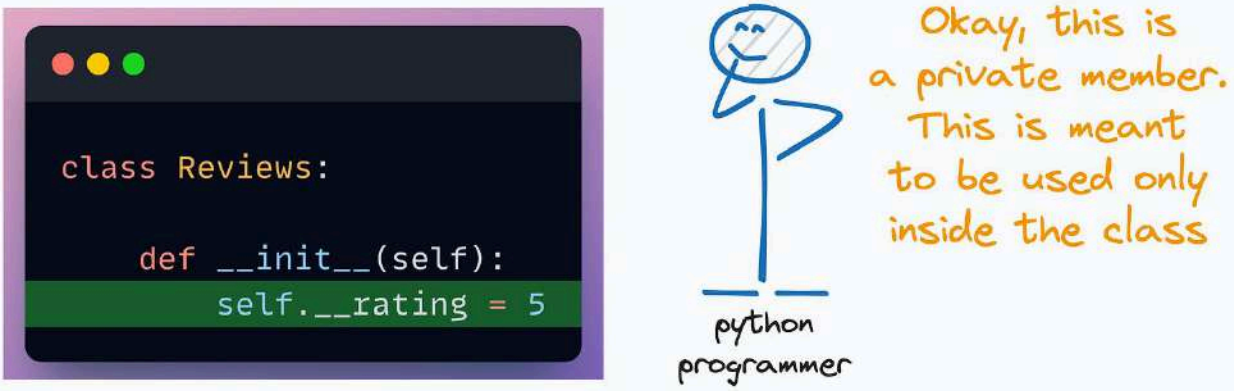


As demonstrated in all the examples above, protected members and private members (with name mangling) can be accessed like public members.

Thus, the point that every Python programmer using OOP must remember here is that Python never enforces encapsulation.

Instead, leveraging encapsulation procedures in Python mainly relies on conventions.

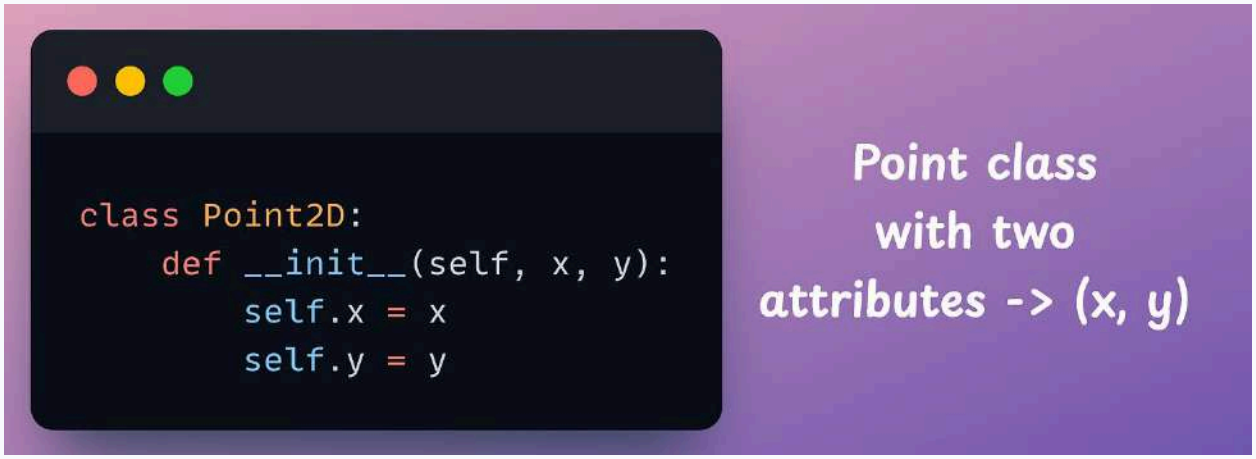
They are used to communicate the accessibility protocols of class members to other programmers.



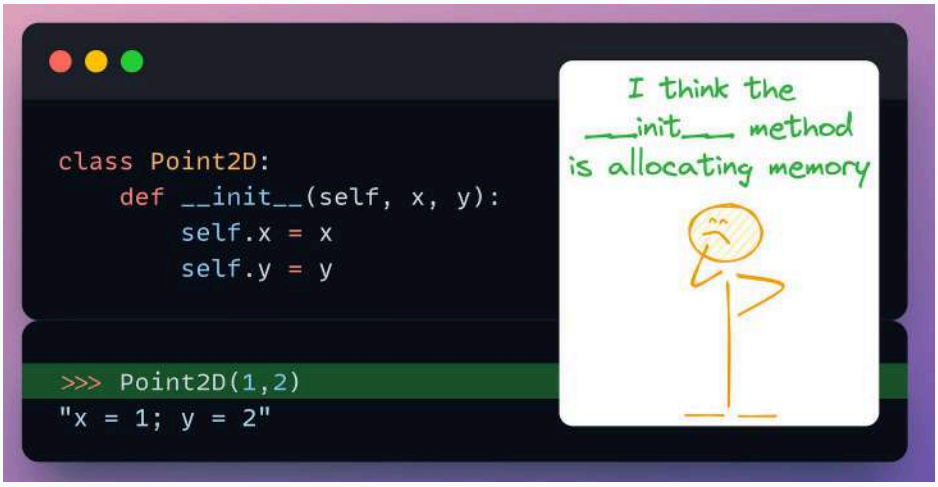
Thus, it is the programmer's responsibility to obey these conventions.

A Common Misconception About __init__()

Most Python programmers misinterpret the __init__() magic method in Python OOP. They think that it creates a new object, i.e., allocates memory to it. For instance, consider the Point2D class below:



Assume that there is also a __repr__() method, as it is not shown here. When we create an object (shown below), programmers believe that in the background, it is the __init__() method that is allocating memory to their object:

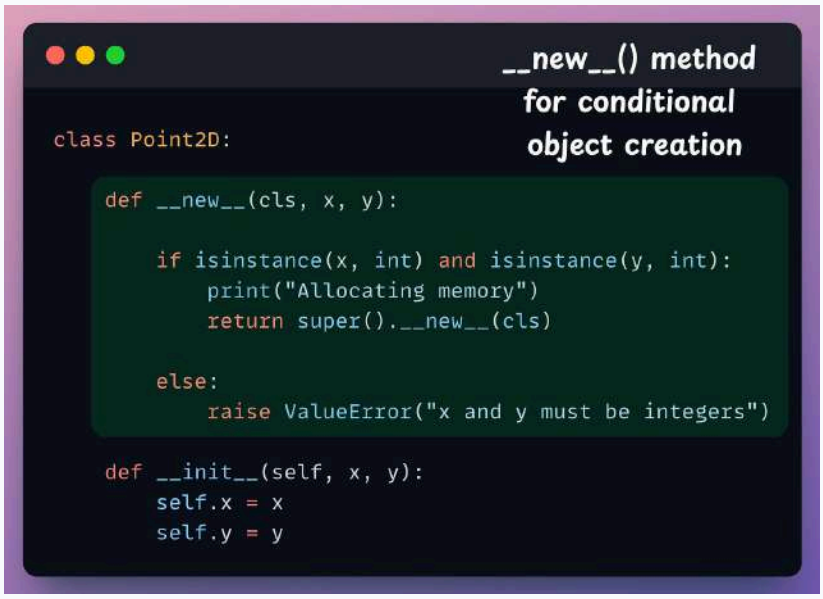


But that is not true.

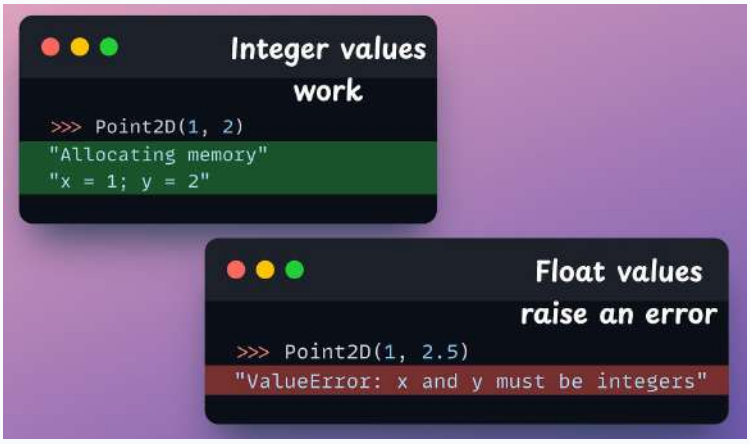
When we create ANY object in Python, the __init__() method NEVER allocates memory to it.

As the name suggests, `__init__()` only assigns value to an object's attributes, i.e., initialize the attributes.

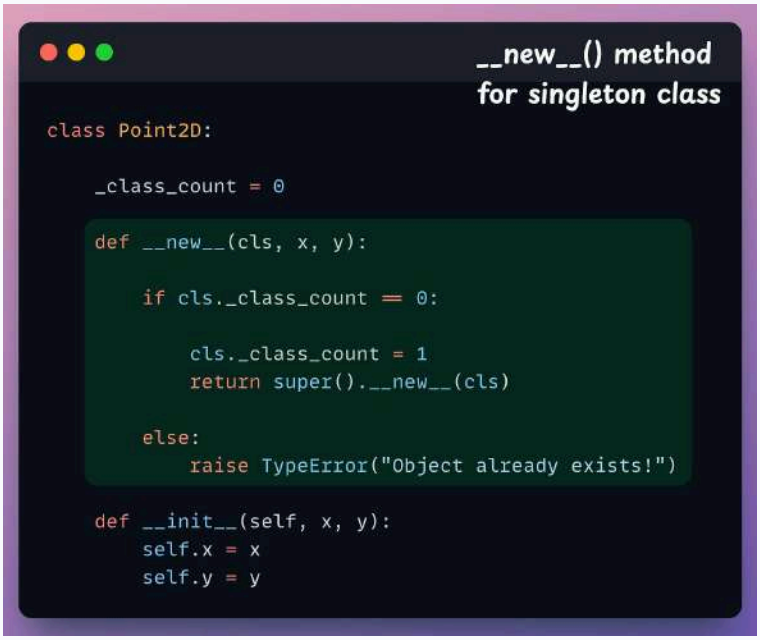
Instead, it's the `__new__()` magic method that creates a new object and allocates memory to it. To understand better, consider the class implementation below.



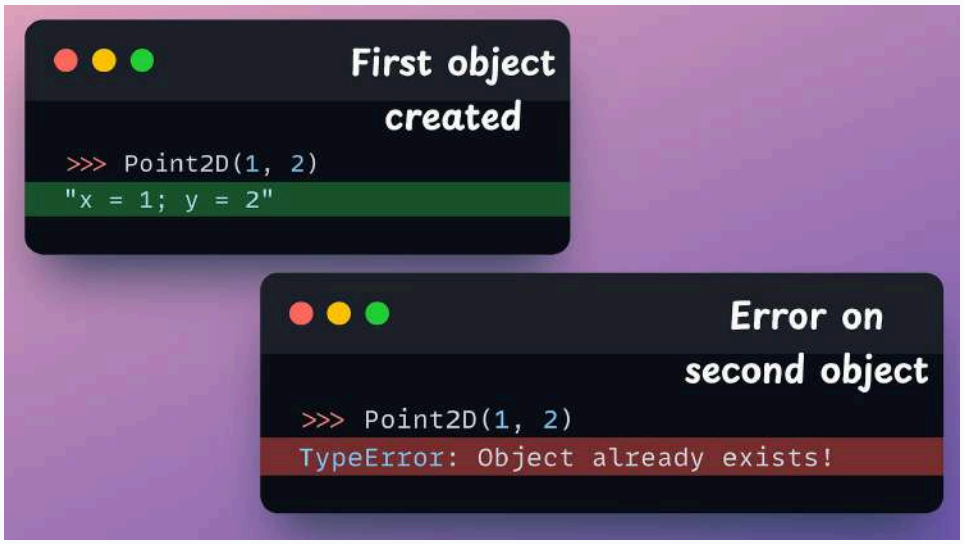
Here, we have implemented the `__new__()` method, which checks if the passed arguments are of integer type. Now, if we try to create an object of this class, Python would first validate the checks specified in the `__new__()` method and create a new object only when the specified conditions are true. This is evident from the image below:



Another popular use case of the `__new__` method is to define singleton classes — classes that can only have one object. For instance, consider the following class implementation:



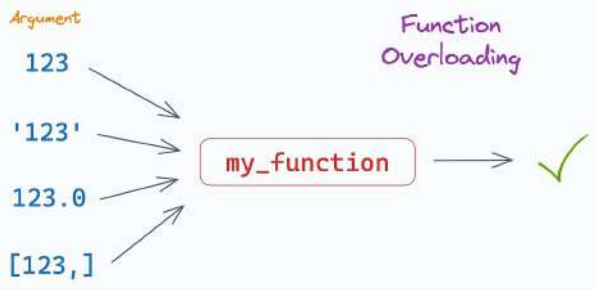
In the above code, the `__new__` method define a class variable `_class_count`. When a new object is instantiated, if the value of `_class_count=0`, the value of `_class_count` is updated to 1 and the object is returned. After creating the first object, a new object can never be created because the value of _class_count will never be 0. This is evident from the image below:



Function Overloading in Python

Function overloading is critical to polymorphism, wherein, we may have multiple functions with:

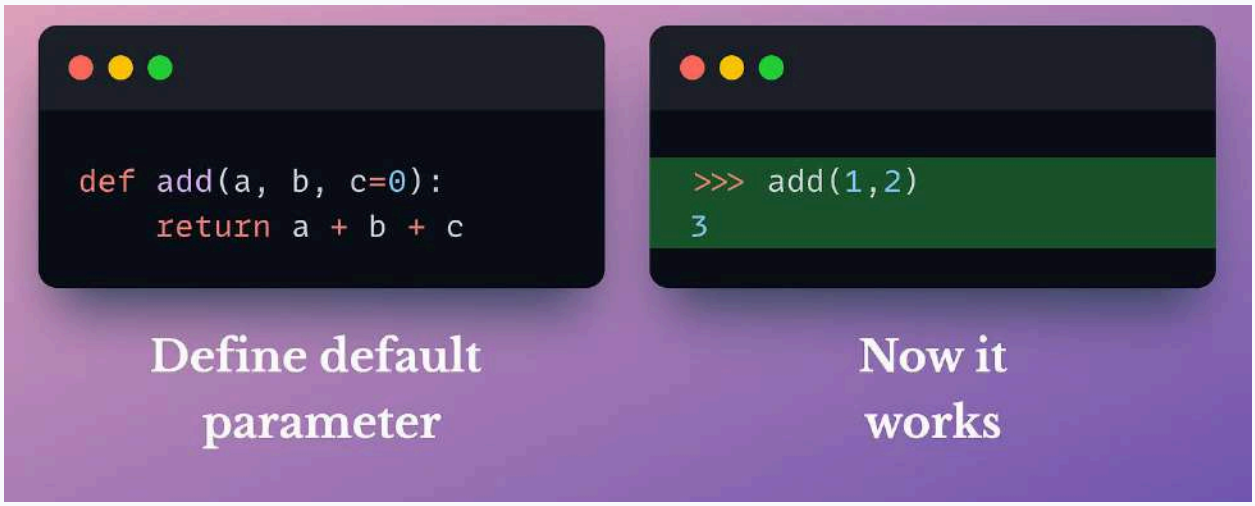
- Same name, and
- Different number (or type) of parameters.



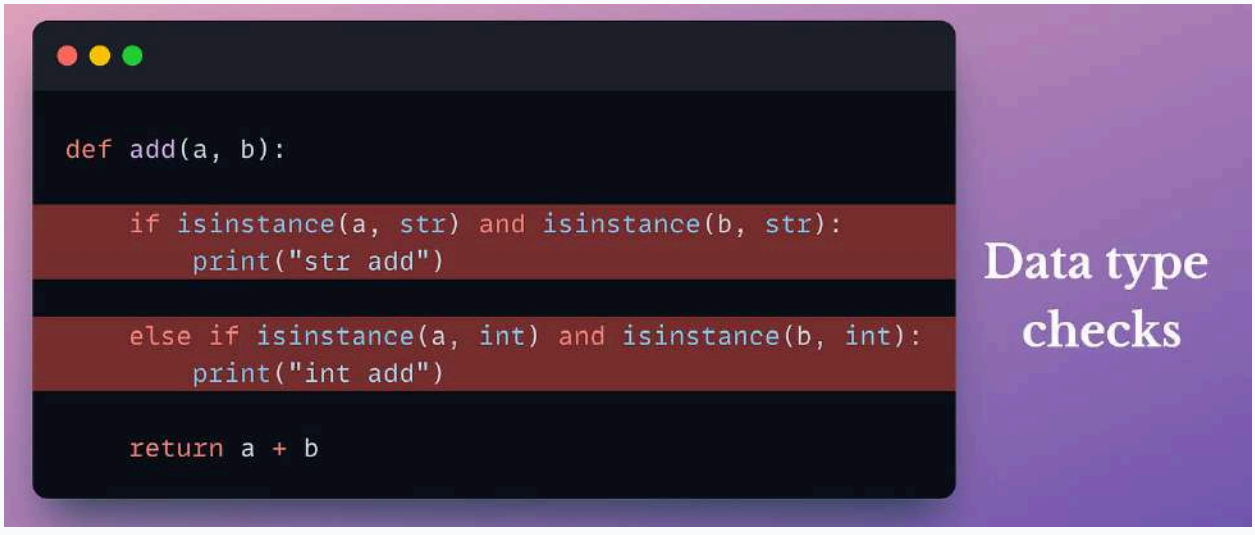
However, Python provides no native support for function overloading. In other words, if we define two (or more) functions with the same name and different parameters, Python will only consider the latest definition corresponding to that function name. This is depicted below.



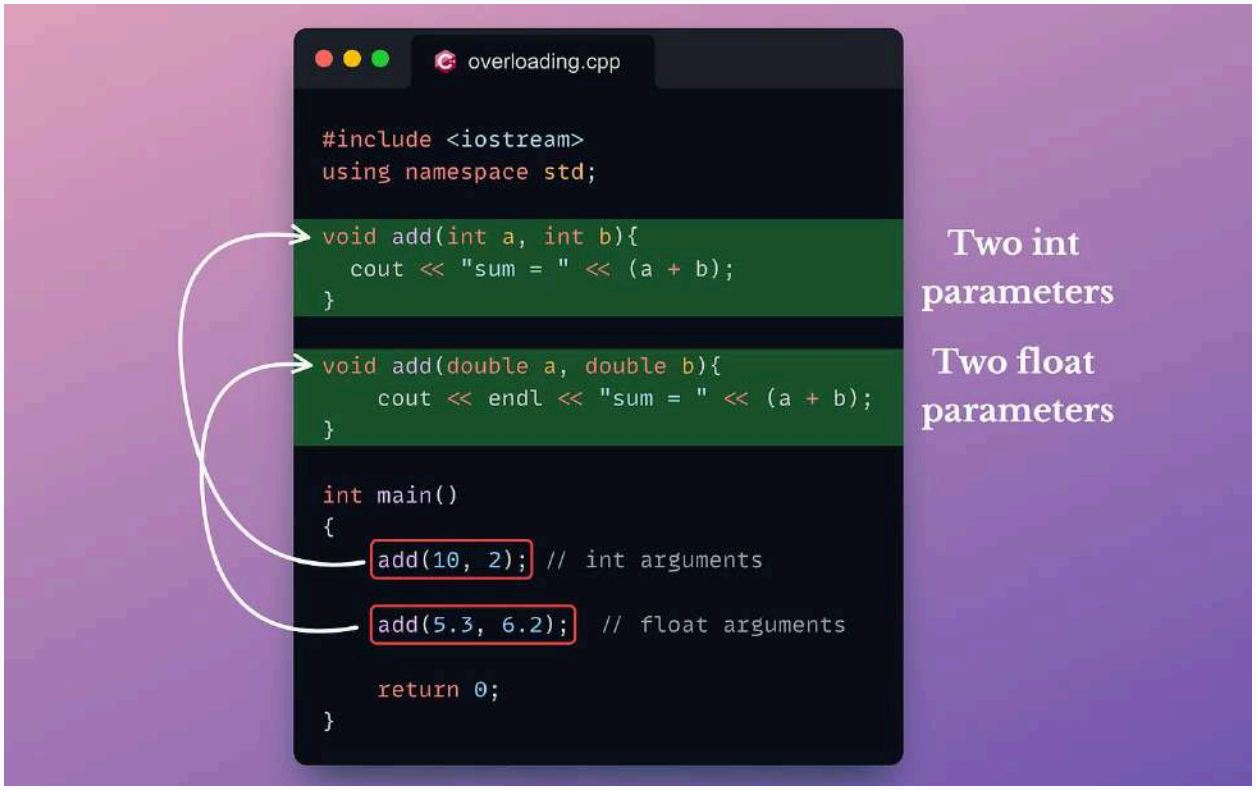
As depicted above, the latest definition of add() had three parameters. That is why passing two arguments raised an error. This restricts us from writing good polymorphic code in Python. Of course, there are ways to prevent this error, as depicted below:



Also, if the same parameter can take multiple data types, we can write (somewhat) polymorphic code by adding if statements using `isinstance()`.

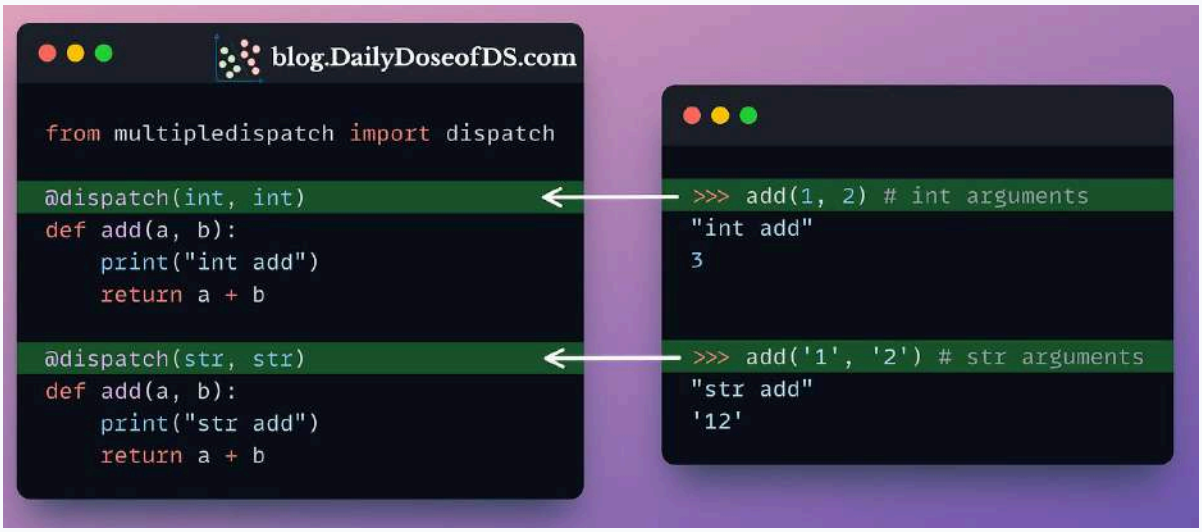


But this isn't as elegant as defining multiple functions with different data types like we can do in, say, C++, is it?



C++ automatically invokes the correct method corresponding to the argument data type. While exploring this for one of my projects, I found a pretty handy way to enable function overloading in Python

The @dispatch decorator from the Multidispatch library allows us to leverage the standard and elegant function overloading in Python, as demonstrated below:



As depicted above, we have multiple functions with the same name and different parameters. The `@dispatch` decorator allows us to invoke the correct function corresponding to the parameters passed during the function call.

Another cool thing about the `@dispatch` decorator is that it raises an error when the function call does not match any of the function definitions:



This allows us to identify errors which can often go unnoticed.

In a dummy experimentation, I did notice a slight increment in the run time. But the order of increase was in nanoseconds or so, which can be safely ignored.